



ChatGPT

Middle JavaScript Developer Курс-квест

Модуль 1 - Вступление и базовая диагностика

Урок 1 - Что должен знать Middle-разработчик

Шаг 01:

Миссия 1

Максим работает над веб-страницей и получает баг: кнопка не реагирует на клик в iOS Safari. Он думает, может ли решением быть просто игнор багов в старых браузерах? Что должен выбрать Middle-разработчик?

Подсказка: Подумайте о пользователях и кроссбраузерной совместимости.

Ответ:

- Проанализировать, почему на Safari кнопка не работает, и найти совместимый способ, либо полифилл
- Сказать, что современные браузеры важнее, пусть iOS Safari игнорируется
- Удалить кнопку для всех пользователей Safari

Шаг 02:

Миссия 2

Света пишет модуль, который должен обрабатывать JSON-ответы с API. Иногда приходит значение null вместо объекта — что делать?

Подсказка: Надёжность важнее «быстрого решения» без проверки.

Ответ:

- Проверять, что значение не null перед доступом к его полям

- Предполагать, что никогда null не придёт
- Ловить ошибку на уровне catch и игнорировать

🔗 Шаг 03:

Миссия 3

Артём внедряет библиотеку для форматирования дат. Он может использовать стороннюю библиотеку, или написать свою небольшую функцию. Что более оправдано для Middle-разработчика?

Подсказка: Задумайтесь: что важнее — надёжность, поддержка и повторное использование.

Ответ:

- **Выбрать проверенную библиотеку, если нужно поддерживать разные локали и форматы**
- Сразу писать свою, чтобы было легче контролировать
- Использовать библиотеку только частично, смешивая с собственным кодом без тестов

🔗 Шаг 04:

Миссия 4

Сопоставьте понятия и определения, важные для Middle-JS-разработчика:

Понятия:

Ответ:

- Замыкания (closure)
- Промисы (Promise)
- Асинхронная функция (async/await)
- Обработчик событий (event handler)

Определения:

1. Способ описать функцию, которая может приостановить выполнение до завершения асинхронной операции
2. Функция, которая сохраняет доступ к переменным своей внешней области даже после того, как эта область завершила выполнение
3. Объект, представляющий результат асинхронной операции и позволяющий навешивать .then/.catch
4. Функция, вызываемая при наступлении определённого события, например, click или input

Подсказка:

Подумай, что делает промис, как async/await над ним, что такое closure — часто связанные с областью видимости, и как события вызываются в браузере.

- **Замыкания (closure)** — Функция, которая сохраняет доступ к переменным своей внешней области даже после того, как эта область завершила выполнение
- **Промисы (Promise)** — Объект, представляющий результат асинхронной операции и позволяющий навешивать .then/.catch
- **Асинхронная функция (async/await)** — Способ описать функцию, которая может приостановить выполнение до завершения асинхронной операции
- **Обработчик событий (event handler)** — Функция, вызываемая при наступлении определённого события, например, click или input

🔗 Шаг 05:

Миссия 5

Сопоставьте термины, связанных с архитектурой фронтенда:

Термины:

Ответ:

- Single Page Application (SPA)
- Server-Side Rendering (SSR)
- Progressive Web App (PWA)
- Static Site Generation (SSG)

Описание:

1. Приложение, работающее в офлайн-режиме, с пуш-уведомлениями и обновлениями, как будто как нативное приложение
2. Генерация страниц заранее на сервере или во время сборки, а не на клиенте при каждом запросе
3. Рендеринг HTML на сервере, чтобы контент был готов до отправки клиенту
4. SPA: клиент загружает единственный HTML, потом навигация и рендер на стороне клиента без перезагрузки страницы

Подсказка:

Обратите внимание на разницу, где и когда происходит рендеринг и когда нужен офлайн-режим.

- **Single Page Application (SPA)** — SPA: клиент загружает единственный HTML, потом навигация и рендер на стороне клиента без перезагрузки страницы
- **Server-Side Rendering (SSR)** — Рендеринг HTML на сервере, чтобы контент был готов до отправки клиенту
- **Progressive Web App (PWA)** — Приложение, работающее в офлайн-режиме, с пуш-уведомлениями и обновлениями, как будто как нативное приложение
- **Static Site Generation (SSG)** — Генерация страниц заранее на сервере или во время сборки, а не на клиенте при каждом запросе

Шаг 06:

Миссия 6

Сопоставьте процессы разработки, которые Middle-разработчик должен понимать:

Процессы:

Ответ:

- Code Review
- Continuous Integration (CI)
- Version Control / Git Workflow
- Automated Testing

Описание:

1. Проверка кода другими членами команды перед слиянием в основную ветку
2. Система, в которой при добавлении изменений автоматически запускаются сборки и проверки (линейка тестов, линтеры и т.д.)
3. Использование коммитов, веток, pull/merge запросов, разрешение конфликтов
4. Использование тестов, написанных заранее, которые автоматически проверяют корректность работы модулей или функций

Подсказка:

Подумайте, как часто баги появляются из-за плохой интеграции, и как тесты и ревью могут помочь их избежать.

- **Code Review** — Проверка кода другими членами команды перед слиянием в основную ветку
- **Continuous Integration (CI)** — Система, в которой при добавлении изменений автоматически запускаются сборки и проверки (линейка тестов, линтеры и т.д.)
- **Version Control / Git Workflow** — Использование коммитов, веток, pull/merge запросов, разрешение конфликтов
- **Automated Testing** — Использование тестов, написанных заранее, которые автоматически проверяют корректность работы модулей или функций

Шаг 07:

Миссия 7

Вопрос: Юля готовит рефактор кода: старый компонент настолько громоздкий, что много логики прямо в JSX. Расположите шаги, как это сделать аккуратно, как Middle-разработчик:

1. Выделить повторяющуюся часть логики в отдельную функцию или хук
2. Проверить, что все тесты проходят до изменений
3. Удалить дубли и ненужные зависимости из компонента
4. Провести ревью изменений с коллегами
5. Запустить функциональное тестирование после изменений в staging / dev среде

Подсказка:

Сначала — убедиться, что ничего не сломается, потом улучшать, затем проверять, затем показывать, потом запускать.

Ответ:

- Проверить, что все тесты проходят до изменений
- Выделить повторяющуюся часть логики в отдельную функцию или хук
- Удалить дубли и ненужные зависимости из компонента
- Провести ревью изменений с коллегами
- Запустить функциональное тестирование после изменений в staging / dev среде

Шаг 08:

Миссия 8

Вопрос: Сергей хочет оптимизировать загрузку страницы: есть тяжёлые скрипты, шрифты, изображения. Расположите шаги как Middle-разработчик, чтобы оптимизировать безопасно и эффективно:

1. Проанализировать текущие показатели времени загрузки (Lighthouse, DevTools и др.)
2. Выявить ресурсы, которые тормозят (скрипты, изображения, блокирующие CSS и др.)
3. Использовать lazy loading, отложенную загрузку, разделение кода (code splitting)
4. Проверить, что после изменений страница корректно отображается на разных устройствах
5. Мониторить производительность после деплоя и в реальных условиях

Подсказка:

Оптимизация — это не просто уменьшить вес, а измерить, улучшить, проверить, и потом следить.

Ответ:

- Проанализировать текущие показатели времени загрузки (Lighthouse, DevTools и др.)

- Выявить ресурсы, которые тормозят (скрипты, изображения, блокирующие CSS и др.)
- Использовать lazy loading, отложенную загрузку, разделение кода (code splitting)
- Проверить, что после изменений страница корректно отображается на разных устройствах
- Мониторить производительность после деплоя и в реальных условиях

🔗 Шаг 09:

Миссия 9

Вопрос: Лена работает в команде, где новые фишки появляются быстро, но баги тоже. Она хочет внедрить процесс, чтобы качество не падало. Расположите шаги, как Middle-разработчик выстроит устойчивый рабочий процесс:

1. Внедрить линтер и форматтер, чтобы код соответствовал стилю команды
2. Настроить автоматические тесты (юнит, интеграция)
3. Организовать code review на каждую задачу, даже маленькую
4. Добавить CI, чтобы сборка и тесты запускались при каждом pull request
5. Проводить ретроспективы и собирать обратную связь по качеству после релизов

Подсказка:

Качество — это не одно действие, а серия связанных шагов: стиль, тесты, проверка, автоматизация и обратная связь.

Ответ:

- Внедрить линтер и форматтер, чтобы код соответствовал стилю команды
- Настроить автоматические тесты (юнит, интеграция)
- Организовать code review на каждую задачу, даже маленькую
- Добавить CI, чтобы сборка и тесты запускались при каждом pull request
- Проводить ретроспективы и собирать обратную связь по качеству после релизов

🔗 Шаг 10:

Миссия 10

Вопрос: Команда игры «Grow a Garden» добавляет функционал: игрок может сохранять прогресс, но иногда данные теряются при обновлении версии. Расположите шаги, как Middle-разработчик внедрит бы миграцию данных безопасно:

1. Спланировать структуру новой версии данных и определить, как старые записи будут преобразованы
2. Написать тесты, которые симулируют существующие данные и новые случаи
3. Написать миграционные скрипты, которые преобразуют старые данные в новый формат
4. Развернуть миграцию на тестовой / staging среде и проверить, что данные корректно сохраняются и читаются старым кодом, новым кодом
5. После успешного тестирования выполнить миграцию на продакшн и отслеживать ошибки и отклонения

Подсказка:

Миграция — это риск: сначала план, потом тесты, потом постепенное развёртывание, потом мониторинг.

Ответ:

- Спланировать структуру новой версии данных и определить, как старые записи будут преобразованы
- Написать тесты, которые симулируют существующие данные и новые случаи

- Написать миграционные скрипты, которые преобразуют старые данные в новый формат
- Развернуть миграцию на тестовой / staging среде и проверить, что данные корректно сохраняются и читаются старым кодом, новым кодом
- После успешного тестирования выполнить миграцию на продакшн и отслеживать ошибки и отклонения

🔗 Шаг 11:

Миссия 11

Аня создает небольшую утилиту на JavaScript и впервые использует `const`. Она не уверена, когда и где следует использовать `const` вместо `let`. Что правильнее приоритетно выбрать?

Подсказка: Подумай о безопасности и неизменности данных.

Ответ:

- **Использовать `const` по умолчанию, когда значение не планируется изменяться, и `let` только там, где нужна изменемость**
- Всегда использовать `let`, так как `const` может ограничивать гибкость
- Использовать `var`, потому что старый код поддерживает его лучше

🔗 Шаг 12:

Миссия 12

Борис пишет функцию, которая делает HTTP запрос к API. Иногда API возвращает ошибку, иногда — данные. Что он должен делать, чтобы обработка ошибок была надежной?

Подсказка: Подумай о том, что может пойти не так: сети, формат данных, статус ответа.

Ответ:

- **Проверять `status` ответа, ловить исключения при парсинге и учитывать случай плохой сети**
- Предполагать, что ответ всегда 200 OK и валиден
- Просто подавлять ошибки и возвращать пустой объект

🔗 Шаг 13:

Миссия 13

Кристина работает в команде с другими Middle разработчиками. У неё есть предложение: писать каждый компонент React с `PropTypes`, даже если используется TypeScript. Что она должна выбрать?

Подсказка: Подумай о том, какая гарантия типов уже есть и какая нагрузка на поддержку.

Ответ:

- **Использовать TypeScript типы, а `PropTypes` только там, где TS не охватывает (например, в сторонних модулях или динамическом JS)**
- Всегда использовать `PropTypes`, даже при TypeScript
- **Не использовать никакой проверки типов** — «работает же»

🔗 Шаг 14:

Миссия 14

Влад должен решить, как организовать структуру проекта: несколько файлов, папок, разделение по фичам или по слоям (`components`, `services` и др.). Что более подходит для Middle-уровня?

Подсказка: Подумай о масштабируемости и удобстве поддержки.

Ответ:

- **Организовать проект по модулям/фичам, так чтобы компоненты, сервисы и тесты одной фичи были рядом**
- Смотреть только на слои (components/services) без разделения по фичам
- Сложно, пусть каждый пишет как удобно

 **Шаг 15:**

Миссия 15

Женя замечает, что один из компонентов React часто перерендеривается без нужды. Что ей скорее всего стоит сделать?

Подсказка: Подумай про props, state и сравнения.

Ответ:

- **Использовать React.memo или проверку shouldComponentUpdate, чтобы предотвратить ненужные перерендеры**
- Игнорировать, пока не будет проблем с производительностью
- **Всегда перерендеривать** — безопаснее

 **Шаг 16:**

Миссия 16

Сергей планирует писать документацию к API, но думает: «Может обойтись просто комментариями в коде?» Что выберет Middle-разработчик?

Подсказка: Подумай, как новые коллеги или ты через год поймёшь, что делает метод.

Ответ:

- **Создать отдельную документацию: описание API, примеры, контракты, помимо комментариев**
- Положиться только на комментарии в коде
- Не писать ничего, используя только сам код как документацию

Урок 2 - Самодиагностика (тест + задачи)

 **Шаг 01:**

Миссия 1

Олег работает над интерфейсом и видит, что где-то используется `var`, где-то `let` и `const` без явной причины. Что сделает Middle-разработчик?

Подсказка: Подсказка: `const` и `let` появились, чтобы улучшить читаемость и безопасность кода.

Ответ:

- **Провести ревизию кода и стандартизировать использование let/const, убрать var**
- Оставить всё как есть, var всё ещё работает
- Заменить все var на let, даже там, где должно быть const

 **Шаг 02:**

Миссия 2

Марина пишет функцию, которая может получить неопределённый параметр. Что предпочтительнее?

Подсказка: Подсказка: безопасный код умеет обрабатывать случаи отсутствия данных.

Ответ:

- Проверить параметр на `undefined` и задать значение по умолчанию
- Предположить, что параметр всегда передаётся
- Бросить ошибку без проверки

Шаг 03:

Миссия 3

Илья интегрирует стороннюю библиотеку, через которую возможно много ошибок в консоли. Как он поступит?

Подсказка: Подумай, как показать проблему, но не мешать пользователям.

Ответ:

- Активировать уровни логирования, чтобы показывать ошибки только в дев/стейдж, а в продакшене предупреждения
- Отключить библиотеку, пока все баги не исправятся
- Игнорировать ошибки и не проверять, что они значимы

Шаг 04:

Миссия 4

Катя обнаружила, что в проекте нет модульных тестов для базовых утилит. Что её шаг как Middle-разработчика?

Подсказка: Тестирование — это гарантия, что утилиты не сломаются при изменениях.

Ответ:

- Написать модульные тесты для утилит и интегрировать их в CI
- Счесть, что утилиты простые, и тесты не нужны
- Оставить тесты на будущее, когда будет больше времени

Шаг 05:

Миссия 5

Андрей замечает, что функция очень большая — более 200 строк, делает много задач сразу. Как он поступит?

Подсказка: Подумай о single responsibility и поддержке кода.

Ответ:

- Разбить функцию на несколько функций с понятными именами, каждая делает одну задачу
- Оставить как есть, раз пока работает
- Добавить комментариев, которые объясняют каждую часть, но не разделять

Шаг 06:

Миссия 6

Сопоставьте понятия JavaScript с их описаниями:

Понятия:

Ответ:

- Замыкание (closure)
- Hoisting
- Прототипное наследование

- Event Loop

Описание:

1. Когда объявления функций и переменных поднимаются вверх своей области до выполнения кода
2. Механизм, по которому объекты наследуют свойства от прототипа
3. Функция помнит своё лексическое окружение после выхода из функции, в которой была создана
4. Цикл обработки задач, колбэков и очередей событий в JavaScript

Подсказка:

Подумай, что делает each: hoisting — о поднимании, closure — об окружении, прототип наследует, event loop управляет асинхронностью.

- **Замыкание (closure)** — Функция помнит своё лексическое окружение после выхода из функции, в которой была создана
- **Hoisting** — Когда объявления функций и переменных поднимаются вверх своей области до выполнения кода
- **Прототипное наследование** — Механизм, по которому объекты наследуют свойства от прототипа
- **Event Loop** — Цикл обработки задач, колбэков и очередей событий в JavaScript

🔗 Шаг 07:

Миссия 7

Сопоставьте методы массивов JS с их поведением:

Методы:

Ответ:

- map
- filter
- reduce
- forEach

Описание:

1. Создаёт новый массив, применяя функцию к каждому элементу
2. Создаёт новый массив из элементов, прошедших тест
3. Проходит по всем элементам, выполняя функцию, но не возвращает новый массив
4. Сокращает массив к одному значению (сумма, объект и др.)

Подсказка:

Подумай, что делают каждый метод: map, filter, forEach, reduce — и возвращают ли они что-то.

- **map** — Создаёт новый массив, применяя функцию к каждому элементу
- **filter** — Создаёт новый массив из элементов, прошедших тест
- **forEach** — Проходит по всем элементам, выполняя функцию, но не возвращает новый массив
- **reduce** — Сокращает массив к одному значению (сумма, объект и др.)

🔗 Шаг 08:

Миссия 8

Сопоставьте концепции асинхронности в JS:

Концепции:

Ответ:

- Callback Hell
- Promise chaining
- async/await
- Ошибка unhandled promise

Описание:

1. Когда промисы не обрабатываются, и исключения остаются без catch
2. Использование then-и catch последовательно для нескольких асинхронных шагов
3. Когда вложенные колбэки запутывают код и усложняют ошибки
4. Синтаксический сахар над промисами, упрощающий чтение асинхронного кода

Подсказка:

Подумай про каждое: где callback слишком вложен, где используются then/catch, где try/catch через async/await.

- **Callback Hell** — Когда вложенные колбэки запутывают код и усложняют ошибки
- **Promise chaining** — Использование then-и catch последовательно для нескольких асинхронных шагов
- **async/await** — Синтаксический сахар над промисами, упрощающий чтение асинхронного кода
- **Ошибка unhandled promise** — Когда промисы не обрабатываются, и исключения остаются без catch

Шаг 09:

Миссия 9

Сопоставьте принципы хорошего кода:

Принципы:

Ответ:

- DRY
- KISS
- YAGNI
- Separation of Concerns

Описание:

1. Не усложняй: простой и понятный код предпочтительнее сложного
2. Не пишите то, что не требуется сейчас
3. Разделение ответственности между модулями, слоями и компонентами
4. **Don't Repeat Yourself** — избегайте дублирования кода

Подсказка:

Каждый принцип — ответ на типичную проблему: дублирование, избыточность, сложность, смешение обязанностей.

- **DRY** — Don't Repeat Yourself — избегайте дублирования кода
- **KISS** — Не усложняй: простой и понятный код предпочтительнее сложного
- **YAGNI** — Не пишите то, что не требуется сейчас
- **Separation of Concerns** — Разделение ответственности между модулями, слоями и компонентами

Шаг 10:

Миссия 10

Сопоставьте виды ошибок в JS с типичными примерами:

Виды ошибок:

Ответ:

- `TypeError`
- `ReferenceError`
- `SyntaxError`
- `RangeError`

Примеры:

1. Вызов несуществующего метода у `null` или `undefined`
2. Обращение к переменной, которая не объявлена
3. Неправильный синтаксис, например лишняя запятая или скобка
4. Передача числа вне допустимого диапазона например для массива или рекурсии глубокой

Подсказка:

Подумай про каждый: что неверно объявлено, что не существует, что выходит за пределы, что синтаксически неправильно.

- **`TypeError`** — Вызов несуществующего метода у `null` или `undefined`
- **`ReferenceError`** — Обращение к переменной, которая не объявлена
- **`SyntaxError`** — Неправильный синтаксис, например лишняя запятая или скобка
- **`RangeError`** — Передача числа вне допустимого диапазона например для массива или рекурсии глубокой

Шаг 11:

Миссия 11

Вопрос: Ира получает задачу рефакторить старый код, который никто не трогал год. Что будет её шагами, если она действует как Middle-разработчик?

1. Проверить, что есть покрытие тестами или написать базовые тесты
2. Понять назначение кода, просмотреть документацию и комментарии
3. Выделить части кода, которые можно реиспользовать или упростить
4. Рефакторить маленькие участки, не меняя поведение
5. Запустить полный набор тестов и убедиться, что всё работает как прежде

Подсказка:

Рефакторинг — это не спринт: сначала убедиться, потом понять, потом аккуратно менять, затем проверять.

Ответ:

- Проверить, что есть покрытие тестами или написать базовые тесты
- Понять назначение кода, просмотреть документацию и комментарии
- Выделить части кода, которые можно реиспользовать или упростить
- Рефакторить маленькие участки, не меняя поведение
- Запустить полный набор тестов и убедиться, что всё работает как прежде

Шаг 12:

Миссия 12

Вопрос: Саша внедряет систему логирования в браузере и на сервере. Какие шаги он выполнит в правильном порядке?

1. Определить, какие события логировать и на каких уровнях (info, warning, error)
2. Выбрать библиотеку или сервис логирования (например, Winston, bunyan, внешние сервисы)
3. Добавить логирование в ключевых точках кода (обработчики ошибок, точки входа и выхода функций)
4. Настроить хранение или отправку логов, формат сообщения, трассировку стека ошибок
5. Проанализировать логи после запуска, настроить алерты или уведомления при ошибках критического уровня

Подсказка:

Логи без плана бесполезны; сначала определить, потом собирать, затем реагировать.

Ответ:

- Определить, какие события логировать и на каких уровнях (info, warning, error)
- Выбрать библиотеку или сервис логирования (например, Winston, bunyan, внешние сервисы)
- Добавить логирование в ключевых точках кода (обработчики ошибок, точки входа и выхода функций)
- Настроить хранение или отправку логов, формат сообщения, трассировку стека ошибок
- Проанализировать логи после запуска, настроить алерты или уведомления при ошибках критического уровня

 Шаг 13:

Миссия 13

Вопрос: Люба делает деплой новой версии фронтенда. Как она выстроит безопасный процесс?

1. Создать тестовую или staging среду, зеркальную продакшну
2. Запустить автоматизированные тесты и проверить, что сборка успешна
3. Подготовить инструкцию отката (rollback) на случай проблем
4. Провести ревью кода изменений, конфигураций и сборки CI/CD
5. Развернуть на продакшн и мониторить ошибки и производительность

Подсказка:

Деплой — риск. Лучше подготовиться: тесты, ревью, откат, мониторинг.

Ответ:

- Создать тестовую или staging среду, зеркальную продакшну
- Запустить автоматизированные тесты и проверить, что сборка успешна
- Подготовить инструкцию отката (rollback) на случай проблем
- Провести ревью кода изменений, конфигураций и сборки CI/CD
- Развернуть на продакшн и мониторить ошибки и производительность

 Шаг 14:

Миссия 14

Вопрос: Денис работает над оптимизацией загрузки и отображения страниц. Как он поступит, если хочет уменьшить время до First Contentful Paint и Time to Interactive?

1. Проанализировать метрики загрузки с помощью Lighthouse или Web Vitals
2. Выявить блокирующие ресурсы: тяжелый JS, CSS, шрифты
3. Отложить или лениво загружать скрипты и изображения, использовать code splitting

4. Минимизировать и объединить CSS/JS где возможно, сократить зависимости
5. Проверить, что изменения не ухудшают работу на мобильных устройствах и в слабых сетях

Подсказка:

Оптимизация рендеринга требует метрик, затем устранения узких мест, потом проверка на разных устройствах.

Ответ:

- Проанализировать метрики загрузки с помощью Lighthouse или Web Vitals
- Выявить блокирующие ресурсы: тяжелый JS, CSS, шрифты
- Отложить или лениво загружать скрипты и изображения, использовать code splitting
- Минимизировать и объединить CSS/JS где возможно, сократить зависимости
- Проверить, что изменения не ухудшают работу на мобильных устройствах и в слабых сетях

 Шаг 15:

Миссия 15

Вопрос: Оля должна обновить зависимости проекта до новых версий. Как она это сделает безопасно?

1. Посмотреть release notes зависимостей на предмет breaking changes
2. Обновить зависимости в dev / staging и запустить тесты
3. Обратиться к команде, чтобы обсудить возможные риски обновлений
4. Провести ручное тестирование ключевого функционала после обновления
5. Деплоить обновлённые зависимости на продакшн и следить за ошибками

Подсказка:

Обновление библиотек может ломать код — сначала читать, тестировать, обсуждать и только потом выпускать.

Ответ:

- Посмотреть release notes зависимостей на предмет breaking changes
- Обновить зависимости в dev / staging и запустить тесты
- Обратиться к команде, чтобы обсудить возможные риски обновлений
- Провести ручное тестирование ключевого функционала после обновления
- Деплоить обновлённые зависимости на продакшн и следить за ошибками

 Шаг 16:

Миссия 16

Вопрос: Команда Grow a Garden хочет улучшить качество кода и процесс выпуска. Расположите шаги, которые Middle-разработчик предложит внедрить:

1. Ввести линтер и форматтер, чтобы стиль кода был единым
2. Настроить CI/CD, чтобы каждый PR автоматически проверялся тестами и линтингом
3. Организовать регулярное Code Review, с критериями качества кода
4. Вести документацию по архитектуре, стандартам и стилю кода
5. Проводить пост-релизный анализ: что пошло не так, баги, производительность, отзывы

Подсказка:

Качество — это не одна проверка, а цепочка: стандарты, автоматизация, проверка, документирование и анализ.

Ответ:

- Ввести линтер и форматтер, чтобы стиль кода был единым
- Настроить CI/CD, чтобы каждый PR автоматически проверялся тестами и линтингом
- Организовать регулярное Code Review, с критериями качества кода
- Вести документацию по архитектуре, стандартам и стилю кода
- Проводить пост-релизный анализ: что пошло не так, баги, производительность, отзывы

Урок 3 - Как работать с GitHub, Node, DevTools

Шаг 01:

Миссия 1

Вика только что скачала курс с GitHub. Что ей нужно сделать сразу, чтобы начать работать локально?

Подсказка: Подумай, что нужно, чтобы код работал на твоём компьютере.

Ответ:

- **Склонировать репозиторий, установить зависимости через `npm install` или `yarn install`**
- Сразу начать изменять код, не проверив, работает ли он
- Запустить код без установки Node.js

Шаг 02:

Миссия 2

Паша внес изменения в курс на локальной копии и хочет отправить их на GitHub. Какой шаг следующий после работы над кодом?

Подсказка: Подумай о базовых командах Git.

Ответ:

- **Сделать `git add`, затем `git commit` с понятным сообщением, и потом `git push`**
- Просто скопировать файлы в папку GitHub на компьютере
- Удалить репозиторий и заново загрузить изменения

Шаг 03:

Миссия 3

Лера пытается запустить курс, но получает ошибку, что Node.js версии слишком старый. Что делать?

Подсказка: Обрати внимание на требования курса и версии программ.

Ответ:

- **Установить требуемую версию Node.js, например через `nvm` или другую версиюционную утилиту**
- Игнорировать предупреждение и надеяться, что всё заработает
- Удалить Node.js вовсе и попробовать без него

Шаг 04:

Миссия 4

Максим хочет дебажить код курса при помощи DevTools: скрипты не выполняются как надо. Что он должен проверить первым?

Подсказка: DevTools могут не видеть файлы, если настройки сервера или sourcemaps не корректны.

Ответ:

- Проверить, что sourcemaps включены и путь к исходникам корректен
- Удалить DevTools и начать заново
- Считать, что DevTools всегда работают одинаково на всех браузерах

Шаг 05:

Миссия 5

Игорь хочет настроить прт-скрипт для запуска DevTools серверного окружения. Что он добавляет в package.json?

Подсказка: Подумай, как прт-скрипты запускаются из package.json и как указать команду.

Ответ:

- В поле "scripts" добавить строку, например "start": "node server.js" или "dev": "nodemon index.js"
- Добавить прт скрипт без указания команды
- Написать скрипт в readme, чтобы каждый вручную копировал

Шаг 06:

Миссия 6

Саша хочет сохранить прогресс в Git, но не хочет добавлять файлы с node_modules. Что он должен сделать?

Подсказка: Есть специальный файл, который говорит Git, что игнорировать.

Ответ:

- Добавить node_modules в .gitignore и убедиться, что они не в индексе
- Удалить node_modules из проекта
- Закоммитить node_modules, чтобы всё было вместе

Шаг 07:

Миссия 7

Сопоставьте инструменты DevTools с их применением:

Инструменты:

Ответ:

- Network panel
- Elements / DOM inspector
- Console
- Sources / Debugger

Применение:

1. Проверка HTTP-запросов и ответов, загрузки ресурсов
2. Просмотр и изменение структуры HTML / CSS на лету
3. Вывод логов, ошибок и предупреждений, выполнение JavaScript в консоли
4. Установка точек останова, пошаговое выполнение кода

Подсказка:

DevTools имеет вкладки — каждая отвечает за свой тип задач: инспектирование, сеть, отладка и консоль.

- **Network panel** — Проверка HTTP-запросов и ответов, загрузки ресурсов
- **Elements / DOM inspector** — Просмотр и изменение структуры HTML / CSS на лету
- **Console** — Вывод логов, ошибок и предупреждений, выполнение JavaScript в консоли
- **Sources / Debugger** — Установка точек останова, пошаговое выполнение кода

🔮 Шаг 08:

Миссия 8

Сопоставьте флаги или команды Git с их действием:

Команды / флаги:

Ответ:

- git status
- git log
- git diff
- git reset --hard

Описание:

1. Показать истории коммитов
2. Отменить все незакомиченные изменения, вернуться к последнему коммиту
3. Показать, какие файлы изменились и как именно содержимое внутри файлов изменилось
4. Показать текущий статус: какие файлы изменены, какие не закомичены и т.п.

Подсказка:

git status — состояние, log — история, diff — что внутри, reset — откат.

- **git status** — Показать текущий статус: какие файлы изменены, какие не закомичены и т.п.
- **git log** — Показать истории коммитов
- **git diff** — Показать, какие файлы изменились и как именно содержимое внутри файлов изменилось
- **git reset --hard** — Отменить все незакомиченные изменения, вернуться к последнему коммиту

🔮 Шаг 09:

Миссия 9

Вопрос: Ольга готовит окружение: клонирование, установка, запуск. Расположите шаги, как это сделает Middle-разработчик.

1. git clone
2. cd в директорию проекта
3. npm install или yarn install зависимостей
4. Запустить npm run dev или аналогичный скрипт
5. Открыть DevTools / браузер и проверить консоль на ошибки

Подсказка:

Сначала получить код, затем собрать зависимости, затем запустить, потом проверить работу.

Ответ:

- git clone

- cd в директорию проекта
- npm install или yarn install зависимостей
- Запустить npm run dev или аналогичный скрипт
- Открыть DevTools / браузер и проверить консоль на ошибки

Шаг 10:

Миссия 10

Вопрос: Игорь настраивает Git ветки: он хочет внести фичу, не мешая основной ветке, затем вернуть на продакшн. Каков порядок действий?

1. Создать новую ветку feature/x
2. Работать в ветке feature/x, коммитить изменения
3. Переключиться на основную ветку (master/main)
4. Слить feature/x в main через pull request или merge после ревью
5. Удалить feature ветку, когда фича принята и протестирована

Подсказка:

Работа над фичей должна быть изолирована, потом слияние, потом уборка.

Ответ:

- Создать новую ветку feature/x
- Работать в ветке feature/x, коммитить изменения
- Переключиться на основную ветку (master/main)
- Слить feature/x в main через pull request или merge после ревью
- Удалить feature ветку, когда фича принята и протестирована

Модуль 2 - Углубление в Core JavaScript (Объекты, Контекст, Функции)

Урок 1 - Объекты и ссылки тонкости сравнения и мутаций

Шаг 01:

Миссия 1

Аня создала два объекта: `const a = { x: 1 }; const b = { x: 1 };` Она спрашивает: `a === b` — будет ли true?

Подсказка: Оператор `===` для объектов сравнивает не содержимое, а что именно?

Ответ:

- **Да** — потому что у них одинаковые свойства и значения
- **Нет** — потому что они разные объекты, разные ссылки

Шаг 02:

Миссия 2

Вася делает “clone” объекта так: `const original = { a: 1, b: { c: 2 } }; const copy = { ...original };` Если после этого он изменит `copy.b.c = 5`, повлияет ли это на `original.b.c`?

Подсказка: Оператор `spread` делает «поверхностную копию».

Ответ:

- **Да** — свойство b изменится, потому что вложенный объект ссылается на ту же ссылку
- **Нет** — `copy` полностью независим от `original`

🔗 Шаг 03:

Миссия 3

Игорь передаёт объект в функцию: `function f(o) { o.x = 10 }` и вызывает `f(obj)`. Что произойдет с `obj`, если `obj` — объект?

Подсказка: Объекты передаются по ссылке/ссылка копируется.

Ответ:

- **Поле `x` у `obj` изменится внутри и снаружи функции**
- `obj` не изменится, т.к. функция получает копию объекта

🔗 Шаг 04:

Миссия 4

Лена делает сравнение объектов так: `Object.is(obj1, obj2)` и `obj1 === obj2`. В каком случае они дадут `true`?

Подсказка: Подумай, что `Object.is` делает дополнительно по сравнению с `===`.

Ответ:

- **Когда обе переменные ссылаются на один и тот же объект (одна и та же ссылка)**
- Когда объекты имеют идентичные свойства по значению
- Когда объекты созданы независимо, но выглядят одинаково

🔗 Шаг 05:

Миссия 5

Саша хочет клонировать объект глубоко (вместе со вложенными объектами), чтобы изменение в копии не влияло на оригинал. Что он должен использовать?

Подсказка: Подумай, что делает «глубокое копирование» и какие есть способы в JS.

Ответ:

- **Метод `JSON.parse(JSON.stringify(obj))`**
- **Рекурсивная функция, копирующая вложенные объекты**
- **Просто оператор `spread` — `{...obj}`**

🔗 Шаг 06:

Миссия 6

Сопоставьте понятия сравнения объектов в JS с их описаниями:

Понятия:

Ответ:

- Сравнение по ссылке (referential equality)
- shallow сравнение (shallow equality)
- deep сравнение (deep equality)
- `Object.is()`

Описание:

1. **Оператор, который считает, что два объекта равны только если они — один и тот же экземпляр (одна ссылка)**
2. Сравнение только первого уровня свойств без проверки вложенных объектов
3. Полное сравнение структуры и значений, включая вложенные объекты
4. Метод, похожий на `===`, но с чуть более тонким поведением для `NaN` и `-0`

Подсказка: Подумай, чем отличаются === и Object.is, и что значит сравнение вложенных структур.

- **Сравнение по ссылке (referential equality)** — Оператор, который считает, что два объекта равны только если они — один и тот же экземпляр (одна ссылка)
- **shallow сравнение (shallow equality)** — Сравнение только первого уровня свойств без проверки вложенных объектов
- **deep сравнение (deep equality)** — Полное сравнение структуры и значений, включая вложенные объекты
- **Object.is()** — Метод, похожий на ===, но с чуть более тонким поведением для NaN и -0

Шаг 07:

Миссия 7

Сопоставьте методы копирования объектов с тем, что они делают:

Методы / Способы:

Ответ:

- Object.assign({}, obj)
- Spread оператор {...obj}
- JSON.parse(JSON.stringify(obj))
- Рекурсивная функция копирования

Описание:

1. Создание нового объекта на основе существующего, копируя только первые уровни и ссылки на вложенные объекты
2. Работает только для сериализуемых данных, не переносит методы и свойства, не поддерживает циклические ссылки
3. Глубокое копирование, включая вложенные объекты, до примитивов
4. То же, что и spread, но стандартный вариант с assign

Подсказка: Подумай, что делают разные копии, особенно что происходит с вложенными объектами и методами.

- **Object.assign({}, obj)** — Создание нового объекта на основе существующего, копируя только первые уровни и ссылки на вложенные объекты
- **Spread оператор {...obj}** — То же, что и spread, но стандартный вариант с assign
- **JSON.parse(JSON.stringify(obj))** — Работает только для сериализуемых данных, не переносит методы и свойства, не поддерживает циклические ссылки
- **Рекурсивная функция копирования** — Глубокое копирование, включая вложенные объекты, до примитивов

Шаг 08:

Миссия 8

Сопоставьте сценарий с тем, как будет работать сравнение объектов:

Сценарии:

Ответ:

- Два объекта с одинаковыми ключами и примитивными значениями
- Объект с вложенным объектом, одинаковым по структуре, но разной ссылке
- Два вложенных объекта, которые ссылаются на один и тот же объект
- Использование Object.is для NaN и -0

Результат сравнения (===):

1. true только если ссылки одинаковы
2. false, даже если содержимое вложенного объекта одинаково, потому что он другая ссылка
3. false, если это не один и тот же объект, даже если структура выглядит одинаково
4. true, если **Object.is** применяется и сравнивает NaN с NaN — true

Подсказка: Представьте, что сравнение по === смотрит только на указатель (ссылку) и не “заглядывает” внутрь объектов, кроме особых случаев.

- Два объекта с одинаковыми ключами и примитивными значениями — false, если разные ссылки
- Объект с вложенным объектом, одинаковым по структуре, но разной ссылке — false, даже если содержимое вложенного объекта одинаково
- Два вложенных объекта, которые ссылаются на один и тот же объект — true, если === и ссылка та же
- Использование **Object.is** для NaN и -0 — true, Object.is считает NaN === NaN, в отличие от ===

🧙 Шаг 09:

Миссия 9

Сопоставьте термин с примером мутации объекта:

Термины:

Ответ:

- Иммутабельность
- Мутация
- Чистая функция
- Побочные эффекты

Примеры:

1. Функция изменяет свойство объекта, переданного ей как параметр
2. Функция возвращает новый объект, не меняя аргумент
3. **Объект не должен изменяться после создания** — изменение разрешено только через клонирование или копирование
4. Любое изменение внешнего состояния из функции или переменной вне функции

Подсказка: Подумай, что такое побочные эффекты и как связаны мутация и чистота функций.

- **Иммутабельность** — Объект не должен изменяться после создания — изменение разрешено только через клонирование или копирование
- **Мутация** — Функция изменяет свойство объекта, переданного ей как параметр
- **Чистая функция** — Функция возвращает новый объект, не меняя аргумент
- **Побочные эффекты** — Любое изменение внешнего состояния из функции или переменной вне функции

🧙 Шаг 10:

Миссия 9

Аня написала код и хочет понять, как сравниваются объекты в JavaScript. Что верно про сравнение объектов с помощью оператора ===?

Подсказка: Помните, что объекты сравниваются по ссылкам, а не по содержимому.

Ответ:

- **Два объекта равны только если они указывают на один и тот же объект в памяти**
- Два объекта равны, если у них одинаковые ключи и значения
- Объекты всегда равны, если у них одинаковый набор ключей

🔗 **Шаг 11:**

Миссия 10

Иван создал объект и присвоил его другой переменной. Что произойдет, если изменить свойство объекта через новую переменную?

Подсказка: Обратите внимание на то, что переменные хранят ссылки на объекты.

Ответ:

- **Изменение отразится в обоих переменных, так как они ссылаются на один объект**
- Изменение отразится только в новой переменной
- Произойдет ошибка, так как объекты нельзя изменять через разные переменные
- Объекты клонируются автоматически при присваивании

🔗 **Шаг 12:**

Миссия 11

Катя пытается понять, как клонировать объект без изменения оригинала. Что из перечисленного создаст поверхностную копию объекта?

Подсказка: Поверхностная копия копирует только первые уровни объекта.

Ответ:

- **Object.assign({}, obj)**
- JSON.parse(JSON.stringify(obj))
- Присвоение через =
- Использование spread оператора с массивом

🔗 **Шаг 13:**

Миссия 12

Сопоставьте способы копирования объектов с их особенностями.

Способы копирования:

Ответ:

- Object.assign()
- JSON.parse(JSON.stringify())
- Присваивание по ссылке
- Глубокое клонирование с рекурсией

Описание:

1. Создает новую структуру без ссылок на вложенные объекты
2. Копирует только первый уровень объекта, вложенные объекты остаются ссылками
3. Переменная содержит ссылку на исходный объект
4. Реализуется с помощью специальных библиотек или рекурсивных функций

Подсказка: Внимательно подумайте, какие методы работают с копированием глубоко, а какие – поверхностно.

- **Object.assign()** — Копирует только первый уровень объекта, вложенные объекты остаются ссылками
- **JSON.parse(JSON.stringify())** — Создает новую структуру без ссылок на вложенные объекты
- **Присваивание по ссылке** — Переменная содержит ссылку на исходный объект
- **Глубокое клонирование с рекурсией** — Реализуется с помощью специальных библиотек или рекурсивных функций

🔮 Шаг 14:

Миссия 13

Сопоставьте методы с их поведением при мутации вложенных объектов.

Методы:

Ответ:

- Поверхностное копирование
- Глубокое копирование
- Присваивание по ссылке

Поведение:

1. Изменения отражаются и в исходном объекте
2. Изменения не влияют на исходный объект
3. Вложенные объекты остаются общими, изменения видны в обоих

Подсказка: Разберитесь, как изменение вложенных объектов влияет на копии.

- **Поверхностное копирование** — Вложенные объекты остаются общими, изменения видны в обоих
- **Глубокое копирование** — Изменения не влияют на исходный объект
- **Присваивание по ссылке** — Изменения отражаются и в исходном объекте

🔮 Шаг 15:

Миссия 14

Сопоставьте понятия с их описаниями.

Ответ:

- Ссылка (reference)
 - Мутация объекта
 - Поверхностное копирование
 - Глубокое копирование
1. Изменение существующего объекта без создания нового
 2. Копия объекта, включая все вложенные структуры
 3. Указатель на место в памяти, где хранится объект
 4. Копия объекта только первого уровня

Подсказка: Вспомните, что происходит при работе с объектами в памяти.

- **Ссылка (reference)** — Указатель на место в памяти, где хранится объект
- **Мутация объекта** — Изменение существующего объекта без создания нового
- **Поверхностное копирование** — Копия объекта только первого уровня
- **Глубокое копирование** — Копия объекта, включая все вложенные структуры

🔧 Шаг 16:

Миссия 15

Максим создаёт функцию, которая принимает объект и изменяет его поле. Что произойдет с объектом после вызова функции?

Подсказка: Функции получают ссылки на объекты, а не их копии.

Ответ:

- **Поле объекта изменится вне функции, так как объект передан по ссылке**
- Объект останется неизменным, так как изменения локальны
- Произойдет ошибка, потому что объекты нельзя изменять внутри функции

📖 Урок 2 - Контекст выполнения (this) и его привязка (bind, call, apply)

🔧 Шаг 01:

Миссия 1

Сэм пишет функцию: `function greet() { return this.name; }` и вызывает её просто как `greet()`, вне объекта. Что вернётся, если включён строгий режим?

Подсказка: Подумай, что значит «контекст вызова» и как `strict mode` влияет на `this`.

Ответ:

- **undefined**
- Глобальный объект (`window` или `global`)

🔧 Шаг 02:

Миссия 2

Наташа создала объект: `const user = { name: 'Alice', getName: function() { return this.name; } }`; и присвоила метод в переменную: `const fn = user.getName`; Что вернёт `fn()` (в не строгом режиме)?

Подсказка: Подумай, что именно стоит перед точкой при вызове метода.

Ответ:

- `'Alice'`
- **Глобальный объект, либо `undefined` в строгом режиме**

🔧 Шаг 03:

Миссия 3

Вадим хочет вызвать функцию с контекстом другого объекта напрямую. Что из следующего подойдёт?

Подсказка: Методы `call`, `apply` и `bind` — разные способы задать контекст; один из них возвращает новую функцию.

Ответ:

- **`func.call(otherObj, arg1, arg2)`**
- **`func.apply(otherObj, [arg1, arg2])`**
- `func.bind(otherObj, arg1, arg2)()` всегда без аргументов

🔧 Шаг 04:

Миссия 4

Ксюша делает `bind`: `const bound = obj.method.bind(otherObj)`; Когда она вызывает `bound()`, чему будет равен `this` внутри `method`?

Подсказка: bind фиксирует контекст заранее.

Ответ:

- **otherObj**
- obj

🔮 Шаг 05:

Миссия 5

Борис пишет функцию-конструктор Person, затем делает: Person.apply(null, ['John']); Что произойдёт с this внутри Person?

Подсказка: Контекст при apply устанавливается явно; что такое this, если его передать null?

Ответ:

- **this будет глобальным объектом (или undefined в строгом режиме)**
- this будет новым объектом Person

🔮 Шаг 06:

Миссия 6

Сопоставьте метод с тем, как он работает с this:

Методы:

Ответ:

- call()
- apply()
- bind()
- Обычный вызов метода через объект (obj.method())

Описание:

1. Вызывает функцию немедленно, передавая аргументы через список после контекста
2. Возвращает новую функцию с фиксированным контекстом, которую можно вызвать позже
3. Аргументы передаются в виде массива
4. **this внутри метода** — объект, через который был вызван метод

Подсказка: Подумай про разницу между немедленным вызовом и возвращаемой функцией, и про способ передачи аргументов.

- **call()** — Вызывает функцию немедленно, передавая аргументы через список после контекста
- **apply()** — Аргументы передаются в виде массива
- **bind()** — Возвращает новую функцию с фиксированным контекстом, которую можно вызвать позже
- **Обычный вызов метода через объект (obj.method())** — this внутри метода — объект, через который был вызван метод

🔮 Шаг 07:

Миссия 7

Сопоставьте вызов функции с правилом, которое определяет this:

Вызовы:

Ответ:

- new Constructor()

- `func.call(obj)`
- arrow function внутри метода
- `setTimeout(obj.method, 0)`

Как вычисляется `this` в каждом случае:

1. **Новый объект, `this`** — экземпляр Constructor
2. Контекст явно установлен через `obj`
3. `this` берётся из внешней области, arrow не имеет собственного `this`
4. **Вызов функции без объекта-приёмника** — `this` будет глобальным или `undefined` в strict

Подсказка: Arrow-функции наследуют `this`; при использовании `new`, `bind`, `call`, `apply` — правила разные.

- **`new Constructor()`** — Новый объект, `this` — экземпляр Constructor
- **`func.call(obj)`** — Контекст явно установлен через `obj`
- **arrow function внутри метода** — `this` берётся из внешней области, arrow не имеет собственного `this`
- **`setTimeout(obj.method, 0)`** — Вызов функции без объекта-приёмника — `this` будет глобальным или `undefined` в strict

🔮 Шаг 08:

Миссия 8

Сопоставьте способы передачи аргументов и их эффект при `bind/call/apply`:

Способы:

Ответ:

- `bind(obj, arg1, arg2)`
- `call(obj, arg1, arg2)`
- `apply(obj, [arg1, arg2])`
- `bind(obj)` без аргументов

Эффект:

1. Создаёт новую функцию, `this= obj`, параметры `arg1`, `arg2` зафиксированы
2. Немедленно вызывает функцию, `this= obj`, с `arg1`, `arg2` как отдельные аргументы
3. Немедленно вызывает функцию, `this= obj`, аргументы передаются массивом
4. Создаёт новую функцию, `this= obj`, параметры не фиксированы; при вызове можно передать

Подсказка: Обрати внимание, когда параметры фиксированы заранее, и когда функция вызывается сразу vs позже.

- **`bind(obj, arg1, arg2)`** — Создаёт новую функцию, `this= obj`, параметры `arg1`, `arg2` зафиксированы
- **`call(obj, arg1, arg2)`** — Немедленно вызывает функцию, `this= obj`, с `arg1`, `arg2` как отдельные аргументы
- **`apply(obj, [arg1, arg2])`** — Немедленно вызывает функцию, `this= obj`, аргументы передаются массивом
- **`bind(obj)` без аргументов** — Создаёт новую функцию, `this= obj`, параметры не фиксированы; при вызове можно передать

🔮 Шаг 09:

Миссия 9

Вопрос: Олег хочет использовать метод `borrow`: у объекта A есть метод, он хочет применить его к объекту B, затем передать аргументы. Как он сделает это?

1. Найти метод на объекте A, например `A.method`
2. Вызвать его через `call/apply`, передав B как `this` и нужные аргументы
3. Если метод понадобится позже, создать `bound`-версию через `bind(B, ...)`
4. Протестировать, что внутри метода `this === B` и аргументы применены правильно
5. Использовать этот подход для случаев, где метод не принадлежит объекту B

Подсказка: Метод `borrow` — просто заимствование метода; `call/apply` — однократный вызов, `bind` — подготовка функции.

Ответ:

- Найти метод на объекте A, например `A.method`
- Вызвать его через `call/apply`, передав B как `this` и нужные аргументы
- Если метод понадобится позже, создать `bound`-версию через `bind(B, ...)`
- Протестировать, что внутри метода `this === B` и аргументы применены правильно
- Использовать этот подход для случаев, где метод не принадлежит объекту B

Шаг 10:

Миссия 10

Вопрос: Лена реализует обработчик события, передавая метод объекта как `callback`. Иногда `this` теряется. Как она выстроит цепочку, чтобы `this` не терялся?

1. Определить метод, который должен быть обработчиком, на объекте, а не как отдельная функция
2. Вызвать `addEventListener` с методом, предварительно привязанным через `bind`, например `obj.method.bind(obj)`
3. Убедиться, что при удалении обработчика используется та же версия функции, которая была привязана `bind`, чтобы правильно отписать
4. Проверить, что внутри метода `this` указывает на объект `obj`, а не на элемент DOM или глобал
5. Написать тест или лог, вызывающий обработчик, и убедиться, что `this` ведёт себя как ожидается

Подсказка: В `callback`'ах `this` часто теряется; `bind` помогает сохранить нужный контекст.

Ответ:

- Определить метод, который должен быть обработчиком, на объекте, а не как отдельная функцию
- Вызвать `addEventListener` с методом, предварительно привязанным через `bind`, например `obj.method.bind(obj)`
- Убедиться, что при удалении обработчика используется та же версия функции, которая была привязана `bind`, чтобы правильно отписать
- Проверить, что внутри метода `this` указывает на объект `obj`, а не на элемент DOM или глобал
- Написать тест или лог, вызывающий обработчик, и убедиться, что `this` ведёт себя как ожидается

Шаг 11:

Миссия 11

Вопрос: Игорь создаёт функцию, которую хочет применять к разным объектам (borrow method). Как он настроит её, так чтобы её можно было и сразу вызвать, и позднее использовать с фиксированным контекстом?

1. Написать функцию в общем месте, не привязанную к конкретному объекту
2. При необходимости вызвать её с контекстом через call или apply
3. Если потребуется многократное использование на одном объекте, создать bound-версию через bind
4. Проверить, что аргументы передаются правильно и this указывает на нужный объект
5. Использовать эту функцию в разных местах, где требуется одинаковый контекст

Подсказка: Подумай, когда удобнее вызвать сразу и когда выгодно подготовить функцию заранее для повторного использования.

Ответ:

- Написать функцию в общем месте, не привязанную к конкретному объекту
- При необходимости вызвать её с контекстом через call или apply
- Если потребуется многократное использование на одном объекте, создать bound-версию через bind
- Проверить, что аргументы передаются правильно и this указывает на нужный объект
- Использовать эту функцию в разных местах, где требуется одинаковый контекст

Шаг 12:

Миссия 12

Вопрос: Маша пишет утилиту, которая будет вычислять площадь и периметр. Она хочет, чтобы одна функция могла быть вызвана как метод объекта Shape, так и отдельно, используя bind/call. Как она спланирует архитектуру?

1. Определить общие функции расчёта, которые принимают this и аргументы (например, this.width, this.height)
2. Не прикреплять их сразу как методы ко всем объектам, а держать отдельно
3. При вызове для объекта Shape: object.method(...) использовать обычный метод
4. При необходимости использования функции отдельно, использовать bind(shapeObj) или call(shapeObj, ...)
5. Написать тесты: один вызов как метод, другой через bind или call, проверить, что результаты совпадают

Подсказка: Утилита должна работать гибко и не дублировать код; bind/call помогают использовать одну и ту же логику для разных контекстов.

Ответ:

- Определить общие функции расчёта, которые принимают this и аргументы (например, this.width, this.height)
- Не прикреплять их сразу как методы ко всем объектам, а держать отдельно
- При вызове для объекта Shape: object.method(...) использовать обычный метод
- При необходимости использования функции отдельно, использовать bind(shapeObj) или call(shapeObj, ...)
- Написать тесты: один вызов как метод, другой через bind или call, проверить, что результаты совпадают

Шаг 13:

Миссия 13

Вопрос: Петя создал объект `car` с методом `drive`, но хочет передавать этот метод как `callback` в `setTimeout`, при этом сохранить контекст `car`. Как он устроит вызовы?

1. `const boundDrive = car.drive.bind(car)`
2. Передать `boundDrive` в `setTimeout`, например `setTimeout(boundDrive, 1000)`
3. Убедиться, что внутри `drive` `this === car` даже через таймер
4. При необходимости очистки таймера или отмены, сохранить ссылку `boundDrive`, чтобы можно было `clearTimeout(boundDrive)`
5. Написать проверку/лог, что `this.name` или `this.whatever` в `drive` соответствует `car`

Подсказка: `callback` + таймер = контекст может быть потерян; `bind` помогает сохранить его.

Ответ:

- `const boundDrive = car.drive.bind(car)`
- Передать `boundDrive` в `setTimeout`, например `setTimeout(boundDrive, 1000)`
- Убедиться, что внутри `drive` `this === car` даже через таймер
- При необходимости очистки таймера или отмены, сохранить ссылку `boundDrive`, чтобы можно было `clearTimeout(boundDrive)`
- Написать проверку/лог, что `this.name` или `this.whatever` в `drive` соответствует `car`

Шаг 14:

Миссия 14

Вопрос: Катя хочет, чтобы метод одного объекта мог быть заимствован другим, и чтобы аргументы передавались гибко. Как она настроит цепочку использования `call` / `apply` / `bind`?

1. Создать метод в объекте `A`, принимающий аргументы
2. Для единичного вызова использовать `A.method.call(B, arg1, arg2)`
3. Если аргументы уже в массиве, использовать `apply(B, [arg1, arg2])`
4. Если метод понадобится позже многократно, создать `bound` версии: `const bound = A.method.bind(B, ...presetArgs)`
5. Использовать `bound` когда нужен постоянно этот контекст, и проверять, что `this` указывает на `B`, а аргументы корректны

Подсказка: Выбор `call`, `apply` или `bind` зависит от того, когда и как ты вызываешь функцию.

Ответ:

- Создать метод в объекте `A`, принимающий аргументы
- Для единичного вызова использовать `A.method.call(B, arg1, arg2)`
- Если аргументы уже в массиве, использовать `apply(B, [arg1, arg2])`
- Если метод понадобится позже многократно, создать `bound` версии: `const bound = A.method.bind(B, ...presetArgs)`
- Использовать `bound` когда нужен постоянно этот контекст, и проверять, что `this` указывает на `B`, а аргументы корректны

Урок 3 - Функции декларации, выражения, IIFE

Шаг 01:

Миссия 1

Саша пишет функцию-декларацию: `function foo() { return 42; }` Она вызывает её до места определения: `console.log(foo());` перед объявлением. Что будет?

Подсказка: Функции-декларации поднимаются (`hoisted`).

Ответ:

- **42** — работает, потому что объявление функции поднято вверх
- Ошибка, что `foo` не определена

Шаг 02:

Миссия 2

Илья делает функциональное выражение: `const bar = function() { return 7; };` и вызывает `bar()` до объявления. Что произойдёт?

Подсказка: Функциональные выражения не поднимаются как декларации.

Ответ:

- **Ошибка, что `bar` не функция или `undefined`**
- Вернётся 7

Шаг 03:

Миссия 3

Катя хочет выполнить код сразу при загрузке страницы и не засорять глобальную область. Что подойдёт?

Подсказка: IIFE позволяет выполнить функцию сразу без присвоения.

Ответ:

- **`(function() { console.log('Hello'); })()`;**
- **`((() => { console.log('Hello'); })())`;**
- `function hello() { console.log('Hello'); }`

Шаг 04:

Миссия 4

Олег пишет IIFE с параметром: `(function(a, b) { return a + b; })(2,3)`; Что вернёт выражение?

Подсказка: IIFE может принимать аргументы как обычная функция.

Ответ:

- **5**
- NaN
- undefined

Шаг 05:

Миссия 5

Чего не будет у функционального выражения:

Подсказка: Сравни объявление и выражение; подумай о `hoisting` и именах.

Ответ:

- **Будет доступна до места объявления (до определения)**
- **Имя функции, если это анонимное выражение**
- Функция `declaration-style hoisting`

Шаг 06:

Миссия 6

Сопоставьте понятие с описанием:

Понятия:

Ответ:

- Function Declaration
- Function Expression
- IIFE (Immediately Invoked Function Expression)
- Anonymous Function

Описание:

1. Объявление функции с именем, поднимается в область видимости и может быть использовано до определения
2. Выражение, присвоенное переменной или аргументу, может быть именованным или анонимным
3. Выражение функции, которое вызывается сразу после объявления
4. Функция без имени, обычно используется в выражениях или IIFE

Подсказка: Проверь, что делает декларация, что выражение, и что означает „сразу вызывается“.

- **Function Declaration** — Объявление функции с именем, поднимается в область видимости и может быть использовано до определения
- **Function Expression** — Выражение, присвоенное переменной или аргументу, может быть именованным или анонимным
- **IIFE (Immediately Invoked Function Expression)** — Выражение функции, которое вызывается сразу после объявления
- **Anonymous Function** — Функция без имени, обычно используется в выражениях или IIFE

 **Шаг 07:**

Миссия 7

Сопоставьте свойства функции декларации и функции выражения:

Свойства:

Ответ:

- Hoisting
- Name обязательный/необязательный
- Определение доступно до объявления
- Использование как аргумент / возврат функции

Описание:

1. Функция-декларация полностью поднимается, можно вызвать до объявления
2. **У выражения имя может быть анонимным** — необязательным
3. Выражение нельзя вызвать до места, где оно объявлено
4. Выражение можно передать как аргумент в другую функцию или вернуть из функции

Подсказка: Сравни, что декларация делает автоматически, и что выражение требует, чтобы был достигнут код, где оно определяется.

- **Hoisting** — Функция-декларация полностью поднимается, можно вызвать до объявления
- **Name обязательный/необязательный** — У выражения имя может быть анонимным — необязательным
- **Определение доступно до объявления** — Выражение нельзя вызвать до места, где оно объявлено

- **Использование как аргумент / возврат функции** — Выражение можно передать как аргумент в другую функцию или вернуть из функции

🔮 Шаг 08:

Миссия 8

Сопоставьте сценарий с типом функции:

Сценарии:

Ответ:

- Определена как `function foo(){...}`
- Присвоена переменной: `const bar = function(){...}`
- IIFE сразу вызывается после определения
- Анонимная функция передаётся в callback

Тип функции:

1. Function Declaration
2. Function Expression
3. IIFE
4. Anonymous Function Expression

Подсказка: Обрати внимание на синтаксис: объявление vs выражение; где и как вызывается.

- **Определена как `function foo(){...}`** — Function Declaration
- **Присвоена переменной: `const bar = function(){...}`** — Function Expression
- **IIFE сразу вызывается после определения** — IIFE
- **Анонимная функция передаётся в callback** — Anonymous Function Expression

🔮 Шаг 09:

Миссия 9

Вопрос: Оля хочет, чтобы вспомогательная переменная была недоступна из глобальной области и чтобы её значение вычислялось сразу. Как она организует код?

1. Обёрнуть вычисление в IIFE
2. Внутри IIFE объявить вспомогательную переменную через `let / const`
3. Выполнить вычисление и вернуть нужное значение из IIFE
4. Присвоить результат переменной на внешнем уровне
5. Убедиться, что вспомогательная переменная недоступна из глобальной области

Подсказка: IIFE даёт собственную область видимости; возвращаемое значение можно сохранить, но внутреннее останется закрытым.

Ответ:

- Обёрнуть вычисление в IIFE
- Внутри IIFE объявить вспомогательную переменную через `let / const`
- Выполнить вычисление и вернуть нужное значение из IIFE
- Присвоить результат переменной на внешнем уровне
- Убедиться, что вспомогательная переменная недоступна из глобальной области

🔮 Шаг 10:

Миссия 10

Вопрос: Лёша хочет, чтобы функция инициализации модуля вызывалась сразу и создавала приватные данные. Как он сделает это правильно?

1. Объявить IIFE, оборачивая функцию в скобки
2. Внутри IIFE объявить приватные переменные через `let/const`
3. Внутрь IIFE вернуть публичный API (объект функций) или присвоить переменной наружу результат
4. Вызывать код, сразу выполняя IIFE
5. Убедиться, что наружу не выходит приватный контекст

Подсказка: Этот паттерн часто используется для модулей до появления модулей ES; позволяет инкапсулировать данные.

Ответ:

- Объявить IIFE, оборачивая функцию в скобки
- Внутри IIFE объявить приватные переменные через `let/const`
- Внутрь IIFE вернуть публичный API (объект функций) или присвоить переменной наружу результат
- Вызывать код, сразу выполняя IIFE
- Убедиться, что наружу не выходит приватный контекст

Шаг 11:

Миссия 11

Вопрос: Лёша хочет, чтобы функции однострочные и простые могли быть декларациями, а анонимные — выражениями, и сразу вычислял некоторые конфигурации через IIFE. Какой порядок он выберет при проектировании модуля?

1. Выделить часто используемые функции как декларации в начале модуля
2. **Вспомогательные функции внутри кода или callback'ов** — выражения, особенно анонимные
3. Конфигурацию и начальные установки выполнить через IIFE
4. Внешний API модуля сформировать из деклараций или присвоенных выражений
5. Документировать, какие функции декларации, какие выражения, и зачем используется IIFE

Подсказка: В модуле важно однозначно понимать, какие функции доступны сразу, какие приватные, и что выполняется сразу.

Ответ:

- Выделить часто используемые функции как декларации в начале модуля
- **Вспомогательные функции внутри кода или callback'ов** — выражения, особенно анонимные
- Конфигурацию и начальные установки выполнить через IIFE
- Внешний API модуля сформировать из деклараций или присвоенных выражений
- Документировать, какие функции декларации, какие выражения, и зачем используется IIFE

Шаг 12:

Миссия 12

Вопрос: Мария хочет, чтобы временные переменные были скрыты, и не хотелось оставить мусор в глобальной области. Что она сделает?

1. Объявит IIFE, внутри него `let / const` временные переменные

2. Выполнит вычисления в IIFE (например, инициализацию, подготовки данных)
3. Вернёт только нужное наружу, либо ничего не возвращает, если результат не нужен сразу
4. Не присваивает временные переменные глобально
5. Убедится, что глобальные объекты не изменены внутри IIFE (не мутируются случайно)

Подсказка: IIFE помогает изолировать временные переменные и избежать загрязнения глобальной области.

Ответ:

- Объявит IIFE, внутри него let / const временные переменные
- Выполнит вычисления в IIFE (например, инициализацию, подготовки данных)
- Вернёт только нужное наружу, либо ничего не возвращает, если результат не нужен сразу
- Не присваивает временные переменные глобально
- Убедится, что глобальные объекты не изменены внутри IIFE (не мутируются случайно)

Урок 4 - Замыкания и лексическое окружение

Шаг 01:

Миссия 1

Игорь делает так: `function makeCounter() { let count = 0; return function() { count++; return count; } } const counter1 = makeCounter(); counter1(); counter1();` Что будет вторым вызовом?

Подсказка: Замыкание сохраняет состояние между вызовами функции.

Ответ:

- 2
- 0
- undefined

Шаг 02:

Миссия 2

Петя создаёт функцию в цикле с var: `for (var i = 0; i < 3; i++) { setTimeout(function() { console.log(i); }, 100); }` Что выведется три раза?

Подсказка: var не имеет блочного лексического окружения; замыкания будут захватывать одну переменную.

Ответ:

- 3,3,3
- 0,1,2
- 0,0,0

Шаг 03:

Миссия 3

Настя пишет: `function outer() { let x = 5; return function inner() { let x = 10; return x; } }` Вызов `const fn = outer(); fn();` — что выведется?

Подсказка: Обрати внимание, что переменная x объявлена дважды: внутри outer и внутри inner .

Ответ:

- 10
- 5

- undefined

🔮 Шаг 04:

Миссия 4

Сопоставьте понятие с правильным определением:

Понятия:

Ответ:

- Замыкание (Closure)
- Лексическое окружение (Lexical Environment)
- Внешняя функция
- Переменная из внешнего контекста

Определения:

1. Функция, которая «закрывает» доступ к переменным своей внешней функции, даже после её завершения
2. Место, где функция запоминает переменные, доступные ей при создании
3. Функция, внутри которой объявляется вложенная функция
4. Переменная, которая объявлена вне, но используется внутри вложенной функции

Подсказка: Замыкание = функция + окружение; лексическое окружение — это «память» о переменных.

- **Замыкание (Closure)** — Функция, которая «закрывает» доступ к переменным своей внешней функции, даже после её завершения
- **Лексическое окружение (Lexical Environment)** — Место, где функция запоминает переменные, доступные ей при создании
- **Внешняя функция** — Функция, внутри которой объявляется вложенная функция
- **Переменная из внешнего контекста** — Переменная, которая объявлена вне, но используется внутри вложенной функции

🔮 Шаг 05:

Миссия 5

Сопоставьте сценарии с тем, сохраняется ли значение переменной в замыкании:

Сценарии:

Ответ:

- Переменная объявлена через var внутри функции; возвращается вложенная функция
- Переменная объявлена через let внутри блока функции; вложенная функция захватывает её
- Переменная объявлена после вложенной функции; затем вызывается вложенная функция
- Переменная локальная вложенной функции, не объявлена вовне

Сохранится ли значение?

1. **Да** — var в функции будет захвачен замыканием
2. **Да** — let внутри блока тоже зависит от функции, лексическое окружение это поддерживает
3. **Нет** — если переменная не существует к моменту захвата, будет ошибка или undefined
4. **Нет** — если переменная локальна и не связана с внешним контекстом

Подсказка: Подумай, откуда и когда объявляется переменная, и когда вложенная функция её использует.

- **Переменная объявлена через var внутри функции; возвращается вложенная функция** — Да — var в функции будет захвачен замыканием
- **Переменная объявлена через let внутри блока функции; вложенная функция захватывает её** — Да — let внутри блока тоже зависит от функции, лексическое окружение это поддерживает
- **Переменная объявлена после вложенной функции; затем вызывается вложенная функция** — Нет — если переменная не существует к моменту захвата, будет ошибка или undefined
- **Переменная локальная вложенной функции, не объявлена вовне** — Нет — если переменная локальна и не связана с внешним контекстом

🔗 Шаг 06:

Миссия 6

Сопоставьте люди-сценарий с тем, что будет напечатано:

Сценарии:

Ответ:

- Функция возвращает другую функцию, которая использует переменную из внешней функции, после того как внешняя закончила работу
- Вложенная функция вызывается сразу внутри внешней, до её завершения
- Переменная изменяется после создания вложенной функции, потом вызывается вложенная функция
- Переменная локальна вложенной функции и не используется вне неё

Что будет выведено:

1. Старое сохранённое значение переменной на момент создания замыкания
2. Актуальное значение внешней переменной в момент вызова (если завершена внешняя функция и замыкание существует)
3. Новое, если изменена до вызова вложенной функции
4. **Не влияет** — локальная переменная вложенной функции не касается внешнего

Подсказка: Замыкание “запоминает” окружение, но значение переменной может измениться до вызова вложенной функции.

- **Функция возвращает другую функцию, которая использует переменную из внешней функции, после того как внешняя закончила работу** — Старое сохранённое значение переменной на момент создания замыкания
- **Вложенная функция вызывается сразу внутри внешней, до её завершения** — Актуальное значение внешней переменной в момент вызова (если завершена внешняя функция и замыкание существует)
- **Переменная изменяется после создания вложенной функции, потом вызывается вложенная функция** — Новое, если изменена до вызова вложенной функции
- **Переменная локальна вложенной функции и не используется вне неё** — Не влияет — локальной переменной вложенной функции не касается внешнего

🔗 Шаг 07:

Миссия 7

Сопоставьте функции с тем, как они используют лексическое окружение:

Функции:

Ответ:

- makeGreeting(greeting)

- `makeAdder(x)`
- `factory-счётчик`
- `callback` внутри события, использующий внешнюю переменную

Как они работают:

1. Сохраняет “приветствие” и возвращает функцию, которая подставляет имя
2. Возвращает функцию, которая добавляет `x` к аргументу
3. Замыкает значение счётчика, возвращает методы увеличения / чтения
4. Использует значение из окружающего контекста, даже когда событие произойдёт позже

Подсказка: Каждый “фабричный” паттерн замыкания позволяет сохранить состояние или переменные вне времени вызова.

- **`makeGreeting(greeting)`** — Сохраняет “приветствие” и возвращает функцию, которая подставляет имя
- **`makeAdder(x)`** — Возвращает функцию, которая добавляет `x` к аргументу
- **`factory-счётчик`** — Замыкает значение счётчика, возвращает методы увеличения / чтения
- **`callback` внутри события, использующий внешнюю переменную** — Использует значение из окружающего контекста, даже когда событие произойдёт позже

🔗 Шаг 08:

Миссия 8

Сопоставьте термин с примером, связанным с замыканием:

Термины:

Ответ:

- Приватные переменные
- Замыкание через функцию фабрику
- Перекрытие переменных (shadowing)
- Утечка памяти через неправильное использование замыканий

Примеры:

1. Фабрика, которая возвращает объекты с методами, использующими внутренние переменные, недоступные снаружи
2. Объявление переменной с тем же именем внутри вложенной функции, скрывающей внешнюю переменную
3. Сохранение ссылок на замыкаемые переменные после того, как они больше не нужны, и удержание памяти
4. Использование локальных переменных внутри функции, которые не экспонируются, только через возвращаемую функцию

Подсказка: Замыкание может быть полезным, но также может приводить к ошибкам или утечкам, если не освободить контекст.

- **Приватные переменные** — Использование локальных переменных внутри функции, которые не экспонируются, только через возвращаемую функцию
- **Замыкание через функцию фабрику** — Фабрика, которая возвращает объекты с методами, использующими внутренние переменные, недоступные снаружи
- **Перекрытие переменных (shadowing)** — Объявление переменной с тем же именем внутри вложенной функции, скрывающей внешнюю переменную
- **Утечка памяти через неправильное использование замыканий** — Сохранение ссылок на замыкаемые переменные после того, как они больше не нужны, и удержание памяти

Шаг 09:

Миссия 9

Вопрос: Олег делает модуль конфигурации, используя замыкания для частных переменных. Как он спланирует код?

1. Создать функцию `initConfig`, которая объявляет частные параметры
2. Внутри `initConfig` объявить функции `getter` и `setter` к этим параметрам
3. Вернуть объект с методами, но не возвращать сами частные переменные
4. Использовать это API модуля, не допускать прямой модификации частных полей извне
5. Написать тесты, что частные параметры не доступны, и изменения идут только через методы `module`

Подсказка: Модуль с замыканиями — это способ эмулировать частность до ES6 классов.

Ответ:

- Создать функцию `initConfig`, которая объявляет частные параметры
- Внутри `initConfig` объявить функции `getter` и `setter` к этим параметрам
- Вернуть объект с методами, но не возвращать сами частные переменные
- Использовать это API модуля, не допускать прямой модификации частных полей извне
- Написать тесты, что частные параметры не доступны, и изменения идут только через методы `module`

Шаг 10:

Миссия 10

Вопрос: Команда `Grow a Garden` хочет сформировать лучшие практики по использованию замыканий — чтобы код был безопасным и понятным. Какие шаги они предложат?

1. **Обучить всех, что замыкание** — это функция + лексическое окружение, значение внешней переменной сохраняется даже после выполнения внешней функции; использовать `let/const`
2. Идентифицировать сценарии, где замыкания полезны: частные данные, фабрики, `callback`'ы, события
3. **Избегать лишних замыканий, удерживающих большие объекты, особенно DOM-узлы** — чтобы не было утечек памяти
4. Писать модульные тесты, проверяющие, что состояние не доступно извне (частность), и изменения идут только через API
5. Документировать паттерны с примерами: фабрики, частность, замыкания в циклах, способы фиксации значений (IIFE, `let`, аргументы)

Подсказка: Практика + документация + тесты помогают закрепить правильное использование замыканий.

Ответ:

- **Обучить всех, что замыкание** — это функция + лексическое окружение, значение внешней переменной сохраняется даже после выполнения внешней функции; использовать `let/const`
- Идентифицировать сценарии, где замыкания полезны: частные данные, фабрики, `callback`'ы, события
- **Избегать лишних замыканий, удерживающих большие объекты, особенно DOM-узлы** — чтобы не было утечек памяти
- Писать модульные тесты, проверяющие, что состояние не доступно извне (частность), и изменения идут только через API

- Документировать паттерны с примерами: фабрики, приватность, замыкания в циклах, способы фиксации значений (IIFE, let, аргументы)

Шаг 11:

Миссия 10

Сопоставьте понятия, связанные с замыканиями, с их правильным описанием.

Ответ:

- Лексическое окружение
- Замыкание
- Область видимости
- Контекст выполнения

1. Объект, в котором хранятся переменные и функции, доступные на момент выполнения.
2. Функция вместе с сохранённым лексическим окружением, в котором она была создана.
3. Участок кода, где переменные видимы и доступны.
4. Объект, который указывает на текущий this во время вызова функции.

Подсказка: Разберитесь, что сохраняется и откуда берутся переменные в замыкании.

- **Лексическое окружение** — Объект, в котором хранятся переменные и функции, доступные на момент выполнения.
- **Замыкание** — Функция вместе с сохранённым лексическим окружением, в котором она была создана.
- **Область видимости** — Участок кода, где переменные видимы и доступны.
- **Контекст выполнения** — Объект, который указывает на текущий this во время вызова функции.

Шаг 12:

Миссия 11

Сопоставьте примеры использования замыканий с их результатом.

Ответ:

- Функция возвращает приватный счётчик, увеличивающийся при каждом вызове.
- Функция, которая сохраняет параметр и возвращает функцию с доступом к нему.
- Использование замыкания для создания конфигурации с сохранёнными значениями.
- Функция, использующая внешнюю переменную для кэширования результатов.

1. Приватный счётчик
2. Карринг функции
3. Замыкание для настройки
4. Кэширование с помощью замыканий

Подсказка: Обратите внимание, как замыкание сохраняет данные для разных целей.

- **Функция возвращает приватный счётчик, увеличивающийся при каждом вызове.** — Приватный счётчик
- **Функция, которая сохраняет параметр и возвращает функцию с доступом к нему.** — Карринг функции
- **Использование замыкания для создания конфигурации с сохранёнными значениями.** — Замыкание для настройки
- **Функция, использующая внешнюю переменную для кэширования результатов.** — Кэширование с помощью замыканий

Шаг 13:

Миссия 12

Сопоставьте этапы работы замыкания с описанием.

Ответ:

- Создание лексического окружения
- Возврат вложенной функции
- Сохранение доступа к внешним переменным
- Вызов функции с доступом к сохранённым данным

1. При вызове внешней функции создаётся лексическое окружение.
2. Внутри внешней функции создаётся вложенная функция.
3. Вложенная функция сохраняет доступ к окружению после завершения внешней.
4. При вызове вложенной функции используются сохранённые переменные.

Подсказка: Подумайте о жизненном цикле замыкания.

- **Создание лексического окружения** — При вызове внешней функции создаётся лексическое окружение.
- **Возврат вложенной функции** — Внутри внешней функции создаётся вложенная функция.
- **Сохранение доступа к внешним переменным** — Вложенная функция сохраняет доступ к окружению после завершения внешней.
- **Вызов функции с доступом к сохранённым данным** — При вызове вложенной функции используются сохранённые переменные.

Шаг 14:

Миссия 13

Сопоставьте типичные ошибки при работе с замыканиями с их причинами.

Ответ:

- Использование var в цикле для замыкания
- Неявное использование глобальных переменных
- Забыть вызвать функцию-обертку
- Памятные утечки из-за слишком долгого хранения ссылок

1. Переменная var одна для всех итераций цикла.
2. Замыкание захватывает внешние переменные вне ожиданий.
3. Функция возвращает undefined вместо функции.
4. Объекты остаются в памяти из-за сохранённых ссылок.

Подсказка: Ошибки связаны с областью видимости и жизненным циклом данных.

- **Использование var в цикле для замыкания** — Переменная var одна для всех итераций цикла.
- **Неявное использование глобальных переменных** — Замыкание захватывает внешние переменные вне ожиданий.
- **Забыть вызвать функцию-обертку** — Функция возвращает undefined вместо функции.
- **Памятные утечки из-за слишком долгого хранения ссылок** — Объекты остаются в памяти из-за сохранённых ссылок.

Шаг 15:

Миссия 14

Расположите этапы создания и работы замыкания в правильном порядке:

1. Вложенная функция сохраняет ссылку на лексическое окружение.
2. Вызов внешней функции создает новое лексическое окружение.
3. Вложенная функция вызывается и использует сохранённые переменные.
4. Внешняя функция возвращает вложенную функцию.

Подсказка: Подумайте, как создаётся и используется замыкание.

Ответ:

- Вызов внешней функции создает новое лексическое окружение.
- Внешняя функция возвращает вложенную функцию.
- Вложенная функция сохраняет ссылку на лексическое окружение.
- Вложенная функция вызывается и использует сохранённые переменные.

Урок 5 - Hoisting переменные и функции

Шаг 01:

Миссия 1

Аня написала код, где объявила переменную с помощью `var`, но использует её до объявления. Что произойдет?

Подсказка: Подумайте, как работает hoisting с переменными `var`.

Ответ:

- **Переменная будет доступна, но со значением `undefined`**
- Ошибка `ReferenceError`
- Переменная будет иметь своё присвоенное значение

Шаг 02:

Миссия 2

Иван объявил функцию через `function declaration` и вызвал её до объявления в коде. Что произойдет?

Подсказка: Вспомните, как работает hoisting для `function declarations`.

Ответ:

- **Функция выполнится успешно**
- Ошибка `ReferenceError`
- Функция будет `undefined`

Шаг 03:

Миссия 3

Сергей использует `let` для объявления переменной, но пытается обратиться к ней до объявления. Что увидит Сергей в консоли?

Подсказка: Проанализируйте временную мёртвую зону (TDZ) для `let`.

Ответ:

- **`ReferenceError`**
- `undefined`

- null

🔗 Шаг 04:

Миссия 4

Елена объявила функцию как function expression через const . Можно ли вызвать эту функцию до объявления?

Подсказка: Подумайте, что именно поднимается в hoisting для function expressions.

Ответ:

- Да, можно вызвать и функция выполнится
- **Нет, будет ошибка ReferenceError**
- Функция будет undefined

🔗 Шаг 05:

Миссия 5

Пётр видит, что при объявлении переменной с помощью var вне функции, переменная становится глобальной. Что будет, если объявить такую же переменную внутри функции?

Подсказка: Проанализируйте область видимости переменных с var .

Ответ:

- **Переменная будет локальной для функции**
- Переменная станет глобальной
- Переменная будет доступна только внутри блока if или цикла

🔗 Шаг 06:

Миссия 6

Артём пытается понять разницу между hoisting для var и let . Что верно?

Подсказка: Вспомните разницу в инициализации и области видимости.

Ответ:

- **var инициализируется undefined, let не инициализируется до объявления**
- Оба инициализируются undefined
- Оба работают одинаково

🔗 Шаг 07:

Миссия 7

Нужно сопоставить термины, связанные с hoisting, и их определения.

Ответ:

- Hoisting
 - Temporal Dead Zone (TDZ)
 - Function Declaration
 - Function Expression
 - var
 - let/const
1. Область между началом блока и объявлением переменной, где доступ к ней вызывает ошибку.
 2. Автоматическое поднятие объявлений функций и переменных в начало области видимости.

3. Объявление функции, которое поднимается полностью и может быть вызвано до объявления.
4. Объявление функции, присваиваемое переменной, не поднимается.
5. Объявление переменной, поднимается с инициализацией `undefined`.
6. Объявление переменной с блочной областью видимости, не инициализируется до объявления.

Подсказка: Обратите внимание на разницу между `var` и `let/const`, а также на типы функций.

- **Hoisting** — Автоматическое поднятие объявлений функций и переменных в начало области видимости.
- **Temporal Dead Zone (TDZ)** — Область между началом блока и объявлением переменной, где доступ к ней вызывает ошибку.
- **Function Declaration** — Объявление функции, которое поднимается полностью и может быть вызвано до объявления.
- **Function Expression** — Объявление функции, присваиваемое переменной, не поднимается.
- **var** — Объявление переменной, поднимается с инициализацией `undefined`.
- **let/const** — Объявление переменной с блочной областью видимости, не инициализируется до объявления.

Шаг 08:

Миссия 8

Сопоставьте ошибки, которые могут возникнуть, с их причиной в контексте hoisting.

Ответ:

- `ReferenceError` при доступе к переменной
 - Переменная равна `undefined`
 - Функция `undefined` при вызове
 - Функция успешно вызвана до объявления
1. Переменная объявлена через `let/const` и доступ происходит до объявления
 2. Переменная объявлена через `var` и используется до присвоения значения
 3. Функция объявлена через `function expression` и вызвана до присвоения
 4. Функция объявлена через `function declaration`

Подсказка: Вспомните, как работают разные способы объявления функций и переменных.

- **ReferenceError при доступе к переменной** — Переменная объявлена через `let/const` и доступ происходит до объявления
- **Переменная равна undefined** — Переменная объявлена через `var` и используется до присвоения значения
- **Функция undefined при вызове** — Функция объявлена через `function expression` и вызвана до присвоения
- **Функция успешно вызвана до объявления** — Функция объявлена через `function declaration`

Шаг 09:

Миссия 9

Алексей хочет понять, какие объявления поднимаются и в какой форме. Сопоставьте объявления с тем, что поднимается.

Ответ:

- `var x = 10;`
 - `let y = 20;`
 - `function foo() {}`
 - `const bar = function() {}`
1. Объявление и инициализация `undefined`
 2. Нет поднятия, TDZ до объявления
 3. Полное объявление функции
 4. Нет поднятия, TDZ до присвоения

Подсказка: Разберитесь, что именно поднимается для разных типов объявлений.

- **`var x = 10;`** — Объявление и инициализация `undefined`
- **`let y = 20;`** — Нет поднятия, TDZ до объявления
- **`function foo() {}`** — Полное объявление функции
- **`const bar = function() {}`** — Нет поднятия, TDZ до присвоения

 Шаг 10:

Миссия 10

Сопоставьте примеры кода с их поведением при hoisting.

Ответ:

- `console.log(a); var a = 5;`
 - `console.log(b); let b = 5;`
 - `foo(); function foo() { return 1; }`
 - `bar(); var bar = function() { return 2; };`
1. Выведет `undefined`
 2. Выдаст `ReferenceError`
 3. Функция вызовется успешно
 4. Ошибка, `bar` не функция

Подсказка: Проанализируйте, что именно поднимается и когда переменная или функция становятся доступными.

- **`console.log(a); var a = 5;`** — Выведет `undefined`
- **`console.log(b); let b = 5;`** — Выдаст `ReferenceError`
- **`foo(); function foo() { return 1; }`** — Функция вызовется успешно
- **`bar(); var bar = function() { return 2; };`** — Ошибка, `bar` не функция

 Шаг 11:

Миссия 11

Сопоставьте, каким образом объявление влияет на область видимости и hoisting.

Ответ:

- `var`
- `let`
- `const`
- `function declaration`

- function expression

1. Функция и переменные доступны по всему функциональному скоупу, поднимаются с инициализацией undefined (для var)
2. Объявления с блочной областью, не инициализируются до объявления (TDZ)
3. Объявления с блочной областью, неизменяемые после объявления
4. Функция поднимается полностью и доступна до объявления
5. Функция присваивается переменной, не поднимается

Подсказка: Вспомните разницу между функциональной и блочной областью видимости.

- **var** — Функция и переменные доступны по всему функциональному скоупу, поднимаются с инициализацией undefined (для var)
- **let** — Объявления с блочной областью, не инициализируются до объявления (TDZ)
- **const** — Объявления с блочной областью, неизменяемые после объявления
- **function declaration** — Функция поднимается полностью и доступна до объявления
- **function expression** — Функция присваивается переменной, не поднимается

🔗 Шаг 12:

Миссия 11

Василий начал писать код и заметил, что переменная объявлена с помощью var, но при попытке вывести её значение до объявления получил undefined. Почему так происходит?

Подсказка: Подумайте, как JavaScript обрабатывает объявление переменных в фазе компиляции.

Ответ:

- **Переменная объявляется в памяти, но её инициализация происходит позже.**
- Ошибка в коде, нельзя использовать переменную до объявления.
- Переменная имеет значение null по умолчанию.
- Переменная объявляется автоматически в глобальном объекте.

🔗 Шаг 13:

Миссия 12

Ольга объявила функцию так: `function greet() { return 'Hello'; }`. Когда происходит подъем (hoisting) этой функции?

Подсказка: Вспомните, как JavaScript обрабатывает объявления функций во время компиляции.

Ответ:

- **Функция поднимается полностью, включая тело, и доступна везде в области видимости.**
- Только объявление функции поднимается, но тело не доступно до выполнения.
- Функция поднимается только если объявлена с помощью const.

🔗 Шаг 14:

Миссия 13

Пётр использует let для объявления переменной. Что произойдёт, если обратиться к этой переменной до её объявления?

Подсказка: Вспомните понятие "временной мёртвой зоны" (TDZ).

Ответ:

- **Будет ошибка ReferenceError, так как переменная в TDZ.**
- Переменная будет undefined, как с var.

- Переменная будет иметь значение null.

🔗 Шаг 15:

Миссия 14

Алина написала следующий код: `console.log(foo); var foo = 123;` Что выведется в консоль и почему?

Подсказка: Обратите внимание на поведение `var` и порядок выполнения кода.

Ответ:

- **undefined, так как объявление переменной поднимается, а инициализация нет.**
- 123, потому что переменная уже объявлена.
- Ошибка `ReferenceError`, переменная недоступна.

🔗 Шаг 16:

Миссия 15

Иван объявил функцию как функциональное выражение: `var func = function() { return 'Hi'; };` Можно ли вызвать `func()` до её объявления?

Подсказка: Подумайте, что поднимается при `hoisting` — объявление переменной или значение.

Ответ:

- **Нет, будет ошибка, потому что переменная объявлена, но значение — нет.**
- Да, функция доступна сразу.
- Можно, но вернёт `undefined`.

📖 Урок 6 - Практика реализация функции `once`, `memoize`, `debounce`

🔗 Шаг 01:

Миссия 1

Аня разработала функцию, которая должна сработать ровно один раз при клике на кнопку. Какая реализация функции `once` лучше всего подойдет для её задачи?

Подсказка: Подумайте, как сохранить состояние, что функция уже была вызвана.

Ответ:

- **Функция возвращает новую функцию, которая вызывает исходную только при первом вызове, а дальше возвращает кешированный результат.**
- Функция просто вызывает исходную функцию внутри себя при каждом вызове.
- Функция вызывает исходную функцию, но с задержкой в 1 секунду.
- Функция возвращает исходную функцию без изменений.

🔗 Шаг 02:

Миссия 2

Виктор заметил, что некоторые функции вызываются слишком часто и замедляют приложение. Какой подход поможет уменьшить количество вызовов и оптимизировать производительность?

Подсказка: Вспомните, как работает техника `debounce`.

Ответ:

- **Использовать `debounce`** — функция будет вызвана только после того, как прекратятся частые вызовы в течение заданного времени.
- Вызывать функцию сразу при каждом событии без изменений.
- Оборачивать функцию в `setTimeout` с постоянной задержкой, но без сброса таймера.

Шаг 03:

Миссия 3

Иван хочет использовать `memoize` для функции, которая вычисляет факториал. Что будет ключевым результатом использования мемоизации?

Подсказка: Как мемоизация влияет на повторяющиеся вычисления с одинаковыми аргументами?

Ответ:

- **Функция будет возвращать результат из кеша при повторных вызовах с теми же аргументами, экономя время.**
- Функция будет выполнять вычисления заново каждый раз.
- Функция будет вызываться только один раз и больше не будет работать.
- Функция станет работать медленнее из-за дополнительной памяти.

Шаг 04:

Миссия 4

Настя реализовала `debounce` с задержкой 300 мс для обработки ввода в поиске. Что произойдет, если пользователь быстро введет текст без пауз более 300 мс?

Подсказка: Подумайте, когда именно вызовется обработчик после серии событий.

Ответ:

- **Обработчик вызовется один раз после того, как пользователь перестанет вводить текст на 300 мс.**
- Обработчик вызовется сразу при каждом нажатии клавиши.
- Обработчик вызовется ровно 300 раз за 300 мс.

Шаг 05:

Миссия 5

Максим реализует функцию `once` и хочет предотвратить повторное выполнение исходной функции. Как лучше всего сохранить состояние, чтобы это сделать?

Подсказка: Рассмотрите замыкания для хранения значения внутри возвращаемой функции.

Ответ:

- **Использовать переменную-флаг внутри замыкания, чтобы отслеживать, была ли функция вызвана.**
- Вызывать функцию внутри `setInterval`.
- Создавать новый объект при каждом вызове функции.
- Удалять функцию после первого вызова.

Шаг 06:

Миссия 6

Сопоставьте функции с их основным предназначением:

Ответ:

- `once`
- `memoize`
- `debounce`

Определения:

1. Ограничивает количество вызовов функции, вызывая её один раз.

2. Кеширует результаты вызова функции для повторного использования.
3. Отсрочивает вызов функции до тех пор, пока не прекратятся частые события.

Подсказка: Вспомните, как каждая функция помогает оптимизировать производительность.

- **once** — Ограничивает количество вызовов функции, вызывая её один раз.
- **memoize** — Кеширует результаты вызова функции для повторного использования.
- **debounce** — Отсрочивает вызов функции до тех пор, пока не прекратятся частые события.

🔮 Шаг 07:

Миссия 7

Виктор написал мемоизированную функцию. Сопоставьте ситуацию с эффектом мемоизации:

Ответ:

- Функция вызывается с новыми аргументами
 - Функция вызывается с предыдущими аргументами
 - Функция вызывается первый раз
1. Происходит вычисление и сохранение результата
 2. Результат берется из кеша, вычисление не происходит
 3. Функция создаёт кеш и сохраняет результат

Подсказка: Рассмотрите жизненный цикл кеша мемоизации.

- **Функция вызывается с новыми аргументами** — Происходит вычисление и сохранение результата
- **Функция вызывается с предыдущими аргументами** — Результат берется из кеша, вычисление не происходит
- **Функция вызывается первый раз** — Функция создаёт кеш и сохраняет результат

🔮 Шаг 08:

Миссия 8

Сопоставьте типы ошибок при реализации функций `once`, `memoize`, `debounce` с возможными причинами:

Ответ:

- Память растёт бесконтрольно
 - Функция вызывается слишком часто
 - Функция не вызывается вовсе
1. Отсутствие очистки кеша мемоизации
 2. **Ошибки в реализации debounce** — таймер сбрасывается неправильно
 3. Неправильное сохранение флага вызова в `once`

Подсказка: Ошибки чаще связаны с управлением состоянием и временем.

- **Память растёт бесконтрольно** — Отсутствие очистки кеша мемоизации
- **Функция вызывается слишком часто** — Ошибки в реализации `debounce` — таймер сбрасывается неправильно
- **Функция не вызывается вовсе** — Неправильное сохранение флага вызова в `once`

🔮 Шаг 09:

Миссия 9

Сопоставьте параметры debounce с их функцией:

Ответ:

- Задержка
 - Функция обратного вызова
 - Контекст вызова
1. Время ожидания перед вызовом функции
 2. Функция, которая будет вызываться с задержкой
 3. Объект, на который будет привязан this при вызове функции

Подсказка: Обратите внимание на работу с контекстом внутри таймаута.

- **Задержка** — Время ожидания перед вызовом функции
- **Функция обратного вызова** — Функция, которая будет вызываться с задержкой
- **Контекст вызова** — Объект, на который будет привязан this при вызове функции

Шаг 10:

Миссия 10

Расположите шаги создания функции once в правильном порядке:

1. Создать переменную для хранения результата функции
2. Создать переменную-флаг для отслеживания вызова
3. Вернуть новую функцию, которая проверяет флаг
4. Вызвать исходную функцию при первом вызове и сохранить результат
5. При повторных вызовах возвращать сохраненный результат без вызова функции

Подсказка: Функция должна запомнить результат и не выполнять исходную функцию повторно.

Ответ:

- Создать переменную для хранения результата функции
- Создать переменную-флаг для отслеживания вызова
- Вернуть новую функцию, которая проверяет флаг
- Вызвать исходную функцию при первом вызове и сохранить результат
- При повторных вызовах возвращать сохраненный результат без вызова функции

Шаг 11:

Миссия 11

Расположите шаги реализации мемоизации функции вычисления суммы:

1. Создать объект для хранения кеша
2. Проверить, есть ли результат в кеше по аргументам
3. Если нет, вызвать исходную функцию и сохранить результат в кеше
4. Вернуть результат из кеша

Подсказка: Мемоизация помогает избежать повторных вычислений.

Ответ:

- Создать объект для хранения кеша
- Проверить, есть ли результат в кеше по аргументам
- Если нет, вызвать исходную функцию и сохранить результат в кеше
- Вернуть результат из кеша

Шаг 12:

Миссия 12

Расположите шаги debounce, чтобы правильно обрабатывать серию быстрых событий:

1. При каждом событии сбросить текущий таймер
2. Установить новый таймер на заданное время
3. По окончании таймера вызвать функцию обратного вызова

Подсказка: Основная идея — вызвать функцию после паузы в событиях.

Ответ:

- При каждом событии сбросить текущий таймер
- Установить новый таймер на заданное время
- По окончании таймера вызвать функцию обратного вызова

Модуль 3 - Асинхронность и Event Loop

Урок 1 - Синхронный и асинхронный JS

Шаг 01:

Миссия 1

Помогите Денису сопоставить основные понятия асинхронности с их описаниями.

Ответ:

- Event Loop
 - Callback
 - Promise
 - Microtask
1. Цикл, который управляет выполнением задач и колбеков в JS.
 2. Объект, представляющий результат асинхронной операции.
 3. Функция, переданная в другую функцию для выполнения позже.
 4. Очередь задач с приоритетом выше, чем у обычных колбеков.

Подсказка: Разберитесь, как JS упорядочивает выполнение отложенных задач.

- **Event Loop** — Цикл, который управляет выполнением задач и колбеков в JS.
- **Callback** — Функция, переданная в другую функцию для выполнения позже.
- **Promise** — Объект, представляющий результат асинхронной операции.
- **Microtask** — Очередь задач с приоритетом выше, чем у обычных колбеков.

Шаг 02:

Миссия 2

Ирина изучает виды задач в Event Loop. Помогите ей сопоставить типы очередей с их характеристиками.

Ответ:

- Macro task
- Micro task
- Rendering

- **Callback Queue**

1. Задачи с более низким приоритетом, например `setTimeout`, `setInterval`.
2. Задачи с высоким приоритетом, например `Promise.then`.
3. Обновление интерфейса после выполнения задач.
4. Основная очередь для выполнения отложенных колбеков.

Подсказка: Подумайте, как браузер управляет порядком выполнения и рендерингом.

- **Macro task** — Задачи с более низким приоритетом, например `setTimeout`, `setInterval`.
- **Micro task** — Задачи с высоким приоритетом, например `Promise.then`.
- **Rendering** — Обновление интерфейса после выполнения задач.
- **Callback Queue** — Основная очередь для выполнения отложенных колбеков.

Шаг 03:

Миссия 3

Костя столкнулся с проблемой: код внутри промиса не выполняется сразу. Сопоставьте этапы работы с `Promise` с их описаниями.

Ответ:

- **Pending**
 - **Fulfilled**
 - **Rejected**
 - **Then**
1. Состояние ожидания результата.
 2. Состояние успешного завершения.
 3. Состояние ошибки.
 4. Метод для обработки результата или ошибки.

Подсказка: Помните, что промис меняет состояние один раз.

- **Pending** — Состояние ожидания результата.
- **Fulfilled** — Состояние успешного завершения.
- **Rejected** — Состояние ошибки.
- **Then** — Метод для обработки результата или ошибки.

Шаг 04:

Миссия 4

Лена разбирается в разнице между колбеками и промисами. Помогите ей сопоставить эти термины с их преимуществами.

Ответ:

- **Callbacks**
 - **Promises**
 - **Async/Await**
 - **Event Loop**
1. Обеспечивает управление асинхронностью с помощью вложенных функций.
 2. Позволяет управлять последовательностью асинхронных операций более удобно и читаемо.
 3. Синтаксический сахар над промисами для написания асинхронного кода как синхронного.
 4. Управляет очередью выполнения асинхронных задач.

Подсказка: Вспомните, как современные методы упрощают обработку асинхронных операций.

- **Callbacks** — Обеспечивает управление асинхронностью с помощью вложенных функций.
- **Promises** — Позволяет управлять последовательностью асинхронных операций более удобно и читаемо.
- **Async/Await** — Синтаксический сахар над промисами для написания асинхронного кода как синхронного.
- **Event Loop** — Управляет очередью выполнения асинхронных задач.

🔧 Шаг 05:

Миссия 5

Сергей исследует таймлайн выполнения асинхронного кода. Помогите сопоставить событие с правильным моментом в Event Loop.

Ответ:

- `setTimeout`
- `Promise.then`
- UI rendering
- Event handler

1. Выполняется после окончания текущего макротаска.
2. Выполняется в микротасковой очереди после макротаска.
3. Происходит между макротасками для обновления интерфейса.
4. Запускается при взаимодействии пользователя с элементом.

Подсказка: Вспомните приоритеты выполнения различных типов задач.

- **`setTimeout`** — Выполняется после окончания текущего макротаска.
- **`Promise.then`** — Выполняется в микротасковой очереди после макротаска.
- **UI rendering** — Происходит между макротасками для обновления интерфейса.
- **Event handler** — Запускается при взаимодействии пользователя с элементом.

🔧 Шаг 06:

Миссия 6

Владимир пишет асинхронный код с использованием промисов. Расположите правильный порядок обработки промиса:

1. Создание промиса
2. Выполнение асинхронной операции
3. Промис меняет состояние на `fulfilled` или `rejected`
4. Вызов метода `then` или `catch`

Подсказка: Помните, что промис начинается с ожидания, а потом переходит в результат или ошибку.

Ответ:

- Создание промиса
- Выполнение асинхронной операции
- Промис меняет состояние на `fulfilled` или `rejected`
- Вызов метода `then` или `catch`

🔮 Шаг 07:

Миссия 7

Настя хочет понять, как происходит выполнение задач в Event Loop. Расположите этапы в правильном порядке:

1. Выполнение всех микротасков
2. Обработка макротасков
3. Обновление интерфейса
4. Обработка очереди колбеков

Подсказка: Event Loop сначала выполняет задачи с высоким приоритетом, потом обновляет UI, затем переходит к задачам с более низким приоритетом.

Ответ:

- Обработка макротасков
- Выполнение всех микротасков
- Обновление интерфейса
- Обработка очереди колбеков

🔮 Шаг 08:

Миссия 8

Олег отлаживает код с использованием `async/await`. Расположите шаги выполнения асинхронной функции с `await`:

1. Функция вызывается и возвращает Promise
2. Ждет завершения промиса после `await`
3. Выполняется следующий код после `await`
4. Обработка результата или ошибки

Подсказка: Обратите внимание, что `await` приостанавливает выполнение функции, но не блокирует Event Loop.

Ответ:

- Функция вызывается и возвращает Promise
- Ждет завершения промиса после `await`
- Выполняется следующий код после `await`
- Обработка результата или ошибки

🔮 Шаг 09:

Миссия 9

Антон хочет понять порядок вывода в консоль следующего кода: `console.log('Start'); setTimeout(() => console.log('Timeout'), 0); Promise.resolve().then(() => console.log('Promise')); console.log('End');`

Расположите вывод в правильном порядке:

1. Start
2. End
3. Promise
4. Timeout

Подсказка: Микротаски выполняются раньше макротасков, даже если `timeout 0`.

Ответ:

- Start
- End
- Promise
- Timeout

 Шаг 10:

Миссия 10

Валя хочет разобраться, как происходит цепочка промисов. Расположите шаги в правильном порядке:

1. Создание первого промиса
2. Обработка результата первого then
3. Создание следующего промиса на основе результата
4. Обработка результата второго then

Подсказка: Помните, что каждый then возвращает новый промис, который ждёт результата предыдущего.

Ответ:

- Создание первого промиса
- Обработка результата первого then
- Создание следующего промиса на основе результата
- Обработка результата второго then

 Шаг 11:

Миссия 10

Анна пишет код и замечает, что синхронные операции блокируют интерфейс. Что именно происходит в синхронном JavaScript?

Подсказка: Подумайте, как JavaScript обрабатывает задачи по очереди.

Ответ:

- **Код выполняется последовательно, одна операция ждет завершения предыдущей**
- Все операции запускаются одновременно и параллельно
- Код пропускает задачи без задержек
- Синхронный код выполняется в асинхронном режиме

 Шаг 12:

Миссия 11

Владимир добавляет setTimeout с задержкой 0. Что он должен ожидать?

Подсказка: setTimeout ставит задачу в очередь, а не выполняет мгновенно.

Ответ:

- **Функция выполнится после завершения текущего стека вызовов**
- Функция выполнится сразу, игнорируя остальной код
- Функция никогда не выполнится

🔗 Шаг 13:

Миссия 12

Алексей изучает, как работают разные асинхронные механизмы в JS. Помогите ему сопоставить термины с их описаниями.

Термины:

Ответ:

- Call Stack
- Callback Queue
- Event Loop
- Microtask Queue

Определения:

1. Стек вызовов, где выполняется текущий код
2. Очередь колбэков из асинхронных событий
3. Механизм, который проверяет стек и очереди, чтобы решить, что выполнить дальше
4. Очередь микротасков, которые имеют приоритет перед обычными колбэками

Подсказка: Подумайте, что происходит после завершения синхронного кода и как браузер обрабатывает задачи.

- **Call Stack** — Стек вызовов, где выполняется текущий код
- **Callback Queue** — Очередь колбэков из асинхронных событий
- **Event Loop** — Механизм, который проверяет стек и очереди, чтобы решить, что выполнить дальше
- **Microtask Queue** — Очередь микротасков, которые имеют приоритет перед обычными колбэками

🔗 Шаг 14:

Миссия 13

Светлана хочет понять, почему промисы всегда выполняются после текущего стека, даже если они уже решены.

Подсказка: Промисы используют микротаски, которые имеют особый приоритет.

Ответ:

- **Промисы ставятся в очередь микротасков и выполняются после синхронного кода**
- Промисы выполняются синхронно сразу после создания
- Промисы выполняются в колбэках `setTimeout`

🔗 Шаг 15:

Миссия 14

Игорь хочет разобраться, какие задачи относятся к микротаскам, а какие — к макротаскам. Помогите ему сопоставить.

Типы задач:

Ответ:

- Microtasks
- Macrotasks

Примеры:

1. `setTimeout`, `setInterval`, UI события
2. `Promise.then`, `process.nextTick`, `MutationObserver`

Подсказка: Подумайте, какие задачи выполняются быстрее и имеют приоритет.

- **Microtasks** — `Promise.then`, `process.nextTick`, `MutationObserver`
- **Macrotasks** — `setTimeout`, `setInterval`, UI события

Шаг 16:

Миссия 15

Павел заметил, что иногда `setTimeout` с 0 задержкой выполняется позже, чем промисы. Почему?

Подсказка: Вспомните разницу между микротасками и макротасками.

Ответ:

- **Промисы попадают в микротаски и выполняются раньше очереди `setTimeout`**
- `setTimeout` с 0 выполняется мгновенно
- `setTimeout` и промисы выполняются одновременно

Урок 2 - Event Loop, Call Stack, MicrotasksMacrotasks

Шаг 01:

Миссия 1

Сопоставьте понятия и их объяснения в контексте Event Loop и асинхронности:

Понятия:

Ответ:

- Call Stack
- Microtasks
- Macrotasks
- Event Loop

Определения:

1. Цикл, который проверяет и обрабатывает очереди задач
2. Стек вызовов, где выполняется синхронный код
3. Задачи с более высоким приоритетом, например, `Promise callbacks`
4. Задачи с более низким приоритетом, например, `setTimeout`, `setInterval`

Подсказка: Обратите внимание на разницу в приоритетах и способе выполнения.

- **Call Stack** — Стек вызовов, где выполняется синхронный код
- **Microtasks** — Задачи с более высоким приоритетом, например, `Promise callbacks`
- **Macrotasks** — Задачи с более низким приоритетом, например, `setTimeout`, `setInterval`
- **Event Loop** — Цикл, который проверяет и обрабатывает очереди задач

Шаг 02:

Миссия 2

Сопоставьте этапы выполнения асинхронного кода с их описаниями:

Ответ:

- 1. Функция вызывается и помещается в Call Stack

- 2. Асинхронная операция запускается и откладывается
- 3. Синхронный код завершается, Call Stack освобождается
- 4. Callback из Microtask или Macrotask помещается в Call Stack

Определения:

1. Операция откладывается и ожидает результата (например, таймер или промис)
2. Код выполняется по порядку, пока не закончится синхронный блок
3. Обработка очереди задач в Event Loop
4. Обработка callback и продолжение работы программы

Подсказка: Обратите внимание, когда Call Stack занят, а когда освобождается для новых задач.

- **1. Функция вызывается и помещается в Call Stack** — Код выполняется по порядку, пока не закончится синхронный блок
- **2. Асинхронная операция запускается и откладывается** — Операция откладывается и ожидает результата (например, таймер или промис)
- **3. Синхронный код завершается, Call Stack освобождается** — Обработка очереди задач в Event Loop
- **4. Callback из Microtask или Macrotask помещается в Call Stack** — Обработка callback и продолжение работы программы

🔮 Шаг 03:

Миссия 3

Сопоставьте типы задач с примерами:

Ответ:

- Microtasks
 - Macrotasks
1. setTimeout, setInterval
 2. Promise.then, process.nextTick

Подсказка: Microtasks выполняются до перерисовки, а Macrotasks — после.

- **Microtasks** — Promise.then, process.nextTick
- **Macrotasks** — setTimeout, setInterval

🔮 Шаг 04:

Миссия 4

Сопоставьте понятия Event Loop с ключевыми свойствами:

Ответ:

- Приоритет задач
 - Очередь Microtasks
 - Очередь Macrotasks
 - Время выполнения
1. Всегда обрабатывается после очереди Microtasks
 2. Обрабатываются сразу после выполнения текущего стека вызовов
 3. Менее приоритетная, выполняется позже
 4. Синхронный код блокирует обработку

Подсказка: Помните, что Event Loop всегда сначала очищает Microtasks.

- **Приоритет задач** — Менее приоритетная, выполняется позже
- **Очередь Microtasks** — Обработываются сразу после выполнения текущего стека вызовов
- **Очередь Macrotasks** — Всегда обрабатывается после очереди Microtasks
- **Время выполнения** — Синхронный код блокирует обработку

🔮 Шаг 05:

Миссия 5

Задача: Расположите этапы, которые происходят при выполнении кода с Promise:

1. Вызов функции, помещенной в Call Stack
2. Выполнение синхронного кода функции
3. Переход Promise в состояние fulfilled/rejected
4. Помещение callback из .then() в очередь Microtasks
5. Выполнение callback из Microtasks

Подсказка: Подумайте о переходе от синхронного к асинхронному коду.

Ответ:

- Вызов функции, помещенной в Call Stack
- Выполнение синхронного кода функции
- Переход Promise в состояние fulfilled/rejected
- Помещение callback из .then() в очередь Microtasks
- Выполнение callback из Microtasks

🔮 Шаг 06:

Миссия 6

Задача: Расположите правильный порядок действий Event Loop при обработке setTimeout и Promise:

1. Синхронный код выполняется и Call Stack очищается
2. Promise.resolve().then() ставится в очередь Microtasks
3. setTimeout(callback, 0) ставится в очередь Macrotasks
4. Event Loop сначала выполняет все Microtasks
5. Event Loop приступает к выполнению Macrotasks

Подсказка: Помните, что Microtasks выполняются раньше Macrotasks.

Ответ:

- Синхронный код выполняется и Call Stack очищается
- Promise.resolve().then() ставится в очередь Microtasks
- setTimeout(callback, 0) ставится в очередь Macrotasks
- Event Loop сначала выполняет все Microtasks
- Event Loop приступает к выполнению Macrotasks

🔮 Шаг 07:

Миссия 7

Задача: Отсортируйте правильный порядок выполнения кода с несколькими setTimeout и Promise:

1. Вызов синхронной функции и выполнение кода
2. Добавление setTimeout(callback1, 0) в очередь Macrotasks

3. Выполнение `Promise.resolve().then(callback2)` и добавление в очередь `Microtasks`
4. Обработка всех `Microtasks` (`callback2`)
5. Обработка `Macrotasks` (`callback1`)

Подсказка: `Microtasks` выполняются до `Macrotasks`, несмотря на порядок добавления.

Ответ:

- Вызов синхронной функции и выполнение кода
- Добавление `setTimeout(callback1, 0)` в очередь `Macrotasks`
- Выполнение `Promise.resolve().then(callback2)` и добавление в очередь `Microtasks`
- Обработка всех `Microtasks` (`callback2`)
- Обработка `Macrotasks` (`callback1`)

Шаг 08:

Миссия 7

Аня запускает простой скрипт с функцией `setTimeout`, почему она видит, что он выполняется после синхронного кода?

Подсказка: Подумайте, что происходит с кодом в `Call Stack` и где помещаются задачи из `setTimeout`.

Ответ:

- Потому что `setTimeout` помещается в очередь `Macrotasks` после выполнения синхронного кода
- Потому что `setTimeout` выполняется сразу в `Call Stack`
- Потому что `setTimeout` игнорируется движком JavaScript
- Потому что `setTimeout` выполняется параллельно синхронному коду

Шаг 09:

Миссия 8

Максим изучает `Event Loop`. Как он определит, что `Microtasks` имеют приоритет перед `Macrotasks`?

Подсказка: Обратите внимание на порядок, в котором выполняются задачи после завершения `Call Stack`.

Ответ:

- Потому что `Microtasks` выполняются сразу после очистки `Call Stack`, до начала `Macrotasks`
- Потому что `Macrotasks` выполняются раньше `Microtasks`
- `Microtasks` и `Macrotasks` выполняются одновременно
- `Microtasks` откладываются на следующий цикл `Event Loop`

Шаг 10:

Миссия 9

Екатерина заметила, что промисы всегда выполняются в очереди `Microtasks`. Как объяснить этот механизм коллеге?

Подсказка: Вспомните, когда именно обрабатываются `then/catch` в отношении `Call Stack` и других задач.

Ответ:

- Промисы обрабатываются после текущего стека, но до `Macrotasks`, потому что они в `Microtasks`

- Промисы выполняются одновременно с синхронным кодом
- Промисы игнорируются Event Loop и выполняются сразу
- Промисы помещаются в Macrotasks

🔗 Шаг 11:

Миссия 10

Даниил написал код с несколькими `setTimeout` с разными задержками. Как понять, в каком порядке они выполнятся?

Подсказка: Обратите внимание, что задержка задаёт минимальное время для помещения в очередь `Macrotasks`.

Ответ:

- **Сначала выполнится `setTimeout` с меньшей задержкой, но порядок может меняться из-за нагрузки на Call Stack**
- Все `setTimeout` выполняются сразу и одновременно
- Порядок всегда совпадает с написанием кода
- `setTimeout` с большей задержкой выполнится первым

🔗 Шаг 12:

Миссия 11

Сопоставьте понятия из Event Loop и их описание:

Ответ:

- Call Stack
- Macrotasks
- Microtasks
- Event Loop

Определения:

1. Стек, в котором выполняется синхронный код
2. Очередь задач, которая включает `setTimeout` и `setInterval`
3. Очередь задач, которая включает промисы и `process.nextTick`
4. Механизм, который проверяет Call Stack и очереди задач

Подсказка: Обратите внимание, что разные типы задач обрабатываются в разное время.

- **Call Stack** — Стек, в котором выполняется синхронный код
- **Macrotasks** — Очередь задач, которая включает `setTimeout` и `setInterval`
- **Microtasks** — Очередь задач, которая включает промисы и `process.nextTick`
- **Event Loop** — Механизм, который проверяет Call Stack и очереди задач

🔗 Шаг 13:

Миссия 12

Помогите Ивану разобраться с этапами работы Event Loop. Сопоставьте этапы и их описание:

Ответ:

- 1. Выполнение Call Stack
- 2. Очистка Microtasks
- 3. Выполнение Macrotasks
- 4. Обновление UI

Описание:

1. Синхронный код выполняется в стеке вызовов
2. Выполняются задачи из очереди Microtasks
3. Обрабатывается следующая задача из Macrotasks
4. Обновляется интерфейс пользователя

Подсказка: Обратите внимание, что UI обновляется между выполнением разных типов задач.

- **1. Выполнение Call Stack** — Синхронный код выполняется в стеке вызовов
- **2. Очистка Microtasks** — Выполняются задачи из очереди Microtasks
- **3. Выполнение Macrotasks** — Обрабатывается следующая задача из Macrotasks
- **4. Обновление UI** — Обновляется интерфейс пользователя

Шаг 14:

Миссия 13

Расставьте соответствия между видами задач и примерами из JavaScript:

Ответ:

- Macrotasks
- Microtasks
- Синхронный код
- Event Loop

Примеры:

1. Вызовы setTimeout, setInterval
2. Обработка Promise.then
3. Выполнение обычного кода без асинхронности
4. Цикл обработки задач и очередей

Подсказка: Вспомните, какие задачи относятся к микротаскам, а какие — к макротаскам.

- **Macrotasks** — Вызовы setTimeout, setInterval
- **Microtasks** — Обработка Promise.then
- **Синхронный код** — Выполнение обычного кода без асинхронности
- **Event Loop** — Цикл обработки задач и очередей

Шаг 15:

Миссия 14

Алексей хочет объяснить разницу между Microtasks и Macrotasks. Помогите подобрать правильные определения:

Ответ:

- Microtasks
- Macrotasks

Определения:

1. Задачи, которые выполняются после текущего стека и имеют высокий приоритет
2. Задачи, которые планируются на следующий цикл Event Loop и выполняются позже

Подсказка: Вспомните, что Microtasks запускаются до Macrotasks.

- **Microtasks** — Задачи, которые выполняются после текущего стека и имеют высокий приоритет
- **Macrotasks** — Задачи, которые планируются на следующий цикл Event Loop и выполняются позже

Шаг 16:

Миссия 15

Расположите события Event Loop в правильном порядке:

1. Выполнение синхронного кода в Call Stack
2. Выполнение задач из Microtasks
3. Обработка задач из Macrotasks (setTimeout и т.п.)
4. Обновление интерфейса пользователя

Подсказка: Подумайте, что выполняется сразу, а что — с задержкой.

Ответ:

- Выполнение синхронного кода в Call Stack
- Выполнение задач из Microtasks
- Обработка задач из Macrotasks (setTimeout и т.п.)
- Обновление интерфейса пользователя

Урок 3 - Промисы создание, цепочки, catch, finally

Шаг 01:

Миссия 1

Сопоставьте методы промисов с их назначением:

Ответ:

- then
 - catch
 - finally
1. Обрабатывает успешное выполнение промиса
 2. Обрабатывает ошибку промиса
 3. Выполняется всегда после завершения промиса

Подсказка: Подумайте о том, когда вызывается каждый метод.

- **then** — Обрабатывает успешное выполнение промиса
- **catch** — Обрабатывает ошибку промиса
- **finally** — Выполняется всегда после завершения промиса

Шаг 02:

Миссия 2

Сопоставьте состояния промиса с их описанием:

Ответ:

- pending
- fulfilled

- rejected

1. Промис ожидает результата
2. Промис успешно завершён
3. Промис отклонён с ошибкой

Подсказка: Подумайте, что означает каждый статус.

- **pending** — Промис ожидает результата
- **fulfilled** — Промис успешно завершён
- **rejected** — Промис отклонён с ошибкой

Шаг 03:

Миссия 3

Сопоставьте части цепочки промисов с их ролью:

Ответ:

- Возвращение значения в then
 - Поймать ошибку в catch
 - Очистка ресурсов в finally
1. Передача результата следующему then
 2. Обработка исключений
 3. Выполнение завершающих действий

Подсказка: Задумайтесь, зачем нужен каждый этап цепочки.

- **Возвращение значения в then** — Передача результата следующему then
- **Поймать ошибку в catch** — Обработка исключений
- **Очистка ресурсов в finally** — Выполнение завершающих действий

Шаг 04:

Миссия 4

Сопоставьте сценарии использования промисов с правильным примером:

Ответ:

- Асинхронная загрузка данных
 - Обработка ошибок при запросе
 - Выполнение кода после завершения операции
1. then
 2. catch
 3. finally

Подсказка: Подумайте, какой метод подходит под каждую ситуацию.

- **Асинхронная загрузка данных** — then
- **Обработка ошибок при запросе** — catch
- **Выполнение кода после завершения операции** — finally

Шаг 05:

Миссия 5

Расположите шаги обработки промиса в правильном порядке:

1. Создание промиса
2. Обработка успешного результата через then
3. Обработка ошибки через catch
4. Выполнение завершающего кода через finally

Подсказка: Помните, что finally всегда выполняется последним.

Ответ:

- Создание промиса
- Обработка успешного результата через then
- Обработка ошибки через catch
- Выполнение завершающего кода через finally

Шаг 06:

Миссия 6

Расположите этапы работы цепочки промисов, когда происходит ошибка:

1. Выполнение then
2. Возникновение ошибки
3. Вызов catch
4. Вызов finally

Подсказка: Ошибка прерывает then и идет в catch, а finally выполняется всегда.

Ответ:

- Выполнение then
- Возникновение ошибки
- Вызов catch
- Вызов finally

Шаг 07:

Миссия 7

Расположите шаги цепочки промисов, когда в then возвращается новый промис:

1. Первый then возвращает промис
2. Ожидание разрешения возвращённого промиса
3. Выполнение следующего then после разрешения
4. Обработка ошибок в catch

Подсказка: Возвращённый промис влияет на дальнейшее выполнение цепочки.

Ответ:

- Первый then возвращает промис
- Ожидание разрешения возвращённого промиса
- Выполнение следующего then после разрешения
- Обработка ошибок в catch

Шаг 08:

Миссия 8

Расположите этапы обработки промиса при работе с fetch:

1. Вызов fetch
2. Обработка ответа в then
3. Парсинг JSON
4. Обработка ошибок в catch
5. Выполнение finally для очистки

Подсказка: Помните, что fetch возвращает промис и нужно обрабатывать все этапы.

Ответ:

- Вызов fetch
- Обработка ответа в then
- Парсинг JSON
- Обработка ошибок в catch
- Выполнение finally для очистки

Шаг 09:

Миссия 9

Расположите действия разработчика при отладке цепочки промисов:

1. Добавление catch для обработки ошибок
2. Проверка последовательности then
3. Использование finally для логирования
4. Тестирование на неожиданные результаты

Подсказка: Отладка требует проверки и обработки всех этапов цепочки.

Ответ:

- Добавление catch для обработки ошибок
- Проверка последовательности then
- Использование finally для логирования
- Тестирование на неожиданные результаты

Шаг 10:

Миссия 10

Расположите этапы работы с промисом, который получает данные и обновляет интерфейс:

1. Создание промиса для запроса данных
2. Обработка успешного ответа в then
3. Обновление интерфейса
4. Обработка ошибок в catch
5. Выполнение действий после завершения в finally

Подсказка: Обратите внимание на логику от запроса до обновления UI.

Ответ:

- Создание промиса для запроса данных
- Обработка успешного ответа в then
- Обновление интерфейса
- Обработка ошибок в catch
- Выполнение действий после завершения в finally

🔮 Шаг 11:

Миссия 11

Расположите порядок выполнения цепочки промисов с разветвлениями:

1. Первый then выполняется
2. Возвращается новый промис
3. Выполнение следующего then после разрешения нового промиса
4. Обработка ошибки в catch
5. Выполнение finally

Подсказка: Разветвления влияют на последовательность вызовов.

Ответ:

- Первый then выполняется
- Возвращается новый промис
- Выполнение следующего then после разрешения нового промиса
- Обработка ошибки в catch
- Выполнение finally

🔮 Шаг 12:

Миссия 11

Аня написала промис, который должен возвращать результат через 2 секунды. Что из списка создаст корректный промис?

Подсказка: Обратите внимание, что внутри промиса нужно вызвать resolve для успешного завершения.

Ответ:

- **new Promise(resolve => setTimeout(() => resolve('done'), 2000))**
- `setTimeout(() => Promise.resolve('done'), 2000)`
- `Promise('done').then(() => setTimeout(2000))`

🔮 Шаг 13:

Миссия 12

Петя хочет обработать ошибку в промисе. Какой метод нужно использовать?

Подсказка: Ошибки обрабатываются отдельным методом, который следует за then.

Ответ:

- **.catch()**
- .then()
- .finally()

🔮 Шаг 14:

Миссия 13

Коля написал цепочку промисов. Что гарантирует метод finally?

Подсказка: Этот метод вызывается всегда, независимо от результата промиса.

Ответ:

- **Выполнение кода после завершения промиса, независимо от успеха или ошибки**
- Отмена промиса
- Повторное выполнение промиса

🔮 Шаг 15:

Миссия 14

Даша хочет вернуть данные из промиса и передать их дальше. Какой метод в цепочке используется для передачи значения дальше?

Подсказка: В цепочке промисов результат передается следующему then.

Ответ:

- **.then()**
- .catch()
- .finally()

🔮 Шаг 16:

Миссия 15

Иван написал промис, но забыл вернуть его из функции. Что произойдет?

Подсказка: Промис должен быть возвращен, чтобы его можно было обрабатывать дальше.

Ответ:

- **Цепочка then/catch не будет работать, так как промис не передан дальше**
- Промис выполнится, но не будет ошибок
- Код упадет с ошибкой синтаксиса

📖 Урок 4 - AsyncAwait синтаксис, обработка ошибок

🔮 Шаг 01:

Миссия 1

Ирина пишет async функцию: `async function f() { return 5; }` . Что вернёт вызов `f()` ?

Подсказка: Async функция всегда возвращает Promise.

Ответ:

- **Promise, который разрешится значением 5**
- Число 5
- undefined

🔮 Шаг 02:

Миссия 2

Максим написал код: `async function f() { await Promise.resolve('ok'); }` Что делает функция `f`, если её вызвать?

Подсказка: Обратите внимание, что нет явного return.

Ответ:

- **Возвращает Promise, разрешённый с undefined**
- Возвращает Promise, разрешённый со строкой 'ok'
- Возвращает сразу строку 'ok'

🔮 Шаг 03:

Миссия 3

Ольга хочет обрабатывать ошибки внутри async функции. Какой способ правильный?

Подсказка: Используйте конструкцию, которая ловит исключения внутри функции.

Ответ:

- **Использовать try...catch внутри async функции**
- Оборачивать вызов async функции в setTimeout
- Использовать только .then().catch(), не async/await

Шаг 04:

Миссия 4

Сопоставьте части async/await с их свойствами.

Части:

Ответ:

- async ключевое слово
- await выражение
- async функция возвращает Promise
- ошибка в await

Свойства:

1. Приостанавливает выполнение до завершения промиса
2. Функция всегда возвращает Promise, даже если возвращается примитив
3. Если промис отвергается, await выбрасывает исключение
4. Позволяет писать асинхронный код похожим на синхронный

Подсказка: Вспомните, что делает каждое ключевое слово и выражение.

- **async ключевое слово** — Позволяет писать асинхронный код похожим на синхронный
- **await выражение** — Приостанавливает выполнение до завершения промиса
- **async функция возвращает Promise** — Функция всегда возвращает Promise, даже если возвращается примитив
- **ошибка в await** — Если промис отвергается, await выбрасывает исключение

Шаг 05:

Миссия 5

Сопоставьте примеры вызова async функций и способы обработки ошибок с ними.

Примеры:

Ответ:

- async function f() { throw new Error('err'); }
- async function g() { await Promise.reject('oops'); }
- async function h() { return Promise.reject('fail'); }

Обработка ошибок:

1. try...catch внутри вызывающего async function
2. вызов .catch на возвращённом Promise
3. необработанная ошибка, если ни то, ни другое не использовать

Подсказка: Каждая async функция возвращает Promise — подумайте, как ошибки распространяются.

- **async function f() { throw new Error('err'); }** — try...catch внутри вызывающего async function

- **async function g() { await Promise.reject('oops'); }** — вызов `.catch` на возвращённом Promise
- **async function h() { return Promise.reject('fail'); }** — необработанная ошибка, если ни то, ни другое не использовать

🔗 Шаг 06:

Миссия 6

Сопоставьте участки кода с тем, что произойдёт, если убрать обработки ошибок.

Код:

Ответ:

- **async function a() { await fetch('bad-url'); }**
- **async function b() { try { await fetch('bad-url'); } catch(e) { console.log(e) } }**
- **async function c() { return Promise.reject('error'); }**

Что будет:

1. Возврат rejected Promise, ошибка не поймана
2. Ошибка поймана внутри функции, код не падает наружу
3. Promise, возвращённый функцией, будет отклонён, если не обрабатывается извне

Подсказка: Ошибки распространяются по цепочке Promise.

- **async function a() { await fetch('bad-url'); }** — Возврат rejected Promise, ошибка не поймана
- **async function b() { try { await fetch('bad-url'); } catch(e) { console.log(e) } }** — Ошибка поймана внутри функции, код не падает наружу
- **async function c() { return Promise.reject('error'); }** — Promise, возвращённый функцией, будет отклонён, если не обрабатывается извне

🔗 Шаг 07:

Миссия 7

Саша написал функцию `async` и забыл внести `await` перед `Promise.reject`. Что произойдёт при вызове функции?

Подсказка: Обратите внимание: `async` функция возвращает Promise, и `reject` будет частью этого Promise.

Ответ:

- **Функция вернёт Promise, который будет отклонён**
- Функция не сработает и бросит ошибку сразу
- `Promise.resolve` будет возвращён

🔗 Шаг 08:

Миссия 8

Расположите шаги обработки `async/await` с ошибкой в правильном порядке:

1. Вызов `async` функции
2. `await` внутри функции, который вызывает `reject`
3. попадание в блок `catch` внутри `async` функции
4. возврат ошибки (отклонённого Promise) наружу, если не поймано внутри
5. вызов `.catch` на возвращённом Promise, если функция вызвана без обработчика ошибок внутри

Подсказка: Ошибка может быть поймана либо внутри, либо снаружи, порядок важен.

Ответ:

- Вызов `async` функции
- `await` внутри функции, который вызывает `reject`
- попадание в блок `catch` внутри `async` функции
- возврат ошибки (отклонённого `Promise`) наружу, если не поймано внутри
- вызов `.catch` на возвращённом `Promise`, если функция вызвана без обработчика ошибок внутри

Шаг 09:

Миссия 9

Юлия хочет обработать два асинхронных запроса последовательно и ловить ошибки. Расположите порядок операций:

1. `await` первого запроса
2. `await` второго запроса
3. обернуть оба `await` в `try...catch`
4. обработать ошибку, если один из запросов неудачен
5. вернуть результат, если оба запроса успешны

Подсказка: Последовательность важна — ошибки нужно ловить до доступа к результатам.

Ответ:

- `await` первого запроса
- `await` второго запроса
- обернуть оба `await` в `try...catch`
- обработать ошибку, если один из запросов неудачен
- вернуть результат, если оба запроса успешны

Шаг 10:

Миссия 10

Владимир хочет запустить несколько асинхронных операций параллельно и обработать ошибки. Расположите шаги правильного подхода:

1. Создать массив промисов
2. `await Promise.all(promises)`
3. обернуть весь вызов в `try...catch`
4. если один промис `reject`, перейти в `catch`
5. если все промисы `resolve`, вернуть массив результатов

Подсказка: `Promise.all` сразу отвергается, если один из промисов `reject`.

Ответ:

- Создать массив промисов
- `await Promise.all(promises)`
- обернуть весь вызов в `try...catch`
- если один промис `reject`, перейти в `catch`
- если все промисы `resolve`, вернуть массив результатов

Шаг 11:

Миссия 11

Сергей хочет поймать ошибку, возникающую при `await` внутри цикла. Расположите шаги:

1. объявление `async` функции с циклом
2. внутри цикла `await` для каждой итерации
3. поместить `await` в `try...catch` внутри цикла или вокруг всего цикла
4. ловить ошибку, если одна из итераций `reject`
5. продолжить выполнение или прекратить в зависимости от ошибки

Подсказка: Решение зависит, хотите ли обработать каждую итерацию отдельно или всю функцию целиком.

Ответ:

- объявление `async` функции с циклом
- внутри цикла `await` для каждой итерации
- поместить `await` в `try...catch` внутри цикла или вокруг всего цикла
- ловить ошибку, если одна из итераций `reject`
- продолжить выполнение или прекратить в зависимости от ошибки

Шаг 12:

Миссия 12

Алена пишет `nested async` функции. Как правильно передать ошибку наружу, если внутренняя функция бросает ошибку?

1. внутренняя `async` функция бросает ошибку через `throw` или `reject`
2. внешняя `async` функция вызывает внутреннюю через `await`
3. ошибка падает к внешнему `try...catch`, если он есть
4. если внешнего `try...catch` нет, ошибка возвращается как `reject` внешнего `Promise`
5. вызов `.catch` на внешней функции, если её вызвали без обработки ошибок внутри функции

Подсказка: Ошибка может быть поймана на разных уровнях, важно понимать, где именно.

Ответ:

- внутренняя `async` функция бросает ошибку через `throw` или `reject`
- внешняя `async` функция вызывает внутреннюю через `await`
- ошибка падает к внешнему `try...catch`, если он есть
- если внешнего `try...catch` нет, ошибка возвращается как `reject` внешнего `Promise`
- вызов `.catch` на внешней функции, если её вызвали без обработки ошибок внутри функции

Шаг 13:

Миссия 13

Игорь хочет, чтобы код с `async/await` возвращал дефолтное значение при ошибке, но продолжал работу. Расположите шаги:

1. обернуть `await` в `try...catch`
2. в `catch` вернуть дефолтное значение
3. вызвать `async` функцию из внешнего кода
4. использовать результат, даже если была ошибка внутренней части
5. продолжить выполнение остальной логики

Подсказка: Это полезно, когда ошибка не критична и нужно вернуть запасной вариант.

Ответ:

- обернуть `await` в `try...catch`

- в catch вернуть дефолтное значение
- вызвать аsync функцию из внешнего кода
- использовать результат, даже если была ошибка внутренней части
- продолжить выполнение остальной логики

🔮 Шаг 14:

Миссия 14

Юля хочет параллельно запустить три асинхронные операции и правильно обработать ошибки так, чтобы узнать, какая из них не удалась. Расположите шаги:

1. создать массив Promise'ов
2. использовать Promise.allSettled(promises)
3. обработать результаты всех Promise'ов из результата allSettled
4. отфильтровать те, что rejected и те, что fulfilled
5. сообщить об ошибках и вернуть успешные результаты

Подсказка: Promise.all прерывает работу на первой ошибке, allSettled позволяет узнать все ответы.

Ответ:

- создать массив Promise'ов
- использовать Promise.allSettled(promises)
- обработать результаты всех Promise'ов из результата allSettled
- отфильтровать те, что rejected и те, что fulfilled
- сообщить об ошибках и вернуть успешные результаты

🔮 Шаг 15:

Миссия 15

Павел пишет аsync функцию, которая вызывает несколько аsync подфункций. Как обеспечить, чтобы ошибки одной подфункции не останавливали остальные, но всё равно собрать все результаты/ошибки? Расположите шаги:

1. запустить подфункции без await параллельно (например, Promise.allSettled или map + await внутри)
2. дождаться всех результатов
3. собрать как успешные, так и ошибочные результаты отдельно
4. обработка ошибок внутри каждой подфункции или после сбора результатов
5. вернуть или использовать агрегированный результат, содержащий данные и ошибки

Подсказка: Такой подход полезен, когда часть операций может не удалась, но нам важно всё узнать.

Ответ:

- запустить подфункции без await параллельно (например, Promise.allSettled или map + await внутри)
- дождаться всех результатов
- собрать как успешные, так и ошибочные результаты отдельно
- обработка ошибок внутри каждой подфункции или после сбора результатов
- вернуть или использовать агрегированный результат, содержащий данные и ошибки

🔮 Шаг 16:

Миссия 16

Аня вызывает async функцию без await и без .catch(). Что произойдёт, и как правильно исправить? Расположите шаги:

1. async функция возвращает Promise
2. если не использовать await или .then/.catch, ошибка будет необработанной
3. вызвать async функцию из кода, используя await или .then().catch
4. обернуть вызов в try...catch, если используется await
5. если используется .then/.catch, прикрепить catch для обработки ошибок

Подсказка: Даже если внутри async всё верно, вызов снаружи тоже должен обрабатывать ошибки.

Ответ:

- async функция возвращает Promise
- если не использовать await или .then/.catch, ошибка будет необработанной
- вызвать async функцию из кода, используя await или .then().catch
- обернуть вызов в try...catch, если используется await
- если используется .then/.catch, прикрепить catch для обработки ошибок

Урок 5 - Параллельные запросы Promise.all, Promise.race

Шаг 01:

Миссия 1

Сопоставьте методы параллельной обработки промисов с их поведением.

Методы:

Ответ:

- Promise.all
- Promise.race
- Promise.allSettled
- Promise.any

Поведение:

1. Ждёт, пока ****все**** промисы завершатся успешно, иначе reject
2. **Упрощённый вариант: первый завершённый, успешный или через reject** — возвращает его результат
3. Ждёт завершения всех промисов, но возвращает состояния каждого независимо от успеха
4. Возвращает первый ****успешный**** промис, игнорируя reject, если есть успехи

Подсказка: Обратите внимание, что каждый метод решает разные задачи по готовности нескольких промисов.

- **Promise.all** — Ждёт, пока ****все**** промисы завершатся успешно, иначе reject
- **Promise.race** — Первый завершённый, успешный или через reject — возвращает его результат
- **Promise.allSettled** — Ждёт завершения всех промисов, но возвращает состояния каждого независимо от успеха
- **Promise.any** — Возвращает первый ****успешный**** промис, игнорируя reject, если есть успехи

Шаг 02:

Миссия 2

Сопоставьте сценарии с методом, который лучше всего подходит.

Сценарии:

Ответ:

- Три API-запроса, результат нужен только если все успешны
- Две загрузки файлов, нужно показать, кто завершится первым
- Несколько задач, нужно дождаться всех ответов, но не прерывать, если одна из них упала
- Несколько операций, нужен первый успешный результат, остальные игнорируются при успехе

Методы:

1. Promise.all
2. Promise.race
3. Promise.allSettled
4. Promise.any

Подсказка: Подумайте, что делает каждый метод с reject и как он влияет на результат.

- **Три API-запроса, результат нужен только если все успешны** — Promise.all
- **Две загрузки файлов, нужно показать, кто завершится первым** — Promise.race
- **Несколько задач, нужно дождаться всех ответов, но не прерывать, если одна из них упала** — Promise.allSettled
- **Несколько операций, нужен первый успешный результат, остальные игнорируются при успехе** — Promise.any

Шаг 03:

Миссия 3

Сопоставьте поведение метода race с примерами кода:

Примеры:

Ответ:

- Promise.race([fetch(a), fetch(b)])
- Promise.race([delay(1000), Promise.reject('err')])
- Promise.race([Promise.resolve('fast'), delay(2000)])

Что произойдёт первым:

1. Первый успешный fetch или сеть, кто ответит быстрее
2. Мгновенный reject, потому что один из промисов сразу rejected
3. Promise.resolve, так как он мгновенный в сравнении с delay

Подсказка: Метод race решает исход на основании того, кто закончит первым — будь это успех или ошибка.

- **Promise.race([fetch(a), fetch(b)])** — Первый успешный fetch или сеть, кто ответит быстрее
- **Promise.race([delay(1000), Promise.reject('err')])** — Мгновенный reject, потому что один из промисов сразу rejected
- **Promise.race([Promise.resolve('fast'), delay(2000)])** — Promise.resolve, так как он мгновенный в сравнении с delay

Шаг 04:

Миссия 4

Ольга использует `Promise.allSettled` вместо `Promise.all`, чтобы получить результаты всех запросов, даже если одни из них упадут. Что она увидит в результате?

Подсказка: `allSettled` всегда завершится успешно с массивом результатов.

Ответ:

- **Массив с объектами { status: 'fulfilled' или 'rejected', value или reason }**
- Только значения успешных промисов
- `Promise.reject`, если есть хотя бы один `reject`

🔮 Шаг 05:

Миссия 5

Николай использует `Promise.any`, но все промисы отклоняются. Что произойдёт?

Подсказка: `any` требует хотя бы один успешный, иначе — `reject`.

Ответ:

- **`Promise.any` отвергнётся со `AggregateError`**
- Вернётся массив ошибок
- Вернётся `undefined`

🔮 Шаг 06:

Миссия 6

Расположите шаги использования `Promise.all`, чтобы корректно обработать результаты и ошибки:

1. создать массив промисов
2. вызвать `Promise.all` на массиве
3. добавить `.catch` для обработки ошибок
4. добавить `.then` для обработки массива результатов
5. использовать результат, если все промисы успешны

Подсказка: Обратите внимание, что `.then` и `.catch` — методы на возвращённом `Promise`.

Ответ:

- создать массив промисов
- вызвать `Promise.all` на массиве
- добавить `.catch` для обработки ошибок
- добавить `.then` для обработки массива результатов
- использовать результат, если все промисы успешны

🔮 Шаг 07:

Миссия 7

Ксения хочет использовать `Promise.race` и обработать исход, каким бы он ни был (успех или ошибка). Расположите шаги:

1. вызвать `Promise.race` с массивом промисов
2. добавить `.then` для обработки успеха
3. добавить `.catch` для обработки отклонения
4. использовать результат первого завершившегося промиса
5. обрабатывать и успех, и ошибку надлежащим образом

Подсказка: Методы `.then/.catch` вызываются на том же `Promise`, что возвращает `race`.

Ответ:

- вызвать Promise.race с массивом промисов
- добавить .then для обработки успеха
- добавить .catch для обработки отклонения
- использовать результат первого завершившегося промиса
- обрабатывать и успех, и ошибку надлежащим образом

 Шаг 08:**Миссия 8**

Андрей запускает набор API запросов и хочет: если один из них долго выполняется, использовать результат быстрее тех, кто выполнил быстрее. Что он применит и в каком порядке:

1. создать массив промисов запросов
2. вызвать Promise.race на массиве
3. обработать результат первого завершившегося (usuccess or reject)
4. показать пользователю результат или ошибку
5. при необходимости отменить другие запросы / игнорировать их результат

Подсказка: Важно понимать, что race возвращает ****первый**** результат, и остальные можно не ожидать.

Ответ:

- создать массив промисов запросов
- вызвать Promise.race на массиве
- обработать результат первого завершившегося (usuccess or reject)
- показать пользователю результат или ошибку
- при необходимости отменить другие запросы / игнорировать их результат

 Шаг 09:**Миссия 9**

Женя хочет собрать все ответы запросов, но продолжать, даже если некоторые завершились ошибкой. Как будет действовать:

1. создать массив промисов запросов
2. использовать Promise.allSettled на массиве
3. дождаться всех результатов
4. отдельно обработать fulfilled и rejected состояния
5. показать пользователю итоговый отчёт со всеми результатами

Подсказка: allSettled позволяет получить статусы всех промисов.

Ответ:

- создать массив промисов запросов
- использовать Promise.allSettled на массиве
- дождаться всех результатов
- отдельно обработать fulfilled и rejected состояния
- показать пользователю итоговый отчёт со всеми результатами

 Шаг 10:**Миссия 10**

Оля хочет параллельно выполнить 3 запроса, но если один из них не успеет более чем за время таймаута, считать его ошибкой. Расположите шаги:

1. создать три промиса запросов и один промис-таймер, который reject по истечении времени
2. вызвать Promise.race между таймером и группой запросов, либо между каждым запросом с таймером
3. обрабатывать успех, если запрос завершился первым до таймаута
4. обрабатывать ошибку, если таймаут завершён первым или запрос reject
5. сообщить пользователю результат или ошибку

Подсказка: Таймаут можно реализовать как отдельный промис, участвующий в race.

Ответ:

- создать три промиса запросов и один промис-таймер, который reject по истечении времени
- вызвать Promise.race между таймером и группой запросов, либо между каждым запросом с таймером
- обрабатывать успех, если запрос завершился первым до таймаута
- обрабатывать ошибку, если таймаут завершён первым или запрос reject
- сообщить пользователю результат или ошибку

Шаг 11:

Миссия 11

Паша хочет запустить два набора запросов параллельно, один набор быстрее другого, и тогда использовать результат первого набора или таймаута. Расположите операции:

1. создать два массива промисов: быстрые и медленные
2. вызвать Promise.race между быстрым набором и таймаутом и между медленным набором и таймаутом
3. обработать, какой из них завершился первым (быстрый или медленный или таймаут)
4. отменить ожидание другого набора или игнорировать результат, если не нужен
5. отобразить пользователю результат

Подсказка: Можно запустить несколько race, но важно, какой завершён первым.

Ответ:

- создать два массива промисов: быстрые и медленные
- вызвать Promise.race между быстрым набором и таймаутом и между медленным набором и таймаутом
- обработать, какой из них завершился первым (быстрый или медленный или таймаут)
- отменить ожидание другого набора или игнорировать результат, если не нужен
- отобразить пользователю результат

Шаг 12:

Миссия 12

Настя хочет выполнить три API запроса и вернуть ****все**** ответы быстрее, чем один из них упадёт, но если кто-то не отвечает в течение времени, считать его как reject и всё равно собрать остальные. Расположите шаги:

1. создать три промиса запросов + таймаут-помощник для каждого, который reject по истечении времени
2. обернуть каждый запрос с таймаутом в Promise.race

3. использовать `Promise.allSettled` на массив таких `race`-промисов
4. дожидаться всех результатов, даже если некоторые `reject` или таймаут
5. отобразить пользователю список: успешные, ошибки, таймауты

Подсказка: Комбинация `race` + `allSettled` позволяет контролировать таймауты и собрать всё.

Ответ:

- создать три промиса запросов + таймаут-помощник для каждого, который `reject` по истечении времени
- обернуть каждый запрос с таймаутом в `Promise.race`
- использовать `Promise.allSettled` на массив таких `race`-промисов
- дожидаться всех результатов, даже если некоторые `reject` или таймаут
- отобразить пользователю список: успешные, ошибки, таймауты

🔮 Шаг 13:

Миссия 13

Саша хочет начать несколько запросов, показать результат первого успешного, но продолжать слушать остальные на случай, если первый будет ошибкой. Как он поступит:

1. создать массив промисов запросов
2. вызвать `Promise.any` на массиве
3. добавить `.then` для первого успешного результата
4. добавить `.catch`, если все промисы `reject`
5. отобразить успешный результат или сообщение об ошибке, если нет успехов

Подсказка: `any` нужен, когда важно получить хотя бы один успех.

Ответ:

- создать массив промисов запросов
- вызвать `Promise.any` на массиве
- добавить `.then` для первого успешного результата
- добавить `.catch`, если все промисы `reject`
- отобразить успешный результат или сообщение об ошибке, если нет успехов

🔮 Шаг 14:

Миссия 14

Алекс создаёт гибрид: несколько запросов с таймаутами, хочет получить либо первый быстрый, либо все ответы, если никто не успеет. Как он организует:

1. создать массив запросов и отдельный таймаут-промис для `race`
2. вызвать `Promise.race` между запросами и таймаутом
3. если `race` даёт результат запроса до таймаута, использовать его
4. если `race` даёт таймаут, тогда запустить `Promise.allSettled` на запросах без таймаута, чтобы собрать всё
5. обработать результаты: успех или таймаут + возможные ошибки и показать пользователю

Подсказка: Это комбинированный подход: `race` для скорости, `allSettled` — для полноты.

Ответ:

- создать массив запросов и отдельный таймаут-промис для `race`
- вызвать `Promise.race` между запросами и таймаутом
- если `race` даёт результат запроса до таймаута, использовать его

- если `race` даёт таймаут, тогда запустить `Promise.allSettled` на запросах без таймаута, чтобы собрать всё
- обработать результаты: успех или таймаут + возможные ошибки и показать пользователю

Урок 6 - Практика симуляция API, цепочки промисов, UI блокировки

Шаг 01:

Миссия 1

Юлия симулирует API-вызов функцией, которая возвращает промис со `setTimeout`. Как сделать так, чтобы UI не блокировался в это время?

Подсказка: Подумайте, что `setTimeout` делает асинхронным.

Ответ:

- **Использовать промис + `setTimeout`, не блокируя главный поток**
- Выполнить тяжелую работу синхронно перед отображением UI
- Вызывать API в цикле `while` синхронно

Шаг 02:

Миссия 2

Алекс создал цепочку промисов: `fetch(a).then(...).then(...)`. Если первый `fetch` отклонится, как поймать ошибку?

Подсказка: Подумайте, где должен быть `catch`.

Ответ:

- **Добавить `.catch()` после цепочки `then`**
- Добавлять `catch` внутри каждого `then`
- Не нужно `catch`, браузер сам поймает ошибку

Шаг 03:

Миссия 3

Сопоставьте части цепочки промисов с тем, что они делают.

Части:

Ответ:

- `.then(handler)`
- `.catch(handler)`
- `.finally(handler)`
- новый промис внутри `then`

Что делают:

1. Выполняется, когда промис разрешён успешно
2. Обрабатывает ошибку, если промис отклонён выше по цепочке
3. Всегда вызывается после завершения (успех или ошибка)
4. Позволяет продолжить цепочку с новым значением или промисом

Подсказка: Обратите внимание на порядок и на то, что `finally` не меняет результат промиса.

- **`.then(handler)`** — Выполняется, когда промис разрешён успешно
- **`.catch(handler)`** — Обрабатывает ошибку, если промис отклонён выше по цепочке
- **`.finally(handler)`** — Всегда вызывается после завершения (успех или ошибка)

- **новый промис внутри then** — Позволяет продолжить цепочку с новым значением или промисом

🔗 Шаг 04:

Миссия 4

Сопоставьте примеры с эффектом на UI-блокировку.

Примеры:

Ответ:

- Запуск тяжёлого цикла `for` без `await`
- `setTimeout` внутри цикла
- `fetch` + `then` + обработка данных
- `await` внутри `async` функции

Эффект:

1. UI замораживается до завершения цикла
2. UI остаётся отзывчивым, работа отложена
3. UI остаётся отзывчивым, данные приходят позже
4. UI приостанавливается на время `await`, но не блокируется весь поток (способ `non-blocking`)

Подсказка: Обратите внимание, что `await` не делает синхронный цикл асинхронным, он просто вставляет точку, где можно обработать другие задачи.

- **Запуск тяжёлого цикла `for` без `await`** — UI замораживается до завершения цикла
- **`setTimeout` внутри цикла** — UI остаётся отзывчивым, работа отложена
- **`fetch` + `then` + обработка данных** — UI остаётся отзывчивым, данные приходят позже
- **`await` внутри `async` функции** — UI приостанавливается на время `await`, но не блокируется весь поток (способ `non-blocking`)

🔗 Шаг 05:

Миссия 5

Дмитрий симулирует API, используя функцию, возвращающую промис + `setTimeout`. Он хочет сделать цепочку: сначала `fetchUser`, затем `fetchPosts`, затем `fetchComments`. Как гарантировать, что `fetchComments` вызывается после `fetchPosts`?

Подсказка: Каждая следующая операция должна зависеть от предыдущей.

Ответ:

- **Вызывать `fetchUser().then(user => fetchPosts(user)).then(posts => fetchComments(posts))`**
- Запуск всех `fetch` одновременно и потом выбрать `fetchComments`
- Просто вызвать `fetchComments()` после `setTimeout` одного из промисов

🔗 Шаг 06:

Миссия 6

Расположите шаги симуляции API-цепочки без UI блокировки:

1. симуляция API-вызова через промис + `setTimeout`
2. вызов первого API, ожидание ответа
3. после ответа первого, вызов второго API
4. после второго, вызов третьего API
5. отображение результатов на UI после всех ответов

Подсказка: Важно, чтобы каждый следующий запрос ждал предыдущий, и ничего не блокировало основной поток.

Ответ:

- симуляция API-вызова через промис + `setTimeout`
- вызов первого API, ожидание ответа
- после ответа первого, вызов второго API
- после второго, вызов третьего API
- отображение результатов на UI после всех ответов

Шаг 07:

Миссия 7

Ольга хочет симулировать API-цепочку с ошибкой в середине и отобразить ошибку пользователю, но не сбросить UI полностью. Расположите шаги:

1. вызов `fetchUser()`
2. `fetchUser().then(user => fetchPosts(user))`
3. внутри цепочки `.catch(error => показать сообщение об ошибке и остановить дальнейшие запросы)`
4. если ошибки нет, `then(fetchComments)`
5. отобразить результаты или сообщение об ошибке

Подсказка: Обработка ошибок должна быть встроена в цепочку, чтобы UI мог реагировать.

Ответ:

- вызов `fetchUser()`
- `fetchUser().then(user => fetchPosts(user))`
- внутри цепочки `.catch(error => показать сообщение об ошибке и остановить дальнейшие запросы)`
- если ошибки нет, `then(fetchComments)`
- отобразить результаты или сообщение об ошибке

Шаг 08:

Миссия 8

Лиза симулирует API-вызов через `setTimeout` внутри промиса. Что произойдёт с UI, если она вызовет такую функцию без `await` или `then`?

Подсказка: асинхронные отложенные задачи не блокируют главный поток.

Ответ:

- **UI останется отзывчивым, работа продолжится, промис выполнится позже**
- UI замёрзнет до окончания `setTimeout`
- Функция выдаст синхронную ошибку

Шаг 09:

Миссия 9

Максим делает цепочку: `simulateAPI().then(data => process(data)).then(render)`. Какой подход позволит ловить ошибки на любом этапе цепочки?

Подсказка: Подумайте, где добавить `.catch`, чтобы охватить все `.then`.

Ответ:

- **Добавить `.catch()` после последнего `then`**

- Добавлять `catch` внутри каждого `then`
- Не использовать `then`, только `await`
- Использовать `try...catch` вокруг `simulateAPI()` только

🔗 Шаг 10:

Миссия 10

Сопоставьте этапы цепочки промисов с тем, когда UI может быть обновлён или заблокирован.

Этапы:

Ответ:

- симуляция API через `setTimeout`
- `then` с обработкой данных
- рендеринг результата
- обработка ошибки через `catch`

Что происходит на UI:

1. UI отзывчив до ответа API
2. обновление части интерфейса после `then`
3. отображение ошибки, если что-то пошло не так
4. финальный рендеринг только когда данные готовы

Подсказка: Подумайте, что делать в каждом `then` и `catch`, чтобы UI не блокировался и показывал прогресс или ошибки.

- **симуляция API через `setTimeout`** — UI отзывчив до ответа API
- **`then` с обработкой данных** — обновление части интерфейса после `then`
- **обработка ошибки через `catch`** — отображение ошибки, если что-то пошло не так
- **рендеринг результата** — финальный рендеринг только когда данные готовы

🔗 Шаг 11:

Миссия 11

Сопоставьте тип обработки ошибки с удобством для пользователя.

Методы:

Ответ:

- `catch` в конце цепочки
- `try...catch` внутри `async/await`
- обработка в каждом `then`

Плюсы/минусы:

1. проще, но может скрыть, на каком этапе ошибка
2. более явное поведение и контроль на уровне вызова
3. блоки кода громоздкие, если много `then`

Подсказка: Подумайте, какой метод даёт больше контроля и какой меньше усложняет код.

- **`catch` в конце цепочки** — проще, но может скрыть, на каком этапе ошибка
- **`try...catch` внутри `async/await`** — более явное поведение и контроль на уровне вызова
- **обработка в каждом `then`** — блоки кода громоздкие, если много `then`

Шаг 12:

Миссия 12

Расположите шаги запуска цепочки API-симуляции с UI индикатором загрузки:

1. показать spinner загрузки на UI
2. вызвать simulateAPI()
3. await или then для ответа
4. скрыть spinner и отобразить данные или ошибку
5. если ошибка, вывести сообщение ошибки пользователю

Подсказка: UI должен показывать, что идет загрузка, и реагировать на результат или ошибку.

Ответ:

- показать spinner загрузки на UI
- вызвать simulateAPI()
- await или then для ответа
- скрыть spinner и отобразить данные или ошибку
- если ошибка, вывести сообщение ошибки пользователю

Шаг 13:

Миссия 13

Катя симулирует несколько API последовательно, но хочет, чтобы UI мог прерываться или показывать обновления после каждого ответа. Расположите шаги:

1. вызвать первый API, ждать ответ
2. обновить UI частичным результатом
3. вызвать второй API, ждать ответ
4. обновить UI снова
5. последний API и финальный UI-рендеринг

Подсказка: Частичное рендеринг помогает пользователю видеть, что происходит, не ждать всю цепочку.

Ответ:

- вызвать первый API, ждать ответ
- обновить UI частичным результатом
- вызвать второй API, ждать ответ
- обновить UI снова
- последний API и финальный UI-рендеринг

Модуль 4 - Работа с API и сетью

Урок 1 - HTTP-протокол методы, коды, заголовки

Шаг 01:

Миссия 1

Наташа хочет получить данные с сервера, не изменяя их. Какой HTTP-метод наиболее подходит для этого?

Подсказка: метод, который безопасен и не изменяет ресурс.

Ответ:

- **GET**
- POST
- DELETE
- PUT

 **Шаг 02:**

Миссия 2

Сергей хочет обновить информацию о пользователе, заменяя всё новое содержимое. Какой метод он должен использовать?

Подсказка: метод, который полностью заменяет ресурс.

Ответ:

- **PUT**
- GET
- PATCH
- OPTIONS

 **Шаг 03:**

Миссия 3

Ирина делает частичное обновление ресурса (например, только одно поле). Какой HTTP-метод лучше всего?

Подсказка: метод, который изменяет только часть ресурса.

Ответ:

- **PATCH**
- DELETE
- OPTIONS
- HEAD

 **Шаг 04:**

Миссия 4

Сопоставьте HTTP-коды ответа с ситуациями:

Коды:

Ответ:

- 200
- 201
- 400
- 404

Ситуации:

1. Запрос успешно выполнен, ресурс найден
2. Создан новый ресурс
3. Клиент отправил некорректный запрос
4. Ресурс не найден на сервере

Подсказка: Обратите внимание, что 2xx — успешные, 4xx — ошибки клиента.

- **200** — Запрос успешно выполнен, ресурс найден

- **201** — Создан новый ресурс
- **400** — Клиент отправил некорректный запрос
- **404** — Ресурс не найден на сервере

Шаг 05:

Миссия 5

Сопоставьте HTTP-методы с их устойчивостью (idempotency):

Методы:

Ответ:

- GET
- PUT
- POST
- DELETE

Устойчивость:

1. Можно вызывать много раз без изменения результата
2. Вызывает изменение при каждом вызове
3. Вызывает удаление ресурса, устойчивый метод
4. Запрос получения, ничего не изменяет

Подсказка: Методы safe и idempotent часто обсуждаются вместе.

- **GET** — Запрос получения, ничего не изменяет
- **PUT** — Можно вызывать много раз без изменения результата
- **DELETE** — Вызывает удаление ресурса, устойчивый метод
- **POST** — Вызывает изменение при каждом вызове

Шаг 06:

Миссия 6

Сопоставьте заголовки и их роль:

Заголовки:

Ответ:

- Content-Type
- Authorization
- Cache-Control
- Accept

Роли:

1. Указывает формат тела ответа или запроса
2. Информация об авторизации / токене доступа
3. Указания клиенту/прокси как кешировать ответ
4. Какие форматы (MIME) клиент готов принять

Подсказка: Заголовки могут быть как в запросе, так и в ответе.

- **Content-Type** — Указывает формат тела ответа или запроса
- **Authorization** — Информация об авторизации / токене доступа
- **Cache-Control** — Указания клиенту/прокси как кешировать ответ
- **Accept** — Какие форматы (MIME) клиент готов принять

🔮 Шаг 07:

Миссия 7

Пётр отправляет запрос методом POST на создание ресурса. Какой код статуса он скорее всего получит, если всё прошло успешно?

Подсказка: Создание нового ресурса традиционно сопровождается специальным статусом.

Ответ:

- **201 Created**
- 200 OK
- 204 No Content
- 202 Accepted

🔮 Шаг 08:

Миссия 8

Лена делает запрос на удаление ресурса. Если всё удачно, что логично вернуть в ответе?

Подсказка: Часто при удалении ресурс перестает существовать.

Ответ:

- **204 No Content**
- 200 OK
- 202 Accepted
- 404 Not Found

🔮 Шаг 09:

Миссия 9

Расположите шаги работы HTTP-запроса/ответа в хронологическом порядке:

1. Клиент делает запрос с методом и заголовками
2. Сервер обрабатывает запрос
3. Сервер отправляет статусный код и заголовки
4. Сервер отправляет тело ответа (если есть)
5. Клиент получает и интерпретирует ответ

Подсказка: Важно помнить про порядок: метод → заголовки → статус → тело.

Ответ:

- Клиент делает запрос с методом и заголовками
- Сервер обрабатывает запрос
- Сервер отправляет статусный код и заголовки
- Сервер отправляет тело ответа (если есть)
- Клиент получает и интерпретирует ответ

🔮 Шаг 10:

Миссия 10

Расположите группы HTTP-статусов по категориям:

1. 1xx информационные
2. 2xx успех
3. 3xx перенаправление
4. 4xx клиентская ошибка

5. 5xx ошибка сервера

Подсказка: Первая цифра кода определяет категорию.

Ответ:

- 1xx информационные
- 2xx успех
- 3xx перенаправление
- 4xx клиентская ошибка
- 5xx ошибка сервера

Шаг 11:

Миссия 11

Максим хочет узнать, разрешены ли методы POST и GET для ресурса. Какой метод должен использовать клиент?

Подсказка: Существует метод, возвращающий возможные методы ресурса без модификации.

Ответ:

- **OPTIONS**
- HEAD
- TRACE
- PATCH

Шаг 12:

Миссия 12

Ольга делает запрос с заголовком `Accept: application/json`. Что она тем самым указывает серверу?

Подсказка: Заголовок `Accept` сообщает, что клиент может обработать.

Ответ:

- **Клиент хочет получить ответ в формате JSON**
- Клиент отправляет JSON
- Сервер должен принять JSON body
- Ответ будет без заголовков

Шаг 13:

Миссия 13

Сопоставьте коды HTTP с кратким описанием:

Коды:

Ответ:

- 204
- 301
- 403
- 500

Описание:

1. Без тела ответа, но успешный запрос
2. Ресурс перемещён постоянно
3. Доступ запрещён

4. Внутренняя ошибка сервера

Подсказка: Обратите внимание, что 2xx — успех без контента, 3xx — перенаправление, 4xx/5xx — ошибки.

- **204** — Без тела ответа, но успешный запрос
- **301** — Ресурс перемещён постоянно
- **403** — Доступ запрещён
- **500** — Внутренняя ошибка сервера

Шаг 14:

Миссия 14

Расположите классы HTTP-методов по предсказуемости и побочным эффектам:

1. GET
2. HEAD
3. PUT
4. POST
5. DELETE

Подсказка: Некоторые методы безопасны и/или идиempотентны.

Ответ:

- GET
- HEAD
- PUT
- POST
- DELETE

Шаг 15:

Миссия 15

Расположите шаги типичного HTTP ответа с кэшированием:

1. Запрос клиента с заголовками, включая If-None-Match или If-Modified-Since
2. Сервер проверяет, изменился ли ресурс
3. **Если нет изменений** — отвечает 304 Not Modified без тела ответа
4. **Если изменился** — отправляет 200 OK с новым содержимым и заголовками ETag / Last-Modified
5. Клиент обновляет кеш и отображает содержимое

Подсказка: Коды 304 и заголовки ETag / Last-Modified позволяют экономить трафик.

Ответ:

- Запрос клиента с заголовками, включая If-None-Match или If-Modified-Since
- Сервер проверяет, изменился ли ресурс
- **Если нет изменений** — отвечает 304 Not Modified без тела ответа
- **Если изменился** — отправляет 200 OK с новым содержимым и заголовками ETag / Last-Modified
- Клиент обновляет кеш и отображает содержимое

Шаг 16:

Миссия 16

Расположите шаги, которые происходят при редиректе (например, 301):

1. Клиент делает запрос к URL A
2. Сервер отвечает статусом 301 и заголовком Location: URL Б
3. Клиент получает ответ со статусом 301 и читает заголовок Location
4. Клиент автоматически делает новый запрос к URL Б (если браузер / клиент поддерживает редиректы)
5. Сервер Б отвечает содержимым ресурса по новому URL

Подсказка: Редирект — это указание нового местоположения ресурса через заголовок «Location».

Ответ:

- Клиент делает запрос к URL A
- Сервер отвечает статусом 301 и заголовком Location: URL Б
- Клиент получает ответ со статусом 301 и читает заголовок Location
- Клиент автоматически делает новый запрос к URL Б (если браузер / клиент поддерживает редиректы)
- Сервер Б отвечает содержимым ресурса по новому URL

Урок 2 - fetch, axios, формы и отправка данных

Шаг 01:

Миссия 1

Наташа решила получить данные с сервера, используя fetch . Какой метод запроса используется по умолчанию?

Подсказка: Метод должен быть безопасным и не изменять данные.

Ответ:

- GET
- POST
- PUT
- DELETE

Шаг 02:

Миссия 2

Артём хочет отправить форму регистрации на сервер с помощью fetch . Какой метод он должен указать?

Подсказка: Этот метод используется для создания новых ресурсов.

Ответ:

- POST
- GET
- PATCH
- OPTIONS

Шаг 03:

Миссия 3

Ольга делает запрос с fetch и хочет отправить данные формы. Какой заголовок ей нужно установить?

Подсказка: Этот заголовок сообщает серверу о формате данных в теле запроса.

Ответ:

- **Content-Type: application/json**
- Accept: application/json
- Authorization: Bearer token
- Cache-Control: no-cache

Шаг 04:

Миссия 4

После отправки формы Иван получил статус 400 . Что это означает?

Подсказка: Этот код говорит, что ошибка на стороне клиента.

Ответ:

- **Клиент отправил некорректный запрос**
- Сервер временно недоступен
- Доступ запрещён
- Ресурс успешно создан

Шаг 05:

Миссия 5

Саша использует axios для отправки данных. Какой метод он должен использовать, чтобы отправить PATCH-запрос?

Подсказка: В axios у каждого HTTP-метода есть свой метод JavaScript.

Ответ:

- **axios.patch(url, data)**
- axios.send(url, data)
- axios.update(url, data)
- axios.requestPatch(url, data)

Шаг 06:

Миссия 6

Сопоставьте методы axios с их назначением:

Подсказка: Методы axios аналогичны HTTP-методам.

Ответ:

- **axios.get** — Получить данные с сервера
- **axios.post** — Отправить новые данные
- **axios.put** — Полностью заменить ресурс
- **axios.delete** — Удалить ресурс

Шаг 07:

Миссия 7

Марина использует FormData для отправки данных формы. Сопоставьте методы с их действием.

Подсказка: Эти методы позволяют управлять ключами и значениями в форме.

Ответ:

- **append** — Добавляет новое поле формы
- **set** — Устанавливает или заменяет значение поля

- **get** — Получает значение по ключу
- **delete** — Удаляет поле из формы

🔗 Шаг 08:

Миссия 8

Кирилл делает запрос через `fetch('https://api.example.com/data')` . Какой статус ответа считается успехом по умолчанию?

Подсказка: Статусы в диапазоне 200-299 обычно означают успешный ответ.

Ответ:

- **200 OK**
- 404 Not Found
- 500 Internal Server Error
- 301 Moved Permanently

🔗 Шаг 09:

Миссия 9

Даша хочет отправить JSON через `fetch`. Что нужно сделать с телом запроса?

Подсказка: Объекты JavaScript не могут быть переданы как есть — необходимо преобразование.

Ответ:

- **JSON.stringify(object)**
- `object.toString()`
- Отправить объект напрямую

🔗 Шаг 10:

Миссия 10

Сергей использует `axios` и хочет отправить форму с полями и файлами. Какой тип данных удобно использовать?

Подсказка: Данные формы + файлы требуют особого формата.

Ответ:

- **FormData**
- JSON
- XML

🔗 Шаг 11:

Миссия 11

Марина хочет при помощи `fetch` отправить POST запрос и сразу обработать JSON из ответа. Что нужно сделать в `then`?

Подсказка: `fetch` возвращает объект `Response`, который надо преобразовать.

Ответ:

- **response.json()**
- `response.text()`
- `response.data`

🔗 Шаг 12:

Миссия 12

Пётр использует `axios`, а сервер возвращает HTTP-статус 404. Что произойдёт по умолчанию в `axios`?

Подсказка: `axios` автоматически отвергает промис при статусах вне диапазона 2xx.

Ответ:

- **Промис перейдёт в `catch`**
- Будет `then` с пустым объектом
- Игнорируется статус и возвращается как `success`

🔗 Шаг 13:

Миссия 13

Сопоставьте, что делает следующий код с формой:

Метод формы / объект:

Ответ:

- `FormData.append`
- `FormData.set`
- `FormData.get`
- `FormData.delete`

Что делает:

1. Добавляет новое поле без удаления старого
2. Заменяет или устанавливает значение поля, может удалить предыдущий
3. Получает текущее значение поля по имени
4. Удаляет поле из объекта формы

Подсказка: Методы работы с `FormData` аналогичны работе с Map-подобными коллекциями.

- **`FormData.append`** — Добавляет новое поле без удаления старого
- **`FormData.set`** — Заменяет или устанавливает значение поля, может удалить предыдущий
- **`FormData.get`** — Получает текущее значение поля по имени
- **`FormData.delete`** — Удаляет поле из объекта формы

🔗 Шаг 14:

Миссия 14

Сопоставьте части кода с их назначением:

Части кода:

Ответ:

- `fetch(url, { method: 'POST', body: JSON.stringify(data) })`
- `axios.post(url, data)`
- `fetch(url)`
- `axios.get(url, { params: query })`

Назначение:

1. Отправка данных на сервер
2. Автоматический JSON-парсинг ответа
3. Простой GET запрос без тела

4. GET с параметрами запроса

Подсказка: Обратите внимание, что `axios.get` с опцией `params` удобно для `query string`.

- **`fetch(url, { method: 'POST', body: JSON.stringify(data) })`** — Отправка данных на сервер
- **`axios.post(url, data)`** — Автоматический JSON-парсинг ответа
- **`fetch(url)`** — Простой GET запрос без тела
- **`axios.get(url, { params: query })`** — GET с параметрами запроса

Шаг 15:

Миссия 15

Расположите шаги отправки формы и обработки ответа через `fetch`:

1. Собрать данные формы (`input`, файлы) в `FormData` или объект `JS`
2. Настроить `fetch` с методом `POST` и заголовками (если `JSON`) или без (если `FormData`)
3. Отправить `fetch` запрос
4. Обработать ответ: проверить `status`, преобразовать `response.json()` (если `JSON`)
5. Показать пользователю результат или сообщение об ошибке

Подсказка: `JSON` требует заголовков и преобразования, `FormData` — особый формат.

Ответ:

- Собрать данные формы (`input`, файлы) в `FormData` или объект `JS`
- Настроить `fetch` с методом `POST` и заголовками (если `JSON`) или без (если `FormData`)
- Отправить `fetch` запрос
- Обработать ответ: проверить `status`, преобразовать `response.json()` (если `JSON`)
- Показать пользователю результат или сообщение об ошибке

Шаг 16:

Миссия 16

Расположите шаги использования `axios` для отправки `JSON` и обработки ошибки:

1. `axios.post(url, data)`
2. `.then(response => use response.data)`
3. `.catch(error => обработать ошибку)`
4. установить заголовок `Content-Type: application/json`
5. при необходимости показать пользователю статус ошибки

Подсказка: Установка заголовков важна до отправки, `then` и `catch` — после.

Ответ:

- `axios.post(url, data)`
- `.then(response => use response.data)`
- `.catch(error => обработать ошибку)`
- установить заголовок `Content-Type: application/json`
- при необходимости показать пользователю статус ошибки

Урок 3 - Отмена запросов с `AbortController`

Шаг 01:

Миссия 1

Задача: Сопоставьте шаги создания и использования AbortController в правильном порядке.

Подсказка: Подумайте, как правильно организовать этот процесс.

Ответ:

- **Создание экземпляра AbortController** — Первый шаг — нужно создать контроллер.
- **Получение сигнала** — Затем нужно получить сигнал от контроллера.
- **Передача сигнала в fetch** — После этого передаем сигнал в запрос.
- **Вызов abort()** — Когда нужно отменить запрос, вызываем abort().

Шаг 02:

Миссия 2

Задача: Расположите действия по отмене HTTP-запроса в правильном порядке.

Подсказка: Нужно правильно организовать шаги для успешной отмены запроса.

Ответ:

- Создать AbortController
- Создать запрос с fetch
- Передать сигнал в fetch
- Вызвать abort()

Шаг 03:

Миссия 3

Задача: Сопоставьте методы объекта AbortController с их описанием.

Подсказка: Контроллер имеет несколько методов, каждый из которых выполняет свою задачу.

Ответ:

- **abort()** — Метод, который вызывает отмену запроса.
- **signal** — Сигнал, который передается в fetch для контроля отмены.

Шаг 04:

Миссия 4

Задача: Сопоставь шаги создания и использования AbortController в правильном порядке.

Подсказка: Подумай, как правильно организовать этот процесс для отмены запроса.

Ответ:

- **Создать экземпляр AbortController** — Первый шаг — создаем объект для управления отменой.
- **Получить сигнал через controller.signal** — Получаем сигнал, который будет передан в запрос.
- **Передать сигнал в fetch** — Передаем этот сигнал в запрос для отслеживания.
- **Вызвать abort()** — Когда нужно отменить запрос, вызываем abort().

Шаг 05:

Миссия 5

Задача: Расположите действия по отмене HTTP-запроса в правильном порядке.

Подсказка: Подумай, что должно быть первым, а что — последним.

Ответ:

- Создать экземпляр AbortController

- Передать сигнал в fetch
- Вызвать abort()
- Создать запрос через fetch

🔗 Шаг 06:

Миссия 6

Задача: Сопоставьте методы объекта AbortController с их описанием.

Подсказка: Контроллер имеет несколько методов для управления запросами.

Ответ:

- **abort()** — Метод для отмены запроса.
- **signal** — Объект, который передается в fetch для отслеживания состояния запроса.

🔗 Шаг 07:

Миссия 7

Вы только что начали работать над новым проектом и хотите отправить запрос к API. В какой ситуации важно использовать `AbortController`?

Подсказка: Представьте, что вы работаете с большим набором данных, и запрос может занять время.

Ответ:

- **Когда необходимо отменить запрос, если он не успевает завершиться в течение заданного времени**
- Когда API возвращает слишком много данных
- Когда нужно отправить запрос с несколькими заголовками
- Когда API использует сложные методы аутентификации

🔗 Шаг 08:

Миссия 8

Вы пытаетесь отправить запрос на сервер, но время ожидания слишком велико. Какой метод вам поможет отменить запрос?

Подсказка: Это метод, который помогает отменить операции в JavaScript.

Ответ:

- **AbortController**
- fetch
- async/await
- Promise

🔗 Шаг 09:

Миссия 9

Задача: Сопоставьте действия с правильным описанием, связанное с использованием `AbortController`.

Действия:

Ответ:

- Создание нового экземпляра `AbortController`
- Присваивание сигналу запроса
- Отправка запроса с использованием `fetch`

- Отмена запроса

Описания:

1. Метод позволяет отправлять запросы с контролем отмены.
2. Создаёт объект, который сигнализирует о необходимости отмены запроса.
3. Метод привязывает сигнал к запросу, чтобы тот мог быть отменён.
4. Вызывается для остановки запроса, если он не завершился вовремя.

- **Создание нового экземпляра `AbortController`** — Создаёт объект, который сигнализирует о необходимости отмены запроса.
- **Присваивание сигналу запроса** — Метод привязывает сигнал к запросу, чтобы тот мог быть отменён.
- **Отправка запроса с использованием `fetch`** — Метод позволяет отправлять запросы с контролем отмены.
- **Отмена запроса** — Вызывается для остановки запроса, если он не завершился вовремя.

Шаг 10:

Миссия 10

Как вы думаете, что произойдёт, если вы попытаетесь отменить запрос, который уже завершился?

Подсказка: Запросы могут быть успешными или завершиться ошибкой.

Ответ:

- **Ничего не произойдёт, запрос уже завершился**
- Произойдёт ошибка в коде
- Запрос будет отменён, но результат не изменится
- Запрос будет повторно отправлен на сервер

Шаг 11:

Миссия 11

Задача: Сопоставьте корректные методы работы с запросами и их описания.

Методы:

Ответ:

- `fetch`
- `AbortController`
- `setTimeout`

Описания:

1. Метод для отправки HTTP-запросов.
 2. Метод для отмены HTTP-запросов.
 3. Метод для установки таймера отмены запроса.
- **`fetch`** — Метод для отправки HTTP-запросов.
 - **`AbortController`** — Метод для отмены HTTP-запросов.
 - **`setTimeout`** — Метод для установки таймера отмены запроса.

Шаг 12:

Миссия 11

Вы начали работать с внешним API. На что стоит обратить внимание, если запрос может занять много времени?

Подсказка: Иногда запросы могут зависать из-за медленного ответа сервера.

Ответ:

- **Использовать `AbortController` для отмены запроса по таймауту**
- Увеличить тайм-аут ожидания по умолчанию
- Использовать большее количество параллельных запросов
- Повторить запрос через несколько секунд

Шаг 13:

Миссия 12

Представьте, что ваш API-запрос зависит от других данных. Что делать, если первый запрос задерживается, а второй не может продолжиться без него?

Подсказка: Иногда отмена запросов помогает избежать лишней нагрузки на сервер.

Ответ:

- **Отменить запрос через `AbortController`, если он не успел завершиться вовремя**
- Отправить запрос без проверки времени
- Завершить второй запрос в любом случае
- Удвоить время ожидания ответа от сервера

Шаг 14:

Миссия 13

Задача: Сопоставьте действия с их правильным объяснением при использовании `AbortController`.

Действия:

Ответ:

- Создание экземпляра `AbortController`
- Присвоение сигнала запросу
- Отмена запроса

Объяснения:

1. Связывает запрос с сигналом отмены.
 2. Создаёт объект, который отслеживает необходимость отмены.
 3. Останавливает выполнение запроса, если он не завершился вовремя.
- **Создание экземпляра `AbortController`** — Создаёт объект, который отслеживает необходимость отмены.
 - **Присвоение сигнала запросу** — Связывает запрос с сигналом отмены.
 - **Отмена запроса** — Останавливает выполнение запроса, если он не завершился вовремя.

Шаг 15:

Миссия 14

Что произойдёт, если запрос был отменён с помощью `AbortController`, но сервер всё равно отправил ответ?

Подсказка: Иногда сервер может обработать запрос, даже если он был отменён клиентом.

Ответ:

- **Ответ не будет обработан, так как запрос был отменён**
- Ответ будет проигнорирован клиентом, но сервер его отправит
- Запрос будет повторно отправлен на сервер после отмены
- Ответ будет принят, но запрос не будет обработан корректно

🔗 **Шаг 16:**

Миссия 15

Задача: Сопоставьте методы, связанные с отменой запросов, с их описаниями.

Методы:

Ответ:

- AbortController
- fetch
- clearTimeout

Объяснения:

1. Метод для отмены асинхронных операций.
 2. Метод для выполнения HTTP-запросов.
 3. Метод для удаления таймера, установленного ранее.
- **AbortController** — Метод для отмены асинхронных операций.
 - **fetch** — Метод для выполнения HTTP-запросов.
 - **clearTimeout** — Метод для удаления таймера, установленного ранее.

📖 Урок 4 - Кеширование и стратегия повторов

🔗 **Шаг 01:**

Миссия 1

Вы разрабатываете сайт, который делает API-запросы к серверу. Как вы уменьшите количество повторных запросов к одному и тому же ресурсу?

Подсказка: Иногда данные на сервере не меняются часто, и их можно сохранить локально для повторного использования.

Ответ:

- **Использовать кеширование данных на клиенте**
- Использовать повторные запросы каждый раз, когда пользователь обращается к сайту
- Запрашивать данные с сервера каждый раз, даже если они не изменились
- Игнорировать запросы, если они не критичны

🔗 **Шаг 02:**

Миссия 2

Предположим, что ваш сайт делает запросы к одному и тому же ресурсу несколько раз в течение короткого времени. Какую стратегию повторов вы можете использовать для минимизации нагрузки на сервер?

Подсказка: Есть способы избежать множества одинаковых запросов за короткое время.

Ответ:

- **Использовать дебаунсинг запросов**
- Запрашивать данные только один раз в день

- Запрашивать данные каждый раз, но с увеличением времени ожидания
- Отключить кеширование для всех запросов

Шаг 03:

Миссия 3

Задача: Сопоставьте различные стратегии кеширования с их описаниями.

Стратегии кеширования:

Ответ:

- Memory cache
- LocalStorage
- IndexedDB

Описания:

1. Хранение данных в памяти браузера для быстрого доступа, но данные исчезают после перезагрузки страницы.
2. Простое хранилище, которое сохраняет данные между сессиями, но объём ограничен.
3. Продвинутое хранилище для больших объемов данных, которое сохраняется даже после перезагрузки страницы.

- **Memory cache** — Хранение данных в памяти браузера для быстрого доступа, но данные исчезают после перезагрузки страницы.
- **LocalStorage** — Простое хранилище, которое сохраняет данные между сессиями, но объём ограничен.
- **IndexedDB** — Продвинутое хранилище для больших объемов данных, которое сохраняется даже после перезагрузки страницы.

Шаг 04:

Миссия 4

Какая стратегия кеширования на клиенте подходит для сохранения данных с ограниченной жизнью, которые не должны храниться долго?

Подсказка: Для временных данных лучше использовать лёгкие решения, которые не требуют постоянного хранения.

Ответ:

- **Memory cache**
- LocalStorage
- IndexedDB
- Cookies

Шаг 05:

Миссия 5

Задача: Сопоставьте типы кеширования с их преимуществами и недостатками.

Типы кеширования:

Ответ:

- LocalStorage
- Memory cache
- IndexedDB

Преимущества и недостатки:

1. Малый объём хранилища, но данные доступны после перезагрузки.
 2. Быстрый доступ к данным, но данные теряются при перезагрузке страницы.
 3. Подходит для хранения больших объёмов данных, но доступ к данным может быть медленным.
- **LocalStorage** — Малый объём хранилища, но данные доступны после перезагрузки.
 - **Memory cache** — Быстрый доступ к данным, но данные теряются при перезагрузке страницы.
 - **IndexedDB** — Подходит для хранения больших объёмов данных, но доступ к данным может быть медленным.

Шаг 06:

Миссия 6

Ты только что получил задание: нужно организовать кэширование для данных, получаемых с сервера. Какую стратегию кэширования ты выберешь?

Подсказка: Для часто запрашиваемых данных важно сохранить их в памяти для ускорения доступа.

Ответ:

- **Использовать стратегию кэширования на основе времени жизни данных (TTL)**
- Не использовать кэширование
- Кэшировать только в локальном хранилище браузера
- Использовать кэширование в сессионной памяти браузера

Шаг 07:

Миссия 7

Ты решил реализовать стратегию повторов запросов при неудачных попытках получить данные с сервера. Какой подход тебе подходит?

Подсказка: Убедись, что система не будет постоянно перегружать сервер, отправляя повторные запросы без перерыва.

Ответ:

- **Использовать экспоненциальную задержку между повторными попытками**
- Отправить запросы без задержки, пока не получим результат
- Использовать фиксированную задержку для всех попыток
- Пытаться сразу несколько раз без задержки, а затем прекратить попытки

Шаг 08:

Миссия 8

Сопоставьте стратегии кэширования и их применения:

Ответ:

- LRU (Least Recently Used)
- TTL (Time To Live)
- Запросы без кэширования

Определения:

1. Стратегия, при которой кэшируются только недавно использованные данные.

2. Стратегия, при которой данные кэшируются до определенного времени, после чего они удаляются.
3. Не используется кэширование для хранения данных.

Подсказка:

Каждая стратегия подходит для разных типов данных. Подумайте, как часто будут запрашиваться данные и насколько важно их обновление.

- **LRU (Least Recently Used)** — Стратегия, при которой кэшируются только недавно использованные данные.
- **TTL (Time To Live)** — Стратегия, при которой данные кэшируются до определенного времени, после чего они удаляются.
- **Запросы без кэширования** — Не используется кэширование для хранения данных.

Шаг 09:

Миссия 9

Ты работаешь над проектом, где нужно отправить несколько запросов одновременно. Какую стратегию ты выберешь для их повторного выполнения при ошибке?

Подсказка: Понять, когда стоит повторить запрос, а когда — лучше прекратить, очень важно.

Ответ:

- **Использовать стратегию параллельных повторных запросов с экспоненциальной задержкой**
- Отправить все запросы одновременно без повторных попыток
- Отправить запросы поочередно и повторить каждый при ошибке
- Пытаться повторить запросы без задержек, пока все не пройдут успешно

Шаг 10:

Миссия 10

Расположите шаги реализации стратегии кэширования в правильном порядке:

1. Определение данных, которые нужно кэшировать
2. Настройка времени жизни кэша (TTL)
3. Кэширование данных в памяти
4. Использование данных из кэша при последующих запросах

Подсказка:

Процесс начинается с выбора данных для кэширования и их параметров. Затем важно настроить систему так, чтобы кэшированные данные использовались эффективно.

Ответ:

- Определение данных, которые нужно кэшировать
- Настройка времени жизни кэша (TTL)
- Кэширование данных в памяти
- Использование данных из кэша при последующих запросах

Шаг 11:

Вопрос 5

Что является принципом "чистого кода" при рефакторинге?

Подсказка: Уменьшение дублирования — ключ к чистоте кода.

Ответ:

- **Вынос повторяющегося кода в отдельные функции**
- Добавление длинных комментариев
- Смешивание логики и интерфейса
- Использование непонятных сокращений

Урок 5 - WebSockets и EventSource (введение)

Шаг 01:

Миссия 1

Алекс начал работать над чатом для команды. Он слышал про WebSockets, но запутался. Что отличает WebSocket от обычного HTTP-запроса?

Подсказка: Подумай о том, как долго соединение остаётся открытым.

Ответ:

- **WebSocket поддерживает постоянное двустороннее соединение**
- HTTP-запросы быстрее, чем WebSocket
- WebSocket можно использовать только для загрузки картинок
- WebSocket закрывается сразу после первого сообщения

Шаг 02:

Миссия 2

София пишет онлайн-игру и хочет, чтобы все игроки сразу видели движения друг друга. Какой протокол ей подойдёт?

Подсказка: Выбирай протокол, который умеет обновлять данные в реальном времени.

Ответ:

- **WebSocket**
- FTP
- SMTP
- POP3

Шаг 03:

Миссия 3

Иван заметил, что сервер присылает обновления только тогда, когда они есть. Это напоминает ему механизм подписки. Какой API для этого существует в браузерах?

Подсказка: Он используется для событий, похожих на ленту новостей.

Ответ:

- **EventSource (Server-Sent Events)**
- WebRTC
- GraphQL
- SOAP

Шаг 04:

Миссия 4

Мария разрабатывает доску для онлайн-обсуждений. Ей нужно, чтобы пользователи мгновенно видели новые сообщения. Что проще использовать?

Подсказка: Подумай: нужно только сервер → клиент или двустороннее соединение?

Ответ:

- **EventSource, если нужны только обновления от сервера**
- WebSocket, если нужен только односторонний поток
- REST API
- SMTP

Шаг 05:

Миссия 5

Алексей случайно написал в коде для WebSocket: `new EventSource('ws://example.com')` . Почему это ошибка?

Подсказка: У разных API разные протоколы и классы.

Ответ:

- **EventSource работает только с http/https, а не с ws://**
- WebSocket не требует URL
- EventSource автоматически конвертирует ws в http
- Это допустимо, ошибки не будет

Шаг 06:

Миссия 6

Оля хочет узнать, что произойдёт, если закрыть вкладку с открытым WebSocket-соединением.

Подсказка: Подумай о жизненном цикле соединения.

Ответ:

- **Соединение будет закрыто**
- Соединение будет работать в фоне бесконечно
- Соединение перейдёт в спящий режим
- Ничего не изменится

Шаг 07:

Миссия 7

Программисты сделали WebSocket-сервер для онлайн-игры. Но игроки жалуются: при потере сети соединение рвётся. Что можно сделать на клиенте?

Подсказка: Вспомни про повторные подключения.

Ответ:

- **Реализовать автоматическое переподключение**
- Ничего нельзя сделать
- Использовать только EventSource
- Отключить обработку ошибок

Шаг 08:

Миссия 8

Сопоставь событие WebSocket с его назначением.

Подсказка: Вспомни, какие события есть у соединения: открытие, получение сообщений, ошибки и закрытие.

Ответ:

- **open** — Соединение успешно установлено
- **message** — Получены данные от сервера
- **error** — Произошла ошибка при работе соединения
- **close** — Соединение закрыто

🔗 **Шаг 09:**

Миссия 9

Сопоставь класс API с его назначением.

Подсказка: Один класс для двусторонних чатов, другой для новостных лент.

Ответ:

- **WebSocket** — Подходит для чата и игр с обменом данными в обе стороны
- **EventSource** — Подходит для односторонних уведомлений, например, новостной ленты

🔗 **Шаг 10:**

Миссия 10

У Кати есть список событий EventSource. Сопоставь их с описаниями.

Подсказка: Они похожи на события WebSocket, но отличаются деталями.

Ответ:

- **open** — Соединение с сервером установлено
- **message** — Получено новое событие с данными
- **error** — Соединение было прервано или возникла ошибка

🔗 **Шаг 11:**

Миссия 11

Разработчики обсуждают, как браузеры работают с WebSocket и EventSource. Сопоставь фразу с правильным API.

Подсказка: Подумай, где доступна отправка сообщений с клиента.

Ответ:

- **Клиент может отправлять и получать данные** — WebSocket
- **Клиент может только получать данные** — EventSource

🔗 **Шаг 12:**

Миссия 12

Сопоставь протокол и его тип использования.

Подсказка: Укажи, какой работает поверх TCP и какой всегда односторонний.

Ответ:

- **WebSocket** — Двусторонняя передача данных поверх TCP
- **EventSource** — Односторонняя подписка на события от сервера

🔗 **Шаг 13:**

Миссия 13

Расположи шаги подключения WebSocket-клиента в правильном порядке.

Подсказка: Сначала создаём объект, потом слушаем события, потом отправляем данные.

Ответ:

- Создать объект WebSocket с адресом
- Назначить обработчики событий (onopen, onmessage и т.д.)
- Отправить сообщение на сервер
- Получить ответ от сервера

 Шаг 14:

Миссия 14

Расположи действия при работе с EventSource в верном порядке.

Подсказка: Сначала создаётся подписка, потом назначаются обработчики, затем обрабатываются сообщения.

Ответ:

- Создать объект EventSource с URL
- Назначить обработчик события message
- Обработать полученные данные
- Закрыть соединение при необходимости

 Шаг 15:

Миссия 15

Игровая команда хочет организовать чат через WebSocket. Расставь шаги по порядку.

Подсказка: Сначала открывается соединение, потом идёт общение.

Ответ:

- Открыть соединение
- Обменяться сообщениями
- Закрыть соединение

 Шаг 16:

Миссия 16

Расположи этапы обработки ошибок WebSocket.

Подсказка: Сначала ошибка фиксируется, потом предпринимается действие.

Ответ:

- Обработчик события error срабатывает
- Показать сообщение пользователю
- Попробовать переподключиться

Модуль 5 - Структуры данных и алгоритмы

Урок 1 - Сложность алгоритмов Big-O

 Шаг 01:

Миссия 1

Аня пишет функцию, которая просто проходит по массиву из n элементов один раз, выполняя базовую операцию на каждом элементе. Какова временная сложность этой операции?

Подсказка: Подумай, как количество шагов зависит от размера массива.

Ответ:

- **$O(n)$**
- $O(1)$
- $O(n^2)$

Шаг 02:

Миссия 2

Борис реализует вложенные циклы: внешний цикл по n , внутренний — тоже по n . Какая временная сложность этой части кода?

Подсказка: Каждый элемент внешнего цикла вызывает полный проход внутреннего.

Ответ:

- **$O(n^2)$**
- $O(n \log n)$
- $O(n)$

Шаг 03:

Миссия 3

Саша сортирует массив из n элементов, используя алгоритм быстрой сортировки (QuickSort) в среднем случае. Какова средняя временная сложность?

Подсказка: Быстрая сортировка обычно делит задачу пополам.

Ответ:

- **$O(n \log n)$**
- $O(n^2)$
- $O(\log n)$

Шаг 04:

Миссия 4

Даша проверяет, есть ли элемент в хэш-таблице — операция поиска по ключу. Какова временная сложность в среднем случае?

Подсказка: Хэш-таблица позволяет прямой доступ по ключу.

Ответ:

- **$O(1)$**
- $O(n)$
- $O(\log n)$

Шаг 05:

Миссия 5

Игорь делает рекурсивную функцию, которая делит задачу пополам на каждом шагу, пока не достигнет базового случая. Какова временная сложность?

Подсказка: Подумай, как глубина рекурсии связана с $\log_2 n$.

Ответ:

- **$O(\log n)$**
- $O(n)$
- $O(n \log n)$

🔗 Шаг 06:

Миссия 6

Катя использует алгоритм, который сначала сортирует массив ($O(n \log n)$), а затем выполняет линейный проход по нему. Каков порядок роста всего процесса?

Подсказка: Сложность определяется самой медленной частью, когда n большое.

Ответ:

- $O(n \log n)$
- $O(n^2)$
- $O(n)$

🔗 Шаг 07:

Миссия 7

Задача: Сопоставьте алгоритм с его временной сложностью в оптимальном или среднем случае.

Алгоритмы:

Ответ:

- Линейный поиск в неотсортированном массиве
- Бинарный поиск в отсортированном массиве
- Сортировка слиянием (Merge Sort)
- Поиск по хеш-таблице

Сложности:

1. $O(n \log n)$
2. $O(\log n)$
3. $O(1)$
4. $O(n)$

🔗 Подсказка:

Подумайте, какие структуры данных позволяют быстрый доступ, какие — требуют полного перебора.

- **Линейный поиск в неотсортированном массиве** — $O(n)$
- **Бинарный поиск в отсортированном массиве** — $O(\log n)$
- **Сортировка слиянием (Merge Sort)** — $O(n \log n)$
- **Поиск по хеш-таблице** — $O(1)$

🔗 Шаг 08:

Миссия 8

Задача: Сопоставьте рост сложности с ростом данных N :

Описание:

Ответ:

- Константная сложность
- Логарифмическая сложность
- Линейная сложность
- Квадратичная сложность

Когда показывает себя:

1. Количество шагов не зависит от N

2. Увеличение на один шаг при удвоении размера данных
3. Количество шагов растёт прямо пропорционально N
4. Каждый элемент сравнивается с каждым другим

🔗 **Подсказка:**

Старайся визуализировать: если ты удваиваешь n — как меняется число операций?

- **Константная сложность** — Количество шагов не зависит от N
- **Логарифмическая сложность** — Увеличение на один шаг при удвоении размера данных
- **Линейная сложность** — Количество шагов растёт прямо пропорционально N
- **Квадратичная сложность** — Каждый элемент сравнивается с каждым другим

🔗 **Шаг 09:**

Миссия 9

Задача: Сопоставьте названия функций с их вычислительной сложностью, основанных на известных алгоритмах.

Функции / Алгоритмы:

Ответ:

- Поиск минимального элемента в неотсортированном массиве
- Выборка топ- k элементов через частичную сортировку
- Сортировка вставками (Insertion Sort) на почти отсортированном массиве
- Двоичное дерево поиска сбалансированное, операция поиска

Сложности:

1. $O(n)$
2. $O(n + k \log n)$
3. $O(n^2)$ в худшем, но близко к $O(n)$ на почти отсортированных данных
4. $O(\log n)$

🔗 **Подсказка:**

Подумай, что происходит, если данные почти в порядке; топ- k — сортировка всех ли элементов?

- **Поиск минимального элемента в неотсортированном массиве** — $O(n)$
- **Выборка топ- k элементов через частичную сортировку** — $O(n + k \log n)$
- **Сортировка вставками (Insertion Sort) на почти отсортированном массиве** — $O(n^2)$ в худшем, но близко к $O(n)$ на почти отсортированных данных
- **Двоичное дерево поиска сбалансированное, операция поиска** — $O(\log n)$

🔗 **Шаг 10:**

Миссия 10

Вопрос: Упорядочите следующие функции по возрастанию временной сложности:

1. $O(n \log n)$
2. $O(n^2)$
3. $O(\log n)$
4. $O(n)$

🔗 **Подсказка:**

Начните с тех, что растут медленнее всего, когда N становится очень большим.

Ответ:

- $O(\log n)$
- $O(n)$
- $O(n \log n)$
- $O(n^2)$

🔮 Шаг 11:

Миссия 11

Вопрос: Разработчик Виктор сравнивает сложности нескольких алгоритмов: $O(2^n)$, $O(n!)$, $O(n^2)$, $O(2^{\log n})$. Упорядочите их по возрастанию сложности.

🔮 Подсказка:

Чем выше экспонента или факториал — тем быстрее растёт сложность при большом n .

Ответ:

- $O(n^2)$
- $O(2^{\log n})$
- $O(2^n)$
- $O(n!)$

🔮 Шаг 12:

Миссия 12

Вопрос: Женя рассматривает алгоритмы: $O(n)$, $O(n\sqrt{n})$, $O(n^2)$, $O(n \log n)$, $O(\log n)$. Упорядочите по возрастанию.

🔮 Подсказка:

Убедись, что $\sqrt{n} < \log n$ или $> \log n$ зависит от роста — представь пример при большом n .

Ответ:

- $O(\log n)$
- $O(n)$
- $O(n \log n)$
- $O(n\sqrt{n})$
- $O(n^2)$

🔮 Шаг 13:

Миссия 13

Вопрос: Упорядочите следующие алгоритмы по возрастанию временной сложности, сравнивая: линейный проход, бинарный поиск, полный перебор всех пар элементов, а также логарифмическую и константную сложность.

1. Поиск двух одинаковых элементов через два вложенных цикла
2. Поиск элемента через бинарный поиск в отсортированном массиве
3. Линейный проход по массиву
4. Постоянная операция (например, доступ по индексу)

🔮 Подсказка:

Самое лёгкое — когда время не зависит от N , затем медленно растущие, затем быстрые взлёты.

Ответ:

- Постоянная операция (например, доступ по индексу)

- Бинарный поиск в отсортированном массиве
- Линейный проход по массиву
- Поиск двух одинаковых элементов через два вложенных цикла

Урок 2 - Массивы и методы (reduce, flatMap, sort)

Шаг 01:

Миссия 1

Герой Лена решила посчитать сумму очков в игре по массиву [10, 20, 30] . Какой метод массива ей поможет?

Подсказка: Этот метод аккумулирует значения в одно.

Ответ:

- **reduce()**
- map()
- filter()
- forEach()

Шаг 02:

Миссия 2

Программист Игорь хочет объединить массивы друзей из разных чатов в один без вложенности. Какой метод лучше?

Подсказка: Этот метод работает как map, но ещё разворачивает результат.

Ответ:

- **flatMap()**
- reduce()
- map()
- concat()

Шаг 03:

Миссия 3

Геймер Артём сортирует очки игроков [50, 100, 10] . Какой метод массива он должен использовать?

Подсказка: Этот метод изменяет массив на месте и умеет работать с компаратором.

Ответ:

- **sort()**
- reduce()
- map()
- filter()

Шаг 04:

Миссия 4

Игрок Вика хочет отсортировать имена пользователей по алфавиту. Какой метод ей подойдёт?

Подсказка: Этот метод без компаратора сортирует строки по Unicode.

Ответ:

- **sort()**

- flatMap()
- reduce()
- map()

🔗 Шаг 05:

Миссия 5

Сопоставьте методы массивов с их назначением.

Подсказка: Вспомните: одни методы сворачивают, другие разворачивают, а некоторые изменяют порядок.

Ответ:

- **reduce()** — Сворачивает массив в одно значение
- **flatMap()** — Преобразует и разворачивает массив
- **sort()** — Сортирует элементы массива
- **map()** — Создаёт новый массив, преобразуя каждый элемент

🔗 Шаг 06:

Миссия 6

Программист собирает статистику. Сопоставьте ситуацию с правильным методом массива.

Подсказка: Вспомните реальные задачи: подсчёт, объединение, упорядочивание.

Ответ:

- **Подсчитать сумму баллов** — reduce()
- **Разбить строку на слова и объединить в общий массив** — flatMap()
- **Упорядочить список друзей** — sort()
- **Изменить каждый элемент массива на его длину** — map()

🔗 Шаг 07:

Миссия 7

Андрей работает над магазином. Сопоставьте товары и действия с методами.

Подсказка: Каждый метод отвечает за свой тип операции: свёртка, сортировка, разворачивание.

Ответ:

- **Сумма цен в корзине** — reduce()
- **Развёрнуть массив категорий с подкатегориями** — flatMap()
- **Отсортировать товары по цене** — sort()
- **Создать массив названий товаров в верхнем регистре** — map()

🔗 Шаг 08:

Миссия 8

Даша анализирует массив пользователей. Сопоставьте задачу с методом.

Подсказка: Обратите внимание, что методы могут изменять массив, а могут возвращать новое значение.

Ответ:

- **Получить средний возраст пользователей** — reduce()
- **Сделать список всех хобби из вложенных массивов** — flatMap()
- **Отсортировать имена по алфавиту** — sort()
- **Создать массив первых букв имён** — map()

🔗 Шаг 09:

Миссия 9

Компания делает отчёт. Сопоставьте пример отчётной операции с методом массива.

Подсказка: Сравните задачи: агрегирование, сортировка, разворачивание, преобразование.

Ответ:

- **Сумма всех заказов** — `reduce()`
- **Список всех товаров из заказов** — `flatMap()`
- **Отсортировать заказы по дате** — `sort()`
- **Поменять названия товаров на заглавные** — `map()`

🔗 Шаг 10:

Миссия 10

Разработчик Арсений работает с данными студентов. Сопоставьте задачу и метод.

Подсказка: Метод с аккумулятором полезен для вычислений, а метод сортировки — для порядка.

Ответ:

- **Средний балл студентов** — `reduce()`
- **Все курсы студентов в один список** — `flatMap()`
- **Список студентов по алфавиту** — `sort()`
- **Добавить к имени студента его возраст** — `map()`

🔗 Шаг 11:

Миссия 11

В проекте о фильмах нужно выполнить разные операции. Сопоставьте их с методами.

Подсказка: Вспомните, какой метод возвращает одно значение, а какой — массив.

Ответ:

- **Общая сумма рейтингов** — `reduce()`
- **Список всех актёров из фильмов** — `flatMap()`
- **Отсортировать фильмы по рейтингу** — `sort()`
- **Список названий фильмов в верхнем регистре** — `map()`

🔗 Шаг 12:

Миссия 12

Определи порядок действий для подсчёта суммы элементов массива с помощью `reduce`.

Подсказка: Вспомни: сначала берётся начальное значение, затем перебор элементов, добавление их в аккумулятор, и в конце возвращается результат.

Ответ:

- Задать начальное значение аккумулятора
- Перебрать элементы массива
- Добавить значение элемента к аккумулятору
- Вернуть результат свёртки

🔗 Шаг 13:

Миссия 13

Расположи шаги использования sort с функцией сравнения для чисел.

Подсказка: Сначала пишется функция-компаратор, затем она применяется к элементам, и в конце массив меняется.

Ответ:

- Определить функцию сравнения
- Применить функцию сравнения к парам элементов
- Изменить порядок элементов в массиве
- Вернуть отсортированный массив

Шаг 14:

Миссия 14

Определи порядок применения flatMap для преобразования массива предложений в слова.

Подсказка: Сначала идёт разбиение, потом разворачивание, а затем объединение в новый массив.

Ответ:

- Применить функцию разбиения к каждому элементу
- Развернуть полученные массивы
- Объединить всё в один массив
- Вернуть результат

Шаг 15:

Миссия 15

Расположи шаги для нахождения максимального значения с помощью reduce.

Подсказка: Сравнивай аккумулятор с текущим элементом и сохраняй большее.

Ответ:

- Задать начальное значение аккумулятора
- Сравнить аккумулятор с текущим элементом
- Сохранить большее значение в аккумулятор
- Вернуть финальный результат

Шаг 16:

Миссия 16

Определи правильный порядок сортировки массива строк по длине.

Подсказка: Для этого пишут функцию сравнения, которая возвращает разницу длин.

Ответ:

- Написать функцию сравнения по длине
- Применить её к парам строк
- Изменить порядок элементов
- Вернуть отсортированный массив

Урок 3 - Объекты, Map, Set, WeakMap

Шаг 01:

Миссия 1

Алексей хранит список пользователей в объекте, но заметил, что ключи могут случайно пересекаться со свойствами прототипа. Какую структуру данных стоит использовать вместо объекта?

Подсказка: Вспомни структуру, которая позволяет безопасно хранить любые ключи, включая объекты.

Ответ:

- **Map**
- Object
- Array
- Set

Шаг 02:

Миссия 2

Ирина создала Set для хранения логинов. Какое свойство отличает Set от массива?

Подсказка: Подумай, что делает Set уникальным.

Ответ:

- **Хранит только уникальные значения**
- Имеет индексацию как массив
- Автоматически сортирует элементы
- Позволяет обращаться к значениям по ключу

Шаг 03:

Миссия 3

Марк хранит данные с привязкой к DOM-элементам. Какую структуру он должен использовать, чтобы элементы удалялись автоматически при сборке мусора?

Подсказка: Есть структура, которая слабо держит ссылки.

Ответ:

- **WeakMap**
- Map
- WeakSet
- Object

Шаг 04:

Миссия 4

Оля пытается перебрать значения Map. Какой метод лучше всего использовать?

Подсказка: Map поддерживает несколько методов итерации.

Ответ:

- **map.values()**
- map[0]
- map.getValues()
- map.toArray()

Шаг 05:

Миссия 5

Петя хочет проверить, есть ли ключ в Map. Какой метод используется?

Подсказка: Вспомни метод, который возвращает true или false.

Ответ:

- **map.has(key)**
- map.includes(key)
- map.contains(key)
- map.exists(key)

🧭 Шаг 06:

Миссия 6

Даша хочет удалить элемент из Set. Какой метод ей нужен?

Подсказка: Вспомни метод, который возвращает true, если элемент был удален.

Ответ:

- **set.delete(value)**
- set.remove(value)
- set.pop(value)
- set.clear(value)

🧭 Шаг 07:

Миссия 7

Сопоставь структуру с её особенностью:

Подсказка: Вспомни, какая структура работает только с уникальными значениями, а какая хранит слабые ссылки.

Ответ:

- **Set** — Хранит только уникальные значения
- **Map** — Позволяет использовать ключами любые типы
- **WeakMap** — Ключи — только объекты, удаляются при сборке мусора
- **Object** — Ключи автоматически приводятся к строкам

🧭 Шаг 08:

Миссия 8

Сопоставь метод с его назначением:

Подсказка: Подумай, какой метод возвращает все ключи, а какой — все значения.

Ответ:

- **map.keys()** — Возвращает все ключи
- **map.values()** — Возвращает все значения
- **map.entries()** — Возвращает массив пар [ключ, значение]
- **map.get(key)** — Возвращает значение по ключу

🧭 Шаг 09:

Миссия 9

Сопоставь действие с корректным методом Set:

Подсказка: Подумай, где используется add, а где clear.

Ответ:

- **Добавить элемент** — set.add(value)

- **Удалить все элементы** — `set.clear()`
- **Проверить наличие элемента** — `set.has(value)`
- **Удалить конкретный элемент** — `set.delete(value)`

🔮 Шаг 10:

Миссия 10

Сопоставь структуру с примером применения:

Подсказка: Вспомни, где важны уникальные значения, а где — безопасная работа с объектами.

Ответ:

- **Set** — Хранение списка уникальных логинов
- **Map** — Хранение связки ID пользователя и его профиля
- **WeakMap** — Хранение данных, связанных с DOM-элементом
- **Object** — Простая структура с ключами-строками

🔮 Шаг 11:

Миссия 11

Сопоставь метод Map с его результатом:

Подсказка: Думай о том, что возвращает каждый вызов.

Ответ:

- **map.set(key, value)** — Добавляет или обновляет значение
- **map.has(key)** — Проверяет наличие ключа
- **map.size** — Количество элементов
- **map.delete(key)** — Удаляет элемент по ключу

🔮 Шаг 12:

Миссия 12

Сопоставь структуру с её ограничением:

Подсказка: Подумай, почему WeakMap нельзя перебирать.

Ответ:

- **Object** — Ключи приводятся к строкам
- **Map** — Можно итерировать по ключам и значениям
- **Set** — Нет доступа по индексу
- **WeakMap** — Нельзя перебирать элементы напрямую

🔮 Шаг 13:

Миссия 13

Расположи шаги добавления и проверки значения в Set в правильном порядке:

Подсказка: Вспомни: сначала создаём Set, потом добавляем, затем проверяем.

Ответ:

- Создать Set
- Добавить элемент через `add`
- Проверить наличие через `has`

Шаг 14:

Миссия 14

Расставь шаги работы с Map в правильном порядке:

Подсказка: Подумай: сначала создаём Map, потом добавляем, потом читаем.

Ответ:

- Создать Map
- Добавить пару через set
- Получить значение через get

Шаг 15:

Миссия 15

Определи порядок действий при использовании WeakMap для хранения состояния DOM-элемента:

Подсказка: Сначала создаём WeakMap, потом устанавливаем данные, затем читаем.

Ответ:

- Создать WeakMap
- Установить данные через set
- Получить данные через get

Шаг 16:

Миссия 16

Расположи шаги очистки Set:

Подсказка: Вспомни, что сначала добавляем элементы, потом вызываем clear.

Ответ:

- Создать Set
- Добавить несколько элементов
- Очистить с помощью clear

Урок 4 - Очередь, стек, связный список (на JS)

Шаг 01:

Миссия 1

Программист Лена пишет игру, где игроки становятся в очередь за ресурсами. Какой метод массива лучше всего подходит для реализации добавления элемента в конец очереди?

Подсказка: Вспомни, что очередь работает по принципу FIFO: первым вошёл — первым вышел.

Ответ:

- **push()**
- pop()
- shift()
- unshift()

Шаг 02:

Миссия 2

Игрок Костя написал функцию для стека. Какой метод массива используется, чтобы убрать верхний элемент стека?

Подсказка: Стек — это LIFO: последним зашёл — первым вышел.

Ответ:

- `shift()`
- **`pop()`**
- `unshift()`
- `slice()`

Шаг 03:

Миссия 3

Во время турнира разработчик Денис моделирует очередь игроков. Какой метод массива удаляет первый элемент очереди?

Подсказка: Представь очередь у кассы: первым забирает покупку тот, кто стоял первым.

Ответ:

- **`shift()`**
- `pop()`
- `push()`
- `slice()`

Шаг 04:

Миссия 4

Катя хочет построить стек для отмены действий в приложении. Какой метод добавит новый элемент наверх стека?

Подсказка: Представь стопку книг: кладёшь сверху — убираешь тоже сверху.

Ответ:

- **`push()`**
- `shift()`
- `unshift()`
- `splice()`

Шаг 05:

Миссия 5

Программист Артём делает симуляцию парковки с очередью машин. Как лучше проверить, пустая ли очередь?

Подсказка: В очереди пусто, если элементов нет совсем.

Ответ:

- **`queue.length === 0`**
- `queue === null`
- `queue[0] === undefined`
- `queue.isEmpty()`

Шаг 06:

Миссия 6

Виталий создаёт игру с undo/redo. Какой метод массива чаще всего применяют, чтобы получить последний элемент стека, не удаляя его?

Подсказка: Индекс последнего элемента = длина массива минус 1.

Ответ:

- **stack[stack.length - 1]**
- stack.last()
- stack.pop()
- stack.slice(-2)

🔮 Шаг 07:

Миссия 7

Сопоставь тип структуры данных и её характеристику.

Подсказка: Вспомни порядок работы: FIFO и LIFO.

Ответ:

- **Очередь** — Первым вошёл — первым вышел
- **Стек** — Последним вошёл — первым вышел
- **Связный список** — Элементы связаны указателями на следующий
- **Массив** — Элементы хранятся в непрерывном блоке памяти

🔮 Шаг 08:

Миссия 8

Связный список активно применяют в разработке. Сопоставь термин и его описание.

Подсказка: Каждый элемент списка хранит не только значение, но и ссылку.

Ответ:

- **Узел** — Содержит данные и ссылку на следующий элемент
- **Голова** — Первый элемент списка
- **Хвост** — Последний элемент списка
- **Ссылка** — Указатель на следующий узел

🔮 Шаг 09:

Миссия 9

Разработчик Юля моделирует очередь заданий на сервере. Сопоставь метод массива с его действием.

Подсказка: Подумай, какие методы добавляют, а какие убирают элементы.

Ответ:

- **push()** — Добавляет элемент в конец
- **pop()** — Удаляет элемент из конца
- **shift()** — Удаляет элемент из начала
- **unshift()** — Добавляет элемент в начало

🔮 Шаг 10:

Миссия 10

Игроки обсуждают стек. Сопоставь действие и правильное описание.

Подсказка: В стеке важна вершина — последний элемент.

Ответ:

- **peek** — Получение верхнего элемента без удаления
- **push** — Добавление элемента наверх
- **pop** — Удаление верхнего элемента
- **length** — Количество элементов в стеке

🔮 **Шаг 11:**

Миссия 11

Разработчик Андрей строит связный список для инвентаря игрока. Сопоставь операцию и её описание.

Подсказка: Вспомни, что удаление или добавление в связном списке требует переназначения ссылок.

Ответ:

- **Добавление в начало** — Новая голова указывает на прежнюю
- **Добавление в конец** — Предыдущий хвост указывает на новый узел
- **Удаление головы** — Новой головой становится следующий элемент
- **Удаление хвоста** — Предпоследний элемент указывает на null

🔮 **Шаг 12:**

Миссия 12

Виктор тестирует разные структуры. Сопоставь структуру и её сложность доступа к элементу.

Подсказка: В массиве доступ по индексу быстрый, в списке нужно пройти по цепочке.

Ответ:

- **Массив** — $O(1)$ доступ по индексу
- **Связный список** — $O(n)$ доступ к элементу
- **Стек** — $O(1)$ доступ к вершине
- **Очередь** — $O(1)$ доступ к первому элементу

🔮 **Шаг 13:**

Миссия 13

Упорядочь действия для добавления элемента в очередь.

Подсказка: Вспомни принцип FIFO.

Ответ:

- Создать элемент
- Добавить в конец очереди
- При необходимости обновить хвост
- Проверить длину очереди

🔮 **Шаг 14:**

Миссия 14

Расставь шаги удаления элемента из стека.

Подсказка: Стек работает по принципу LIFO.

Ответ:

- Проверить, пуст ли стек

- Взять последний элемент
- Удалить его
- Вернуть значение

🔮 Шаг 15:

Миссия 15

Игрок Петя реализует связный список. Расставь шаги добавления элемента в конец.

Подсказка: Не забудь про обновление ссылки хвоста.

Ответ:

- Создать новый узел
- Найти хвост списка
- Указать, что `хвост.next` = новый узел
- Обновить хвост

🔮 Шаг 16:

Миссия 16

Расположи шаги обхода связного списка в правильном порядке.

Подсказка: Начиная с головы и двигайся по ссылкам.

Ответ:

- Взять текущий элемент
- Вывести его значение
- Перейти к следующему узлу
- Повторять, пока не достигнут `null`

📖 Урок 5 - Поиск, фильтрация, сортировка

🔮 Шаг 01:

Миссия 1

Алиса готовит список участников хакатона. Ей нужно быстро проверить, есть ли имя в массиве. Какой метод массива в JS для этого подойдёт лучше всего?

Подсказка: Вспомни, какой метод проверяет наличие элемента в массиве и возвращает `true/false`.

Ответ:

- **`includes()`**
- `filter()`
- `map()`
- `sort()`

🔮 Шаг 02:

Миссия 2

Боб ищет первое число больше 10 в массиве очков. Какой метод ему лучше использовать?

Подсказка: Этот метод возвращает первый найденный элемент, удовлетворяющий условию.

Ответ:

- **`find()`**
- `filter()`

- `map()`
- `forEach()`

🔮 Шаг 03:

Миссия 3

Катя хочет отфильтровать только активных пользователей из списка. Какой метод массива ей поможет?

Подсказка: Этот метод создаёт новый массив только с подходящими элементами.

Ответ:

- **`filter()`**
- `find()`
- `map()`
- `reduce()`

🔮 Шаг 04:

Миссия 4

Данил сортирует список книг по алфавиту. Какой метод массива он должен использовать?

Подсказка: Этот метод изменяет исходный массив, упорядочивая элементы.

Ответ:

- **`sort()`**
- `map()`
- `reduce()`
- `filter()`

🔮 Шаг 05:

Миссия 5

Игрок в онлайн-игре проверяет, есть ли в массиве очков хотя бы одно значение больше 100. Какой метод ему стоит выбрать?

Подсказка: Этот метод возвращает `true/false`, если хотя бы один элемент удовлетворяет условию.

Ответ:

- **`some()`**
- `every()`
- `includes()`
- `find()`

🔮 Шаг 06:

Миссия 6

Сергей проверяет, что все студенты сдали экзамен (каждый имеет балл выше 50). Какой метод массива использовать?

Подсказка: Этот метод проверяет условие для всех элементов массива.

Ответ:

- **`every()`**
- `some()`
- `filter()`

- `map()`

🔮 Шаг 07:

Миссия 7

Лена хочет получить индекс элемента массива, равного 42. Какой метод ей использовать?

Подсказка: Этот метод возвращает индекс первого найденного элемента или -1, если его нет.

Ответ:

- **`indexOf()`**
- `find()`
- `map()`
- `sort()`

🔮 Шаг 08:

Миссия 8

Антон ищет последнего победителя в массиве результатов. Какой метод поможет ему найти последний индекс элемента?

Подсказка: Этот метод похож на `indexOf`, но ищет с конца массива.

Ответ:

- **`lastIndexOf()`**
- `indexOf()`
- `find()`
- `some()`

🔮 Шаг 09:

Миссия 10

Сопоставьте методы поиска с их результатами.

Подсказка: Подумай: `indexOf` и `lastIndexOf` возвращают числа, а `includes` — булево.

Ответ:

- **`indexOf()`** — Возвращает индекс первого найденного элемента
- **`lastIndexOf()`** — Возвращает индекс последнего найденного элемента
- **`includes()`** — Возвращает `true/false` при проверке наличия элемента

🔮 Шаг 10:

Миссия 11

Сопоставьте методы с их типичными сценариями.

Подсказка: Сфокусируйся на том, что нужно получить в итоге: массив, элемент или булево значение.

Ответ:

- **`filter()`** — Получить только активных пользователей
- **`find()`** — Найти первую книгу дороже 1000 рублей
- **`some()`** — Проверить, есть ли среди оценок двойки
- **`every()`** — Убедиться, что все студенты сдали экзамен

🔗 Шаг 11:

Миссия 12

Сопоставьте методы сортировки/поиска с их особенностями.

Подсказка: Подумай, что изменяет массив, а что возвращает новый.

Ответ:

- **sort()** — Изменяет исходный массив, сортируя элементы
- **map()** — Создаёт новый массив с преобразованными элементами
- **filter()** — Создаёт новый массив с отобранными элементами

🔗 Шаг 12:

Миссия 13

Сопоставьте методы поиска с типом возвращаемого результата.

Подсказка: Раздели методы: одни возвращают массив, другие — элемент, третьи — число.

Ответ:

- **find()** — Элемент массива
- **filter()** — Новый массив
- **indexOf()** — Число (индекс)
- **includes()** — Булево значение

🔗 Шаг 13:

Миссия 14

Расположите шаги поиска книги по названию в правильном порядке.

1. Сравнить каждое название книги с искомым
2. Пройти по массиву с названиями книг
3. Найти совпадение и вернуть результат
4. **Если ничего не найдено** — вернуть undefined

Подсказка: Подумай, как работает метод find: он идёт по массиву, сравнивает и возвращает первый результат.

Ответ:

- Пройти по массиву с названиями книг
- Сравнить каждое название книги с искомым
- Найти совпадение и вернуть результат
- **Если ничего не найдено** — вернуть undefined

🔗 Шаг 14:

Миссия 15

Расположите шаги фильтрации студентов по возрасту.

1. Сравнить возраст каждого студента с условием
2. Создать новый массив
3. Добавить подходящего студента в новый массив
4. Вернуть массив с результатом

Подсказка: Метод filter формирует новый массив, куда добавляются только подходящие элементы.

Ответ:

- Создать новый массив
- Сравнить возраст каждого студента с условием
- Добавить подходящего студента в новый массив
- Вернуть массив с результатом

🔮 **Шаг 15:**

Миссия 16

Расположите шаги сортировки чисел по возрастанию.

1. Сравнить два числа
2. Определить порядок (больше/меньше)
3. Поменять элементы местами при необходимости
4. Продолжить, пока весь массив не будет отсортирован

Подсказка: Метод `sort` использует функцию сравнения, чтобы переставить элементы.

Ответ:

- Сравнить два числа
- Определить порядок (больше/меньше)
- Поменять элементы местами при необходимости
- Продолжить, пока весь массив не будет отсортирован

📖 **Урок 6 - Практика задачи Leetcode-like на JS**

🔮 **Шаг 01:**

Миссия 1

Ты ищешь первый отрицательный элемент в массиве. Какой метод подойдёт лучше всего?

Подсказка: Нужен метод, который возвращает одно значение, а не массив.

Ответ:

- **`arr.find(x => x < 0)`**
- `arr.filter(x => x < 0)`
- `arr.includes(-1)`
- `arr.some(x => x < 0)`

🔮 **Шаг 02:**

Миссия 2

Разработчик ищет, есть ли в массиве хотя бы один чётный элемент. Какой метод поможет?

Подсказка: Подумай о булевом результате.

Ответ:

- **`arr.some(x => x % 2 === 0)`**
- `arr.every(x => x % 2 === 0)`
- `arr.find(x => x % 2 === 0)`
- `arr.filter(x => x % 2 === 0)`

🔮 **Шаг 03:**

Миссия 3

У тебя список студентов, и нужно проверить, все ли они сдали проект. Какой метод подойдёт?
Подсказка: Он возвращает true, только если условие выполнено для каждого элемента.

Ответ:

- **arr.every(student => student.submitted)**
- arr.some(student => student.submitted)
- arr.filter(student => student.submitted)
- arr.find(student => student.submitted)

Шаг 04:

Миссия 4

Тебе нужно отсортировать массив чисел по возрастанию. Какой синтаксис будет правильным?
Подсказка: По умолчанию sort сортирует как строки.

Ответ:

- **arr.sort((a, b) => a - b)**
- arr.sort()
- arr.sort((a, b) => b - a)
- arr.reverse()

Шаг 05:

Миссия 5

Сопоставь методы поиска с их описанием.

Подсказка: Один метод ищет первое совпадение, другой проверяет наличие элемента, третий фильтрует.

Ответ:

- **find** — Возвращает первый элемент, удовлетворяющий условию
- **includes** — Проверяет, есть ли элемент в массиве
- **filter** — Возвращает массив элементов, удовлетворяющих условию

Шаг 06:

Миссия 6

Сопоставь методы работы с массивами с их результатами.

Подсказка: Подумай, какие методы возвращают логическое значение, а какие массив или элемент.

Ответ:

- **some** — Возвращает true, если хотя бы один элемент соответствует условию
- **every** — Возвращает true, если все элементы соответствуют условию
- **map** — Возвращает новый массив, преобразованный по функции

Шаг 07:

Миссия 7

Сопоставь методы сортировки и их описание.

Подсказка: Один используется по умолчанию, другой для чисел, третий разворачивает массив.

Ответ:

- **arr.sort()** — Сортирует элементы как строки

- **arr.sort((a,b)=>a-b)** — Сортирует числа по возрастанию
- **arr.reverse()** — Меняет порядок элементов на противоположный

🔗 Шаг 08:

Миссия 8

Сопоставь методы массивов с их типичным использованием в задачах.

Подсказка: Один метод часто используют для поиска индекса, другой для проверки условия, а третий для удаления лишних элементов.

Ответ:

- **findIndex** — Возвращает индекс первого подходящего элемента
- **some** — Проверяет, есть ли хотя бы одно совпадение
- **filter** — Удаляет ненужные элементы, оставляя только подходящие

🔗 Шаг 09:

Миссия 9

Сопоставь методы массивов и результат, который они возвращают.

Подсказка: Раздели их на методы, которые возвращают массив, число или булево значение.

Ответ:

- **filter** — Возвращает массив
- **findIndex** — Возвращает число
- **every** — Возвращает true/false

🔗 Шаг 10:

Миссия 10

Сопоставь задачи с подходящими методами.

Подсказка: Для проверки условий есть методы, для поиска элемента — другие.

Ответ:

- **Найти первое слово длиннее 5 букв** — find
- **Проверить, все ли числа положительные** — every
- **Проверить, есть ли в массиве отрицательное число** — some

🔗 Шаг 11:

Миссия 11

Расположи шаги для поиска минимального элемента в массиве вручную.

Подсказка: Сначала инициализируй переменную, потом перебери массив.

Ответ:

- Инициализировать min первым элементом
- Пройтись по массиву в цикле
- Сравнить текущий элемент с min
- Обновить min при необходимости

🔗 Шаг 12:

Миссия 12

Расположи шаги алгоритма пузырьковой сортировки.

Подсказка: Нужно многократно проходить по массиву и менять местами элементы.

Ответ:

- Повторять проход по массиву несколько раз
- Сравнить соседние элементы
- Поменять их местами, если первый больше второго
- Продолжить до конца массива

Шаг 13:

Миссия 13

Расположи шаги алгоритма бинарного поиска.

Подсказка: Деление пополам — ключевая идея.

Ответ:

- Установить границы поиска (левая и правая)
- Вычислить середину
- Сравнить середину с искомым элементом
- Сдвинуть границы влево или вправо в зависимости от результата

Шаг 14:

Миссия 14

Расположи шаги алгоритма быстрой сортировки (quicksort).

Подсказка: Вспомни про опорный элемент и разбиение массива.

Ответ:

- Выбрать опорный элемент
- Разделить массив на два подмассива: меньше и больше опорного
- Рекурсивно отсортировать подмассивы
- Объединить результат

Шаг 15:

Миссия 15

Расположи шаги алгоритма сортировки вставками.

Подсказка: Каждый новый элемент вставляется в уже отсортированную часть массива.

Ответ:

- Начать со второго элемента
- Сравнить с предыдущими
- Сдвигать элементы вправо до подходящей позиции
- Вставить элемент на нужное место

Шаг 16:

Миссия 16

Расположи шаги алгоритма сортировки выбором.

Подсказка: Нужно каждый раз находить минимальный элемент в оставшейся части массива.

Ответ:

- Найти минимальный элемент в неотсортированной части
- Поменять его местами с первым элементом неотсортированной части

- Сдвинуть границу отсортированной части вправо
- Повторить шаги до конца массива

Модуль 6 - ООП в JavaScript

Урок 1 - Прототипное наследование

Шаг 01:

Миссия 1

Аня решила написать игру, где у каждого персонажа есть базовые свойства. Она создала объект `hero` и хочет, чтобы все новые герои наследовали от него. Какой способ в JS позволяет создать объект с прототипом?

Подсказка: Вспомни метод, который принимает объект в качестве аргумента и создаёт новый на его основе.

Ответ:

- **`Object.create(hero)`**
- `new hero()`
- `hero.clone()`
- `Object.assign({}, hero)`

Шаг 02:

Миссия 2

Дима пишет чат для своей игры. Он хочет, чтобы у объекта `Message` был доступ к методу `toString()`. Где этот метод хранится по умолчанию?

Подсказка: Все объекты в JS наследуют базовые методы из одного места.

Ответ:

- **`Object.prototype`**
- `Message.prototype`
- `Array.prototype`
- `Function.prototype`

Шаг 03:

Миссия 3

Катя строит иерархию классов для своей игры. Она создала конструктор `Player`. Как правильно добавить общий метод в прототип?

Подсказка: Чтобы все экземпляры могли использовать метод, нужно добавить его в объект-прототип конструктора.

Ответ:

- **`Player.prototype.attack = function() {...}`**
- `Player.attack = function() {...}`
- `new Player().attack = function() {...}`
- `Object.assign(Player, {attack(){...}})`

Шаг 04:

Миссия 4

Игорь сделал объект `mage`, а затем написал `mage.__proto__ = hero`. Что это означает?

Подсказка: Свойство **proto** указывает на то, у кого объект будет искать свойства и методы.

Ответ:

- **Теперь mage наследует свойства и методы объекта hero**
- mage стал копией объекта hero
- mage заменил hero в памяти
- mage перестал иметь собственные свойства

Шаг 05:

Миссия 5

Оля делает игру с NPC. У нее есть объект npc с методом speak(). Новый объект villager создан через Object.create(npc). Как villager получит метод speak?

Подсказка: Новый объект ищет свойства сначала у себя, а потом у прототипа.

Ответ:

- **Через цепочку прототипов**
- Метод автоматически копируется в villager
- Метод становится встроенным в движок JS
- villager создаёт собственный speak() при вызове

Шаг 06:

Миссия 6

Петя заметил, что метод hasOwnProperty() доступен у любого объекта. Почему?

Подсказка: Этот метод не копируется в каждый объект отдельно, а находится выше по цепочке.

Ответ:

- **Потому что он определён в Object.prototype**
- Потому что JS добавляет его в каждый объект при создании
- Потому что движок дублирует метод во все объекты
- Потому что это ключевое слово языка

Шаг 07:

Миссия 7

Вася создал объект car и задал ему прототип vehicle. Если у обоих есть свойство wheels, какое значение увидит car?

Подсказка: При совпадении свойств всегда выигрывает то, что ближе.

Ответ:

- **Свойство из самого объекта car**
- Свойство из vehicle
- Свойство из Object.prototype
- Ошибка при обращении

Шаг 08:

Миссия 8

Настя хочет проверить, принадлежит ли метод объекту напрямую, а не его прототипу. Как это сделать?

Подсказка: Вспомни встроенный метод, который возвращает true только для «собственных» свойств.

Ответ:

- `obj.hasOwnProperty('methodName')`
- `'methodName' in obj`
- `obj.propertyIsEnumerable('methodName')`
- `typeof obj.methodName !== 'undefined'`

🔗 **Шаг 09:**

Миссия 9

Саша создал цепочку прототипов: `student` → `person` → `Object`. Где завершится поиск свойства, если оно не найдено?

Подсказка: В конце цепочки есть глобальный объект-прототип.

Ответ:

- В `Object.prototype`
- В `null`
- В `window`
- В глобальной переменной `prototype`

🔗 **Шаг 10:**

Миссия 10

Сопоставь термины с их объяснением. Программисты обсуждают наследование в JS и приводят примеры.

Подсказка: Обрати внимание: одни понятия связаны с объектами, другие с конструкторами.

Ответ:

- **prototype** — Объект, от которого наследуют экземпляры конструктора
- ****proto**** — Ссылка на прототип объекта
- **Object.create** — Метод, создающий объект с заданным прототипом
- **constructor** — Функция, создающая объекты

🔗 **Шаг 11:**

Миссия 11

В офисе программисты спорят, что значит «цепочка прототипов». Сопоставь понятия с описанием.

Подсказка: Думай о том, как движок JS ищет свойства.

Ответ:

- **Цепочка прототипов** — Механизм поиска свойств в объектах вверх по иерархии
- **Собственные свойства** — Принадлежат только самому объекту
- **Наследуемые свойства** — Приходят из прототипа
- **Конец цепочки** — `Object.prototype`

🔗 **Шаг 12:**

Миссия 12

Разработчики сравнивают наследование в JS и классическое ООП. Сопоставь понятия.

Подсказка: JS работает на прототипах, а не на строгих классах.

Ответ:

- **Классическое ООП** — Основано на классах и экземплярах
- **Прототипное наследование** — Основано на объектах и ссылках на их прототипы
- **extends** — Ключевое слово для наследования в классах JS
- **Object.setPrototypeOf** — Метод изменения прототипа объекта

 **Шаг 13:**

Миссия 13

Сопоставь методы и их описание. Глеб пишет движок игры и использует разные способы работы с прототипами.

Подсказка: Методы Object работают именно с прототипами.

Ответ:

- **Object.getPrototypeOf** — Возвращает прототип объекта
- **Object.setPrototypeOf** — Устанавливает новый прототип объекта
- **Object.create** — Создает новый объект с заданным прототипом
- **Object.hasOwnProperty** — Проверяет наличие собственного свойства

 **Шаг 14:**

Миссия 14

Артём пишет игру и хочет разобраться, чем отличается прототип конструктора и свойство **__proto__** у объекта. Сопоставь термины.

Подсказка: Один указывает, что получают будущие экземпляры, а другой — откуда наследует конкретный объект.

Ответ:

- **Function.prototype** — Прототип, который получают все новые объекты от конструктора
- **__proto__** — Ссылка конкретного объекта на его прототип
- **Instance** — Объект, созданный через конструктор
- **Inheritance** — Механизм получения свойств и методов через цепочку

 **Шаг 15:**

Миссия 15

Упорядочь шаги: Маша создает прототипное наследование в игре для персонажей.

Подсказка: Сначала нужен конструктор, потом прототип, потом экземпляр.

Ответ:

- Создать функцию-конструктор
- Добавить методы в prototype конструктора
- Создать объект через new
- Вызвать метод у созданного объекта

Урок 2 - Классы **constructor**, **extends**, **super**

 **Шаг 01:**

Миссия 1

Ты помогаешь другу-программисту написать класс Car с конструктором. Что обязательно нужно указать в конструкторе?

Подсказка: Вспомни: конструктор служит для инициализации объекта.

Ответ:

- **Параметры, которые инициализируют свойства объекта**
- Названия всех методов класса
- Набор импортируемых библиотек
- Ключевое слово `extends`

Шаг 02:

Миссия 2

Твоя команда делает игру: у персонажа есть класс `Hero` . Как вызвать конструктор этого класса при создании нового героя?

Подсказка: Вспомни синтаксис создания объекта через класс.

Ответ:

- `new Hero()`
- `Hero.create()`
- `Hero.constructor()`
- `call Hero()`

Шаг 03:

Миссия 3

Разработчик сделал класс `Student` , а потом решил унаследовать класс `HighSchoolStudent` . Как правильно объявить наследование?

Подсказка: Подумай, каким ключевым словом указывают родителя.

Ответ:

- **`class HighSchoolStudent extends Student`**
- `class HighSchoolStudent inherit Student`
- `class HighSchoolStudent implements Student`
- `class HighSchoolStudent with Student`

Шаг 04:

Миссия 4

Ты создаешь класс `Warrior` , который наследуется от `Hero` . Как вызвать конструктор родительского класса внутри `Warrior` ?

Подсказка: Вспомни ключевое слово для вызова родительских методов.

Ответ:

- **`super()`**
- `parent()`
- `Hero.constructor()`
- `base()`

Шаг 05:

Миссия 5

Коллега написал класс `Player` и наследника `VIPPlayer` . Но в конструкторе `VIPPlayer` он забыл вызвать `super()` . Что произойдет?

Подсказка: Вспомни, что без вызова родительского конструктора нельзя обращаться к `this` .

Ответ:

- Будет ошибка при обращении к **this**
- Родительский конструктор вызовется автоматически
- Объект создастся без ошибок
- Класс перестанет наследоваться

🔗 Шаг 06:

Миссия 6

Ты делаешь класс `Animal` и наследника `Cat`. Как правильно передать параметр `name` в родительский конструктор?

Подсказка: Подумай, как прокидывают параметры в `super()`.

Ответ:

- **`super(name)`**
- `this(name)`
- `Animal(name)`
- `parent(name)`

🔗 Шаг 07:

Миссия 7

В проекте есть класс `Book`. Нужно унаследовать `Magazine`, добавив поле `issue`. Как правильно написать конструктор?

Подсказка: Сначала вызови родительский конструктор, потом инициализируй новое поле.

Ответ:

- **`constructor(title, issue){ super(title); this.issue = issue; }`**
- `constructor(issue){ this.issue = issue; super(title); }`
- `constructor(title){ this.title = title; this.issue = issue; }`
- `constructor(){ issue = super.issue; }`

🔗 Шаг 08:

Миссия 8

Сопоставь элементы синтаксиса классов с их назначением:

Подсказка: Вспомни: `constructor` — для инициализации, `extends` — для наследования, `super` — для вызова родителя.

Ответ:

- **`constructor`** — Инициализирует объект и задает свойства
- **`extends`** — Указывает, что класс наследует другой
- **`super`** — Вызывает родительский конструктор
- **`this`** — Ссылается на текущий объект

🔗 Шаг 09:

Миссия 9

Сопоставь класс и правильный способ его наследования:

Подсказка: У каждого класса должен быть потомок с `extends`.

Ответ:

- **class Animal** — class Dog extends Animal
- **class Vehicle** — class Car extends Vehicle
- **class Account** — class PremiumAccount extends Account
- **class User** — class Admin extends User

🔗 **Шаг 10:**

Миссия 10

Сопоставь действие с правильным ключевым словом:

Подсказка: Каждое слово выполняет свою роль в классе.

Ответ:

- **Вызвать конструктор родителя** — super()
- **Создать наследника** — extends
- **Объявить метод инициализации** — constructor
- **Сослаться на свойства объекта** — this

🔗 **Шаг 11:**

Миссия 11

Сопоставь ошибки с причинами:

Подсказка: Вспомни, когда возникает ошибка при использовании this или super.

Ответ:

- **Ошибка: Cannot access 'this' before calling super** — Не вызван super() в конструкторе наследника
- **Ошибка: Unexpected super** — super() используется вне конструктора
- **Ошибка: Class constructor cannot be invoked without 'new'** — Попытка вызвать класс как обычную функцию
- **Ошибка: Duplicate constructor** — В классе объявлено два конструктора

🔗 **Шаг 12:**

Миссия 12

Сопоставь описание с корректным кодом:

Подсказка: Внимательно смотри на синтаксис классов.

Ответ:

- **Класс User с конструктором name** — `class User { constructor(name){ this.name = name; } }`
- **Класс Admin, наследующий User** — `class Admin extends User { }`
- **Конструктор Admin вызывает super** — `constructor(name){ super(name); }`
- **Admin добавляет поле role** — `this.role = 'admin';`

🔗 **Шаг 13:**

Миссия 13

Расставь шаги создания класса с наследованием в правильном порядке:

Подсказка: Сначала объявляется родитель, потом наследник, затем вызов super и добавление новых свойств.

Ответ:

- Объявить класс-родитель
- Создать класс-наследник с extends
- Вызвать super() внутри конструктора наследника
- Добавить новые свойства и методы

 Шаг 14:

Миссия 14

Расставь действия программиста при отладке ошибки с super() в правильном порядке:

Подсказка: Сначала читай код, потом проверь конструктор, затем вызов super, и только потом запускай снова.

Ответ:

- Прочитать сообщение об ошибке
- Проверить, есть ли конструктор в наследнике
- Убедиться, что super() вызван первым в конструкторе
- Перезапустить выполнение кода

 Шаг 15:

Миссия 15

Упорядочь этапы добавления нового поля в класс-наследник:

Подсказка: Сначала вызов super(), потом работа с новым полем.

Ответ:

- Вызвать super() с параметрами
- Инициализировать новое свойство
- Добавить методы для работы с новым свойством
- Проверить работу объекта

 Урок 3 - Инкапсуляция символы, приватные поля

 Шаг 01:

Миссия 1

Анна создала класс BankAccount и хочет спрятать баланс, чтобы никто не менял его напрямую. Какой способ в ES2022 позволяет объявить приватное поле?

Подсказка: Вспомни про знак, который ставится перед именем свойства, чтобы скрыть его.

Ответ:

- **Использовать синтаксис #balance**
- Использовать _balance
- Создать переменную balance вне класса
- Добавить геттер getBalance()

 Шаг 02:

Миссия 2

Программист Дима пишет класс Player . Он хочет хранить секретный токен доступа. Что ему лучше использовать для инкапсуляции?

Подсказка: Иногда для приватности можно применять уникальные идентификаторы, которые недоступны извне.

Ответ:

- **Символ (Symbol)**
- Обычная строка
- Константа вне класса
- Глобальная переменная

Шаг 03:

Миссия 3

Игрок Вася добавил приватное поле `#hp` в класс `Monster`. Как правильно получить его значение внутри класса?

Подсказка: Приватные поля доступны только методам того же класса.

Ответ:

- **`this.#hp`**
- `this.hp`
- `Monster.#hp`
- `get hp()`

Шаг 04:

Миссия 4

В коде Пети есть символ `const key = Symbol('secret')`. Зачем использовать символ как ключ для свойства объекта?

Подсказка: Символы создают уникальные значения, даже если описание совпадает.

Ответ:

- **Чтобы свойство было скрытым и не пересекалось с другими**
- Чтобы быстрее выполнялся код
- Чтобы сделать свойство глобальным
- Чтобы добавить комментарий к коду

Шаг 05:

Миссия 5

Сергей хочет скрыть пароль в объекте `User`. Какой способ наиболее корректный?

Подсказка: Подумай о современных возможностях языка для приватных данных.

Ответ:

- **Использовать приватное поле `#password`**
- Сохранять пароль в обычном свойстве
- Создать глобальную переменную `password`
- Писать пароль в комментарии

Шаг 06:

Миссия 6

Катя заметила, что приватные поля с `#` нельзя просмотреть через `Object.keys()`. Почему?

Подсказка: Эти поля специально сделаны так, чтобы их нельзя было перечислять.

Ответ:

- Потому что они **исключены из стандартного перечисления свойств**
- Потому что это баг JavaScript
- Потому что они автоматически становятся глобальными
- Потому что они конвертируются в null

🔗 Шаг 07:

Миссия 7

Разработчик Никита хочет хранить внутреннее состояние только для методов класса. Что лучше выбрать?

Подсказка: Сравни Symbol и приватные поля: что проще и современнее?

Ответ:

- **Приватные поля (#field)**
- Symbol без ограничений
- Свойства с нижним подчёркиванием
- Комментарии в коде

🔗 Шаг 08:

Миссия 8

Сопоставь технику инкапсуляции с её описанием.

Подсказка: Символы и приватные поля решают схожие задачи, но делают это по-разному.

Ответ:

- **Приватные поля (#)** — Ограничены внутри класса, недоступны извне.
- **Символы** — Создают уникальные ключи, свойства скрыты от обычного перебора.
- **Подчёркивание в имени** — Просто соглашение, не даёт реальной защиты.
- **Object.keys()** — Не возвращает приватные поля с #.

🔗 Шаг 09:

Миссия 9

Ира изучает разницу между приватными полями и символами. Помогите сопоставить.

Подсказка: Символы более гибкие, а приватные поля — встроенный синтаксис.

Ответ:

- **Приватное поле** — Объявляется через знак #.
- **Symbol** — Создаётся через функцию Symbol().
- **Недоступность снаружи** — Гарантируется приватным полем.
- **Уникальность ключа** — Гарантируется символом.

🔗 Шаг 10:

Миссия 10

Вася создал класс с приватным полем #coins . Соотнеси действие и результат.

Подсказка: Проверь, где код упадёт, а где сработает.

Ответ:

- **Вызов this.#coins внутри метода** — Работает корректно.
- **Вызов obj.#coins снаружи** — Ошибка.

- **Object.keys(obj)** — Не покажет #coins.
- **Геттер внутри класса** — Может вернуть значение приватного поля.

🔮 Шаг 11:

Миссия 11

Сопоставь поле и его свойство в плане инкапсуляции.

Подсказка: Сравни синтаксис и поведение.

Ответ:

- **#level** — Приватное поле, доступное только внутри класса.
- **_level** — Публичное свойство, скрывание условное.
- **Symbol('level')** — Уникальный ключ, не виден в Object.keys().
- **level** — Обычное публичное свойство.

🔮 Шаг 12:

Миссия 12

Расположи шаги добавления приватного поля в класс.

Подсказка: Сначала объявление, потом использование, затем доступ через метод.

Ответ:

- Объявить приватное поле через #
- Использовать его внутри конструктора или метода
- Создать публичный метод-геттер
- Вызвать геттер снаружи класса

🔮 Шаг 13:

Миссия 13

Упорядочи шаги работы с Symbol в объекте.

Подсказка: Сначала создаём символ, затем используем его как ключ, потом читаем значение.

Ответ:

- Создать символ через Symbol()
- Использовать символ как ключ при создании свойства
- Записать значение в это свойство
- Прочитать значение по символу

🔮 Шаг 14:

Миссия 14

Расставь шаги для инкапсуляции пароля в классе User .

Подсказка: Действия должны идти от объявления до использования.

Ответ:

- Объявить #password в классе
- Инициализировать его в конструкторе
- Создать метод для проверки пароля
- Вызвать метод снаружи

Урок 4 - Композиция против наследования

Шаг 01:

Миссия 1

Программист Алиса пишет игру, где у персонажей есть здоровье и броня. Ей нужно решить: наследовать класс Player от Armor, или использовать композицию. Что правильнее?

Подсказка: Подумайте: броня — это часть игрока, но игрок сам по себе не является бронёй.

Ответ:

- **Использовать композицию: сделать броню свойством игрока**
- Сделать класс Player наследником Armor
- Сделать броню родителем игрока
- Игнорировать броню и хранить её отдельно от игрока

Шаг 02:

Миссия 2

Боб создаёт систему уведомлений. Уведомление может отправляться по SMS, Email или Push. Как лучше построить архитектуру?

Подсказка: Подумайте: новые способы отправки сообщений могут добавляться в будущем.

Ответ:

- **Использовать композицию: Notification содержит объект Channel**
- Сделать класс SMS родителем Notification
- Notification наследует Email и Push
- Хранить все методы отправки прямо внутри Notification

Шаг 03:

Миссия 3

Катя пишет приложение для доставки еды. Ей нужен класс Order, в котором есть адрес доставки. Стоит ли Order наследовать от Address?

Подсказка: Вопрос в том, что представляет собой заказ: он включает адрес, но не является им.

Ответ:

- **Нет, использовать композицию: у Order есть Address**
- Да, Order наследует Address
- Order и Address должны наследовать общий класс BaseEntity
- Нужно полностью игнорировать Address

Шаг 04:

Миссия 4

Программист Денис моделирует транспорт: у него есть классы Car, Bicycle и Engine. Как лучше связать их?

Подсказка: Не у всех транспортных средств есть двигатель.

Ответ:

- **Car содержит Engine через композицию**
- Car наследует Engine
- Engine наследует Bicycle
- **Engine** — родитель всего транспорта

🔗 Шаг 05:

Миссия 5

Олег строит чат-приложение. Сообщение может содержать текст, картинку и видео. Как правильно организовать?

Подсказка: Разные вложения — это части сообщения, но сообщение не должно быть их наследником.

Ответ:

- **Message содержит вложения через композицию**
- Message наследует Image и Video
- Image наследует Message
- Каждый тип вложения хранится в отдельном приложении

🔗 Шаг 06:

Миссия 6

Настя разрабатывает магазин игр. Каждая игра может иметь скидку. Что лучше?

Подсказка: Игра сама по себе не является скидкой.

Ответ:

- **Game содержит Discount через композицию**
- Game наследует Discount
- Discount наследует Game
- Игра и скидка должны быть одним классом

🔗 Шаг 07:

Миссия 7

Дима пишет систему для фитнеса. У него есть классы Workout и Timer . Как лучше связать?

Подсказка: Тренировка не является таймером, но может использовать его.

Ответ:

- **Workout содержит Timer через композицию**
- Workout наследует Timer
- Timer наследует Workout
- Оба класса наследуют общий родитель TimeBase

🔗 Шаг 08:

Миссия 8

Игорь проектирует библиотеку для рисования. Есть классы Canvas и Brush . Как правильно их связать?

Подсказка: Холст — это не кисть, но они работают вместе.

Ответ:

- **Canvas использует Brush через композицию**
- Canvas наследует Brush
- Brush наследует Canvas
- Они должны быть объединены в один класс

🔮 Шаг 09:

Миссия 9

Расположите шаги проектирования классов с композицией:

Подсказка: Сначала определяем сущности, потом связи, а затем реализуем код.

Ответ:

- Определить ключевые сущности
- Решить, какие свойства лучше вынести в отдельные классы
- Связать сущности через композицию
- Реализовать методы взаимодействия

🔮 Шаг 10:

Миссия 10

Расположите шаги проектирования с наследованием:

Подсказка: Важно идти от общего к частному.

Ответ:

- Определить общий базовый класс
- Определить наследников
- Добавить уникальные свойства и методы
- Проверить корректность иерархии

🔮 Шаг 11:

Миссия 11

Расположите шаги выбора между композицией и наследованием:

Подсказка: Сначала нужно понять отношения между объектами.

Ответ:

- Определить, является ли объект разновидностью другого
- **Если нет, проверить** — может ли он быть частью другого
- Принять решение: наследование или композиция
- Реализовать соответствующую архитектуру

🔮 Шаг 12:

Миссия 12

Алиса создает игру и хочет использовать функциональность персонажей без создания длинной цепочки наследования. Что лучше применить?

Подсказка: Подумайте о гибкости и повторном использовании кода без сложной иерархии.

Ответ:

- **Композиция**
- Наследование от базового класса
- Глобальные функции
- Миксины без структуры

🔮 Шаг 13:

Миссия 13

Боб создает разные типы персонажей: воинов и магов. Он хочет, чтобы они могли использовать общие способности без создания общего класса. Что использовать?

Подсказка: Подумайте о возможности объединять функции по желанию.

Ответ:

- **Композиция модулей с отдельными функциями**
- Создание большого родительского класса
- Использование наследования цепочкой
- Глобальные объекты для всех способностей

Шаг 14:

Миссия 14

Катя хочет объединить поведение нескольких типов объектов без создания сложной иерархии. Что правильнее использовать?

Подсказка: Подумайте о "сборке" поведения из маленьких частей.

Ответ:

- **Композицию через функции и объекты**
- Наследование по глубокой иерархии
- Использование глобальных переменных для хранения состояний
- Миксины без структуры

Шаг 15:

Миссия 15

Денис создает игрового персонажа с оружием и магией. Он хочет, чтобы возможности можно было комбинировать по необходимости. Что выбрать?

Подсказка: Подумайте о независимых модулях, которые можно подключать к объекту.

Ответ:

- **Композицию из отдельных функций/модулей**
- Наследование от одного большого базового класса
- Прямое копирование методов между объектами
- Глобальные методы для всех персонажей

Шаг 16:

Миссия 16

Олег хочет, чтобы его объект "автомобиль" имел функции движения и сигнализации, не создавая два наследника. Что использовать?

Подсказка: Подумайте о добавлении поведения без наследования.

Ответ:

- **Композицию** — подключение функций к объекту
- Создание подкласса для каждой функции
- Прямое изменение прототипа базового класса
- Глобальные функции без объектов

Урок 5 - SOLID-принципы в JS

Шаг 01:

Миссия 1

Алиса создает систему оплаты, где каждый метод оплаты должен быть независимым. Какой принцип SOLID ей стоит применять в первую очередь?

Подсказка: Подумайте о том, чтобы изменения в одном классе не ломали другие.

Ответ:

- **Принцип единственной ответственности (SRP)**
- Принцип инверсии зависимостей (DIP)
- Принцип открытости/закрытости (OCP)
- Принцип подстановки Лисков (LSP)

Шаг 02:

Миссия 2

Боб разрабатывает игровую платформу. Каждый тип пользователя имеет разные права. Как лучше структурировать классы пользователей?

Подсказка: Подумайте, чтобы новые типы пользователей легко добавлялись без изменения существующего кода.

Ответ:

- **Использовать открытость/закрытость (OCP)**
- Все права хранить в одном классе User
- Делать наследование от общего класса без интерфейсов
- Создавать один глобальный объект для всех пользователей

Шаг 03:

Миссия 3

Катя строит систему уведомлений. Уведомления могут быть Email и SMS. Как применить принцип Liskov для этих классов?

Подсказка: Подумайте, чтобы замена одного типа уведомления на другой не ломала программу.

Ответ:

- **Email и SMS должны быть подтипами общего Notification**
- Notification наследует Email и SMS
- Email и SMS должны быть независимыми и не иметь общего интерфейса
- Создать отдельный метод для каждого типа уведомления без общего класса

Шаг 04:

Миссия 4

Денис пишет систему заказа еды. Каждый заказ может быть оплачен картой или наличными. Какой принцип SOLID поможет легко добавлять новые способы оплаты?

Подсказка: Новый тип оплаты не должен ломать существующую логику.

Ответ:

- **Принцип открытости/закрытости (OCP)**
- Принцип единственной ответственности (SRP)
- Принцип разделения интерфейса (ISP)

- Принцип инверсии зависимостей (DIP)

🔗 Шаг 05:

Миссия 5

Олег создает библиотеку графики. Методы для рисования прямых и кругов должны быть независимыми. Какой принцип поможет избежать большого интерфейса?

Подсказка: Подумайте, чтобы классы реализовывали только нужные методы.

Ответ:

- **Принцип разделения интерфейса (ISP)**
- Принцип открытости/закрытости (OCP)
- Принцип подстановки Лисков (LSP)
- Принцип единственной ответственности (SRP)

🔗 Шаг 06:

Миссия 6

Настя разрабатывает библиотеку для работы с базой данных. Каждый модуль отвечает за отдельную таблицу. Какой принцип следует применять?

Подсказка: Подумайте: один модуль — одна ответственность.

Ответ:

- **Принцип единственной ответственности (SRP)**
- Принцип открытости/закрытости (OCP)
- Принцип инверсии зависимостей (DIP)
- Принцип разделения интерфейса (ISP)

🔗 Шаг 07:

Миссия 7

Дима проектирует платежный сервис. Классы должны зависеть от абстракций, а не конкретных реализаций. Какой принцип SOLID это описывает?

Подсказка: Подумайте, чтобы модуль можно было легко заменить другой реализацией.

Ответ:

- **Принцип инверсии зависимостей (DIP)**
- Принцип открытости/закрытости (OCP)
- Принцип подстановки Лисков (LSP)
- Принцип единственной ответственности (SRP)

🔗 Шаг 08:

Миссия 8

Игорь пишет игру, где персонажи могут летать и бегать. Применяя принцип Liskov, что нужно сделать?

Подсказка: Замена одного типа персонажа на другой не должна ломать логику игры.

Ответ:

- **Все подтипы персонажей должны корректно работать через общий интерфейс Character**
- Создать отдельные несвязанные классы для каждого типа персонажа
- Наследовать только FlyingCharacter от Character, остальные нет

- Игнорировать LSP и проверять каждый тип вручную

Шаг 09:

Миссия 9

Алиса разрабатывает систему управления персонажами и хочет, чтобы каждый класс имел только одну ответственность. Какой принцип SOLID здесь применим?

Подсказка: Подумайте о том, чтобы класс делал что-то одно и выполнял это хорошо.

Ответ:

- **Принцип единственной ответственности (SRP)**
- Принцип открытости/закрытости (OCP)
- Принцип подстановки Барбары Лисков (LSP)
- Принцип разделения интерфейса (ISP)

Шаг 10:

Миссия 10

Боб создает платежную систему, и новые методы оплаты будут добавляться часто. Как сделать так, чтобы существующий код не ломался?

Подсказка: Подумайте о расширяемости без изменения существующих классов.

Ответ:

- **Принцип открытости/закрытости (OCP)**
- Принцип единственной ответственности (SRP)
- Принцип подстановки Барбары Лисков (LSP)
- Принцип инверсии зависимостей (DIP)

Шаг 11:

Миссия 11

Катя создает классы животных, и хочет, чтобы любой подкласс мог использовать методы базового класса без ошибок. Какой принцип использовать?

Подсказка: Подумайте о том, чтобы подклассы корректно заменяли базовый класс.

Ответ:

- **Принцип подстановки Лисков (LSP)**
- Принцип единственной ответственности (SRP)
- Принцип открытости/закрытости (OCP)
- Принцип разделения интерфейса (ISP)

Шаг 12:

Миссия 12

Денис создает интерфейс для разных типов отчетов, и хочет, чтобы классы не были вынуждены реализовывать методы, которые им не нужны. Какой принцип применить?

Подсказка: Подумайте о разделении интерфейсов на маленькие логические части.

Ответ:

- **Принцип разделения интерфейса (ISP)**
- Принцип единственной ответственности (SRP)
- Принцип открытости/закрытости (OCP)
- Принцип инверсии зависимостей (DIP)

Шаг 13:

Миссия 13

Олег создает сервисы, которые зависят от конкретных классов. Он хочет уменьшить связанность между ними. Какой принцип поможет?

Подсказка: Подумайте о том, чтобы зависимости были абстрактными, а не конкретными.

Ответ:

- **Принцип инверсии зависимостей (DIP)**
- Принцип единственной ответственности (SRP)
- Принцип открытости/закрытости (OCP)
- Принцип подстановки Лисков (LSP)

Шаг 14:

Миссия 14

Сопоставьте принципы SOLID с описаниями:

Подсказка: Подумайте, какой принцип отвечает за каждую задачу.

Ответ:

- **SRP** — Класс должен иметь только одну причину для изменения
- **OCP** — Код должен быть открыт для расширения, но закрыт для изменения
- **LSP** — Подклассы должны заменять базовый класс без ошибок
- **ISP** — Классы не должны зависеть от методов, которые они не используют
- **DIP** — Зависимости должны быть от абстракций, а не от конкретных классов

Шаг 15:

Миссия 15

Сопоставьте ситуации с применимым принципом SOLID:

Подсказка: Ситуации помогут понять, какой принцип использовать.

Ответ:

- **Добавление новых методов оплаты без изменения старого кода** — OCP
- **Класс Отчет пытается реализовать все методы интерфейса сразу** — ISP
- **Класс Животное с подклассами Птица и Рыба** — LSP
- **Сервис зависит от конкретного класса логирования** — DIP
- **Класс Пользователь отвечает и за хранение, и за отправку уведомлений** — SRP

Модуль 7 - Функциональное программирование

Урок 1 - Чистые функции и иммутабельность

Шаг 01:

Миссия 1

Алиса создает систему подсчета очков для игры. Она хочет, чтобы функция вычисляла результат, не изменяя исходные данные игроков. Какое свойство функции ей нужно использовать?

Подсказка: Подумайте о том, что функция не должна менять внешний мир.

Ответ:

- **Чистая функция**

- Асинхронная функция
- Функция с побочными эффектами
- Стрелочная функция

🔗 Шаг 02:

Миссия 2

Боб работает с массивом игроков и хочет создать новый массив с их результатами, не меняя исходный массив. Что он должен использовать?

Подсказка: Подумайте, как создать новый объект вместо изменения старого.

Ответ:

- **Методы map, filter, reduce с созданием нового массива**
- forEach с изменением исходного массива
- Использовать глобальные переменные для хранения результата
- Изменять массив напрямую через push или pop

🔗 Шаг 03:

Миссия 3

Катя пишет функцию для вычисления бонусов. Она должна возвращать одинаковый результат при одинаковых входных данных и не иметь побочных эффектов. Какое это свойство?

Подсказка: Подумайте о предсказуемости функции.

Ответ:

- **Детерминированность чистой функции**
- Асинхронность
- Глобальная изменяемость
- Функция с побочными эффектами

🔗 Шаг 04:

Миссия 4

Денис работает с объектами персонажей и хочет добавить новый уровень без изменения исходного объекта. Какой принцип он применяет?

Подсказка: Подумайте о создании нового объекта вместо изменения старого.

Ответ:

- **Иммутабельность данных**
- Прямое изменение исходного объекта
- Использование глобальных переменных
- Побочные эффекты внутри функции

🔗 Шаг 05:

Миссия 5

Олег создает функцию подсчета опыта игрока. Он хочет, чтобы эта функция не зависела от внешних переменных. Как называется такое свойство?

Подсказка: Подумайте, что функция должна использовать только свои аргументы.

Ответ:

- **Изоляция и чистота функции**
- Использование глобальных переменных

- Асинхронная функция
- Функция с побочными эффектами

🔮 Шаг 06:

Миссия 6

Настя пишет функцию для преобразования списка предметов в игре. Она хочет, чтобы исходный список остался неизменным. Какое правило следует соблюдать?

Подсказка: Думайте о сохранении исходных данных.

Ответ:

- **Иммутабельность**
- Модификация исходного массива
- Использование глобальных переменных
- Побочные эффекты внутри функции

🔮 Шаг 07:

Миссия 7

Игорь создает функцию для расчета урона в игре. Он хочет, чтобы каждый вызов функции с одинаковыми параметрами давал один и тот же результат. Какое свойство это описывает?

Подсказка: Подумайте о предсказуемости функции.

Ответ:

- **Детерминированность**
- Асинхронность
- Побочный эффект
- Глобальная зависимость

🔮 Шаг 08:

Миссия 8

Даша пишет функцию суммирования очков. Она хочет, чтобы она не изменяла исходный объект игрока, а возвращала новый. Какой подход использовать?

Подсказка: Думайте о создании новых объектов вместо изменения существующих.

Ответ:

- **Создавать новые объекты (иммутабельность)**
- Изменять исходный объект
- Использовать глобальные переменные
- Вызывать побочные эффекты внутри функции

🔮 Шаг 09:

Миссия 9

Алиса написала функцию, которая возвращает сумму двух чисел без изменения внешних данных. Что это за функция?

Подсказка: Подумайте, делает ли функция что-то кроме возвращения значения?

Ответ:

- **Чистая функция**
- Императивная функция
- Функция с побочным эффектом

- Асинхронная функция

🔮 Шаг 10:

Миссия 10

Боб обновляет объект состояния игры напрямую в функции. Почему это плохая практика в функциональном программировании?

Подсказка: Подумайте, что может произойти, если другие части программы используют этот объект.

Ответ:

- **Функция нарушает иммутабельность**
- Функция является чистой
- Функция оптимизирована для производительности
- Функция синхронная

🔮 Шаг 11:

Миссия 11

Катя создает функцию, которая принимает массив и возвращает новый массив с отсортированными элементами, не меняя исходный. Какая концепция реализована?

Подсказка: Подумайте, сохраняет ли функция исходные данные без изменений.

Ответ:

- **Иммутабельность**
- Побочный эффект
- Мутация массива
- Асинхронная обработка

🔮 Шаг 12:

Миссия 12

Денис вызывает функцию несколько раз с одинаковыми аргументами и всегда получает один и тот же результат. Как называется такое поведение?

Подсказка: Подумайте о предсказуемости функции.

Ответ:

- **Детерминированность**
- Побочный эффект
- Императивность
- Асинхронность

🔮 Шаг 13:

Миссия 13

Олег использует объект как глобальное состояние и меняет его во многих функциях. Что нарушается?

Подсказка: Подумайте о том, что глобальные изменения усложняют предсказуемость.

Ответ:

- **Чистота функции и иммутабельность**
- Оптимизация производительности
- Асинхронная обработка

- Событийная модель

🔗 Шаг 14:

Миссия 14

Сопоставьте понятия с примерами:

Подсказка: Подумайте, что делает функцию чистой и какие данные остаются неизменными.

Ответ:

- **Чистая функция** — Возвращает значение без изменения внешних переменных
- **Функция с побочным эффектом** — Изменяет глобальное состояние или аргументы
- **Иммутабельность** — Исходные объекты или массивы не изменяются
- **Детерминированность** — Всегда возвращает одинаковый результат при одинаковых входных данных

🔗 Шаг 15:

Миссия 15

Сопоставьте принципы иммутабельности с преимуществами:

Подсказка: Подумайте, как неизменность данных помогает избежать ошибок.

Ответ:

- **Иммутабельность** — Предотвращает случайные изменения данных
- **Чистая функция** — Упрощает тестирование и предсказуемость
- **Детерминированность** — Функция всегда возвращает одинаковый результат
- **Функция с побочным эффектом** — Может изменять внешние объекты и глобальное состояние

🔗 Шаг 16:

Миссия 16

Расположите шаги создания чистой функции:

1. Определить входные данные
2. Вычислить результат без изменения внешних данных
3. Вернуть новый результат
4. Проверить, что исходные данные не изменились

Подсказка: Сначала обрабатываются входные данные, потом вычисления и проверка неизменности.

Ответ:

- Определить входные данные
- Вычислить результат без изменения внешних данных
- Вернуть новый результат
- Проверить, что исходные данные не изменились

📖 Урок 2 - Каррирование, композиция

🔗 Шаг 01:

Миссия 1

Вы работаете над приложением для заказа еды, и нужно создать функцию, которая добавляет налог к цене. Какой подход проще реализовать с помощью каррирования?

Подсказка: Подумайте, как можно заранее зафиксировать часть аргументов функции.

Ответ:

- Создать функцию **addTax(taxRate)(price)**
- Создать глобальную переменную `taxRate`
- Создать обычную функцию `addTax(price, taxRate)`

🔮 Шаг 02:

Миссия 2

Вы делаете игру, где персонаж собирает очки. Нужно создать функцию увеличения очков. Как каррирование поможет фиксировать множитель очков?

Подсказка: Множитель можно зафиксировать заранее, а потом применять к разным значениям.

Ответ:

- Создать функцию **increasePoints(multiplier)(points)**
- Создать функцию `increasePoints(points, multiplier)`
- Создать массив `multipliers` и умножать вручную

🔮 Шаг 03:

Миссия 3

Вы разрабатываете чат-бота. Нужно применять несколько фильтров к сообщениям: убрать пробелы, привести к нижнему регистру и убрать спецсимволы. Как лучше это сделать с функциями высшего порядка?

Подсказка: Композиция позволяет соединять функции в цепочку.

Ответ:

- Составить **compose(trim, toLowerCase, removeSymbols)**
- Вызывать каждую функцию отдельно для каждого сообщения
- Использовать только один фильтр

🔮 Шаг 04:

Миссия 4

Вы создаёте магазин одежды. Есть функция `applyDiscount`, которая применяет скидку к цене. Как можно с помощью каррирования создавать разные функции для разных категорий товаров?

Подсказка: Зафиксируйте скидку для каждой категории один раз.

Ответ:

- Создать **applyDiscount(categoryDiscount)(price)**
- Создавать отдельные функции для каждой категории без каррирования
- Применять скидку вручную каждый раз

🔮 Шаг 05:

Миссия 5

В приложении для планирования задач нужно комбинировать несколько функций: фильтрацию по дате, сортировку по приоритету и отображение только активных задач. Как лучше это сделать?

Подсказка: Подумайте о `compose`, чтобы объединить функции в одну цепочку.

Ответ:

- Составить **compose(filterByDate, sortByPriority, filterActive)**

- Вызывать функции по отдельности в любом порядке
- Объединять функции в один огромный цикл

🔗 Шаг 06:

Миссия 6

Формулировка: Сопоставьте термин с его определением в функциональном программировании.

Понятия:

Ответ:

- Каррирование
- Композиция функций
- Чистая функция
- Функция высшего порядка

Определения:

1. Функция, которая принимает одну часть аргументов и возвращает новую функцию для оставшихся.
2. Функция, которая вызывает другие функции и/или возвращает функцию.
3. **Соединение нескольких функций в цепочку, где результат одной** — аргумент другой.
4. Функция без побочных эффектов и предсказуемым результатом.

Подсказка: Подумайте, как эти концепции помогают писать чистый и гибкий код.

- **Каррирование** — Функция, которая принимает одну часть аргументов и возвращает новую функцию для оставшихся.
- **Композиция функций** — Соединение нескольких функций в цепочку, где результат одной — аргумент другой.
- **Чистая функция** — Функция без побочных эффектов и предсказуемым результатом.
- **Функция высшего порядка** — Функция, которая вызывает другие функции и/или возвращает функцию.

🔗 Шаг 07:

Миссия 7

Формулировка: Сопоставьте сценарий с подходящей функцией в FP.

Сценарии:

Ответ:

- Создать функцию добавления налога с фиксированной ставкой
- Объединить фильтры для обработки текста чата
- Функция, которая возвращает новую функцию с фиксированным множителем очков
- Функция без побочных эффектов

Функции:

1. Чистая функция
2. Каррированная функция
3. Композиция функций
4. Функция высшего порядка

Подсказка: Обратите внимание на разницу между каррированием и композицией.

- **Создать функцию добавления налога с фиксированной ставкой** — Каррированная функция
- **Объединить фильтры для обработки текста чата** — Композиция функций
- **Функция, которая возвращает новую функцию с фиксированным множителем очков** — Функция высшего порядка
- **Функция без побочных эффектов** — Чистая функция

🔮 Шаг 08:

Миссия 8

Формулировка: Сопоставьте примеры с типами функций.

Примеры:

Ответ:

- `price => price * 0.9`
- `filterActive(tasks) -> sortByPriority`
- `addTax(10)(price)`
- `function that returns function`

Типы функций:

1. Чистая функция
2. Композиция функций
3. Каррированная функция
4. Функция высшего порядка

Подсказка: Обратите внимание на использование аргументов и возвращаемые функции.

- **`price => price * 0.9`** — Чистая функция
- **`filterActive(tasks) -> sortByPriority`** — Композиция функций
- **`addTax(10)(price)`** — Каррированная функция
- **`function that returns function`** — Функция высшего порядка

🔮 Шаг 09:

Миссия 9

Формулировка: Сопоставьте преимущества функционального подхода с описанием.

Преимущества:

Ответ:

- Повторное использование кода
- Простота тестирования
- Отсутствие побочных эффектов
- Удобная композиция

Описание:

1. Функции можно комбинировать для создания новых
2. Функции легко проверять, так как они предсказуемы
3. Один и тот же код можно использовать в разных местах
4. Функции не изменяют внешнее состояние

Подсказка: Подумайте, как FP помогает поддерживать чистоту и гибкость кода.

- **Повторное использование кода** — Один и тот же код можно использовать в разных местах
- **Простота тестирования** — Функции легко проверять, так как они предсказуемы
- **Отсутствие побочных эффектов** — Функции не изменяют внешнее состояние
- **Удобная композиция** — Функции можно комбинировать для создания новых

🔗 Шаг 10:

Миссия 10

Формулировка: Сопоставьте термины функционального программирования с их описанием.

Понятия:

Ответ:

- Каррирование
- Композиция функций
- Чистая функция
- Иммутабельность

Определения:

1. Функция, которая возвращает новую функцию с частично применёнными аргументами
2. Процесс объединения нескольких функций в одну
3. Функция, результат которой зависит только от входных данных и не имеет побочных эффектов
4. Невозможность изменения исходных данных

🔗 Подсказка:

Обратите внимание на термины: один описывает способ вызова функций, другой — их комбинацию, третий — свойства функции, четвертый — свойства данных.

- **Каррирование** — Функция, которая возвращает новую функцию с частично применёнными аргументами
- **Композиция функций** — Процесс объединения нескольких функций в одну
- **Чистая функция** — Функция, результат которой зависит только от входных данных и не имеет побочных эффектов
- **Иммутабельность** — Невозможность изменения исходных данных

🔗 Шаг 11:

Миссия 11

Формулировка: Сопоставьте примеры применения каррирования и композиции функций.

Понятия:

Ответ:

- Каррирование
- Композиция функций

Определения:

1. Создание специализированной функции для умножения на 2
2. Соединение функций фильтрации и сортировки массива

🔗 Подсказка:

Каррирование часто применяется для создания новых функций с фиксированными параметрами, композиция — для объединения разных шагов обработки.

- **Каррирование** — Создание специализированной функции для умножения на 2
- **Композиция функций** — Соединение функций фильтрации и сортировки массива

🔮 Шаг 12:

Миссия 12

Формулировка: Сопоставьте типы функций с их особенностями.

Понятия:

Ответ:

- Чистая функция
- Функция с побочными эффектами

Определения:

1. Всегда возвращает одинаковый результат для одинаковых входных данных
2. Изменяет состояние программы или внешние данные

🔮 Подсказка:

Подумайте, что значит «чистая»: она не меняет ничего вокруг себя.

- **Чистая функция** — Всегда возвращает одинаковый результат для одинаковых входных данных
- **Функция с побочными эффектами** — Изменяет состояние программы или внешние данные

🔮 Шаг 13:

Миссия 13

Формулировка: Сопоставьте термины композиции функций с примерами в жизни программиста.

Понятия:

Ответ:

- Композиция функций
- Каррирование

Определения:

1. Объединение нескольких функций для обработки HTTP-запроса
2. Создание функции для добавления налога с фиксированной ставкой

🔮 Подсказка:

Композиция объединяет несколько шагов, каррирование фиксирует часть аргументов для будущей функции.

- **Композиция функций** — Объединение нескольких функций для обработки HTTP-запроса
- **Каррирование** — Создание функции для добавления налога с фиксированной ставкой

🔮 Шаг 14:

Миссия 14

Формулировка: Сопоставьте ситуации с типом функции.

Понятия:

Ответ:

- Иммутабельные данные
- Мутируемые данные

Определения:

1. Создание нового объекта при изменении данных
2. Прямое изменение свойств объекта

 Подсказка:

Иммутабельность означает «неизменность», а мутируемые данные меняются прямо.

- **Иммутабельные данные** — Создание нового объекта при изменении данных
- **Мутируемые данные** — Прямое изменение свойств объекта

 Шаг 15:

Миссия 15

Разработчик Алекс создаёт маленькую утилиту для магазина. Ему нужно создать функцию, которая добавляет налог к цене. Какой подход проще всего использовать с каррированием?

Подсказка: Попробуйте представить функцию как серию шагов, каждый из которых принимает только один аргумент.

Ответ:

- **Создать функцию, которая возвращает другую функцию, принимающую цену**
- Создать одну функцию с двумя параметрами
- Использовать цикл для расчета налога
- Использовать объект вместо функции

 Шаг 16:

Миссия 16

Боб хочет фильтровать список пользователей по возрасту, используя каррирование. Что ему поможет?

Подсказка: Подумайте, как создать универсальную функцию, которую можно будет легко применять к разным возрастам.

Ответ:

- **Функция, принимающая возраст и возвращающая функцию для фильтрации**
- Функция, сразу фильтрующая весь массив
- Функция без параметров
- Прямое использование цикла for без функции

 Урок 3 - Паттерны map, filter, reduce на практике

 Шаг 01:

Миссия 1

Разработчик Алекс собирает список цен товаров и хочет получить их квадратные значения. Какой метод лучше всего использовать?

Подсказка: Подумайте, как применить операцию ко всем элементам массива одновременно.

Ответ:

- **Использовать map для создания нового массива**

- Использовать `filter` для создания нового массива
- Использовать `reduce` для создания нового массива
- Использовать `forEach` для изменения массива на месте

🔗 Шаг 02:

Миссия 2

Боб хочет отфильтровать список пользователей старше 18 лет. Какой метод подойдет?

Подсказка: Подумайте о создании нового массива с элементами, которые удовлетворяют условию.

Ответ:

- **Использовать `filter` с проверкой возраста**
- Использовать `map` с проверкой возраста
- Использовать `reduce` без условия
- Использовать `forEach` и пушить элементы вручную

🔗 Шаг 03:

Миссия 3

Лена хочет посчитать общую сумму цен в корзине. Какой метод JavaScript ей поможет?

Подсказка: Подумайте, как аккумулировать значения из массива в одно число.

Ответ:

- **Использовать `reduce` для суммирования цен**
- Использовать `map` для суммирования цен
- Использовать `filter` для суммирования цен
- Использовать `forEach` без аккумулятора

🔗 Шаг 04:

Миссия 4

Игорь хочет получить массив имен пользователей с учетом только тех, кто активен. Какой подход проще всего?

Подсказка: Можно комбинировать методы для фильтрации и трансформации данных.

Ответ:

- **Сначала `filter` для активных, потом `map` для имен**
- Сначала `map`, потом `filter`
- Использовать только `reduce` без фильтрации
- Использовать `forEach` с `push` вручную

🔗 Шаг 05:

Миссия 5

Аня хочет получить новый массив, в котором цены товаров увеличены на 10%, но только для товаров дороже 100. Какой подход правильный?

Подсказка: Подумайте о последовательном применении фильтрации и преобразования.

Ответ:

- **Сначала `filter (>100)`, потом `map (*1.1)`**
- Сначала `map (*1.1)`, потом `filter (>100)`
- Использовать только `map` без условия

- Использовать `reduce` без проверки

🔮 Шаг 06:

Миссия 6

Формулировка: Сопоставьте методы массива с их назначением.

Методы:

Ответ:

- `map`
- `filter`
- `reduce`

Назначение:

1. Создает новый массив, применяя функцию к каждому элементу
2. Возвращает новый массив с элементами, прошедшими проверку
3. Сводит массив к одному значению, аккумулируя результаты

Подсказка: Подумайте о том, какой метод возвращает массив, а какой — одно значение.

- **`map`** — Создает новый массив, применяя функцию к каждому элементу
- **`filter`** — Возвращает новый массив с элементами, прошедшими проверку
- **`reduce`** — Сводит массив к одному значению, аккумулируя результаты

🔮 Шаг 07:

Миссия 7

Сопоставьте сценарии с методами массивов:

Сценарии:

Ответ:

- Выбираем только активных пользователей
- Получаем список их email
- Считаем сумму всех заказов

Методы:

1. `map`
2. `filter`
3. `reduce`

Подсказка: Подумайте, где нужна фильтрация, где преобразование, а где суммирование.

- **Выбираем только активных пользователей** — `filter`
- **Получаем список их email** — `map`
- **Считаем сумму всех заказов** — `reduce`

🔮 Шаг 08:

Миссия 8

Сопоставьте цепочки методов с их результатом:

Цепочки:

Ответ:

- `filter + map`

- `map + reduce`
- `reduce`

Результаты:

1. Массив, элементы которого прошли проверку, затем преобразованы
2. Одно значение, агрегированное из массива после преобразования
3. Одно значение, агрегированное из массива

Подсказка: Обратите внимание на порядок применения методов и тип возвращаемого значения.

- **`filter + map`** — Массив, элементы которого прошли проверку, затем преобразованы
- **`map + reduce`** — Одно значение, агрегированное из массива после преобразования
- **`reduce`** — Одно значение, агрегированное из массива

Шаг 09:

Миссия 9

Вопрос: Расположите шаги получения суммы цен товаров старше 100 единиц:

1. Фильтрация товаров с ценой > 100
2. Извлечение цены каждого товара
3. Суммирование цен

Подсказка: Сначала выбираем, потом преобразуем, потом аккумулируем.

Ответ:

- Фильтрация товаров с ценой > 100
- Извлечение цены каждого товара
- Суммирование цен

Шаг 10:

Миссия 10

Расположите шаги получения списка имен активных пользователей в алфавитном порядке:

1. Фильтрация активных пользователей
2. Извлечение имен
3. Сортировка по алфавиту

Подсказка: Логика: сначала выбираем, потом преобразуем, потом упорядочиваем.

Ответ:

- Фильтрация активных пользователей
- Извлечение имен
- Сортировка по алфавиту

Шаг 11:

Миссия 11

Разработчик Алекс хочет увеличить все цены товаров на сайте на 10% с помощью `map`. Что нужно сделать?

Подсказка: Подумайте, как `map` изменяет каждый элемент массива отдельно.

Ответ:

- **Использовать `map` и в функции умножить каждый элемент на 1.1**

- Использовать `filter` для изменения цен
- Использовать `reduce` для изменения каждого элемента
- Пройтись циклом `forEach` и изменить исходный массив

🔗 Шаг 12:

Миссия 12

Боб получает список пользователей и хочет оставить только тех, кто старше 18 лет с помощью `filter`. Как правильно поступить?

Подсказка: `Filter` возвращает новый массив с элементами, которые соответствуют условию.

Ответ:

- **Использовать `filter` и проверять возраст каждого пользователя**
- Использовать `map` для фильтрации
- Использовать `reduce` для выбора пользователей
- Менять исходный массив через `forEach`

🔗 Шаг 13:

Миссия 13

Лена хочет посчитать общую сумму заказов клиентов с помощью `reduce`. Как правильно это сделать?

Подсказка: `Reduce` аккумулирует значение через каждый элемент массива.

Ответ:

- **Использовать `reduce` с суммированием каждого элемента**
- Использовать `map` для суммирования
- Использовать `filter` для суммирования
- Использовать `forEach` и `return` внутри функции

🔗 Шаг 14:

Миссия 14

Игорь хочет сделать массив с именами пользователей в верхнем регистре с помощью `map`. Что выбрать?

Подсказка: `Map` позволяет преобразовать каждый элемент массива в новый вид.

Ответ:

- **Использовать `map` и внутри функции применять `toUpperCase()` к каждому имени**
- Использовать `filter` с условием для преобразования
- Использовать `reduce` для изменения регистра
- Менять массив вручную через `forEach`

🔗 Шаг 15:

Миссия 15

Аня хочет оставить только заказы со статусом 'оплачен' с помощью `filter`. Что выбрать?

Подсказка: Условие внутри `filter` должно возвращать `true` для нужных элементов.

Ответ:

- **Использовать `filter` с проверкой `order.status === 'оплачен'`**
- Использовать `map` для фильтрации
- Использовать `reduce` для фильтрации

- Менять исходный массив вручную через цикл

🔮 Шаг 16:

Миссия 16

Сопоставьте паттерны JavaScript с их назначением:

Ответ:

- map
- filter
- reduce

1. Создает новый массив, изменяя каждый элемент исходного
2. Оставляет только элементы, удовлетворяющие условию
3. Аккумулирует массив в одно значение

Подсказка: Подумайте, какой метод возвращает массив, какой — одно значение.

- **map** — Создает новый массив, изменяя каждый элемент исходного
- **filter** — Оставляет только элементы, удовлетворяющие условию
- **reduce** — Аккумулирует массив в одно значение

📖 Урок 4 - Lodash Ramda краткий обзор

🔮 Шаг 01:

Миссия 1

Алекс использует Lodash и хочет объединить два массива пользователей. Какой метод подойдет лучше всего?

Подсказка: Подумайте, какой метод создает новый массив, объединяя элементы исходных массивов.

Ответ:

- **_.concat(array1, array2)**
- **_.merge(array1, array2)**
- **_.filter(array1, array2)**
- **_.map(array1, array2)**

🔮 Шаг 02:

Миссия 2

Боб хочет удалить все ложные значения из массива с помощью Lodash. Какой метод выбрать?

Подсказка: В Lodash есть метод, который оставляет только истинные значения.

Ответ:

- **_.compact(array)**
- **_.filter(array, Boolean)**
- **_.remove(array)**
- **_.flatten(array)**

🔮 Шаг 03:

Миссия 3

Лена хочет получить список уникальных имён пользователей с помощью Lodash. Какой метод выбрать?

Подсказка: Вспомните метод, который убирает дубликаты.

Ответ:

- **_.uniq(array)**
- `_.unique(array)`
- `_.flatten(array)`
- `_.compact(array)`

Шаг 04:

Миссия 4

Игорь хочет взять первые 5 элементов массива с помощью Ramda. Какой метод использовать?

Подсказка: Подумайте о методе, который выбирает элементы с начала массива.

Ответ:

- **R.take(5, array)**
- `R.drop(5, array)`
- `R.slice(0,5, array)`
- `R.head(array)`

Шаг 05:

Миссия 5

Аня хочет применить функцию ко всем элементам массива с помощью Ramda. Какой метод использовать?

Подсказка: Функция должна преобразовать каждый элемент, возвращая новый массив.

Ответ:

- **R.map(fn, array)**
- `R.filter(fn, array)`
- `R.reduce(fn, acc, array)`
- `R.forEach(fn, array)`

Шаг 06:

Миссия 6

Сопоставьте методы Lodash и Ramda с их назначением:

Ответ:

- `_.concat`
- `_.compact`
- `_.uniq`
- `R.take`
- `R.map`

1. Создает новый массив без ложных значений
2. Объединяет массивы
3. Возвращает первые N элементов массива
4. Создает массив только с уникальными элементами
5. Применяет функцию ко всем элементам массива

Подсказка: Обратите внимание, какой метод работает с `Lodash`, какой с `Ramda`, и как они трансформируют массив.

- **`_.concat`** — Объединяет массивы
- **`_.compact`** — Создает новый массив без ложных значений
- **`_.uniq`** — Создает массив только с уникальными элементами
- **`R.take`** — Возвращает первые N элементов массива
- **`R.map`** — Применяет функцию ко всем элементам массива

Шаг 07:

Миссия 7

Вопрос: Расположите действия при использовании `Ramda` и `Lodash` для очистки и трансформации массива в правильном порядке:

1. Объединить несколько массивов в один
2. Удалить ложные значения
3. Взять только уникальные элементы
4. Применить функцию к каждому элементу
5. Взять первые N элементов

Подсказка: Сначала объединяем данные, затем чистим их, затем оставляем уникальные, потом трансформируем и, наконец, берём нужное количество.

Ответ:

- Объединить несколько массивов в один
- Удалить ложные значения
- Взять только уникальные элементы
- Применить функцию к каждому элементу
- Взять первые N элементов

Шаг 08:

Миссия 8

Разработчик Алекс использует `Lodash`, чтобы получить уникальные значения из массива чисел. Какой метод лучше всего использовать?

Подсказка: Подумайте о функции, которая удаляет повторяющиеся элементы.

Ответ:

- **`_.uniq(array)`**
- `_.flatten(array)`
- `_.map(array)`
- `_.filter(array)`

Шаг 09:

Миссия 9

Боб хочет преобразовать массив строк в массив их длин с помощью `Ramda`. Что поможет?

Подсказка: Подумайте о функции, которая применяет трансформацию к каждому элементу.

Ответ:

- **`R.map(s => s.length, array)`**
- `R.filter(s => s.length > 0, array)`
- `R.reduce((a,b)=>a+b,array,0)`

- `R.uniq(array)`

🔗 Шаг 10:

Миссия 10

Лена хочет отфильтровать массив заказов, оставив только те, что дороже 100, используя Lodash. Какой метод выбрать?

Подсказка: Filter позволяет оставить только элементы, соответствующие условию.

Ответ:

- `_.filter(orders, o => o.price > 100)`
- `_.map(orders, o => o.price)`
- `_.reduce(orders, (sum,o)=>sum+o.price,0)`
- `_.uniq(orders)`

🔗 Шаг 11:

Миссия 11

Игорь хочет посчитать суммарную стоимость товаров с помощью Ramda. Какой метод лучше использовать?

Подсказка: Reduce аккумулирует значение через каждый элемент массива.

Ответ:

- `R.reduce((sum, item) => sum + item.price, 0, cart)`
- `R.map(item => item.price, cart)`
- `R.filter(item => item.price > 0, cart)`
- `R.uniq(cart)`

🔗 Шаг 12:

Миссия 12

Аня хочет объединить массивы заказов нескольких клиентов в один массив с помощью Lodash. Какой метод выбрать?

Подсказка: Flatten помогает объединить вложенные массивы.

Ответ:

- `_.flatten(orderArrays)`
- `_.map(orderArrays, x => x)`
- `_.filter(orderArrays, x => x.length > 0)`
- `_.reduce(orderArrays, (a,b)=>a+b)`

🔗 Шаг 13:

Миссия 13

Сопоставьте функции Lodash/Ramda с их назначением:

Ответ:

- `_.uniq`
- `_.flatten`
- `R.map`
- `R.reduce`

1. Создает новый массив, изменяя каждый элемент

2. Удаляет повторяющиеся элементы
3. Аккумулирует массив в одно значение
4. Объединяет вложенные массивы в один

Подсказка: Смотрите на результат: массив, одно значение, объединение или уникальные элементы?

- **_.uniq** — Удаляет повторяющиеся элементы
- **_.flatten** — Объединяет вложенные массивы в один
- **R.map** — Создает новый массив, изменяя каждый элемент
- **R.reduce** — Аккумулирует массив в одно значение

Шаг 14:

Миссия 14

Сопоставьте ситуацию с методом:

Ответ:

- Удаление дубликатов пользователей
- Суммирование всех цен товаров
- Преобразование массива имен в длины строк
- Объединение массивов заказов

1. `_.flatten`
2. `_.uniq`
3. `R.map`
4. `R.reduce`

Подсказка: Думайте о цели: фильтр, суммирование, трансформация или объединение.

- **Удаление дубликатов пользователей** — `_.uniq`
- **Суммирование всех цен товаров** — `R.reduce`
- **Преобразование массива имен в длины строк** — `R.map`
- **Объединение массивов заказов** — `_.flatten`

Шаг 15:

Миссия 15

Расположите шаги обработки данных с использованием Lodash / Ramda в правильном порядке:

1. Объединить массивы заказов разных клиентов
2. Удалить повторяющиеся заказы
3. Преобразовать заказы в стоимость каждого заказа
4. Посчитать общую сумму всех заказов

 **Подсказка:**

Сначала нужно объединить данные, потом очистить их, преобразовать в нужный вид и только в конце суммировать.

Ответ:

- Объединить массивы заказов разных клиентов
- Удалить повторяющиеся заказы
- Преобразовать заказы в стоимость каждого заказа
- Посчитать общую сумму всех заказов

Модуль 8 - Модули и сборщики

Урок 1 - ESM и CommonJS

Шаг 01:

Миссия 1

Сопоставьте синтаксис модулей с их описанием:

Синтаксис:

Ответ:

- `import/export`
- `require/module.exports`
- `export default`
- `module.exports =`

Описание:

1. ESM, импорт и экспорт отдельных элементов
2. CommonJS, экспорт всего объекта
3. ESM, экспорт одного значения по умолчанию
4. CommonJS, экспорт всего модуля

Подсказка: Смотрите, какой синтаксис связан с ``import`` и какой с ``require``.

- **`import/export`** — ESM, импорт и экспорт отдельных элементов
- **`require/module.exports`** — CommonJS, экспорт всего объекта
- **`export default`** — ESM, экспорт одного значения по умолчанию
- **`module.exports =`** — CommonJS, экспорт всего модуля

Шаг 02:

Миссия 2

Сопоставьте ошибку с причиной при смешивании ESM и CommonJS:

Ответ:

- `SyntaxError: Cannot use import statement outside a module`
- `TypeError: require is not a function`
- `Unexpected token 'export'`
- `ReferenceError: module is not defined`

1. Файл не отмечен как ESM
2. Попытка использовать `require` в ESM
3. Попытка использовать `export` в CommonJS
4. Попытка использовать `module` в ESM

Подсказка: Ошибки появляются при неправильном сочетании синтаксиса модулей.

- **`SyntaxError: Cannot use import statement outside a module`** — Файл не отмечен как ESM
- **`TypeError: require is not a function`** — Попытка использовать `require` в ESM
- **`Unexpected token 'export'`** — Попытка использовать `export` в CommonJS
- **`ReferenceError: module is not defined`** — Попытка использовать `module` в ESM

Шаг 03:

Миссия 3

Сопоставьте термин и пример:

Ответ:

- Named export
- Default export
- CommonJS export
- Dynamic import

1. `export function sum() {}`
2. `export default class Calculator {}`
3. `module.exports = {sum}`
4. `import('fs').then(fs => ...)`

Подсказка: Вспомните, какой синтаксис позволяет экспортировать несколько элементов и какой — один.

- **Named export** — `export function sum() {}`
- **Default export** — `export default class Calculator {}`
- **CommonJS export** — `module.exports = {sum}`
- **Dynamic import** — `import('fs').then(fs => ...)`

Шаг 04:

Миссия 4

Сопоставьте персонажей и их любимый модульный стандарт:

Ответ:

- **Иван** — Node.js ностальгик
- **Мария** — любитель современных стандартов
- **Сергей** — пишет универсальные пакеты
- **Ольга** — экспериментирует с динамическими импортами

1. CommonJS
2. ESM
3. CommonJS + ESM
4. Dynamic import

Подсказка: Подумайте о привычках и стиле программирования каждого персонажа.

- **Иван** — Node.js ностальгик — CommonJS
- **Мария** — любитель современных стандартов — ESM
- **Сергей** — пишет универсальные пакеты — CommonJS + ESM
- **Ольга** — экспериментирует с динамическими импортами — Dynamic import

Шаг 05:

Миссия 5

Расположите шаги для преобразования проекта с CommonJS в ESM:

1. Переименовать файлы в `.mjs` или указать `"type":"module"`
2. Заменить `require` на `import`
3. Заменить `module.exports` на `export` / `export default`

4. Проверить работу проекта

Подсказка: Начинайте с конфигурации, потом меняйте синтаксис, а в конце тестируйте.

Ответ:

- Переименовать файлы в .mjs или указать "type":"module"
- Заменить require на import
- Заменить module.exports на export / export default
- Проверить работу проекта

Шаг 06:

Миссия 6

Упорядочите действия при динамическом импорте модуля:

1. Вызываем import('module')
2. Используем промис для получения содержимого
3. Работаем с импортированными функциями или объектами
4. Обрабатываем ошибки

Подсказка: Динамический импорт работает как асинхронная операция с промисом.

Ответ:

- Вызываем import('module')
- Используем промис для получения содержимого
- Работаем с импортированными функциями или объектами
- Обрабатываем ошибки

Шаг 07:

Миссия 7

Расставьте шаги по проверке совместимости ESM и CommonJS в проекте:

1. Создать тестовые файлы с импортом разных модулей
2. Запустить Node.js с указанием type:module
3. Проверить, что функции работают как ожидается
4. Исправить найденные ошибки импорта/экспорта

Подсказка: Сначала создаём тесты, потом проверяем и исправляем ошибки.

Ответ:

- Создать тестовые файлы с импортом разных модулей
- Запустить Node.js с указанием type:module
- Проверить, что функции работают как ожидается
- Исправить найденные ошибки импорта/экспорта

Шаг 08:

Миссия 8

Формулировка: Сопоставьте тип модуля с его особенностью.

Понятия:

Ответ:

- ESM
- CommonJS

- `dynamic import`
- `require()`

Определения:

1. Стандартная система модулей JavaScript с поддержкой `import/export`.
2. Функция для динамической загрузки модулей в ESM.
3. Синхронная система модулей Node.js с `module.exports` и `require`.
4. Функция для загрузки CommonJS модулей.

🔗 **Подсказка:**

Подумайте, какие синтаксисы относятся к современным стандартам, а какие к старым.

- **ESM** — Стандартная система модулей JavaScript с поддержкой `import/export`.
- **CommonJS** — Синхронная система модулей Node.js с `module.exports` и `require`.
- **dynamic import** — Функция для динамической загрузки модулей в ESM.
- **require()** — Функция для загрузки CommonJS модулей.

🔗 **Шаг 09:**

Миссия 9

Формулировка: Сопоставьте ошибки при использовании модулей с их объяснением.

Понятия:

Ответ:

- Смешивание `import` и `require`
- Пропуск `"type": "module"` в `package.json`
- Использование `export` в CommonJS
- Попытка `require` ESM-модуля

Определения:

1. Node.js не знает, что файлы должны быть ESM.
2. Нельзя использовать `export` в модулях CommonJS.
3. Нельзя использовать `require` для ESM-модулей.
4. Может вызвать ошибку времени выполнения из-за разных систем модулей.

🔗 **Подсказка:**

Внимательно смотрите, какой синтаксис предназначен для какого типа модулей.

- **Смешивание `import` и `require`** — Может вызвать ошибку времени выполнения из-за разных систем модулей.
- **Пропуск `"type": "module"` в `package.json`** — Node.js не знает, что файлы должны быть ESM.
- **Использование `export` в CommonJS** — Нельзя использовать `export` в модулях CommonJS.
- **Попытка `require` ESM-модуля** — Нельзя использовать `require` для ESM-модулей.

🔗 **Шаг 10:**

Миссия 10

Формулировка: Сопоставьте команды сборщиков с их особенностями для работы с модулями.

Понятия:

Ответ:

- Webpack

- Rollup
- Parcel
- ESBuild

Определения:

1. Быстрая сборка с минимальной конфигурацией, поддержка ESM.
2. Оптимизация для библиотек, tree-shaking, поддержка ESM.
3. Конфигурируемый сборщик, подходит для больших проектов.
4. Zero-config, автоматически собирает зависимости и модули.

🔗 Подсказка:

Подумайте, какой сборщик чаще используется для библиотек, а какой для приложений без конфигурации.

- **Webpack** — Конфигурируемый сборщик, подходит для больших проектов.
- **Rollup** — Оптимизация для библиотек, tree-shaking, поддержка ESM.
- **Parcel** — Zero-config, автоматически собирает зависимости и модули.
- **ESBuild** — Быстрая сборка с минимальной конфигурацией, поддержка ESM.

🔗 Шаг 11:

Миссия 11

Формулировка: Сопоставьте понятие с его применением.

Понятия:

Ответ:

- tree-shaking
- dynamic import
- module bundling
- default export

Определения:

1. Удаление неиспользуемого кода при сборке.
2. Сборка нескольких модулей в один файл.
3. Импорт модуля по необходимости во время выполнения.
4. Экспорт одного значения по умолчанию из модуля.

🔗 Подсказка:

Обратите внимание, какие функции оптимизации выполняются на этапе сборки, а какие во время выполнения.

- **tree-shaking** — Удаление неиспользуемого кода при сборке.
- **dynamic import** — Импорт модуля по необходимости во время выполнения.
- **module bundling** — Сборка нескольких модулей в один файл.
- **default export** — Экспорт одного значения по умолчанию из модуля.

🔗 Шаг 12:

Миссия 12

Вопрос: Расположите шаги использования ESM в Node.js проекте:

1. Добавление "type": "module" в package.json
2. Импорт функций с использованием import

3. Создание модулей с export
4. Запуск приложения и проверка работы модулей

 Подсказка:

Сначала нужно указать Node.js, что проект ESM, затем создавать модули, импортировать и проверять.

Ответ:

- Добавление "type": "module" в package.json
- Создание модулей с export
- Импорт функций с использованием import
- Запуск приложения и проверка работы модулей

 Шаг 13:

Миссия 13

Вопрос: Расположите шаги использования CommonJS:

1. Экспорт функций через module.exports
2. Импорт модулей с помощью require
3. Создание файла с функциями
4. Запуск приложения и проверка работы модулей

 Подсказка:

Сначала создаются функции, потом экспорт, импорт, и проверка работы.

Ответ:

- Создание файла с функциями
- Экспорт функций через module.exports
- Импорт модулей с помощью require
- Запуск приложения и проверка работы модулей

 Шаг 14:

Миссия 14

Вопрос: Расположите шаги по устранению ошибки при смешивании ESM и CommonJS:

1. Определить, какой файл вызывает ошибку
2. Разделить модули по типу (ESM/CommonJS)
3. Исправить синтаксис импорта/экспорта
4. Перезапустить приложение и проверить работу

 Подсказка:

Внимательно следите за типами модулей и синтаксисом.

Ответ:

- Определить, какой файл вызывает ошибку
- Разделить модули по типу (ESM/CommonJS)
- Исправить синтаксис импорта/экспорта
- Перезапустить приложение и проверить работу

Урок 2 - Сборка с Webpack

Шаг 01:

Миссия 1

Формулировка: Сопоставьте понятие с его ролью в Webpack.

Понятия:

Ответ:

- entry
- output
- loaders
- plugins

Определения:

1. Определяет, с какого файла Webpack начинает сборку.
2. Конфигурирует путь и имя выходного файла сборки.
3. Обрабатывает файлы перед сборкой (например, Babel, CSS).
4. Расширяет функциональность Webpack (например, минификация, копирование файлов).

Подсказка:

Обратите внимание на разделение между обработкой файлов и расширением функциональности.

- **entry** — Определяет, с какого файла Webpack начинает сборку.
- **output** — Конфигурирует путь и имя выходного файла сборки.
- **loaders** — Обрабатывает файлы перед сборкой (например, Babel, CSS).
- **plugins** — Расширяет функциональность Webpack (например, минификация, копирование файлов).

Шаг 02:

Миссия 2

Формулировка: Сопоставьте тип loader с его назначением.

Понятия:

Ответ:

- babel-loader
- style-loader
- css-loader
- file-loader

Определения:

1. Преобразует современный JS в совместимый код для старых браузеров.
2. Добавляет CSS в DOM через тег `<style>`.
3. Разбирает CSS и позволяет импортировать его в JS.
4. Импортирует файлы (картинки, шрифты) как модули.

Подсказка:

Подумайте, какой loader отвечает за JS, а какой за стили и файлы.

- **babel-loader** — Преобразует современный JS в совместимый код для старых браузеров.
- **style-loader** — Добавляет CSS в DOM через тег .

- **css-loader** — Разбирает CSS и позволяет импортировать его в JS.
- **file-loader** — Импортирует файлы (картинки, шрифты) как модули.

🔗 Шаг 03:

Миссия 3

Формулировка: Сопоставьте плагин Webpack с его функциональностью.

Понятия:

Ответ:

- HtmlWebpackPlugin
- MiniCssExtractPlugin
- CleanWebpackPlugin
- DefinePlugin

Определения:

1. Создает HTML-файл и вставляет в него сборку JS.
2. Извлекает CSS в отдельные файлы.
3. Удаляет старые файлы сборки перед новой сборкой.
4. Позволяет задавать глобальные константы на этапе сборки.

🔗 Подсказка:

Подумайте, какой плагин отвечает за HTML, CSS и очистку сборки.

- **HtmlWebpackPlugin** — Создает HTML-файл и вставляет в него сборку JS.
- **MiniCssExtractPlugin** — Извлекает CSS в отдельные файлы.
- **CleanWebpackPlugin** — Удаляет старые файлы сборки перед новой сборкой.
- **DefinePlugin** — Позволяет задавать глобальные константы на этапе сборки.

🔗 Шаг 04:

Миссия 4

Формулировка: Сопоставьте термин Webpack с его описанием.

Понятия:

Ответ:

- tree-shaking
- code splitting
- hot module replacement (HMR)
- bundle

Определения:

1. Удаление неиспользуемого кода из сборки.
2. Разделение кода на несколько файлов для оптимизации загрузки.
3. Обновление модулей в браузере без полной перезагрузки.
4. Объединение всех модулей в один или несколько файлов для использования в браузере.

🔗 Подсказка:

Подумайте, что оптимизирует размер, что скорость загрузки и что облегчает разработку.

- **tree-shaking** — Удаление неиспользуемого кода из сборки.
- **code splitting** — Разделение кода на несколько файлов для оптимизации загрузки.

- **hot module replacement (HMR)** — Обновление модулей в браузере без полной перезагрузки.
- **bundle** — Объединение всех модулей в один или несколько файлов для использования в браузере.

Шаг 05:

Миссия 5

Вопрос: Расположите шаги сборки проекта с Webpack:

1. Установка Webpack и необходимых loader'ов
2. Создание конфигурационного файла webpack.config.js
3. Определение entry и output
4. Настройка loader'ов и плагинов
5. Запуск команды сборки и проверка работы

Подсказка:

Сначала устанавливаем инструменты, потом конфигурируем и запускаем.

Ответ:

- Установка Webpack и необходимых loader'ов
- Создание конфигурационного файла webpack.config.js
- Определение entry и output
- Настройка loader'ов и плагинов
- Запуск команды сборки и проверка работы

Шаг 06:

Миссия 6

Формулировка: Сопоставьте свойства конфигурации Webpack с их назначением.

Понятия:

Ответ:

- entry
- output
- module.rules
- plugins

Определения:

1. Указывает точки входа для сборки.
2. Определяет, куда и как сохраняются собранные файлы.
3. Настройка загрузчиков для обработки различных файлов.
4. Подключение дополнительных возможностей, например, минификация или генерация HTML.

Подсказка:

Подумайте, что нужно настроить для файлов, что для расширений возможностей, а что для точек входа.

- **entry** — Указывает точки входа для сборки.
- **output** — Определяет, куда и как сохраняются собранные файлы.
- **module.rules** — Настройка загрузчиков для обработки различных файлов.

- **plugins** — Подключение дополнительных возможностей, например, минификация или генерация HTML.

🔗 Шаг 07:

Миссия 7

Формулировка: Сопоставьте плагины Webpack с их функциями.

Понятия:

Ответ:

- HtmlWebpackPlugin
- MiniCssExtractPlugin
- CleanWebpackPlugin
- DefinePlugin

Определения:

1. Автоматическая генерация HTML-файла и подключение бандлов.
2. Вынесение CSS в отдельные файлы.
3. Очистка папки output перед сборкой.
4. Определение глобальных констант для приложения.

🔗 Подсказка:

Смотрите на действия плагина: что он создает, удаляет или настраивает.

- **HtmlWebpackPlugin** — Автоматическая генерация HTML-файла и подключение бандлов.
- **MiniCssExtractPlugin** — Вынесение CSS в отдельные файлы.
- **CleanWebpackPlugin** — Очистка папки output перед сборкой.
- **DefinePlugin** — Определение глобальных констант для приложения.

🔗 Шаг 08:

Миссия 8

Формулировка: Сопоставьте режим Webpack с его назначением.

Понятия:

Ответ:

- development
- production

Определения:

1. Оптимизация кода, минимизация и tree-shaking для релиза.
2. Быстрая сборка с source maps и поддержкой hot reload для разработки.

🔗 Подсказка:

Смотрите на цель: готовый релиз или удобство разработки.

- **development** — Быстрая сборка с source maps и поддержкой hot reload для разработки.
- **production** — Оптимизация кода, минимизация и tree-shaking для релиза.

🔗 Шаг 09:

Миссия 9

Вопрос: Расположите шаги настройки Webpack для проекта:

1. Создание webpack.config.js
2. Установка зависимостей (webpack, webpack-cli, loaders, plugins)
3. Определение entry и output
4. Настройка module.rules и plugins
5. Запуск сборки

 Подсказка:

Сначала создаем файл, потом ставим зависимости, настраиваем сборку и только потом запускаем.

Ответ:

- Создание webpack.config.js
- Установка зависимостей (webpack, webpack-cli, loaders, plugins)
- Определение entry и output
- Настройка module.rules и plugins
- Запуск сборки

 Шаг 10:

Миссия 10

Формулировка: Сопоставьте понятия Webpack с их назначением.

Понятия:

Ответ:

- entry
- output
- loader
- plugin

Определения:

1. Файл или файлы, с которых начинается сборка
2. Место и имя итогового бандла
3. Трансформация определенного типа файлов
4. Расширение возможностей сборки, например, оптимизация или генерация HTML

 Подсказка:

Подумайте, что делает каждый элемент конфигурации и на каком этапе работы Webpack он применяется.

- **entry** — Файл или файлы, с которых начинается сборка
- **output** — Место и имя итогового бандла
- **loader** — Трансформация определенного типа файлов
- **plugin** — Расширение возможностей сборки, например, оптимизация или генерация HTML

 Шаг 11:

Миссия 11

Формулировка: Сопоставьте ладеры с их назначением.

Ответ:

- babel-loader

- css-loader
- style-loader
- file-loader

Определения:

1. Транспирирует современный JS в совместимый с браузерами код
2. Позволяет импортировать CSS как модули
3. Вставляет CSS в DOM через style-теги
4. Импортирует изображения и файлы в бандл

 **Подсказка:**

Подумайте, какой лоадер нужен для JavaScript, а какой — для стилей и ресурсов.

- **babel-loader** — Транспирирует современный JS в совместимый с браузерами код
- **css-loader** — Позволяет импортировать CSS как модули
- **style-loader** — Вставляет CSS в DOM через style-теги
- **file-loader** — Импортирует изображения и файлы в бандл

 **Шаг 12:**

Миссия 12

Формулировка: Сопоставьте плагины Webpack с их функцией.

Ответ:

- HtmlWebpackPlugin
- MiniCssExtractPlugin
- CleanWebpackPlugin
- DefinePlugin

Определения:

1. Создает HTML файл с подключенными бандлами
2. Извлекает CSS в отдельные файлы
3. Удаляет старые файлы перед сборкой
4. Определяет глобальные переменные на этапе сборки

 **Подсказка:**

Подумайте, какие плагины управляют HTML, CSS, очисткой или переменными.

- **HtmlWebpackPlugin** — Создает HTML файл с подключенными бандлами
- **MiniCssExtractPlugin** — Извлекает CSS в отдельные файлы
- **CleanWebpackPlugin** — Удаляет старые файлы перед сборкой
- **DefinePlugin** — Определяет глобальные переменные на этапе сборки

 **Шаг 13:**

Миссия 13

Формулировка: Сопоставьте режимы сборки Webpack с их особенностями.

Ответ:

- development
- production

Определения:

1. Оптимизированная сборка с минимизацией и tree-shaking
2. Быстрая сборка с source maps для отладки

🔗 **Подсказка:**

Подумайте, какой режим используют для разработки, а какой для выпуска на продакшн.

- **development** — Быстрая сборка с source maps для отладки
- **production** — Оптимизированная сборка с минимизацией и tree-shaking

🔗 **Шаг 14:**

Миссия 14

Вопрос: Расположите шаги сборки проекта с Webpack:

1. Создать конфигурационный файл webpack.config.js
2. Установить зависимости и ладеры
3. Запустить команду webpack для сборки
4. Проверить работу итогового бандла в браузере

🔗 **Подсказка:**

Сначала конфигурация и установка, затем сборка и проверка.

Ответ:

- Создать конфигурационный файл webpack.config.js
- Установить зависимости и ладеры
- Запустить команду webpack для сборки
- Проверить работу итогового бандла в браузере

🔗 **Шаг 15:**

Миссия 15

Вопрос: Расположите шаги подключения Babel к Webpack:

1. Установить babel-loader и @babel/core
2. Настроить module.rules с babel-loader
3. Создать .babelrc или babel.config.js
4. Запустить сборку и проверить совместимость JS

🔗 **Подсказка:**

Подумайте, что нужно сначала установить, потом настроить, затем проверить.

Ответ:

- Установить babel-loader и @babel/core
- Настроить module.rules с babel-loader
- Создать .babelrc или babel.config.js
- Запустить сборку и проверить совместимость JS

🔗 **Шаг 16:**

Миссия 16

Вопрос: Расположите шаги подключения CSS в проект с Webpack:

1. Установить css-loader и style-loader

2. Настроить module.rules для CSS файлов
3. Импортировать CSS в JS файле
4. Запустить сборку и проверить отображение стилей

🔗 **Подсказка:**

Сначала установка лоадеров, потом конфигурация и импорт, затем проверка.

Ответ:

- Установить css-loader и style-loader
- Настроить module.rules для CSS файлов
- Импортировать CSS в JS файле
- Запустить сборку и проверить отображение стилей

📖 **Урок 3 - Babel, транспиляция, полифиллы**

🔗 **Шаг 01:**

Миссия 1

Формулировка: Сопоставьте понятия Babel с их назначением.

Ответ:

- Babel
- Preset
- Plugin
- Полифилл

Определения:

1. Транспилюет современный JS в старые версии для совместимости
2. Набор правил для преобразования кода
3. Добавляет новые возможности или трансформации в код
4. Добавляет функциональность в средах, где она отсутствует

🔗 **Подсказка:**

Подумайте, какой элемент управляет совместимостью, а какой расширяет функциональность.

- **Babel** — Транспилюет современный JS в старые версии для совместимости
- **Preset** — Набор правил для преобразования кода
- **Plugin** — Добавляет новые возможности или трансформации в код
- **Полифилл** — Добавляет функциональность в средах, где она отсутствует

🔗 **Шаг 02:**

Миссия 2

Формулировка: Сопоставьте виды полифиллов с их назначением.

Ответ:

- core-js
- regenerator-runtime

Определения:

1. Добавляет поддержку новых методов и объектов ES6+ (Promise, Map, Set)
2. Добавляет поддержку async/await и генераторов

🔗 **Подсказка:**

Подумайте, что нужно для синхронного кода, а что для асинхронного.

- **core-js** — Добавляет поддержку новых методов и объектов ES6+ (Promise, Map, Set)
- **regenerator-runtime** — Добавляет поддержку async/await и генераторов

🔗 Шаг 03:

Миссия 3

Формулировка: Сопоставьте конфигурационные параметры Babel с их задачами.

Ответ:

- presets
- plugins
- targets

Определения:

1. Определяют набор трансформаций для кода
2. Добавляют новые возможности в процессе транспиляции
3. Указывают, для каких браузеров/сред предназначен код

🔗 Подсказка:

Подумайте, какие параметры управляют набором правил, какие добавляют функциональность, а какие задают совместимость.

- **presets** — Определяют набор трансформаций для кода
- **plugins** — Добавляют новые возможности в процессе транспиляции
- **targets** — Указывают, для каких браузеров/сред предназначен код

🔗 Шаг 04:

Миссия 4

Формулировка: Сопоставьте понятия Babel с их функцией.

Ответ:

- Transpiling
- Preset
- Plugin
- Polyfill

Определения:

1. Преобразование нового JS кода в старый стандарт
2. Набор плагинов для определенной цели (например, поддержка ES2015)
3. Расширяет возможности Babel, добавляя конкретные преобразования
4. Добавляет недостающий функционал в старые окружения

🔗 Подсказка:

Подумайте, что делает Babel, что позволяет использовать новые фичи и что нужно для старых браузеров.

- **Transpiling** — Преобразование нового JS кода в старый стандарт
- **Preset** — Набор плагинов для определенной цели (например, поддержка ES2015)
- **Plugin** — Расширяет возможности Babel, добавляя конкретные преобразования
- **Polyfill** — Добавляет недостающий функционал в старые окружения

🔮 Шаг 05:

Миссия 5

Формулировка: Сопоставьте пресеты Babel с их назначением.

Ответ:

- @babel/preset-env
- @babel/preset-react
- @babel/preset-typescript

Определения:

1. Транспилиция современного JS с учетом целевых браузеров
2. Поддержка JSX и React кода
3. Транспилиция TypeScript в JavaScript

🔮 Подсказка:

Подумайте, какой пресет нужен для браузеров, для React и для TS.

- **@babel/preset-env** — Транспилиция современного JS с учетом целевых браузеров
- **@babel/preset-react** — Поддержка JSX и React кода
- **@babel/preset-typescript** — Транспилиция TypeScript в JavaScript

🔮 Шаг 06:

Миссия 6

Формулировка: Сопоставьте плагины Babel с их назначением.

Ответ:

- @babel/plugin-transform-arrow-functions
- @babel/plugin-transform-async-to-generator
- @babel/plugin-proposal-class-properties
- @babel/plugin-transform-runtime

Определения:

1. Преобразует стрелочные функции в обычные функции
2. Преобразует async/await в генераторы
3. Позволяет использовать свойства классов
4. Избегает дублирования полифиллов

🔮 Подсказка:

Подумайте, какой плагин нужен для стрелок, асинхронного кода, классов и оптимизации полифиллов.

- **@babel/plugin-transform-arrow-functions** — Преобразует стрелочные функции в обычные функции
- **@babel/plugin-transform-async-to-generator** — Преобразует async/await в генераторы
- **@babel/plugin-proposal-class-properties** — Позволяет использовать свойства классов
- **@babel/plugin-transform-runtime** — Избегает дублирования полифиллов

🔮 Шаг 07:

Миссия 7

Программист Алекс создает проект на современном JavaScript, но клиент использует старый браузер. Что поможет сделать код совместимым?

Подсказка: Подумайте о том, как современные функции можно использовать в старых средах.

Ответ:

- **Транспиляция через Babel**
- Игнорировать ошибки браузера
- Писать код только для современного браузера

Шаг 08:

Миссия 8

Команда Джейсона хочет использовать стрелочные функции в проекте, который будет работать в IE11. Какой подход безопасен?

Подсказка: Старые браузеры не знают новых синтаксических конструкций.

Ответ:

- **Использовать Babel для транспиляции кода**
- Использовать стрелочные функции напрямую
- Отключить поддержку IE11

Шаг 09:

Миссия 9

Лейла хочет использовать Promise и async/await, но ее проект должен работать на старых браузерах. Что следует добавить?

Подсказка: Некоторые современные функции отсутствуют в старых движках JS.

Ответ:

- **Полифиллы для Promise**
- Игнорировать async/await
- Использовать только колбэки

Шаг 10:

Миссия 10

Николай подключает Babel к сборке проекта. Какой файл отвечает за настройки транспиляции?

Подсказка: Обычно конфигурация находится в корне проекта и описывает правила преобразования кода.

Ответ:

- **.babelrc или babel.config.js**
- package.json без секции Babel
- webpack.config.json

Шаг 11:

Миссия 11

Программист Дима хочет добавить полифилл для функции Array.includes, чтобы код работал в IE11. Что он должен сделать?

Подсказка: Полифиллы расширяют функциональность, которой нет в старых браузерах.

Ответ:

- **Подключить соответствующий полифилл через core-js**
- Игнорировать поддержку IE11

- Переписать весь код без includes

🔗 Шаг 12:

Миссия 12

Сопоставьте термины с их объяснением:

Ответ:

- Транспилиция
 - Полифилл
 - Babel
 - Современный JS
1. Процесс преобразования кода из нового стандарта в старый
 2. Функция или объект, отсутствующий в старых браузерах, который добавляется через скрипт
 3. Инструмент для транспилиции кода
 4. Версия JavaScript с новыми возможностями и синтаксисом

Подсказка: Подумайте, что делает Babel и зачем нужны полифиллы.

- **Транспилиция** — Процесс преобразования кода из нового стандарта в старый
- **Полифилл** — Функция или объект, отсутствующий в старых браузерах, который добавляется через скрипт
- **Babel** — Инструмент для транспилиции кода
- **Современный JS** — Версия JavaScript с новыми возможностями и синтаксисом

🔗 Шаг 13:

Миссия 13

Сопоставьте типы полифиллов с их задачей:

Ответ:

- ES6 Promise
 - Array.from
 - String.includes
 - fetch
1. Добавляет поддержку промисов для асинхронных операций
 2. Преобразует объект в массив, если он не поддерживается
 3. Проверяет наличие подстроки в строке
 4. Позволяет делать HTTP-запросы в старых браузерах

Подсказка: Подумайте, какие методы отсутствуют в старых версиях браузеров и что полифилл добавляет.

- **ES6 Promise** — Добавляет поддержку промисов для асинхронных операций
- **Array.from** — Преобразует объект в массив, если он не поддерживается
- **String.includes** — Проверяет наличие подстроки в строке
- **fetch** — Позволяет делать HTTP-запросы в старых браузерах

🔗 Шаг 14:

Миссия 14

Сопоставьте конфигурации Babel с их назначением:

Ответ:

- @babel/preset-env
 - @babel/plugin-transform-arrow-functions
 - @babel/plugin-transform-async-to-generator
 - @babel/polyfill
1. Настройка для автоматического выбора нужных транспиляций
 2. Транспиляция стрелочных функций
 3. Транспиляция async/await в генераторы
 4. Добавление полифиллов для старых браузеров

Подсказка: Подумайте, что делает preset и отдельные плагины.

- **@babel/preset-env** — Настройка для автоматического выбора нужных транспиляций
- **@babel/plugin-transform-arrow-functions** — Транспиляция стрелочных функций
- **@babel/plugin-transform-async-to-generator** — Транспиляция async/await в генераторы
- **@babel/polyfill** — Добавление полифиллов для старых браузеров

 Шаг 15:

Миссия 15

Сопоставьте термины с их примером использования:

Ответ:

- Полифилл
 - Транспиляция
 - Babel
 - ES6 модуль
1. `import { map } from 'lodash';`
 2. `window.Promise = ...`
 3. Конвертация стрелочной функции в обычную функцию
 4. Инструмент, который выполняет конвертацию кода

Подсказка: Смотрите, что добавляет совместимость, а что преобразует код.

- **Полифилл** — `window.Promise = ...`
- **Транспиляция** — Конвертация стрелочной функции в обычную функцию
- **Babel** — Инструмент, который выполняет конвертацию кода
- **ES6 модуль** — `import { map } from 'lodash';`

 Шаг 16:

Миссия 16

Сопоставьте понятия с их преимуществами:

Ответ:

- Транспиляция
 - Полифилл
 - Современный JS
 - Babel
1. Позволяет писать новый код без ограничений старых браузеров
 2. Обеспечивает совместимость кода с устаревшими браузерами

3. Добавляет недостающий функционал
4. Инструмент, облегчающий процесс конвертации

Подсказка: Подумайте о роли инструментов и новых возможностей языка.

- **Транспиляция** — Обеспечивает совместимость кода с устаревшими браузерами
- **Полифилл** — Добавляет недостающий функционал
- **Современный JS** — Позволяет писать новый код без ограничений старых браузеров
- **Babel** — Инструмент, облегчающий процесс конвертации

Урок 4 - NPM скрипты и package.json

Шаг 01:

Миссия 1

Формулировка: Сопоставьте термины NPM с их назначением.

Ответ:

- dependencies
- devDependencies
- scripts
- main

Определения:

1. Библиотеки, необходимые для работы приложения в продакшене
2. Библиотеки, используемые только для разработки
3. Команды, которые можно запускать через npm run
4. Главный файл проекта

Подсказка:

Подумайте, что используется в продакшене, что для разработки и где прописаны команды.

- **dependencies** — Библиотеки, необходимые для работы приложения в продакшене
- **devDependencies** — Библиотеки, используемые только для разработки
- **scripts** — Команды, которые можно запускать через npm run
- **main** — Главный файл проекта

Шаг 02:

Миссия 2

Формулировка: Сопоставьте команды NPM с их назначением.

Ответ:

- npm install
- npm run
- npm update
- npm uninstall

Определения:

1. Установка пакета
2. Запуск скрипта из package.json
3. Обновление пакета до последней версии
4. Удаление пакета

Подсказка:

Подумайте о последовательности действий: добавить, использовать, обновить, удалить.

- **npm install** — Установка пакета
- **npm run** — Запуск скрипта из package.json
- **npm update** — Обновление пакета до последней версии
- **npm uninstall** — Удаление пакета

Шаг 03:

Миссия 3

Формулировка: Сопоставьте поля package.json с их функцией.

Ответ:

- name
- version
- license
- repository

Определения:

1. Имя проекта
2. Версия проекта
3. Тип лицензии проекта
4. URL репозитория проекта

Подсказка:

Подумайте о том, что идентифицирует проект, его версию, лицензию и где хранится код.

- **name** — Имя проекта
- **version** — Версия проекта
- **license** — Тип лицензии проекта
- **repository** — URL репозитория проекта

Шаг 04:

Миссия 4

Формулировка: Сопоставьте разделы package.json с их назначением.

Ответ:

- dependencies
 - devDependencies
 - scripts
 - main
1. Пакеты, нужные в продакшене
 2. Пакеты, нужные только для разработки
 3. Определение команд npm run
 4. Главный файл проекта

Подсказка:

Подумайте, какие зависимости нужны для разработки, какие для работы, и где хранить скрипты.

- **dependencies** — Пакеты, нужные в продакшене

- **devDependencies** — Пакеты, нужные только для разработки
- **scripts** — Определение команд npm run
- **main** — Главный файл проекта

🔧 Шаг 05:

Миссия 5

Формулировка: Сопоставьте команды npm с их действием.

Ответ:

- npm init
- npm install
- npm run
- npm uninstall

1. Создает новый package.json
2. Устанавливает зависимости
3. Запускает скрипт из package.json
4. Удаляет пакет

🔧 Подсказка:

Подумайте, какие команды создают, устанавливают, запускают или удаляют пакеты.

- **npm init** — Создает новый package.json
- **npm install** — Устанавливает зависимости
- **npm run** — Запускает скрипт из package.json
- **npm uninstall** — Удаляет пакет

🔧 Шаг 06:

Миссия 6

Формулировка: Сопоставьте ключи package.json с их описанием.

Ответ:

- name
- version
- license
- author

1. Имя проекта
2. Версия проекта
3. Лицензия проекта
4. Автор проекта

🔧 Подсказка:

Подумайте, где хранится информация о названии, версии, лицензии и авторе.

- **name** — Имя проекта
- **version** — Версия проекта
- **license** — Лицензия проекта
- **author** — Автор проекта

Шаг 07:

Миссия 7

Алексей хочет быстро запустить локальный сервер для своего проекта через NPM. Какой способ самый простой?

Подсказка: Подумайте, как NPM скрипты могут автоматизировать повторяющиеся команды.

Ответ:

- **Создать скрипт start в package.json**
- Запускать сервер вручную каждый раз
- Создать отдельный bash-файл

Шаг 08:

Миссия 8

Маша хочет установить пакет lodash и использовать его в проекте. Как правильно добавить его через NPM?

Подсказка: Подумайте о разнице между локальной и глобальной установкой пакета.

Ответ:

- **npm install lodash --save**
- npm install lodash -g
- npm add lodash

Шаг 09:

Миссия 9

Дмитрий хочет запустить несколько команд последовательно через NPM скрипт. Как это сделать?

Подсказка: Подумайте о синтаксисе команд в package.json для последовательного выполнения.

Ответ:

- **Использовать оператор && между командами**
- Создать отдельный скрипт на Python
- Запускать каждую команду вручную

Шаг 10:

Миссия 10

Катя хочет обновить все пакеты проекта до последних версий. Какой NPM скрипт или команда поможет ей в этом?

Подсказка: Существует встроенная команда для обновления зависимостей.

Ответ:

- **npm update**
- npm install all
- npm upgrade project

Шаг 11:

Миссия 11

Игорь хочет запустить тесты проекта через NPM. Где лучше всего хранить команду теста?

Подсказка: NPM скрипты позволяют стандартизировать команды проекта для всех участников.

Ответ:

- В разделе **scripts** в **package.json**
- В отдельном **bash**-файле
- В **README.md**

Шаг 12:

Миссия 12

Сопоставьте термины NPM с их объяснением:

Ответ:

- **dependencies**
 - **devDependencies**
 - **scripts**
 - **package.json**
1. Пакеты, необходимые только для разработки
 2. Скрипты для запуска команд
 3. Список пакетов, которые нужны для работы приложения
 4. Файл конфигурации проекта NPM

Подсказка: Подумайте, какие зависимости нужны только во время разработки, а какие — всегда.

- **dependencies** — Список пакетов, которые нужны для работы приложения
- **devDependencies** — Пакеты, необходимые только для разработки
- **scripts** — Скрипты для запуска команд
- **package.json** — Файл конфигурации проекта NPM

Шаг 13:

Миссия 13

Сопоставьте NPM команды с их функцией:

Ответ:

- **npm install**
 - **npm run**
 - **npm uninstall**
 - **npm update**
1. Запуск скриптов из **package.json**
 2. Удаление установленного пакета
 3. Установка всех зависимостей проекта
 4. Обновление пакетов до последних версий

Подсказка: Подумайте о разнице между установкой, обновлением и удалением пакета.

- **npm install** — Установка всех зависимостей проекта
- **npm run** — Запуск скриптов из **package.json**
- **npm uninstall** — Удаление установленного пакета
- **npm update** — Обновление пакетов до последних версий

Шаг 14:

Миссия 14

Сопоставьте типы скриптов NPM с их назначением:

Ответ:

- start
- test
- build
- preinstall

1. Сборка проекта для продакшена
2. Запуск приложения
3. Запуск тестов
4. Команда, выполняемая до установки пакетов

Подсказка: Подумайте, когда каждая команда запускается.

- **start** — Запуск приложения
- **test** — Запуск тестов
- **build** — Сборка проекта для продакшена
- **preinstall** — Команда, выполняемая до установки пакетов

Шаг 15:

Миссия 15

Сопоставьте термины package.json с примерами:

Ответ:

- main
- version
- license
- repository

1. Путь к главному файлу проекта
2. Номер версии проекта
3. Тип лицензии проекта
4. Ссылка на репозиторий

Подсказка: Подумайте, какие поля отвечают за идентификацию и распространение проекта.

- **main** — Путь к главному файлу проекта
- **version** — Номер версии проекта
- **license** — Тип лицензии проекта
- **repository** — Ссылка на репозиторий

Шаг 16:

Миссия 16

Сопоставьте типы зависимостей с их примером:

Ответ:

- dependencies

- devDependencies

1. React для фронтенда в продакшене
2. Webpack для сборки проекта

Подсказка: Подумайте, какие пакеты нужны только во время разработки.

- **dependencies** — React для фронтенда в продакшене
- **devDependencies** — Webpack для сборки проекта

Урок 5 - Разделение кода (Code Splitting)

Шаг 01:

Миссия 1

Алекс решает оптимизировать загрузку своего веб-приложения. Что из перечисленного поможет ускорить первый рендер?

Подсказка: Подумайте о том, как уменьшить размер загружаемого кода для первой страницы.

Ответ:

- **Разделение кода (Code Splitting)**
- Минификация CSS
- Добавление комментариев в JS

Шаг 02:

Миссия 2

Маша хочет загрузить библиотеку lodash только когда она реально нужна на странице. Как лучше организовать импорт?

Подсказка: Подумайте о динамическом импорте модулей.

Ответ:

- **Использовать import() внутри функции при необходимости**
- Добавить lodash в bundle сразу
- Подключить lodash через CDN всегда

Шаг 03:

Миссия 3

Дима хочет разделить код приложения на логические части: пользовательский интерфейс, API и утилиты. Что поможет реализовать такое разделение?

Подсказка: Подумайте, как современные сборщики поддерживают разбивку на чанки.

Ответ:

- **Создать отдельные entry points и динамический импорт**
- Объединить весь код в один файл
- Удалить ненужные комментарии

Шаг 04:

Миссия 4

Катя хочет уменьшить размер основного bundle приложения. Что для этого использовать?

Подсказка: Подумайте о разделении кода и загрузке только нужного.

Ответ:

- **Code Splitting + динамический импорт**
- Увеличение числа функций в одном файле
- Использование alert вместо console.log

Шаг 05:

Миссия 5

Игорь хочет, чтобы пользователи сначала загрузили только страницу логина, а остальное подгружалось по мере необходимости. Как реализовать это?

Подсказка: Подумайте, как делить приложение на маршруты и чанки.

Ответ:

- **Ленивая загрузка маршрутов через dynamic import**
- Загрузить весь код сразу на первую страницу
- Использовать setTimeout для отложенной загрузки

Шаг 06:

Миссия 6

Сопоставьте понятия Code Splitting с их описанием:

Ответ:

- Entry Point
 - Chunk
 - Lazy Loading
 - Bundle
1. Главный файл приложения, с которого начинается сборка
 2. Независимый фрагмент кода, который можно загрузить отдельно
 3. Загрузка кода по необходимости, а не сразу
 4. Объединённые файлы приложения после сборки

Подсказка: Подумайте о том, что загружается сразу, а что — позже.

- **Entry Point** — Главный файл приложения, с которого начинается сборка
- **Chunk** — Независимый фрагмент кода, который можно загрузить отдельно
- **Lazy Loading** — Загрузка кода по необходимости, а не сразу
- **Bundle** — Объединённые файлы приложения после сборки

Шаг 07:

Миссия 7

Сопоставьте типы Code Splitting с примером:

Ответ:

- Entry Point Splitting
 - Route-based Splitting
 - Component-based Splitting
1. Разделение приложения по основным файлам входа
 2. Разделение кода по маршрутам (страницам)
 3. Разделение отдельных компонентов для динамической загрузки

Подсказка: Подумайте, что загружается первым, что по маршруту, а что по компоненту.

- **Entry Point Splitting** — Разделение приложения по основным файлам входа
- **Route-based Splitting** — Разделение кода по маршрутам (страницам)
- **Component-based Splitting** — Разделение отдельных компонентов для динамической загрузки

Шаг 08:

Миссия 8

Сопоставьте термины Webpack и Code Splitting:

Ответ:

- Dynamic Import
 - Optimization SplitChunks
 - Async Module
1. Импорт модуля по необходимости во время выполнения
 2. Автоматическое разбиение кода на чанки для уменьшения размера bundle
 3. Модуль, который загружается асинхронно

Подсказка: Подумайте о том, что Webpack умеет делать автоматически, а что нужно вызвать вручную.

- **Dynamic Import** — Импорт модуля по необходимости во время выполнения
- **Optimization SplitChunks** — Автоматическое разбиение кода на чанки для уменьшения размера bundle
- **Async Module** — Модуль, который загружается асинхронно

Шаг 09:

Миссия 9

Алекс хочет, чтобы его веб-приложение загружалось быстрее. Какой подход поможет сразу разделить код на части?

Подсказка: Подумайте, как можно загружать код только по мере необходимости.

Ответ:

- **Использовать Code Splitting через динамический import**
- Объединить весь код в один файл
- Удалить все комментарии из кода

Шаг 10:

Миссия 10

Мария использует Webpack. Какой метод помогает разбить код на чанки для разных страниц?

Подсказка: Подумайте о настройках Webpack и разделении точек входа.

Ответ:

- **Настроить entry points и использовать dynamic import**
- Добавить все скрипты в один entry point
- Отключить source maps

Шаг 11:

Миссия 11

Игорь хочет, чтобы библиотека, используемая редко, загружалась только при необходимости. Что ему выбрать?

Подсказка: Подумайте о преимуществах динамической загрузки модулей.

Ответ:

- **Динамический `import()` для этой библиотеки**
- Подключить библиотеку через `script` тег в HTML
- Встроить библиотеку в главный бандл

🔗 Шаг 12:

Миссия 12

Катя использует React и хочет разбивать страницы на отдельные чанки. Что из ниже перечисленного подходит?

Подсказка: Подумайте о `React.lazy` и `Suspense`.

Ответ:

- **`React.lazy()` с динамическим `import`**
- Использовать `setState` для загрузки компонентов
- Вставлять все компоненты на одной странице

🔗 Шаг 13:

Миссия 13

Дмитрий хочет, чтобы библиотека `moment.js` грузилась только на страницах с календарем. Как это реализовать?

Подсказка: Подумайте, как отложенная загрузка уменьшает размер главного бандла.

Ответ:

- **Импортировать через `import()` внутри компонента календаря**
- Добавить `moment.js` в основной бандл
- Подключить момент через CDN на всех страницах

🔗 Шаг 14:

Миссия 14

Сопоставьте термины Code Splitting с их объяснением:

Ответ:

- Entry Points
 - Dynamic Import
 - Vendor Chunk
 - Lazy Loading
1. Файл с зависимостями сторонних библиотек
 2. Точка входа для сборки Webpack
 3. Загрузка модуля только при необходимости
 4. Динамическое подключение модуля через `import()`

Подсказка: Подумайте о том, как Webpack разбивает код и когда модули реально загружаются.

- **Entry Points** — Точка входа для сборки Webpack
- **Dynamic Import** — Динамическое подключение модуля через `import()`
- **Vendor Chunk** — Файл с зависимостями сторонних библиотек

- **Lazy Loading** — Загрузка модуля только при необходимости

Шаг 15:

Миссия 15

Сопоставьте типы разделения кода с их назначением:

Ответ:

- Entry Split
- Route Split
- Component Split
- Vendor Split

1. Разделение по страницам приложения
2. Разделение сторонних библиотек
3. Разделение отдельных компонентов для отложенной загрузки
4. Отдельные бандлы для каждой точки входа

Подсказка: Подумайте, какие бандлы создаются для страниц, компонентов и библиотек.

- **Entry Split** — Отдельные бандлы для каждой точки входа
- **Route Split** — Разделение по страницам приложения
- **Component Split** — Разделение отдельных компонентов для отложенной загрузки
- **Vendor Split** — Разделение сторонних библиотек

Шаг 16:

Миссия 16

Сопоставьте термины Webpack с их функцией в Code Splitting:

Ответ:

- chunkFilename
- optimization.splitChunks
- import()
- output.filename

1. Шаблон имени для чанков, созданных динамическим импортом
2. Конфигурация разделения кода на отдельные чанки
3. Динамическое подключение модуля
4. Имя основного бандла

Подсказка: Подумайте, как Webpack настраивает имена файлов и управление чанками.

- **chunkFilename** — Шаблон имени для чанков, созданных динамическим импортом
- **optimization.splitChunks** — Конфигурация разделения кода на отдельные чанки
- **import()** — Динамическое подключение модуля
- **output.filename** — Имя основного бандла

Модуль 9 - Тестирование

Урок 1 - Введение в тестирование

Шаг 01:

Миссия 1

Аня пишет первую функцию для калькулятора. Какой тип теста лучше всего подойдет для проверки, что функция правильно складывает числа?

Подсказка: Подумайте, какой тест проверяет конкретную функциональность без других зависимостей.

Ответ:

- **Unit-тест**
- E2E-тест
- Интеграционный тест

Шаг 02:

Миссия 2

Иван хочет проверить работу кнопки отправки формы на сайте. Какой тип теста ему использовать?

Подсказка: Подумайте о тесте, который проверяет сценарий от начала до конца.

Ответ:

- **E2E-тест**
- Unit-тест
- Снапшот-тест

Шаг 03:

Миссия 3

Светлана использует Jest. Какой способ позволит проверить, что функция возвращает ожидаемый результат для определенных входных данных?

Подсказка: Подумайте о проверке конкретного значения функции.

Ответ:

- **expect(value).toBe(expected)**
- console.log(value)
- alert(value)

Шаг 04:

Миссия 4

Петр хочет проверить взаимодействие двух модулей. Какой тип теста подойдет лучше всего?

Подсказка: Подумайте о тесте, который проверяет работу нескольких компонентов вместе.

Ответ:

- **Интеграционный тест**
- Unit-тест
- UI-тест

Шаг 05:

Миссия 5

Ольга пишет тесты для API. Какой подход позволит проверять, что сервер возвращает правильный статус и данные?

Подсказка: Подумайте о тестах, которые взаимодействуют с внешним сервисом.

Ответ:

- **Интеграционные тесты API**
- Unit-тест функции клиента
- Спаршот тест

Шаг 06:

Миссия 6

Сопоставьте тип теста с его описанием:

Ответ:

- Unit-тест
 - Интеграционный тест
 - E2E-тест
 - Снапшот-тест
1. Проверяет отдельную функцию или модуль
 2. Проверяет взаимодействие нескольких модулей
 3. Проверяет сценарий от начала до конца на уровне приложения
 4. Сравнивает текущий вывод с сохранённым эталоном

Подсказка: Подумайте о масштабе тестирования и его цели.

- **Unit-тест** — Проверяет отдельную функцию или модуль
- **Интеграционный тест** — Проверяет взаимодействие нескольких модулей
- **E2E-тест** — Проверяет сценарий от начала до конца на уровне приложения
- **Снапшот-тест** — Сравнивает текущий вывод с сохранённым эталоном

Шаг 07:

Миссия 7

Сопоставьте инструмент тестирования с его типичными задачами:

Ответ:

- Jest
 - Mocha
 - Cypress
 - React Testing Library
1. Тестирование React-компонентов
 2. Юнит и интеграционные тесты, снапшоты
 3. Тестирование браузерных сценариев E2E
 4. Юнит-тесты с гибкой конфигурацией

Подсказка: Обратите внимание на то, где чаще всего используется каждый инструмент.

- **Jest** — Юнит и интеграционные тесты, снапшоты
- **Mocha** — Юнит-тесты с гибкой конфигурацией
- **Cypress** — Тестирование браузерных сценариев E2E
- **React Testing Library** — Тестирование React-компонентов

Шаг 08:

Миссия 8

Аня только начала изучать тестирование. Что из нижеперечисленного является основной целью тестов?

Подсказка: Подумайте, для чего программист проверяет код.

Ответ:

- **Убедиться, что код работает корректно**
- Сделать код красивым
- Добавить комментарии к функциям

Шаг 09:

Миссия 9

Борис хочет, чтобы ошибки в коде обнаруживались автоматически. Что ему нужно использовать?

Подсказка: Подумайте о программах, которые проверяют код без участия человека.

Ответ:

- **Автоматические тесты**
- Ручное чтение кода
- Проверка кода через консоль браузера

Шаг 10:

Миссия 10

Света хочет убедиться, что функция сложения всегда возвращает правильный результат. Какой вид теста ей подходит?

Подсказка: Подумайте о проверке конкретной функции с заранее известными результатами.

Ответ:

- **Юнит-тест**
- Интеграционный тест
- Системный тест

Шаг 11:

Миссия 11

Денис хочет протестировать работу всей формы регистрации сразу, включая взаимодействие всех полей. Какой вид теста использовать?

Подсказка: Подумайте о тестах, которые проверяют систему целиком.

Ответ:

- **Интеграционный тест**
- Юнит-тест
- Стресс-тест

Шаг 12:

Миссия 12

Елена хочет, чтобы тесты запускались при каждом изменении кода автоматически. Как это сделать?

Подсказка: Подумайте о средствах непрерывной проверки кода.

Ответ:

- **Настроить watch mode для тестов**
- Запускать тесты вручную раз в неделю
- Добавить alert() после каждой функции

Шаг 13:

Миссия 13

Сопоставьте термины тестирования с их описанием:

Ответ:

- Юнит-тест
 - Интеграционный тест
 - Системный тест
 - Регрессионный тест
1. Проверка отдельных функций или модулей
 2. Проверка взаимодействия нескольких компонентов
 3. Проверка всей системы целиком
 4. Повторная проверка после изменений, чтобы убедиться, что старый код не сломался

Подсказка: Подумайте, какие тесты проверяют только часть кода, а какие — весь проект.

- **Юнит-тест** — Проверка отдельных функций или модулей
- **Интеграционный тест** — Проверка взаимодействия нескольких компонентов
- **Системный тест** — Проверка всей системы целиком
- **Регрессионный тест** — Повторная проверка после изменений, чтобы убедиться, что старый код не сломался

Шаг 14:

Миссия 14

Сопоставьте типы тестов с целью их применения:

Ответ:

- Smoke Test
 - Acceptance Test
 - Stress Test
 - Performance Test
1. Проверка основных функций системы без глубокого анализа
 2. Проверка, соответствует ли продукт требованиям заказчика
 3. Проверка системы при экстремальных нагрузках
 4. Проверка быстродействия и использования ресурсов

Подсказка: Подумайте, какие тесты направлены на стабильность, а какие — на требования или производительность.

- **Smoke Test** — Проверка основных функций системы без глубокого анализа
- **Acceptance Test** — Проверка, соответствует ли продукт требованиям заказчика
- **Stress Test** — Проверка системы при экстремальных нагрузках
- **Performance Test** — Проверка быстродействия и использования ресурсов

Урок 2 - Jest Vitest Testing Library

Шаг 01:

Миссия 1

Аня только начала изучать тестирование. Что является основной целью использования Jest или Vitest?

Подсказка: Подумайте, для чего программисты пишут тесты.

Ответ:

- **Проверить корректность работы функций**
- Сделать код красивым
- Добавить комментарии

Шаг 02:

Миссия 2

Борис хочет запускать тесты автоматически при каждом изменении кода. Что ему поможет?

Подсказка: Подумайте о средствах непрерывной проверки кода.

Ответ:

- **Watch mode в Jest или Vitest**
- Ручной запуск команд
- Консольный alert после функций

Шаг 03:

Миссия 3

Света хочет проверить, что функция сложения возвращает правильный результат для разных входных данных. Какой тип теста использовать?

Подсказка: Подумайте о проверке одной функции на наборе входных значений.

Ответ:

- **Юнит-тест**
- Интеграционный тест
- E2E-тест

Шаг 04:

Миссия 4

Денис хочет проверить всю форму регистрации целиком, включая все поля и кнопки. Какой тип теста выбрать?

Подсказка: Подумайте о тестах, которые проверяют взаимодействие нескольких компонентов.

Ответ:

- **Интеграционный тест**
- Юнит-тест
- Системный тест

Шаг 05:

Миссия 5

Елена хочет проверять корректность рендеринга кнопки и её событий клика с Testing Library. Что ей нужно тестировать?

Подсказка: Подумайте о тестировании поведения UI, а не только функций.

Ответ:

- **Рендер и взаимодействие с элементом через события**
- Только внутреннюю функцию, не UI
- CSS-стили кнопки

Шаг 06:

Миссия 6

Сопоставьте инструменты тестирования с их назначением:

Ответ:

- Jest
 - Vitest
 - Testing Library
 - Playwright
1. Юнит-тесты и снапшоты для JS/TS
 2. Современный быстрый тестовый фреймворк для JS/TS
 3. Тестирование поведения UI-компонентов
 4. End-to-end тестирование веб-приложений

Подсказка: Подумайте, какой инструмент подходит для фронтенда, а какой — для E2E.

- **Jest** — Юнит-тесты и снапшоты для JS/TS
- **Vitest** — Современный быстрый тестовый фреймворк для JS/TS
- **Testing Library** — Тестирование поведения UI-компонентов
- **Playwright** — End-to-end тестирование веб-приложений

Шаг 07:

Миссия 7

Сопоставьте методы Jest с их назначением:

Ответ:

- describe()
 - test()
 - expect()
 - beforeEach()
1. Группирует тесты по смыслу
 2. Определяет отдельный тест-кейс
 3. Сравнивает результат с ожидаемым
 4. Выполняет код перед каждым тестом

Подсказка: Подумайте, какие функции структурируют тесты, а какие проверяют результаты.

- **describe()** — Группирует тесты по смыслу
- **test()** — Определяет отдельный тест-кейс
- **expect()** — Сравнивает результат с ожидаемым
- **beforeEach()** — Выполняет код перед каждым тестом

🔗 Шаг 08:

Миссия 8

Сопоставьте утверждения с понятием Testing Library:

Ответ:

- Фокус на пользователе
 - Тестирование DOM элементов
 - Не проверяет внутренние функции
 - Работает с Jest или Vitest
1. Сосредоточена на поведении и взаимодействии, а не на реализации
 2. Используется для проверки элементов страницы
 3. Позволяет писать тесты без обращения к внутреннему коду компонента
 4. Интегрируется с любым тестовым фреймворком

Подсказка: Подумайте, на что ориентированы тесты Testing Library.

- **Фокус на пользователе** — Сосредоточена на поведении и взаимодействии, а не на реализации
- **Тестирование DOM элементов** — Используется для проверки элементов страницы
- **Не проверяет внутренние функции** — Позволяет писать тесты без обращения к внутреннему коду компонента
- **Работает с Jest или Vitest** — Интегрируется с любым тестовым фреймворком

🔗 Шаг 09:

Миссия 9

Аня создает свой первый юнит-тест с Jest. Что из перечисленного она должна проверить в первую очередь?

Подсказка: Подумайте, с чего обычно начинают тестирование функции.

Ответ:

- **Что функция возвращает ожидаемый результат**
- Как красиво оформлен код функции
- Сколько строк занимает функция

🔗 Шаг 10:

Миссия 10

Борис хочет использовать Vitest для тестирования компонентов React. Какой принцип важен для юнит-тестов?

Подсказка: Подумайте, что делает юнит-тест независимым от остального приложения.

Ответ:

- **Тестировать одну функцию или компонент отдельно**
- Тестировать весь проект сразу
- Игнорировать ошибки в зависимостях

🔗 Шаг 11:

Миссия 11

Света использует Testing Library для тестирования интерфейса. Что она должна проверять?

Подсказка: Подумайте, что важнее для пользователя при взаимодействии с интерфейсом.

Ответ:

- **Правильное отображение элементов и реакция на действия пользователя**
- Внутренние переменные компонента
- Стили CSS компонентов

Шаг 12:

Миссия 12

Денис хочет запускать тесты автоматически при каждом коммите. Какой инструмент ему использовать?

Подсказка: Подумайте о системах, которые интегрируются с git.

Ответ:

- **CI/CD с запуском тестов при коммитах**
- Ручной запуск тестов каждый день
- Проверка кода через alert()

Шаг 13:

Миссия 13

Сопоставьте термины тестирования с их объяснением:

Ответ:

- Jest
 - Vitest
 - Testing Library
 - Mock
1. Фреймворк для юнит-тестов в JavaScript с поддержкой снапшотов
 2. Легковесный инструмент для тестирования, совместимый с Vite
 3. Библиотека для тестирования интерфейса с фокусом на поведение пользователя
 4. Заглушка для зависимостей в тестах

Подсказка: Подумайте, какой инструмент тестирует фронтенд, а какой — backend или модуль.

- **Jest** — Фреймворк для юнит-тестов в JavaScript с поддержкой снапшотов
- **Vitest** — Легковесный инструмент для тестирования, совместимый с Vite
- **Testing Library** — Библиотека для тестирования интерфейса с фокусом на поведение пользователя
- **Mock** — Заглушка для зависимостей в тестах

Шаг 14:

Миссия 14

Сопоставьте функции Jest/Vitest с их назначением:

Ответ:

- describe
 - test / it
 - beforeEach
 - afterEach
1. Группировка связанных тестов
 2. Определение отдельного теста

3. Выполняется перед каждым тестом
4. Выполняется после каждого теста

Подсказка: Подумайте, какие функции управляют порядком выполнения тестов.

- **describe** — Группировка связанных тестов
- **test / it** — Определение отдельного теста
- **beforeEach** — Выполняется перед каждым тестом
- **afterEach** — Выполняется после каждого теста

🔗 Шаг 15:

Миссия 15

Сопоставьте утверждения (assertions) с их назначением:

Ответ:

- `expect(value).toBe(expected)`
 - `expect(value).toEqual(expected)`
 - `expect(array).toContain(item)`
 - `expect(fn).toThrow()`
1. Сравнение примитивных значений
 2. Глубокое сравнение объектов или массивов
 3. Проверка наличия элемента в массиве
 4. Проверка, что функция выбрасывает ошибку

Подсказка: Подумайте, какой метод проверяет массивы, объекты или ошибки.

- **`expect(value).toBe(expected)`** — Сравнение примитивных значений
- **`expect(value).toEqual(expected)`** — Глубокое сравнение объектов или массивов
- **`expect(array).toContain(item)`** — Проверка наличия элемента в массиве
- **`expect(fn).toThrow()`** — Проверка, что функция выбрасывает ошибку

🔗 Шаг 16:

Миссия 16

Расположите шаги написания теста с Testing Library в правильном порядке:

1. Рендер компонента
2. Взаимодействие с элементами (клик, ввод текста)
3. Проверка ожидаемого результата
4. Очистка после теста

Подсказка: Подумайте, сначала нужно показать компонент, потом совершить действия, проверить результат и очистить после теста.

Ответ:

- Рендер компонента
- Взаимодействие с элементами (клик, ввод текста)
- Проверка ожидаемого результата
- Очистка после теста

Урок 3 - МокаЧайСинай

Шаг 01:

Миссия 1

Аня тестирует функцию с использованием Mock. Что она может замещать с помощью Mock?

Подсказка: Подумайте, зачем временно подменять зависимости.

Ответ:

- **Внешние сервисы и функции, чтобы тестировать компонент изолированно**
- Встроенные переменные функции
- CSS стили компонента

Шаг 02:

Миссия 2

Борис использует Spy для отслеживания вызовов функции. Что он хочет узнать?

Подсказка: Spy не изменяет поведение функции, а лишь наблюдает.

Ответ:

- **Сколько раз и с какими аргументами была вызвана функция**
- Изменить реализацию функции
- Проверить правильность CSS

Шаг 03:

Миссия 3

Света создает stub для функции fetch. Какова цель stub?

Подсказка: Подумайте, чем stub отличается от mock.

Ответ:

- **Возвращать предсказуемые данные вместо реального вызова**
- Следить за вызовами функции
- Изменять CSS компонента

Шаг 04:

Миссия 4

Денис использует Sinon для тестирования асинхронной функции. Что он может сделать с помощью Sinon?

Подсказка: Sinon позволяет подменять и наблюдать поведение функций.

Ответ:

- **Создать spy, stub или mock для асинхронной функции**
- Изменить CSS компонента
- Создать тест без подключения библиотеки

Шаг 05:

Миссия 5

Сопоставьте термины с их назначением:

Ответ:

- Mock
 - Stub
 - Spy
 - Fake
1. Объект или функция, которая имитирует поведение настоящего, возвращая заранее заданные данные
 2. Объект или функция, позволяющая отслеживать вызовы без изменения поведения
 3. Объект или функция, которая подменяет реальные зависимости и управляет их поведением
 4. Полная имитация системы или компонента с рабочей логикой, но упрощенной

Подсказка: Подумайте, какой инструмент нужен для наблюдения, а какой — для подмены данных.

- **Mock** — Объект или функция, которая подменяет реальные зависимости и управляет их поведением
- **Stub** — Объект или функция, которая имитирует поведение настоящего, возвращая заранее заданные данные
- **Spy** — Объект или функция, позволяющая отслеживать вызовы без изменения поведения
- **Fake** — Полная имитация системы или компонента с рабочей логикой, но упрощенной

 Шаг 06:

Миссия 6

Сопоставьте методы Sinon с их назначением:

Ответ:

- sinon.spy()
 - sinon.stub()
 - sinon.mock()
 - sinon.fake()
1. Создает шпион для отслеживания вызовов функции
 2. Подменяет функцию или метод для возврата заданных значений
 3. Создает объект mock с ожиданиями вызовов
 4. Создает полную имитацию функции с простой логикой

Подсказка: Подумайте, какой метод используется для проверки вызовов, а какой — для имитации поведения.

- **sinon.spy()** — Создает шпион для отслеживания вызовов функции
- **sinon.stub()** — Подменяет функцию или метод для возврата заданных значений
- **sinon.mock()** — Создает объект mock с ожиданиями вызовов
- **sinon.fake()** — Создает полную имитацию функции с простой логикой

 Шаг 07:

Миссия 7

Сопоставьте сценарий с правильным инструментом:

Ответ:

- Проверка вызова функции без изменения логики
- Подмена HTTP-запроса в тесте

- Сложная имитация всей системы для интеграционных тестов
- Возврат предсказуемых данных вместо реальной функции

1. Spy
2. Stub
3. Fake
4. Mock

Подсказка: Подумайте, что важно: наблюдать, подменять или имитировать полностью.

- **Проверка вызова функции без изменения логики** — Spy
- **Подмена HTTP-запроса в тесте** — Stub
- **Сложная имитация всей системы для интеграционных тестов** — Fake
- **Возврат предсказуемых данных вместо реальной функции** — Mock

🔗 Шаг 08:

Миссия 8

Алексей создает мок для функции, которая обращается к API. Что он должен проверить в первую очередь?

Подсказка: Подумайте, какую цель преследуют моки.

Ответ:

- **Функция возвращает ожидаемые данные без реального запроса**
- Функция вызывает реальный API
- Функция выполняется медленнее обычного

🔗 Шаг 09:

Миссия 9

Марина использует Mock для тестирования ошибок API. Какое поведение важно проверить?

Подсказка: Подумайте о самых частых ошибках при вызове внешних сервисов.

Ответ:

- **Что функция корректно обрабатывает ошибки, не падает приложение**
- Что функция всегда возвращает пустой объект
- Что функция запускается в отдельном потоке

🔗 Шаг 10:

Миссия 10

Иван использует библиотеку Chai для ассертов. Какой тип проверки подходит для сравнения объектов?

Подсказка: Подумайте, какой метод проверяет глубокое равенство, а не ссылку на объект.

Ответ:

- **expect(obj).to.deep.equal(expected)**
- expect(obj).to.equal(expected)
- expect(obj).to.be.true

🔗 Шаг 11:

Миссия 11

Сергей хочет создать мок для функции с таймаутом. Что важно учитывать при тестировании?

Подсказка: Подумайте, что может влиять на скорость выполнения теста.

Ответ:

- **Мок должен возвращать данные сразу, не дожидаясь таймаута**
- Мок должен эмулировать полный таймаут
- Мок не должен использовать функции обратного вызова

Шаг 12:

Миссия 12

Сопоставьте термины тестирования с их объяснением:

Ответ:

- Mock
 - Spy
 - Stub
 - Chai
1. Заглушка для имитации поведения функции
 2. Следит за вызовами функции и аргументами
 3. Заглушка, возвращающая заранее определённые значения
 4. Библиотека для выражения ожиданий и ассертов

Подсказка: Подумайте, какая концепция проверяет вызовы, а какая — значения.

- **Mock** — Заглушка для имитации поведения функции
- **Spy** — Следит за вызовами функции и аргументами
- **Stub** — Заглушка, возвращающая заранее определённые значения
- **Chai** — Библиотека для выражения ожиданий и ассертов

Шаг 13:

Миссия 13

Сопоставьте типы проверок с их назначением:

Ответ:

- toBe
 - toEqual
 - toHaveBeenCalled
 - toThrow
1. Проверка примитивных значений
 2. Глубокое сравнение объектов или массивов
 3. Проверка, что функция была вызвана
 4. Проверка, что функция выбрасывает ошибку

Подсказка: Подумайте, какие проверки проверяют вызовы, а какие — значения.

- **toBe** — Проверка примитивных значений
- **toEqual** — Глубокое сравнение объектов или массивов
- **toHaveBeenCalled** — Проверка, что функция была вызвана
- **toThrow** — Проверка, что функция выбрасывает ошибку

Шаг 14:

Миссия 14

Расположите шаги использования мока в правильном порядке:

1. Создание мока для функции или сервиса
2. Настройка возвращаемых значений или поведения
3. Использование мока в тесте
4. Проверка вызовов и результатов

Подсказка: Подумайте, сначала создаём мок, настраиваем, используем, а потом проверяем результаты.

Ответ:

- Создание мока для функции или сервиса
- Настройка возвращаемых значений или поведения
- Использование мока в тесте
- Проверка вызовов и результатов

Шаг 15:

Миссия 15

Расположите шаги тестирования функции с chai и моками:

1. Создать мок функции
2. Вызвать функцию с моками
3. Сравнить результат с ожидаемым с помощью chai
4. Проверить, что мок был вызван корректно

Подсказка: Подумайте: сначала мок, затем вызов, потом ассерты и проверка вызова мока.

Ответ:

- Создать мок функции
- Вызвать функцию с моками
- Сравнить результат с ожидаемым с помощью chai
- Проверить, что мок был вызван корректно

Урок 4 - Покрывание кода, snapshot-тесты

Шаг 01:

Миссия 1

Алекс создает тесты для функции, которая возвращает список пользователей. Что поможет проверить, что все строки кода выполнены?

Подсказка: Подумайте, что показывает покрытие кода.

Ответ:

- **Использовать инструмент покрытия кода (coverage)**
- Прогонять функцию вручную в консоли
- Просто смотреть на код и догадаться

Шаг 02:

Миссия 2

Маша хочет убедиться, что ее компонент React не сломался после изменений. Какой подход подойдет лучше всего?

Подсказка: Подумайте о способе зафиксировать внешний вид компонента.

Ответ:

- **Использовать snapshot-тестирование**
- Прогнать код через ESLint
- Сделать console.log всех элементов

🔮 Шаг 03:

Миссия 3

Игорь видит, что покрытие кода его функций низкое. Что следует сделать в первую очередь?

Подсказка: Подумайте, где чаще всего возникают пробелы в тестах.

Ответ:

- **Написать тесты для ветвлений и обработчиков ошибок**
- Удалить неиспользуемые функции
- Переписать все функции в одну строку

🔮 Шаг 04:

Миссия 4

Анна хочет зафиксировать состояние формы при тестировании. Что для этого используют?

Подсказка: Подумайте о способе сохранять ожидаемый результат в файл.

Ответ:

- **Сделать snapshot-тест компонента**
- Прогонять форму вручную
- Сравнивать объект с undefined

🔮 Шаг 05:

Миссия 5

Сопоставьте понятия с объяснениями:

Ответ:

- Coverage
- Snapshot
- Regression testing
- Branch coverage

1. Фиксация текущего состояния компонента для последующего сравнения
2. Проверка, что новые изменения не ломают старый код
3. Процент выполненного кода при тестах
4. Покрытие всех ветвлений и условий

Подсказка: Обратите внимание, какие понятия связаны с количеством кода, а какие — с тестовыми состояниями.

- **Coverage** — Процент выполненного кода при тестах
- **Snapshot** — Фиксация текущего состояния компонента для последующего сравнения
- **Regression testing** — Проверка, что новые изменения не ломают старый код
- **Branch coverage** — Покрытие всех ветвлений и условий

Шаг 06:

Миссия 6

Сопоставьте типы snapshot-тестов с их применением:

Ответ:

- UI snapshot
 - Data snapshot
 - Function snapshot
 - Full app snapshot
1. Компоненты интерфейса
 2. Результаты функций или методов
 3. Состояние данных (например, JSON)
 4. Весь проект целиком

Подсказка: Подумайте, что проверяется визуально, а что логикой и данными.

- **UI snapshot** — Компоненты интерфейса
- **Data snapshot** — Состояние данных (например, JSON)
- **Function snapshot** — Результаты функций или методов
- **Full app snapshot** — Весь проект целиком

Шаг 07:

Миссия 7

Расположите шаги snapshot-тестирования в правильном порядке:

1. Создание компонента/функции
2. Фиксация текущего состояния (snapshot)
3. Внесение изменений в компонент/функцию
4. Сравнение нового состояния с сохранённым snapshot

Подсказка: Подумайте: сначала создаем, потом сохраняем, затем меняем, в конце — проверяем соответствие.

Ответ:

- Создание компонента/функции
- Фиксация текущего состояния (snapshot)
- Внесение изменений в компонент/функцию
- Сравнение нового состояния с сохранённым snapshot

Шаг 08:

Миссия 8

Расположите шаги повышения покрытия кода:

1. Определить непокрытые ветки и условия
2. Написать тесты для всех ветвлений
3. Прогнать тесты и проверить отчёт покрытия
4. Исправить ошибки и непокрытые участки

Подсказка: Подумайте логически: сначала выявляем пробелы, затем тестируем, проверяем результат и исправляем.

Ответ:

- Определить непокрытые ветки и условия
- Написать тесты для всех ветвлений
- Прогнать тесты и проверить отчёт покрытия
- Исправить ошибки и непокрытые участки

 Шаг 09:

Миссия 9

Алексей добавил первый тест в проект. Зачем важно смотреть на покрытие кода?

Подсказка: Подумайте, как тесты помогают выявлять неиспользуемый код.

Ответ:

- **Чтобы понять, какие части кода тестируются, а какие нет**
- Чтобы увеличить скорость выполнения кода
- Чтобы уменьшить размер проекта

 Шаг 10:

Миссия 10

Марина использует Jest и видит, что coverage низкий. Что лучше сделать в первую очередь?

Подсказка: Подумайте, какие тесты помогают повысить покрытие.

Ответ:

- **Добавить тесты для непокрытых функций**
- Удалить неиспользуемые функции
- Сократить количество существующих тестов

 Шаг 11:

Миссия 11

Иван решил использовать snapshot-тесты для компонента React. Что проверяет snapshot?

Подсказка: Подумайте, что snapshot фиксирует на момент теста.

Ответ:

- **Соответствие рендеринга компонента ожидаемому виду**
- Производительность компонента
- Время загрузки компонента

 Шаг 12:

Миссия 12

Сергей обнаружил, что snapshot-тест сломался после изменения UI. Что нужно сделать?

Подсказка: Подумайте, когда обновлять snapshot безопасно.

Ответ:

- **Проверить изменения и обновить snapshot, если UI изменился корректно**
- Игнорировать тест
- Удалить все snapshot-тесты

 Шаг 13:

Миссия 13

Сопоставьте термины с их объяснением:

Ответ:

- Coverage
- Snapshot-тест
- Branch coverage
- Function coverage

1. Покрытие ветвлений в коде
2. Проверка, что функции вызываются с правильными результатами
3. Процент кода, который выполняется тестами
4. Фиксация текущего состояния компонента или объекта

Подсказка: Обратите внимание, какой термин связан с визуальным состоянием, а какой с метрикой покрытия.

- **Coverage** — Процент кода, который выполняется тестами
- **Snapshot-тест** — Фиксация текущего состояния компонента или объекта
- **Branch coverage** — Покрытие ветвлений в коде
- **Function coverage** — Проверка, что функции вызываются с правильными результатами

Шаг 14:

Миссия 14

Сопоставьте инструменты с их назначением:

Ответ:

- Jest
- Vitest
- Testing Library
- Enzyme

1. Тестирование компонентов с акцентом на пользовательский интерфейс
2. Библиотека для написания юнит-тестов с coverage и snapshot
3. Легковесная альтернатива Jest с поддержкой Vite
4. Утилиты для тестирования React-компонентов

Подсказка: Подумайте, какой инструмент фокусируется на UI, а какой — на юнит-тестах.

- **Jest** — Библиотека для написания юнит-тестов с coverage и snapshot
- **Vitest** — Легковесная альтернатива Jest с поддержкой Vite
- **Testing Library** — Тестирование компонентов с акцентом на пользовательский интерфейс
- **Enzyme** — Утилиты для тестирования React-компонентов

Шаг 15:

Миссия 15

Расположите шаги написания snapshot-теста в правильном порядке:

1. Создать компонент или объект для теста
2. Рендерить компонент и сохранить snapshot
3. Запустить тест и сравнить текущий результат с snapshot
4. Обновить snapshot, если изменения корректны

Подсказка: Сначала создаем объект, рендерим, проверяем и обновляем snapshot.

Ответ:

- Создать компонент или объект для теста
- Рендерить компонент и сохранить snapshot
- Запустить тест и сравнить текущий результат с snapshot
- Обновить snapshot, если изменения корректны

 Шаг 16:

Миссия 16

Расположите шаги анализа coverage после тестов:

1. Посмотреть отчёт coverage
2. Определить непокрытые функции и ветвления
3. Добавить тесты для непокрытого кода
4. Перезапустить тесты и проверить увеличение coverage

Подсказка: Сначала смотрим на отчёт, выявляем пробелы, добавляем тесты и проверяем снова.

Ответ:

- Посмотреть отчёт coverage
- Определить непокрытые функции и ветвления
- Добавить тесты для непокрытого кода
- Перезапустить тесты и проверить увеличение coverage

 Урок 5 - TDD (Test Driven Development)

 Шаг 01:

Миссия 1

Алексей решил попробовать TDD. Что нужно делать первым шагом в TDD?

Подсказка: Подумайте, с чего начинается любой тест.

Ответ:

- **Написать тест, который пока не проходит**
- Сразу написать рабочий код
- Проверить покрытие кода

 Шаг 02:

Миссия 2

Марина пишет тест для новой функции. Как понять, что тест написан правильно?

Подсказка: Вспомните принцип «красный/зелёный» в TDD.

Ответ:

- **Тест должен сначала падать, если функция ещё не реализована**
- Тест сразу должен проходить
- Тест можно не запускать до написания функции

 Шаг 03:

Миссия 3

Иван добавил новый тест и функция прошла тест. Какой следующий шаг в TDD?

Подсказка: Подумайте, как улучшить код после проверки функциональности.

Ответ:

- **Рефакторинг кода, чтобы сделать его чистым и поддерживаемым**
- Удалить тест
- Игнорировать результаты теста

🔮 **Шаг 04:**

Миссия 4

Сергей пишет тесты на существующую функцию. Зачем важно писать тесты до рефакторинга?

Подсказка: Подумайте, как TDD защищает от случайных ошибок.

Ответ:

- **Чтобы убедиться, что функциональность сохраняется после изменений**
- Чтобы ускорить написание кода
- Чтобы уменьшить количество функций

🔮 **Шаг 05:**

Миссия 5

Сопоставьте шаги TDD с их описанием:

Ответ:

- Red
 - Green
 - Refactor
1. Написание минимального кода для прохождения теста
 2. Написание теста, который пока не проходит
 3. Улучшение кода без изменения функциональности

Подсказка: Красный тест сначала падает, зелёный — проходит, а рефакторинг — улучшение кода.

- **Red** — Написание теста, который пока не проходит
- **Green** — Написание минимального кода для прохождения теста
- **Refactor** — Улучшение кода без изменения функциональности

🔮 **Шаг 06:**

Миссия 6

Сопоставьте преимущества TDD с объяснениями:

Ответ:

- Надёжный код
 - Меньше багов
 - Документация через тесты
 - Рефакторинг без страха
1. Тесты показывают, что функции работают правильно
 2. Ошибки выявляются на раннем этапе разработки
 3. Тесты объясняют, как использовать код
 4. Можно улучшать код, не ломая функциональность

Подсказка: TDD помогает сразу увидеть ошибки и поддерживает чистый код.

- **Надёжный код** — Тесты показывают, что функции работают правильно
- **Меньше багов** — Ошибки выявляются на раннем этапе разработки
- **Документация через тесты** — Тесты объясняют, как использовать код
- **Рефакторинг без страха** — Можно улучшать код, не ломая функциональность

Шаг 07:

Миссия 7

Расположите шаги TDD в правильном порядке:

1. Написать тест, который не проходит
2. Написать минимальный код для прохождения теста
3. Запустить тест и убедиться, что он проходит
4. Рефакторинг кода

Подсказка: Сначала тест падает, потом код проходит тест, затем улучшаем код.

Ответ:

- Написать тест, который не проходит
- Написать минимальный код для прохождения теста
- Запустить тест и убедиться, что он проходит
- Рефакторинг кода

Шаг 08:

Миссия 8

Расположите шаги работы с новой функцией по TDD:

1. Написать тест для ожидаемого поведения
2. Запустить тест, чтобы убедиться, что он падает
3. Реализовать минимальную логику для прохождения теста
4. Проверить и улучшить код, сохраняя функциональность

Подсказка: Следуйте циклу Red → Green → Refactor.

Ответ:

- Написать тест для ожидаемого поведения
- Запустить тест, чтобы убедиться, что он падает
- Реализовать минимальную логику для прохождения теста
- Проверить и улучшить код, сохраняя функциональность

Шаг 09:

Миссия 9

Аня решила попробовать TDD. С чего лучше начать разработку функции?

Подсказка: Подумайте, что делает TDD уникальным методом.

Ответ:

- **Написать тест, который проверяет будущую функцию**
- Сначала написать всю функцию, потом тесты
- Игнорировать тесты и сразу запускать код

🔮 Шаг 10:

Миссия 10

Пётр написал тест на сложную функцию, но он не проходит. Что делать первым шагом?

Подсказка: В TDD тест должен управлять разработкой функции.

Ответ:

- **Реализовать минимальный код, чтобы тест начал проходить**
- Удалить тест, чтобы ничего не ломалось
- Добавить новые функции без исправления теста

🔮 Шаг 11:

Миссия 11

Катя использует TDD и хочет проверить крайние случаи функции. Что важно учитывать при написании тестов?

Подсказка: Подумайте о типичных ошибках, которые сложно заметить без тестов.

Ответ:

- **Проверять как обычные, так и крайние значения входных данных**
- Проверять только корректные данные
- Игнорировать крайние случаи, они редко возникают

🔮 Шаг 12:

Миссия 12

Иван добавил тест, функция ещё не реализована. Как должен выглядеть результат теста?

Подсказка: В TDD тесты помогают проверять прогресс разработки.

Ответ:

- **Тест должен падать, так как функция ещё не реализована**
- Тест должен проходить сразу
- Тест не запускается до реализации функции

🔮 Шаг 13:

Миссия 13

Сопоставьте этапы TDD с их описанием:

Ответ:

- Red
 - Green
 - Refactor
1. Тест падает, функция ещё не реализована
 2. Реализуем код, чтобы тест прошёл
 3. Оптимизируем код без изменения поведения, тесты должны проходить

Подсказка: Подумайте о цикле TDD: сначала тест, потом код, потом чистка.

- **Red** — Тест падает, функция ещё не реализована
- **Green** — Реализуем код, чтобы тест прошёл
- **Refactor** — Оптимизируем код без изменения поведения, тесты должны проходить

Шаг 14:

Миссия 14

Сопоставьте преимущества TDD с объяснениями:

Ответ:

- Более надёжный код
- Документация функций через тесты
- Быстрое обнаружение ошибок
- Поддержка рефакторинга

1. Тесты показывают, что код работает корректно и не ломается при изменениях
2. Тесты описывают поведение функций, можно понять что делает код
3. Ошибки выявляются сразу после внесения изменений
4. Позволяет безопасно улучшать код без сломанных функций

Подсказка: Подумайте, как тесты помогают на разных этапах разработки.

- **Более надёжный код** — Тесты показывают, что код работает корректно и не ломается при изменениях
- **Документация функций через тесты** — Тесты описывают поведение функций, можно понять что делает код
- **Быстрое обнаружение ошибок** — Ошибки выявляются сразу после внесения изменений
- **Поддержка рефакторинга** — Позволяет безопасно улучшать код без сломанных функций

Шаг 15:

Миссия 15

Расположите шаги цикла TDD в правильном порядке:

1. Написать тест, который падает
2. Реализовать минимальный код, чтобы тест прошёл
3. Запустить тесты и убедиться, что проходят
4. Рефакторинг кода с сохранением прохождения тестов

Подсказка: Сначала тест, потом код, потом проверка и оптимизация.

Ответ:

- Написать тест, который падает
- Реализовать минимальный код, чтобы тест прошёл
- Запустить тесты и убедиться, что проходят
- Рефакторинг кода с сохранением прохождения тестов

Модуль 10 - Паттерны проектирования

Урок 1 - Модуль, Фабрика, Синглтон

Шаг 01:

Миссия 1

Аня хочет создать модуль для работы с пользователями. Что лучше сделать первым шагом?

Подсказка: Модуль должен быть самостоятельным блоком функциональности.

Ответ:

- **Создать отдельный файл с функциями, связанными с пользователями**
- Писать все функции прямо в основном файле приложения
- Игнорировать модульность и добавлять код хаотично

Шаг 02:

Миссия 2

Пётр создает фабрику для генерации разных видов уведомлений. Что важно учитывать?

Подсказка: Фабрика упрощает создание объектов с одинаковым интерфейсом.

Ответ:

- **Фабрика должна создавать объекты без привязки к конкретным классам**
- Создавать каждый объект вручную вне фабрики
- Игнорировать интерфейсы объектов

Шаг 03:

Миссия 3

Катя разрабатывает синглтон для конфигурации приложения. Как убедиться, что объект один?

Подсказка: Синглтон гарантирует единственный экземпляр класса.

Ответ:

- **Создать класс с приватным конструктором и статическим методом получения экземпляра**
- Каждый раз создавать новый объект при необходимости
- Использовать глобальные переменные вместо синглтона

Шаг 04:

Миссия 4

Иван хочет объединить модуль и фабрику для работы с формами. Как поступить правильно?

Подсказка: Модуль отвечает за структуру, фабрика — за создание объектов.

Ответ:

- **Создать модуль форм и внутри него фабрику для разных видов форм**
- Делать фабрику в глобальном пространстве без модулей
- Игнорировать фабрику и писать формы вручную

Шаг 05:

Миссия 5

Сопоставьте паттерн с его описанием:

Ответ:

- Модуль
 - Фабрика
 - Синглтон
1. Изолированная часть кода с публичными методами и приватными данными
 2. Создает объекты с одинаковым интерфейсом без указания конкретного класса
 3. Обеспечивает единственный экземпляр объекта во всем приложении

Подсказка: Подумайте о назначении каждого паттерна: структура, создание, единственность.

- **Модуль** — Изолированная часть кода с публичными методами и приватными данными
- **Фабрика** — Создает объекты с одинаковым интерфейсом без указания конкретного класса
- **Синглтон** — Обеспечивает единственный экземпляр объекта во всем приложении

Шаг 06:

Миссия 6

Сопоставьте преимущества паттернов с объяснениями:

Ответ:

- Модуль
 - Фабрика
 - Синглтон
1. Инкапсулирует функциональность, облегчает поддержку
 2. Упрощает создание объектов с одинаковым интерфейсом
 3. Гарантирует единственный объект, удобно для настроек

Подсказка: Подумайте, как каждый паттерн облегчает разработку.

- **Модуль** — Инкапсулирует функциональность, облегчает поддержку
- **Фабрика** — Упрощает создание объектов с одинаковым интерфейсом
- **Синглтон** — Гарантирует единственный объект, удобно для настроек

Шаг 07:

Миссия 7

Расположите этапы создания паттерна Синглтон:

1. Создать приватный конструктор
2. Создать статический метод для получения экземпляра
3. **Проверить, существует ли экземпляр, если нет** — создать
4. Использовать экземпляр в приложении

Подсказка: Сначала ограничиваем создание, потом предоставляем доступ, затем используем.

Ответ:

- Создать приватный конструктор
- Создать статический метод для получения экземпляра
- **Проверить, существует ли экземпляр, если нет** — создать
- Использовать экземпляр в приложении

Шаг 08:

Миссия 8

Алексей создает новый модуль для игры. Что нужно сделать первым шагом?

Подсказка: Подумайте о разделении кода на отдельные независимые части.

Ответ:

- **Создать отдельный файл с функционалом модуля**
- Писать весь код в одном файле
- Игнорировать модуль и подключить всё напрямую

Шаг 09:

Миссия 9

Марина хочет использовать фабрику для создания игровых персонажей. Какой принцип важно соблюдать?

Подсказка: Фабрика помогает управлять созданием объектов.

Ответ:

- **Создавать объекты через единый интерфейс фабрики**
- Создавать объекты напрямую в коде
- Дублировать код создания объектов по всему проекту

Шаг 10:

Миссия 10

Игорь хочет применить синглтон для менеджера настроек игры. Что это гарантирует?

Подсказка: Подумайте, что значит «единственный экземпляр».

Ответ:

- **Будет только один экземпляр объекта на всю игру**
- Каждый модуль создает собственный экземпляр
- Объект можно копировать сколько угодно раз

Шаг 11:

Миссия 11

Оля разрабатывает фабрику для разных типов оружия. Почему стоит использовать фабрику вместо прямого создания объектов?

Подсказка: Подумайте о будущем расширении и поддержке кода.

Ответ:

- **Чтобы легко добавлять новые типы объектов без изменения кода клиентов**
- Чтобы усложнить код без пользы
- Чтобы каждый раз вручную создавать экземпляры

Шаг 12:

Миссия 12

Сопоставьте паттерн с его описанием:

Ответ:

- Модуль
 - Фабрика
 - Синглтон
1. Создает объекты через единый интерфейс, упрощая их замену
 2. Разделяет код на независимые части, легко поддерживаемые и импортируемые
 3. Обеспечивает единственный экземпляр объекта во всей программе

Подсказка: Подумайте о задачах каждого паттерна.

- **Модуль** — Разделяет код на независимые части, легко поддерживаемые и импортируемые
- **Фабрика** — Создает объекты через единый интерфейс, упрощая их замену

- **Синглтон** — Обеспечивает единственный экземпляр объекта во всей программе

Шаг 13:

Миссия 13

Сопоставьте преимущества паттернов проектирования с объяснениями:

Ответ:

- Повторное использование
 - Упрощение поддержки
 - Контроль создания объектов
1. Можно применять одинаковый паттерн в разных проектах
 2. Код легче изменять и расширять без нарушения других частей
 3. Фабрика или синглтон управляет количеством создаваемых объектов

Подсказка: Обдумайте как каждый паттерн решает типичные проблемы разработки.

- **Повторное использование** — Можно применять одинаковый паттерн в разных проектах
- **Упрощение поддержки** — Код легче изменять и расширять без нарушения других частей
- **Контроль создания объектов** — Фабрика или синглтон управляет количеством создаваемых объектов

Шаг 14:

Миссия 14

Расположите шаги создания синглтона:

1. Объявить приватный конструктор
2. Создать статический метод для получения экземпляра
3. Инициализировать единственный объект внутри метода
4. Использовать объект в программе через статический метод

Подсказка: Сначала создаём объект, потом даём доступ к нему.

Ответ:

- Объявить приватный конструктор
- Создать статический метод для получения экземпляра
- Инициализировать единственный объект внутри метода
- Использовать объект в программе через статический метод

Урок 2 - Паттерны публикации подписки

Шаг 01:

Миссия 1

Алекс хочет, чтобы его игровая система уведомляла всех подписчиков о новых событиях. Какой паттерн стоит использовать?

Подсказка: Подумайте о шаблоне, где один объект информирует многих.

Ответ:

- **Паттерн публикации/подписки**
- Синглтон
- Фабрика

🔮 Шаг 02:

Миссия 2

Марина разрабатывает чат в игре и хочет, чтобы сообщения автоматически появлялись у всех игроков. Что нужно сделать?

Подсказка: Подумайте, кто отправляет уведомления, а кто их получает.

Ответ:

- **Создать объект-издатель и подписчиков, которые слушают события**
- Каждому игроку отправлять сообщение вручную
- Использовать статический метод без подписчиков

🔮 Шаг 03:

Миссия 3

Игорь хочет, чтобы подписчики могли отписываться от уведомлений. Какой подход лучше использовать?

Подсказка: Подумайте о гибкости управления подписками.

Ответ:

- **Добавить метод unsubscribe для удаления подписчика**
- Удалять всех подписчиков сразу
- Игнорировать отписки

🔮 Шаг 04:

Миссия 4

Сопоставьте понятие паттерна публикации/подписки с его описанием:

Ответ:

- Издатель (Publisher)
 - Подписчик (Subscriber)
 - Событие (Event)
1. Объект, который уведомляет всех подписчиков о изменениях
 2. Объект, который получает уведомления и реагирует на них
 3. Сообщение или сигнал, который отправляется подписчикам

Подсказка: Подумайте, кто отправляет, кто получает, и что передается.

- **Издатель (Publisher)** — Объект, который уведомляет всех подписчиков о изменениях
- **Подписчик (Subscriber)** — Объект, который получает уведомления и реагирует на них
- **Событие (Event)** — Сообщение или сигнал, который отправляется подписчикам

🔮 Шаг 05:

Миссия 5

Сопоставьте преимущества использования паттерна публикации/подписки с объяснениями:

Ответ:

- Слабое связывание
 - Гибкость
 - Масштабируемость
1. Подписчики не зависят от внутренней логики издателя

2. Легко добавлять или удалять подписчиков без изменения издателя
3. Позволяет легко обрабатывать большое количество подписчиков

Подсказка: Подумайте о преимуществах разделения обязанностей.

- **Слабое связывание** — Подписчики не зависят от внутренней логики издателя
- **Гибкость** — Легко добавлять или удалять подписчиков без изменения издателя
- **Масштабируемость** — Позволяет легко обрабатывать большое количество подписчиков

Шаг 06:

Миссия 6

Расположите шаги реализации паттерна публикации/подписки:

1. Создать объект-издатель
2. Создать подписчиков и зарегистрировать их у издателя
3. Издатель генерирует события
4. Подписчики получают уведомления и реагируют на события

Подсказка: Сначала создаем издателя и подписчиков, затем события начинают распространяться.

Ответ:

- Создать объект-издатель
- Создать подписчиков и зарегистрировать их у издателя
- Издатель генерирует события
- Подписчики получают уведомления и реагируют на события

Шаг 07:

Миссия 7

Расположите действия для динамического управления подписками:

1. Подписчик вызывает метод `subscribe` у издателя
2. Издатель добавляет подписчика в список
3. Происходит событие, издатель уведомляет подписчиков
4. Подписчик может вызвать метод `unsubscribe` для отписки

Подсказка: Сначала подписка, потом уведомления, в конце возможна отписка.

Ответ:

- Подписчик вызывает метод `subscribe` у издателя
- Издатель добавляет подписчика в список
- Происходит событие, издатель уведомляет подписчиков
- Подписчик может вызвать метод `unsubscribe` для отписки

Шаг 08:

Миссия 8

Алексей создает систему уведомлений для игры. Какой паттерн поможет разделить издателя и подписчиков?

Подсказка: Подумайте о том, как сделать так, чтобы один объект оповещал множество других.

Ответ:

- **Паттерн публикации/подписки**

- Фабрика
- Синглтон

🔗 Шаг 09:

Миссия 9

Марина хочет, чтобы разные части игры получали уведомления о событиях. Что важно для подписчиков?

Подсказка: Подумайте о том, как подписчики должны реагировать на события без изменения издателя.

Ответ:

- **Подписчики должны реализовывать обработку события самостоятельно**
- Подписчики напрямую вызывают методы издателя
- Подписчики хранят состояние издателя локально

🔗 Шаг 10:

Миссия 10

Игорь внедряет систему событий для чата в игре. Что важно для издателя событий?

Подсказка: Подумайте о том, чтобы издатель не зависел от количества подписчиков.

Ответ:

- **Издатель не должен знать, кто подписан на события**
- Издатель должен хранить ссылки на все подписчики и вызывать их напрямую
- Издатель сам выполняет обработку событий подписчиков

🔗 Шаг 11:

Миссия 11

Сопоставьте роли паттерна публикации/подписки с их описаниями:

Ответ:

- Издатель (Publisher)
 - Подписчик (Subscriber)
 - Событие (Event)
1. Отправляет уведомления о произошедших событиях
 2. Получает уведомления и реагирует на них
 3. Информация о произошедшем действии или изменении состояния

Подсказка: Подумайте, кто инициирует события и кто их получает.

- **Издатель (Publisher)** — Отправляет уведомления о произошедших событиях
- **Подписчик (Subscriber)** — Получает уведомления и реагирует на них
- **Событие (Event)** — Информация о произошедшем действии или изменении состояния

🔗 Шаг 12:

Миссия 12

Сопоставьте преимущества паттерна публикации/подписки с объяснениями:

Ответ:

- Слабое связывание
- Масштабируемость

- Гибкость добавления новых функций

1. Подписчики и издатели не зависят друг от друга напрямую
2. Можно легко добавлять новых подписчиков без изменения издателя
3. Система легко растёт, обрабатывая больше событий и подписчиков

Подсказка: Подумайте, как независимость компонентов облегчает развитие проекта.

- **Слабое связывание** — Подписчики и издатели не зависят друг от друга напрямую
- **Масштабируемость** — Система легко растёт, обрабатывая больше событий и подписчиков
- **Гибкость добавления новых функций** — Можно легко добавлять новых подписчиков без изменения издателя

Шаг 13:

Миссия 13

Расположите шаги реализации паттерна публикации/подписки:

1. Создать объект издателя
2. Добавить методы для подписки и отписки
3. Создать подписчиков и зарегистрировать их у издателя
4. Издатель публикует событие, подписчики получают уведомления

Подсказка: Сначала создаём систему, потом добавляем участников, затем публикуем события.

Ответ:

- Создать объект издателя
- Добавить методы для подписки и отписки
- Создать подписчиков и зарегистрировать их у издателя
- Издатель публикует событие, подписчики получают уведомления

Шаг 14:

Миссия 14

Расположите этапы подписки на события в правильном порядке:

1. Создание подписчика
2. Регистрация подписчика у издателя
3. Ожидание событий
4. Обработка полученных уведомлений

Подсказка: Сначала создаем объект, затем подключаем его к системе, потом ждём и реагируем.

Ответ:

- Создание подписчика
- Регистрация подписчика у издателя
- Ожидание событий
- Обработка полученных уведомлений

Урок 3 - Стратегия, Команда, Декоратор

Шаг 01:

Миссия 1

Игорь создаёт игру, в которой персонаж может менять стиль атаки. Какой паттерн позволит легко менять поведение объекта во время игры?

Подсказка: Подумайте, как менять алгоритмы без изменения самого персонажа.

Ответ:

- **Стратегия (Strategy)**
- Декоратор (Decorator)
- Команда (Command)

🧙 Шаг 02:

Миссия 2

Марина хочет оформить возможность объединять несколько действий в одно целое для персонажей. Какой паттерн ей подойдёт?

Подсказка: Подумайте о том, чтобы команды выполнялись по очереди или группой.

Ответ:

- **Команда (Command)**
- Стратегия (Strategy)
- Декоратор (Decorator)

🧙 Шаг 03:

Миссия 3

Алексей хочет добавить дополнительные эффекты к объектам игры (например, броню, магию), не меняя их исходного кода. Какой паттерн ему нужен?

Подсказка: Подумайте о том, как обернуть объект, чтобы добавить новые возможности.

Ответ:

- **Декоратор (Decorator)**
- Стратегия (Strategy)
- Команда (Command)

🧙 Шаг 04:

Миссия 4

Сопоставьте паттерны с их основным назначением:

Ответ:

- Стратегия
- Команда
- Декоратор

1. Изменение поведения объекта на лету
2. Инкапсуляция действий как объектов для гибкого управления ими
3. Динамическое добавление обязанностей объекту

Подсказка: Подумайте, кто управляет поведением, кто действиями, а кто добавляет новые возможности.

- **Стратегия** — Изменение поведения объекта на лету
- **Команда** — Инкапсуляция действий как объектов для гибкого управления ими
- **Декоратор** — Динамическое добавление обязанностей объекту

Шаг 05:

Миссия 5

Сопоставьте ситуации из игры с подходящим паттерном:

Ответ:

- Персонаж выбирает способ атаки
- Нужно объединить несколько действий в цепочку
- Добавление магических эффектов к броне

1. Стратегия
2. Команда
3. Декоратор

Подсказка: Подумайте, что влияет на поведение, что на выполнение действий, что на добавление возможностей.

- **Персонаж выбирает способ атаки** — Стратегия
- **Нужно объединить несколько действий в цепочку** — Команда
- **Добавление магических эффектов к броне** — Декоратор

Шаг 06:

Миссия 6

Расположите шаги применения паттерна «Стратегия»:

1. Создать интерфейс поведения
2. Создать конкретные стратегии, реализующие интерфейс
3. Интегрировать стратегию в объект
4. Выполнить объект с выбранной стратегией

Подсказка: Сначала определяем контракт, потом конкретные реализации, затем используем их в объекте.

Ответ:

- Создать интерфейс поведения
- Создать конкретные стратегии, реализующие интерфейс
- Интегрировать стратегию в объект
- Выполнить объект с выбранной стратегией

Шаг 07:

Миссия 7

Расположите шаги использования паттерна «Декоратор»:

1. Создать базовый объект
2. Создать обёртки (декораторы) с дополнительными обязанностями
3. Оборачивать объект нужными декораторами
4. Использовать объект с добавленными обязанностями

Подсказка: Сначала есть объект, потом создаем декораторы, оборачиваем объект и используем.

Ответ:

- Создать базовый объект
- Создать обёртки (декораторы) с дополнительными обязанностями
- Оборачивать объект нужными декораторами

- Использовать объект с добавленными обязанностями

🧙 Шаг 08:

Миссия 8

Алексей разрабатывает боевую систему для игры. Он хочет легко менять стратегию атаки персонажей. Какой паттерн использовать?

Подсказка: Подумайте, как можно менять алгоритмы без изменения кода персонажа.

Ответ:

- **Стратегия**
- Команда
- Декоратор

🧙 Шаг 09:

Миссия 9

Марина хочет, чтобы игроки могли комбинировать действия персонажей, не переписывая основной код. Какой паттерн подойдет?

Подсказка: Подумайте о том, как добавить новые возможности объекту динамически.

Ответ:

- **Декоратор**
- Стратегия
- Команда

🧙 Шаг 10:

Миссия 10

Игорь создает систему команд для управления NPC. Он хочет инкапсулировать запросы как объекты. Что использовать?

Подсказка: Подумайте о том, как хранить и передавать действия как объекты.

Ответ:

- **Команда**
- Стратегия
- Декоратор

🧙 Шаг 11:

Миссия 11

Сопоставьте паттерн с его основной задачей:

Ответ:

- Стратегия
 - Команда
 - Декоратор
1. Инкапсулирует алгоритмы для смены поведения объекта
 2. Инкапсулирует действия как объекты и позволяет откладывать выполнение
 3. Динамически добавляет функциональность объекту

Подсказка: Подумайте, какая цель каждого паттерна в игровых механиках.

- **Стратегия** — Инкапсулирует алгоритмы для смены поведения объекта

- **Команда** — Инкапсулирует действия как объекты и позволяет откладывать выполнение
- **Декоратор** — Динамически добавляет функциональность объекту

Шаг 12:

Миссия 12

Сопоставьте паттерн с примером из жизни игры:

Ответ:

- Стратегия
 - Команда
 - Декоратор
1. Разные методы атаки для персонажей
 2. Сохранение действий игрока для повторного воспроизведения
 3. Добавление магического эффекта к оружию

Подсказка: Подумайте, где нужен выбор алгоритма, где команды, где улучшения.

- **Стратегия** — Разные методы атаки для персонажей
- **Команда** — Сохранение действий игрока для повторного воспроизведения
- **Декоратор** — Добавление магического эффекта к оружию

Шаг 13:

Миссия 13

Расположите шаги применения паттерна Стратегия:

1. Создать интерфейс стратегии
2. Реализовать конкретные стратегии
3. Создать объект контекста, использующий стратегию
4. Динамически менять стратегию объекта при необходимости

Подсказка: Сначала создаём интерфейс, потом реализацию, затем объект и смену стратегии.

Ответ:

- Создать интерфейс стратегии
- Реализовать конкретные стратегии
- Создать объект контекста, использующий стратегию
- Динамически менять стратегию объекта при необходимости

Шаг 14:

Миссия 14

Расположите шаги применения паттерна Декоратор:

1. Создать базовый объект
2. Создать декоратор с дополнительной функциональностью
3. Обернуть базовый объект декоратором
4. Вызвать методы объекта с дополнительными функциями

Подсказка: Сначала базовый объект, потом декоратор, затем обертывание и использование.

Ответ:

- Создать базовый объект
- Создать декоратор с дополнительной функциональностью

- Обернуть базовый объект декоратором
- Вызвать методы объекта с дополнительными функциями

Урок 4 - Паттерны фронтенда (MVC, MVVM, Observer)

Шаг 01:

Миссия 1

Алексей разрабатывает интерфейс игры. Ему нужно отделить логику данных от отображения на экране. Какой паттерн лучше использовать?

Подсказка: Подумайте о разделении модели и представления.

Ответ:

- **MVC**
- MVVM
- Observer

Шаг 02:

Миссия 2

Марина создает интерфейс с реактивными элементами. Ей нужно автоматически обновлять UI при изменении данных. Какой паттерн выбрать?

Подсказка: Подумайте о двусторонней привязке данных и уведомлении представления.

Ответ:

- **MVVM**
- MVC
- Observer

Шаг 03:

Миссия 3

Игорь разрабатывает систему уведомлений для игры. Когда меняется состояние игрока, все подписчики должны быть уведомлены. Какой паттерн использовать?

Подсказка: Подумайте о паттерне для уведомления всех зависимых объектов при изменении состояния.

Ответ:

- **Observer**
- MVC
- MVVM

Шаг 04:

Миссия 4

Сопоставьте паттерн фронтенда с его основной задачей:

Ответ:

- MVC
- MVVM
- Observer

1. Разделение данных, логики и представления
2. Двусторонняя привязка данных между моделью и представлением

3. Уведомление всех подписчиков о изменении состояния

Подсказка: Подумайте о том, где данные, где представление, где реактивность.

- **MVC** — Разделение данных, логики и представления
- **MVVM** — Двусторонняя привязка данных между моделью и представлением
- **Observer** — Уведомление всех подписчиков о изменении состояния

Шаг 05:

Миссия 5

Сопоставьте паттерн с примером из жизни фронтенд-разработки:

Ответ:

- MVC
 - MVVM
 - Observer
1. Форма ввода и таблица, обновляющаяся после отправки данных
 2. Компоненты UI автоматически обновляются при изменении модели
 3. Система уведомлений при изменении состояния игры

Подсказка: Где нужно разделение, где реактивность, где подписка?

- **MVC** — Форма ввода и таблица, обновляющаяся после отправки данных
- **MVVM** — Компоненты UI автоматически обновляются при изменении модели
- **Observer** — Система уведомлений при изменении состояния игры

Шаг 06:

Миссия 6

Расположите шаги использования паттерна Observer:

1. Создать объект-субъект с состоянием
2. Создать объекты-подписчики
3. Подписать объекты на субъект
4. Изменить состояние субъекта и уведомить подписчиков

Подсказка: Сначала создаём субъект, потом подписчики, потом подписка и уведомление.

Ответ:

- Создать объект-субъект с состоянием
- Создать объекты-подписчики
- Подписать объекты на субъект
- Изменить состояние субъекта и уведомить подписчиков

Шаг 07:

Миссия 7

Расположите шаги использования паттерна MVVM:

1. Создать модель данных
2. Создать представление
3. Создать ViewModel, связывающую модель и представление
4. Обеспечить двустороннюю привязку данных

Подсказка: Модель, представление, ViewModel, привязка.

Ответ:

- Создать модель данных
- Создать представление
- Создать ViewModel, связывающую модель и представление
- Обеспечить двустороннюю привязку данных

Шаг 08:

Миссия 8

Аня разрабатывает игру с интерфейсом. Она хочет разделить логику, данные и отображение. Какой паттерн использовать?

Подсказка: Подумайте о классическом разделении на Model, View и Controller.

Ответ:

- **MVC**
- MVVM
- Observer

Шаг 09:

Миссия 9

Борис создаёт интерфейс с двусторонней привязкой данных. Он хочет, чтобы изменения в модели сразу отражались в интерфейсе. Какой паттерн использовать?

Подсказка: Подумайте о двухсторонней привязке между View и Model.

Ответ:

- **MVVM**
- MVC
- Observer

Шаг 10:

Миссия 10

Катя хочет, чтобы несколько компонентов интерфейса автоматически обновлялись при изменении данных. Какой паттерн подойдёт?

Подсказка: Подумайте о подписке объектов на изменения состояния.

Ответ:

- **Observer**
- MVC
- MVVM

Шаг 11:

Миссия 11

Сопоставьте паттерн с его основной задачей:

Ответ:

- MVC
- MVVM

- Observer

1. Разделение данных, логики и представления
2. Двусторонняя привязка данных между моделью и интерфейсом
3. Автоматическое уведомление подписчиков об изменениях

Подсказка: Подумайте, какая цель каждого паттерна в интерфейсах приложений.

- **MVC** — Разделение данных, логики и представления
- **MVVM** — Двусторонняя привязка данных между моделью и интерфейсом
- **Observer** — Автоматическое уведомление подписчиков об изменениях

Шаг 12:

Миссия 12

Сопоставьте паттерн с примером из жизни интерфейса игры:

Ответ:

- MVC
 - MVVM
 - Observer
1. Отдельная модель здоровья игрока, кнопки и экран отображения
 2. Прямое обновление шкалы опыта при изменении значения в модели
 3. Все мини-карты получают уведомление, когда игрок двигается

Подсказка: Где важна структура, где двусторонняя привязка, а где уведомления?

- **MVC** — Отдельная модель здоровья игрока, кнопки и экран отображения
- **MVVM** — Прямое обновление шкалы опыта при изменении значения в модели
- **Observer** — Все мини-карты получают уведомление, когда игрок двигается

Шаг 13:

Миссия 13

Расположите шаги применения паттерна Observer:

1. Создать объект-субъект с состоянием
2. Создать объекты-подписчики
3. Подписать объекты на уведомления субъекта
4. При изменении состояния уведомить всех подписчиков

Подсказка: Сначала создаём субъект, потом подписчики, подписку, затем уведомление.

Ответ:

- Создать объект-субъект с состоянием
- Создать объекты-подписчики
- Подписать объекты на уведомления субъекта
- При изменении состояния уведомить всех подписчиков

Шаг 14:

Миссия 14

Расположите шаги применения паттерна MVVM:

1. Создать модель данных

2. Создать ViewModel с привязкой к модели
3. Создать View с привязкой к ViewModel
4. Обновлять интерфейс при изменении модели через ViewModel

Подсказка: Сначала данные, потом ViewModel, View, затем автоматическое обновление.

Ответ:

- Создать модель данных
- Создать ViewModel с привязкой к модели
- Создать View с привязкой к ViewModel
- Обновлять интерфейс при изменении модели через ViewModel

Модуль 11 - Архитектура JS-приложений

Урок 1 - Файл-структура проектов

Шаг 01:

Миссия 1

Иван начинает новый проект и хочет структурировать файлы, чтобы было удобно находить компоненты, утилиты и стили. Что является лучшей практикой?

Подсказка: Подумайте о логическом разделении по функционалу и типу файлов.

Ответ:

- **Разделять проект на папки components, utils, styles, pages**
- Класть все файлы в одну папку src
- Создавать по одной папке для каждого разработчика

Шаг 02:

Миссия 2

Мария хочет, чтобы каждый компонент был самодостаточным и легко переиспользуемым. Какой принцип следует применить при организации файлов?

Подсказка: Подумайте о единице, которую можно импортировать отдельно.

Ответ:

- **Создавать отдельную папку для каждого компонента с его JS, CSS и тестами**
- Класть все компоненты в одну папку
- Писать весь код в одном файле App.js

Шаг 03:

Миссия 3

Пётр создает утилиты для работы с датой и временем. Где лучше их хранить?

Подсказка: Подумайте о местах для вспомогательных функций, которые не связаны напрямую с компонентами.

Ответ:

- **В папке utils или helpers**
- В папке components
- Внутри каждого компонента

Шаг 04:

Миссия 4

Сопоставьте тип папки проекта с его содержимым:

Ответ:

- components
 - utils
 - styles
 - pages
1. Переиспользуемые функции и хелперы
 2. CSS или SCSS файлы для визуального оформления
 3. Отдельные части интерфейса с логикой и разметкой
 4. Главные экраны или маршруты приложения

Подсказка: Подумайте, что относится к логике, что к внешнему виду, а что к страницам.

- **components** — Отдельные части интерфейса с логикой и разметкой
- **utils** — Переиспользуемые функции и хелперы
- **styles** — CSS или SCSS файлы для визуального оформления
- **pages** — Главные экраны или маршруты приложения

Шаг 05:

Миссия 5

Сопоставьте файл с его назначением в проекте:

Ответ:

- App.js
 - index.js
 - theme.css
 - dateUtils.js
1. Точка входа в приложение
 2. Главный компонент приложения
 3. Стили и тема приложения
 4. Вспомогательные функции для работы с датой

Подсказка: Подумайте, где логика, где стили, а где точка входа.

- **App.js** — Главный компонент приложения
- **index.js** — Точка входа в приложение
- **theme.css** — Стили и тема приложения
- **dateUtils.js** — Вспомогательные функции для работы с датой

Шаг 06:

Миссия 6

Расположите шаги создания структуры проекта:

1. Создать папку src
2. Создать подпапки components, pages, utils, styles
3. Добавить файлы для компонентов, страниц и утилит
4. Настроить точку входа index.js и главный App.js

Подсказка: Сначала создаём основную папку, потом подпапки, потом файлы и точку входа.

Ответ:

- Создать папку src
- Создать подпапки components, pages, utils, styles
- Добавить файлы для компонентов, страниц и утилит
- Настроить точку входа index.js и главный App.js

Шаг 07:

Миссия 7

Расположите порядок работы с файлами при добавлении нового компонента:

1. Создать папку нового компонента с JS, CSS и тестами
2. Импортировать компонент в нужные страницы
3. Настроить стили и подключить их в компонент
4. Проверить работу компонента в приложении

Подсказка: Сначала создаём файлы компонента, потом стили, импортируем и проверяем.

Ответ:

- Создать папку нового компонента с JS, CSS и тестами
- Настроить стили и подключить их в компонент
- Импортировать компонент в нужные страницы
- Проверить работу компонента в приложении

Шаг 08:

Миссия 8

Антон создаёт новое JS-приложение и хочет, чтобы структура файлов была понятной. Какой принцип стоит соблюдать при организации папок?

Подсказка: Подумайте, что облегчает понимание проекта другими разработчиками.

Ответ:

- **Разделять код по функциональности и типу файлов**
- Смешивать все файлы в одной папке
- Создавать отдельную папку для каждого файла

Шаг 09:

Миссия 9

Лиза делает проект с компонентами UI и утилитами. Как лучше разделить файлы?

Подсказка: Подумайте о логической группировке файлов по типу и назначению.

Ответ:

- **Создать папки components, utils, styles**
- Все файлы в папке src без подразделов
- Отдельная папка для каждого метода

Шаг 10:

Миссия 10

Дима работает с большим проектом и хочет, чтобы новые разработчики быстро понимали структуру. Что поможет?

Подсказка: Подумайте о ясности и стандартах в названии папок и файлов.

Ответ:

- **Использовать понятные названия папок и файлов**
- Называть файлы как попало
- Ставить длинные номера вместо названий

💡 Шаг 11:

Миссия 11

Сопоставьте папку проекта с её типичным содержимым:

Ответ:

- components
 - utils
 - styles
 - assets
1. JS-файлы повторно используемых блоков интерфейса
 2. Вспомогательные функции и модули
 3. CSS/SCSS файлы, оформление
 4. Изображения, шрифты, медиа

Подсказка: Подумайте, что логически хранится в каждой папке.

- **components** — JS-файлы повторно используемых блоков интерфейса
- **utils** — Вспомогательные функции и модули
- **styles** — CSS/SCSS файлы, оформление
- **assets** — Изображения, шрифты, медиа

💡 Шаг 12:

Миссия 12

Сопоставьте тип файла с его местом в проекте:

Ответ:

- Header.js
 - helpers.js
 - main.scss
 - logo.png
1. components
 2. utils
 3. styles
 4. assets

Подсказка: Подумайте о назначении каждого файла.

- **Header.js** — components
- **helpers.js** — utils
- **main.scss** — styles
- **logo.png** — assets

Шаг 13:

Миссия 13

Расположите шаги организации структуры проекта:

1. Создать папку src
2. Создать подпапки components, utils, styles, assets
3. Поместить файлы в соответствующие папки
4. Использовать понятные имена файлов и папок

Подсказка: Сначала общая папка, затем подпапки, потом файлы, потом стандарты имен.

Ответ:

- Создать папку src
- Создать подпапки components, utils, styles, assets
- Поместить файлы в соответствующие папки
- Использовать понятные имена файлов и папок

Шаг 14:

Миссия 14

Расположите шаги добавления нового компонента Button:

1. Создать файл Button.js
2. Поместить его в папку components
3. Создать файл стилей Button.scss
4. Добавить импорт стилей в Button.js
5. Использовать компонент в проекте

Подсказка: Сначала создаём файлы, затем связываем их и используем в проекте.

Ответ:

- Создать файл Button.js
- Поместить его в папку components
- Создать файл стилей Button.scss
- Добавить импорт стилей в Button.js
- Использовать компонент в проекте

Урок 2 - Чистая архитектура и слои

Шаг 01:

Миссия 1

Сопоставьте слои приложения с их основной задачей:

Ответ:

- UI Layer
 - Domain Layer
 - Data Layer
 - Service Layer
1. Обработка пользовательских интерфейсов и взаимодействие с пользователем
 2. Бизнес-логика и правила приложения
 3. Хранение и извлечение данных

4. Интеграция с внешними API и сервисами

Подсказка: Смотрите на типичные обязанности каждого слоя.

- **UI Layer** — Обработка пользовательских интерфейсов и взаимодействие с пользователем
- **Domain Layer** — Бизнес-логика и правила приложения
- **Data Layer** — Хранение и извлечение данных
- **Service Layer** — Интеграция с внешними API и сервисами

Шаг 02:

Миссия 2

Сопоставьте принципы чистой архитектуры с их описанием:

Ответ:

- Single Responsibility Principle
 - Open/Closed Principle
 - Dependency Inversion Principle
 - Liskov Substitution Principle
1. Классы должны быть открыты для расширения, но закрыты для модификации
 2. Каждый класс отвечает только за одну вещь
 3. Модули верхнего уровня не должны зависеть от модулей нижнего уровня
 4. Подклассы должны заменять базовые классы без нарушения функциональности

Подсказка: Подумайте, какой принцип отвечает за расширяемость, а какой за изоляцию ответственности.

- **Single Responsibility Principle** — Каждый класс отвечает только за одну вещь
- **Open/Closed Principle** — Классы должны быть открыты для расширения, но закрыты для модификации
- **Dependency Inversion Principle** — Модули верхнего уровня не должны зависеть от модулей нижнего уровня
- **Liskov Substitution Principle** — Подклассы должны заменять базовые классы без нарушения функциональности

Шаг 03:

Миссия 3

Сопоставьте типичные ошибки с последствиями:

Ответ:

- Смешение UI и бизнес-логики
 - Дублирование кода
 - Жёсткие зависимости между модулями
 - Отсутствие абстракций
1. Сложно тестировать и модифицировать код
 2. Повышенная вероятность ошибок при изменениях
 3. Трудно заменить один модуль другим
 4. Низкая поддерживаемость и масштабируемость

Подсказка: Думайте о последствиях нарушения слоёв и принципов чистой архитектуры.

- **Смешение UI и бизнес-логики** — Сложно тестировать и модифицировать код
- **Дублирование кода** — Повышенная вероятность ошибок при изменениях

- **Жёсткие зависимости между модулями** — Трудно заменить один модуль другим
- **Отсутствие абстракций** — Низкая поддерживаемость и масштабируемость

🔗 Шаг 04:

Миссия 4

Сопоставьте компоненты слоёв с реальными примерами:

Ответ:

- UI Layer
 - Domain Layer
 - Data Layer
 - Service Layer
1. React-компоненты, шаблоны HTML
 2. Сервисы оплаты и бизнес-правила корзины
 3. База данных, локальное хранилище
 4. HTTP-клиенты, интеграция с внешними API

Подсказка: Смотрите на то, что выполняет каждый слой на практике.

- **UI Layer** — React-компоненты, шаблоны HTML
- **Domain Layer** — Сервисы оплаты и бизнес-правила корзины
- **Data Layer** — База данных, локальное хранилище
- **Service Layer** — HTTP-клиенты, интеграция с внешними API

🔗 Шаг 05:

Миссия 5

Сопоставьте задачи слоёв с их преимуществами:

Ответ:

- Слой данных
 - Слой бизнес-логики
 - Слой представления
 - Слой сервисов
1. Изоляция данных от UI и логики
 2. Централизованная обработка правил приложения
 3. Лёгкая модификация интерфейса без затрагивания логики
 4. Повторное использование интеграций и API

Подсказка: Подумайте, какие преимущества даёт каждый слой.

- **Слой данных** — Изоляция данных от UI и логики
- **Слой бизнес-логики** — Централизованная обработка правил приложения
- **Слой представления** — Лёгкая модификация интерфейса без затрагивания логики
- **Слой сервисов** — Повторное использование интеграций и API

🔗 Шаг 06:

Миссия 6

Расположите слои приложения в порядке их зависимости друг от друга:

1. UI Layer

2. Service Layer
3. Domain Layer
4. Data Layer

Подсказка: Смотрите, кто зависит от кого. UI не должен напрямую обращаться к данным.

Ответ:

- Data Layer
- Domain Layer
- Service Layer
- UI Layer

Шаг 07:

Миссия 7

Расположите этапы внедрения чистой архитектуры:

1. Определение слоёв и их обязанностей
2. Разделение кода по слоям
3. Внедрение зависимостей через абстракции
4. Тестирование и рефакторинг

Подсказка: Сначала планируем, затем структурируем, потом изолируем зависимости и проверяем.

Ответ:

- Определение слоёв и их обязанностей
- Разделение кода по слоям
- Внедрение зависимостей через абстракции
- Тестирование и рефакторинг

Шаг 08:

Миссия 8

Расположите действия при добавлении нового сервиса в чистую архитектуру:

1. Создание интерфейса сервиса
2. Реализация конкретного сервиса
3. Подключение сервиса через Dependency Injection
4. Использование сервиса в UI или Domain Layer

Подсказка: Сначала абстракция, потом реализация, потом подключение и использование.

Ответ:

- Создание интерфейса сервиса
- Реализация конкретного сервиса
- Подключение сервиса через Dependency Injection
- Использование сервиса в UI или Domain Layer

Шаг 09:

Миссия 9

Никита только что присоединился к команде и видит, что фронтенд напрямую обращается к базе данных. Что лучше всего сделать сначала?

Подсказка: Сначала нужно понять, где код нарушает принципы разделения слоев.

Ответ:

- **Создать сервисный слой между фронтом и базой данных**
- Добавить комментарии к существующему коду
- Переписать весь фронтенд с нуля
- Игнорировать проблему на время

Шаг 10:

Миссия 10

Анна видит, что контроллер начинает содержать бизнес-логику. Какое решение будет правильным?

Подсказка: Контроллер должен управлять потоком данных, а не бизнес-правилами.

Ответ:

- **Вынести бизнес-логику в отдельный сервисный слой**
- Добавить больше условий в контроллер
- Объединить контроллер с базой данных
- Игнорировать и оставить как есть

Шаг 11:

Миссия 11

Максим замечает, что в проекте нет отдельного слоя для работы с API. Как это можно исправить?

Подсказка: Подумайте о слоях как о «модулях», которые выполняют отдельные роли.

Ответ:

- **Создать отдельный модуль для работы с внешними API**
- Размещать вызовы API прямо в UI
- Использовать глобальные переменные для хранения ответов API
- Игнорировать и обращаться к API напрямую

Шаг 12:

Миссия 12

Команда хочет уменьшить связность между слоями. Какой приём лучше использовать?

Подсказка: Связность можно снизить с помощью интерфейсов и абстракций.

Ответ:

- **Внедрить интерфейсы между слоями**
- Объединить слои в один модуль
- Использовать глобальные функции для всех слоёв
- Прямо вызывать методы базы данных из UI

Шаг 13:

Миссия 13

Сопоставьте слои с их основной задачей:

Слои:

Ответ:

- Презентационный слой (UI)

- Сервисный слой (Business)
- Слой данных (Data Access)
- Интеграционный слой (API)

Задачи:

1. Обработка пользовательского ввода и отображение данных
2. Выполнение бизнес-правил и логики приложения
3. Взаимодействие с базой данных
4. Взаимодействие с внешними системами и сервисами

Подсказка: Каждый слой отвечает за строго определённую область, не смешивайтесь с другими.

- **Презентационный слой (UI)** — Обработка пользовательского ввода и отображение данных
- **Сервисный слой (Business)** — Выполнение бизнес-правил и логики приложения
- **Слой данных (Data Access)** — Взаимодействие с базой данных
- **Интеграционный слой (API)** — Взаимодействие с внешними системами и сервисами

🔗 Шаг 14:

Миссия 14

Сопоставьте принципы чистой архитектуры с их описанием:

Ответ:

- Разделение ответственности
 - Независимость от фреймворков
 - Тестируемость
 - Инверсия зависимостей
1. Каждый модуль отвечает только за свои задачи
 2. Слой бизнес-логики не должен зависеть от деталей реализации
 3. Слои можно тестировать независимо друг от друга
 4. Высокоуровневые модули не зависят от низкоуровневых

Подсказка: Подумайте о том, как архитектура помогает поддерживать код и облегчает изменения.

- **Разделение ответственности** — Каждый модуль отвечает только за свои задачи
- **Независимость от фреймворков** — Слой бизнес-логики не должен зависеть от деталей реализации
- **Тестируемость** — Слои можно тестировать независимо друг от друга
- **Инверсия зависимостей** — Высокоуровневые модули не зависят от низкоуровневых

🔗 Шаг 15:

Миссия 15

Сопоставьте типы зависимостей с примерами:

Ответ:

- Жёсткая зависимость
 - Слабая зависимость
 - Инверсия зависимостей
1. Класс A напрямую создаёт объект класса B

2. Класс A получает объект класса B через интерфейс
3. Использование фабрик или DI-контейнера для получения объекта

Подсказка: Обратите внимание, как объекты получают свои зависимости.

- **Жёсткая зависимость** — Класс A напрямую создаёт объект класса B
- **Слабая зависимость** — Класс A получает объект класса B через интерфейс
- **Инверсия зависимостей** — Использование фабрик или DI-контейнера для получения объекта

Шаг 16:

Миссия 16

Сопоставьте виды слоёв с их примером:

Ответ:

- UI слой
 - Сервисный слой
 - Слой доступа к данным
 - Интеграционный слой
1. React-компонент, отображающий таблицу пользователей
 2. Класс, вычисляющий стоимость заказа с учетом скидок
 3. ORM-класс, который выполняет запросы к базе данных
 4. Модуль, отправляющий запросы к внешнему API доставки

Подсказка: Подумайте, где логика находится ближе к пользователю, где — к данным.

- **UI слой** — React-компонент, отображающий таблицу пользователей
- **Сервисный слой** — Класс, вычисляющий стоимость заказа с учетом скидок
- **Слой доступа к данным** — ORM-класс, который выполняет запросы к базе данных
- **Интеграционный слой** — Модуль, отправляющий запросы к внешнему API доставки

Урок 3 - Обработка ошибок и логирование

Шаг 01:

Миссия 1

Ирина пишет функцию для загрузки данных с API и хочет защититься от возможных ошибок сети. Какой подход правильный?

Подсказка: Подумайте, как безопасно обработать неожиданные ситуации.

Ответ:

- **Использовать try...catch для обработки ошибок**
- Игнорировать ошибки и продолжить выполнение
- Выводить ошибки в консоль без обработки
- Закомментировать проблемный код

Шаг 02:

Миссия 2

Даниил хочет логировать ошибки приложения в файл. Какой подход будет правильным?

Подсказка: Логи нужны не для пользователя, а для разработчика.

Ответ:

- **Использовать отдельный модуль логирования для записи ошибок в файл**
- Выводить все ошибки пользователю в alert
- Игнорировать ошибки, пока пользователь не сообщит
- Логировать ошибки в глобальной переменной

🔮 Шаг 03:

Миссия 3

Алексей пишет обработчик ошибок для формы регистрации. Что важно учитывать?

Подсказка: Подумайте о том, что пользователь видит и что остается для разработчика.

Ответ:

- **Показывать пользователю дружелюбное сообщение, а детали логировать**
- Показывать стек ошибки пользователю
- Игнорировать ошибки и сбрасывать форму
- Закомментировать обработку ошибок

🔮 Шаг 04:

Миссия 4

Мария хочет централизовать логирование всех модулей приложения. Что будет лучшей практикой?

Подсказка: Централизация помогает легче искать и анализировать ошибки.

Ответ:

- **Создать единый модуль логирования и использовать его во всех модулях**
- Каждый модуль пишет в отдельный файл без координации
- Выводить логи в браузер пользователя
- Игнорировать несущественные ошибки

🔮 Шаг 05:

Миссия 5

Сопоставьте типы ошибок с примерами:

Ответ:

- Синтаксическая ошибка
 - Ошибка времени выполнения
 - Логическая ошибка
1. Неправильное использование оператора или пропущенная скобка
 2. Обращение к несуществующему объекту во время выполнения
 3. Функция возвращает неправильный результат

Подсказка: Подумайте, когда ошибка проявляется — при компиляции, запуске или логике.

- **Синтаксическая ошибка** — Неправильное использование оператора или пропущенная скобка
- **Ошибка времени выполнения** — Обращение к несуществующему объекту во время выполнения
- **Логическая ошибка** — Функция возвращает неправильный результат

Шаг 06:

Миссия 6

Сопоставьте методы логирования с их назначением:

Ответ:

- `console.log`
- `console.error`
- `console.warn`
- `console.info`

1. Вывод обычной информации для отладки
2. Вывод ошибок приложения
3. Вывод предупреждений
4. Вывод информационных сообщений

Подсказка: Обратите внимание на семантику каждого метода.

- **`console.log`** — Вывод обычной информации для отладки
- **`console.error`** — Вывод ошибок приложения
- **`console.warn`** — Вывод предупреждений
- **`console.info`** — Вывод информационных сообщений

Шаг 07:

Миссия 7

Сопоставьте уровни логирования с типом информации:

Ответ:

- `DEBUG`
- `INFO`
- `WARN`
- `ERROR`

1. Подробная информация для разработчика
2. Регулярная информация о работе приложения
3. Предупреждения о потенциальных проблемах
4. Серьёзные ошибки, нарушающие работу

Подсказка: Уровни логов помогают фильтровать сообщения по важности.

- **`DEBUG`** — Подробная информация для разработчика
- **`INFO`** — Регулярная информация о работе приложения
- **`WARN`** — Предупреждения о потенциальных проблемах
- **`ERROR`** — Серьёзные ошибки, нарушающие работу

Шаг 08:

Миссия 8

Сопоставьте подходы обработки ошибок с их описанием:

Ответ:

- `Try-catch`
- `Promise.catch`
- `Global error handler`

- Validation before execution

1. Локальная обработка исключений в блоке кода
2. Обработка ошибок асинхронного кода
3. Обработка всех непойманных ошибок приложения
4. Предотвращение ошибок с помощью проверок

Подсказка: Ошибки можно поймать заранее, локально или глобально.

- **Try-catch** — Локальная обработка исключений в блоке кода
- **Promise.catch** — Обработка ошибок асинхронного кода
- **Global error handler** — Обработка всех непойманных ошибок приложения
- **Validation before execution** — Предотвращение ошибок с помощью проверок

Шаг 09:

Миссия 9

Расположите шаги обработки ошибки в правильном порядке:

1. Вызов кода, где может произойти ошибка
2. Локальная обработка с try-catch
3. Логирование ошибки в модуль логирования
4. Отправка дружелюбного сообщения пользователю

Подсказка: Сначала ошибка возникает, затем её ловят, логируют и информируют пользователя.

Ответ:

- Вызов кода, где может произойти ошибка
- Локальная обработка с try-catch
- Логирование ошибки в модуль логирования
- Отправка дружелюбного сообщения пользователю

Шаг 10:

Миссия 10

Расположите действия при централизованном логировании:

1. Настройка модуля логирования
2. Добавление вызова логирования в каждый модуль
3. Обработка ошибок локально
4. Отправка логов в хранилище или файл

Подсказка: Настройте модуль, используйте его в коде, обрабатывайте ошибки и сохраняйте логи.

Ответ:

- Настройка модуля логирования
- Добавление вызова логирования в каждый модуль
- Обработка ошибок локально
- Отправка логов в хранилище или файл

Шаг 11:

Миссия 11

Расположите порядок действий при работе с асинхронным кодом:

1. Вызов асинхронной функции
2. Обработка ошибок с помощью `.catch()` или `try-catch` внутри `async`
3. Логирование ошибок
4. Информирование пользователя о проблеме

Подсказка: Асинхронные ошибки тоже нужно ловить, логировать и уведомлять пользователя.

Ответ:

- Вызов асинхронной функции
- Обработка ошибок с помощью `.catch()` или `try-catch` внутри `async`
- Логирование ошибок
- Информирование пользователя о проблеме

Шаг 12:

Миссия 12

Расположите действия при анализе логов:

1. Сбор логов из файлов или хранилища
2. Фильтрация по уровню и модулю
3. Анализ и выявление проблем
4. Документирование найденных ошибок

Подсказка: Сначала собираем логи, затем фильтруем, анализируем и документируем.

Ответ:

- Сбор логов из файлов или хранилища
- Фильтрация по уровню и модулю
- Анализ и выявление проблем
- Документирование найденных ошибок

Шаг 13:

Миссия 13

Расположите этапы обработки глобальных ошибок в приложении:

1. Определение глобального обработчика ошибок
2. Локальная обработка критических ошибок
3. Логирование всех ошибок в единый модуль
4. Отправка уведомлений разработчикам

Подсказка: Глобальные ошибки обрабатываются после локальных, затем логируются и отправляются уведомления.

Ответ:

- Определение глобального обработчика ошибок
- Локальная обработка критических ошибок
- Логирование всех ошибок в единый модуль
- Отправка уведомлений разработчикам

Шаг 14:

Миссия 14

Расположите действия при интеграции стороннего API с обработкой ошибок:

1. Вызов API
2. Проверка ответа и статуса
3. Обработка ошибок или таймаутов
4. Логирование проблем
5. Использование данных в приложении

Подсказка: API сначала вызывается, затем проверяется ответ, ошибки обрабатываются и логируются, после чего данные используются.

Ответ:

- Вызов API
- Проверка ответа и статуса
- Обработка ошибок или таймаутов
- Логирование проблем
- Использование данных в приложении

Шаг 15:

Миссия 15

Лена замечает, что при отправке формы иногда падает приложение без объяснения причины. Что нужно сделать в первую очередь?

Подсказка: Подумайте, как можно поймать неожиданные ошибки.

Ответ:

- **Добавить блок try/catch для обработки возможных ошибок**
- Игнорировать ошибку, она возникает редко
- Удалить форму из приложения
- Вывести alert без обработки ошибок

Шаг 16:

Миссия 16

Пётр добавляет логирование в проект. Какой подход будет правильным?

Подсказка: Подумайте, что полезно логировать, а что нет.

Ответ:

- **Логировать ошибки и важные события, но не все внутренние данные**
- Логировать абсолютно всё без фильтров
- Никогда не логировать ошибки
- Использовать console.log для всего в production

Урок 4 - Работа с глобальным состоянием (вне Redux)

Шаг 01:

Миссия 1

Анна заметила, что несколько компонентов приложения одновременно изменяют одну переменную. Что стоит использовать для централизованного управления состоянием?

Подсказка: Подумайте о глобальном объекте, доступном всем компонентам.

Ответ:

- **Создать глобальный объект состояния и методы для его изменения**

- Использовать локальные переменные в каждом компоненте
- Копировать состояние между компонентами вручную
- Не хранить состояние, полагаться на DOM

🔗 Шаг 02:

Миссия 2

Влад хочет, чтобы компоненты автоматически обновлялись при изменении состояния. Как это лучше реализовать без Redux?

Подсказка: Подумайте о подписках или наблюдателях за состоянием.

Ответ:

- **Реализовать паттерн Observer для глобального состояния**
- Каждый компонент сам проверяет состояние через `setInterval`
- Принудительно перерендеривать весь DOM
- Игнорировать обновления состояния

🔗 Шаг 03:

Миссия 3

Даша использует глобальное состояние для хранения темы приложения. Как безопасно обновить его из компонента?

Подсказка: Подумайте о функциях-обёртках для изменения состояния.

Ответ:

- **Создать функцию `setState`, которая обновляет состояние и уведомляет подписчиков**
- Прямо изменять свойства глобального объекта в компоненте
- Копировать объект состояния и модифицировать локально
- Удалить глобальное состояние и использовать только локальное

🔗 Шаг 04:

Миссия 4

Миша хочет добавить логирование изменений глобального состояния. Как правильно это сделать?

Подсказка: Подумайте о промежуточном слое между изменением состояния и подписчиками.

Ответ:

- **В `setState` добавить вызов функции логирования при каждом обновлении**
- Логировать только при ошибках приложения
- Использовать `console.log` в каждом компоненте отдельно
- Не логировать, чтобы не замедлять работу приложения

🔗 Шаг 05:

Миссия 5

Сопоставьте подходы к глобальному состоянию с их описанием:

Ответ:

- Объект состояния
- Observer
- Callback-функции

- Event Bus

1. Компоненты подписываются на изменения состояния и автоматически обновляются
2. Централизованный объект, где хранится вся информация приложения
3. Функции, передаваемые компонентам для уведомления об изменении состояния
4. События рассылаются всем подписанным компонентам

Подсказка: Подумайте, где используется прямая ссылка на объект, а где – механизм уведомления.

- **Объект состояния** — Централизованный объект, где хранится вся информация приложения
- **Observer** — Компоненты подписываются на изменения состояния и автоматически обновляются
- **Callback-функции** — Функции, передаваемые компонентам для уведомления об изменении состояния
- **Event Bus** — События рассылаются всем подписанным компонентам

🔮 Шаг 06:

Миссия 6

Сопоставьте виды глобального состояния с примерами:

Ответ:

- Текстовая тема приложения
 - Список товаров в корзине
 - Флаг авторизации пользователя
 - Счётчик лайков на посте
1. UI-состояние
 2. Состояние данных
 3. Состояние пользователя
 4. Состояние компонента

Подсказка: Подумайте, что влияет на интерфейс, что на данные, а что на пользователя.

- **Текстовая тема приложения** — UI-состояние
- **Список товаров в корзине** — Состояние данных
- **Флаг авторизации пользователя** — Состояние пользователя
- **Счётчик лайков на посте** — Состояние компонента

🔮 Шаг 07:

Миссия 7

Сопоставьте подходы обновления состояния с их особенностями:

Ответ:

- Иммуutableное обновление
 - Прямое изменение объекта
 - Функциональное обновление
 - Паттерн Pub/Sub
1. Создаётся новый объект состояния вместо изменения существующего
 2. Меняем свойства объекта напрямую
 3. Обновление состояния через функцию, получающую старое состояние

4. Компоненты подписываются и получают уведомления при изменении состояния

Подсказка: Подумайте о чистоте данных и способах уведомления компонентов.

- **Иммутабельное обновление** — Создаётся новый объект состояния вместо изменения существующего
- **Прямое изменение объекта** — Меняем свойства объекта напрямую
- **Функциональное обновление** — Обновление состояния через функцию, получающую старое состояние
- **Паттерн Pub/Sub** — Компоненты подписываются и получают уведомления при изменении состояния

🔮 Шаг 08:

Миссия 8

Расположите шаги обновления глобального состояния в правильном порядке:

1. Компонент вызывает функцию обновления состояния
2. Состояние обновляется (новый объект или изменение существующего)
3. Уведомление всех подписчиков об изменении состояния
4. Компоненты получают новое состояние и перерендериваются

Подсказка: Подумайте о цепочке: кто инициирует изменение, кто реагирует.

Ответ:

- Компонент вызывает функцию обновления состояния
- Состояние обновляется (новый объект или изменение существующего)
- Уведомление всех подписчиков об изменении состояния
- Компоненты получают новое состояние и перерендериваются

🔮 Шаг 09:

Миссия 9

Аня создает игру и хочет хранить глобальные настройки приложения. Какой подход будет правильным?

Подсказка: Подумайте, где логично хранить данные, чтобы их можно было использовать в разных частях программы.

Ответ:

- **Создать объект с глобальными настройками и экспортировать его**
- Создавать настройки локально в каждой функции
- Хранить настройки только в localStorage без объекта
- Не использовать глобальные данные, дублировать их везде

🔮 Шаг 10:

Миссия 10

Максим хочет, чтобы изменение глобальной переменной сразу отражалось в нескольких компонентах. Как лучше это реализовать?

Подсказка: Подумайте, как сделать данные реактивными без Redux.

Ответ:

- **Использовать объект состояния и подписчиков для уведомления компонентов**
- Каждый компонент хранит своё локальное состояние и не делится данными
- Просто менять глобальную переменную без уведомлений компонентов

- Использовать prompt для изменения переменной вручную

🔗 Шаг 11:

Миссия 11

Саша хочет избежать непредсказуемых изменений глобального состояния. Что нужно сделать?

Подсказка: Подумайте о контроле доступа и защите данных.

Ответ:

- **Использовать методы для чтения и изменения состояния, а не менять объект напрямую**
- Менять глобальный объект напрямую из любой функции
- Хранить глобальное состояние в переменной window без защиты
- Создать несколько объектов состояния без контроля доступа

🔗 Шаг 12:

Миссия 12

Лена разрабатывает игру и хочет логировать все изменения глобального состояния. Какой подход будет правильным?

Подсказка: Подумайте о точках, где состояние может изменяться.

Ответ:

- **Использовать функцию-обёртку при изменении состояния для логирования**
- Логировать случайным образом из разных компонентов
- Не логировать, только наблюдать через console.log
- Логировать только при запуске приложения

🔗 Шаг 13:

Миссия 13

Сопоставьте понятия с их примерами:

Ответ:

- Глобальное состояние
 - Локальное состояние
 - Подписчик
 - Метод изменения состояния
1. Объект с настройками игры, доступный во всех компонентах
 2. Состояние конкретного компонента формы
 3. Функция, вызываемая при изменении состояния
 4. Функция, которая изменяет значение в глобальном объекте

Подсказка: Подумайте, что доступно только внутри компонента, а что должно быть видимо всем.

- **Глобальное состояние** — Объект с настройками игры, доступный во всех компонентах
- **Локальное состояние** — Состояние конкретного компонента формы
- **Подписчик** — Функция, вызываемая при изменении состояния
- **Метод изменения состояния** — Функция, которая изменяет значение в глобальном объекте

🔗 Шаг 14:

Миссия 14

Сопоставьте типы ошибок с примерами при работе с глобальным состоянием:

Ответ:

- Неинициализированная переменная
- Несанкционированное изменение
- Конфликт подписчиков
- Циклические зависимости

1. Попытка чтения значения до его присвоения
2. Компонент изменяет состояние напрямую, минуя методы
3. Два подписчика вызывают друг друга бесконечно
4. Состояние зависит от другого состояния, которое зависит от первого

Подсказка: Подумайте о правилах доступа и порядке обновления состояния.

- **Неинициализированная переменная** — Попытка чтения значения до его присвоения
- **Несанкционированное изменение** — Компонент изменяет состояние напрямую, минуя методы
- **Конфликт подписчиков** — Два подписчика вызывают друг друга бесконечно
- **Циклические зависимости** — Состояние зависит от другого состояния, которое зависит от первого

🔗 Шаг 15:

Миссия 15

Сопоставьте подходы к глобальному состоянию с их преимуществами:

Ответ:

- Простой объект
 - Функция с приватными переменными
 - EventEmitter/подписки
 - Proxy
1. Прозрачность и прямой доступ к данным
 2. Контроль доступа и инкапсуляция
 3. Уведомление всех подписчиков о изменении
 4. Отслеживание изменений с минимальным кодом

Подсказка: Подумайте о разных способах управления состоянием и их сильных сторонах.

- **Простой объект** — Прозрачность и прямой доступ к данным
- **Функция с приватными переменными** — Контроль доступа и инкапсуляция
- **EventEmitter/подписки** — Уведомление всех подписчиков о изменении
- **Proxy** — Отслеживание изменений с минимальным кодом

🔗 Шаг 16:

Миссия 16

Расположите шаги организации глобального состояния без Redux:

1. Определить данные, которые будут глобальными
2. Создать объект состояния с методами изменения

3. Настроить подписчиков для обновления компонентов
4. Использовать методы для изменения состояния вместо прямого доступа
5. Тестировать корректность изменений

Подсказка: Подумайте, что должно быть сначала: структура данных, затем уведомление компонентов, затем тестирование.

Ответ:

- Определить данные, которые будут глобальными
- Создать объект состояния с методами изменения
- Настроить подписчиков для обновления компонентов
- Использовать методы для изменения состояния вместо прямого доступа
- Тестировать корректность изменений

Модуль 12 - Безопасность во фронтенде

Урок 1 - XSS, CSRF, CORS — теория

Шаг 01:

Миссия 1

Игорь разрабатывает блог и хочет понять, что такое XSS.

Подсказка: Подумайте о том, как пользовательский ввод может повлиять на сайт.

Ответ:

- **Атака, при которой злоумышленник внедряет скрипты на сайт**
- Метод шифрования данных
- Технология для ускорения загрузки сайта
- Тип сетевого протокола

Шаг 02:

Миссия 2

Анна хочет защитить форму авторизации от CSRF. Что нужно сделать?

Подсказка: Подумайте о том, как убедиться, что запрос отправлен с доверенного сайта.

Ответ:

- **Использовать CSRF-токены для проверки запросов**
- Разрешить любые POST-запросы
- Шифровать HTML с помощью CSS
- Использовать только GET-запросы

Шаг 03:

Миссия 3

Сергей разрабатывает SPA и столкнулся с проблемой CORS. Что это значит?

Подсказка: Подумайте о взаимодействии браузера с сервером из разных доменов.

Ответ:

- **Браузер блокирует запросы к другому домену без специальных заголовков**
- Ошибка при подключении CSS
- Неправильный формат JSON
- Сбой в работе localStorage

Шаг 04:

Миссия 4

Сопоставьте термины безопасности с их описанием:

Ответ:

- XSS
 - CSRF
 - CORS
1. Атака, когда злоумышленник внедряет скрипты в страницу
 2. Механизм проверки источника запроса между доменами
 3. Атака, при которой выполняется незапланированный запрос от имени пользователя

Подсказка: Подумайте, какие атаки связаны с кодом на клиенте, а какие — с запросами.

- **XSS** — Атака, когда злоумышленник внедряет скрипты в страницу
- **CSRF** — Атака, при которой выполняется незапланированный запрос от имени пользователя
- **CORS** — Механизм проверки источника запроса между доменами

Шаг 05:

Миссия 5

Сопоставьте типы XSS с примерами:

Ответ:

- Stored XSS
 - Reflected XSS
 - DOM-based XSS
1. Вредоносный код сохраняется в базе данных и отображается на странице
 2. Код появляется только при клике по ссылке и не сохраняется на сервере
 3. Атака происходит при манипуляции DOM через JavaScript

Подсказка: Подумайте, где хранится вредоносный код — на сервере, в ссылке или в DOM.

- **Stored XSS** — Вредоносный код сохраняется в базе данных и отображается на странице
- **Reflected XSS** — Код появляется только при клике по ссылке и не сохраняется на сервере
- **DOM-based XSS** — Атака происходит при манипуляции DOM через JavaScript

Шаг 06:

Миссия 6

Сопоставьте способы защиты с типами атак:

Ответ:

- Экранирование HTML
 - CSRF-токены
 - Access-Control-Allow-Origin
1. Защита от XSS
 2. Защита от CSRF
 3. Разрешение CORS-запросов

Подсказка: Подумайте, что предотвращает внедрение кода, что проверяет источник, а что управляет кросс-доменными запросами.

- **Экранирование HTML** — Защита от XSS
- **CSRF-токены** — Защита от CSRF
- **Access-Control-Allow-Origin** — Разрешение CORS-запросов

Шаг 07:

Миссия 7

Игорь добавляет форму комментариев на сайте. Какой тип атаки он должен учитывать, если пользователь может вставлять HTML?

Подсказка: Подумайте, какие опасности связаны с выполнением чужого кода в браузере.

Ответ:

- **XSS (Cross-Site Scripting)**
- CSRF (Cross-Site Request Forgery)
- CORS ошибка
- SQL Injection

Шаг 08:

Миссия 8

Мария разрабатывает сайт и хочет защититься от подделки запросов от пользователя. Что нужно учитывать?

Подсказка: Подумайте о механизмах, которые проверяют источник запроса.

Ответ:

- **CSRF (Cross-Site Request Forgery)**
- XSS (Cross-Site Scripting)
- CORS ошибка
- Clickjacking

Шаг 09:

Миссия 9

Сергей хочет разрешить доступ к своему API с другого домена. Что нужно учитывать?

Подсказка: Подумайте о политиках браузера, которые ограничивают доступ к ресурсам между доменами.

Ответ:

- **CORS (Cross-Origin Resource Sharing)**
- XSS
- CSRF
- JSONP

Шаг 10:

Миссия 10

Анна добавляет обработку форм на фронтенде и хочет предотвратить XSS. Что нужно делать с пользовательским вводом?

Подсказка: Подумайте о фильтрации и безопасном отображении данных.

Ответ:

- **Экранировать HTML и использовать безопасный рендеринг**
- Просто обрезать теги HTML
- Ничего не делать, доверять пользователю
- Сохранять и вставлять ввод напрямую в innerHTML

🔮 **Шаг 11:**

Миссия 11

Сопоставьте типы атак с их описанием:

Ответ:

- XSS
 - CSRF
 - CORS
 - Clickjacking
1. Вставка вредоносного скрипта на страницу
 2. Подделка запроса от доверенного пользователя
 3. Ограничение браузером доступа к ресурсам с другого домена
 4. Манипуляция пользователем через невидимый iframe

Подсказка: Подумайте о механизмах браузера и поведении пользователей.

- **XSS** — Вставка вредоносного скрипта на страницу
- **CSRF** — Подделка запроса от доверенного пользователя
- **CORS** — Ограничение браузером доступа к ресурсам с другого домена
- **Clickjacking** — Манипуляция пользователем через невидимый iframe

🔮 **Шаг 12:**

Миссия 12

Сопоставьте типы XSS с примером:

Ответ:

- Reflected XSS
 - Stored XSS
 - DOM-based XSS
1. Вредоносный скрипт вставляется и хранится в базе данных сайта
 2. Вредоносный скрипт передается через URL и сразу отображается
 3. Вредоносный скрипт выполняется из-за манипуляции DOM на клиенте

Подсказка: Подумайте о том, где хранится вредоносный код и как он выполняется.

- **Reflected XSS** — Вредоносный скрипт передается через URL и сразу отображается
- **Stored XSS** — Вредоносный скрипт вставляется и хранится в базе данных сайта
- **DOM-based XSS** — Вредоносный скрипт выполняется из-за манипуляции DOM на клиенте

🔮 **Шаг 13:**

Миссия 12

Аня делает фронтенд для формы комментариев. Какой способ защиты от XSS будет безопаснее всего?

Подсказка: Подумайте о том, как обезопасить пользовательский ввод перед отображением на странице.

Ответ:

- **Экранировать спецсимволы и HTML-теги**
- Просто удалять все теги без проверки
- Не проверять ввод и доверять пользователю
- Использовать `eval` для обработки текста

Шаг 14:

Миссия 13

Петя хочет защитить свои формы от CSRF. Что нужно добавить в запросы?

Подсказка: Подумайте о токенах и проверке подлинности запроса на сервере.

Ответ:

- **Уникальный CSRF-токен в каждом запросе**
- Отправка только GET-запросов
- Хранение пароля в `localStorage`
- Отключение проверки сессий

Шаг 15:

Миссия 14

Игорь работает с API на другом домене. Что поможет избежать ошибок при кросс-доменных запросах?

Подсказка: Подумайте о настройках сервера и браузера.

Ответ:

- **Настройка CORS на сервере**
- Игнорирование ошибок в браузере
- Использование `iframe` без атрибутов безопасности
- Отправка запросов через `eval`

Шаг 16:

Миссия 15

Саша хочет обезопасить свои формы. Какой подход поможет предотвратить внедрение вредоносного JS-кода?

Подсказка: Подумайте о безопасной обработке ввода пользователей.

Ответ:

- **Использовать библиотеки для экранирования данных**
- Вставлять данные напрямую в `innerHTML`
- Отправлять данные через `eval`
- Игнорировать проверку ввода

Шаг 17:

Миссия 16

Сопоставьте угрозу с примером:

Ответ:

- XSS
 - CSRF
 - CORS
 - Replay атака
1. Вставка скрипта в форму комментариев
 2. Повторная отправка формы другим пользователем без согласия
 3. Браузер блокирует кросс-доменный запрос
 4. Перехват старого запроса и его повторное выполнение

Подсказка: Думайте о типичных уязвимостях веб-приложений.

- **XSS** — Вставка скрипта в форму комментариев
- **CSRF** — Повторная отправка формы другим пользователем без согласия
- **CORS** — Браузер блокирует кросс-доменный запрос
- **Replay атака** — Перехват старого запроса и его повторное выполнение

Урок 2 - Безопасная работа с API

Шаг 01:

Миссия 1

Алексей создает фронтенд для работы с API. Какой метод передачи токена наиболее безопасен?

Подсказка: Подумайте о том, чтобы токен не был доступен из JavaScript и не сохранялся в localStorage.

Ответ:

- **HTTP-only cookie**
- localStorage
- sessionStorage
- Просто в URL параметре

Шаг 02:

Миссия 2

Мария разрабатывает SPA и хочет защитить API от подделки запросов. Что нужно использовать?

Подсказка: Подумайте о том, чтобы проверять источник запроса на сервере.

Ответ:

- **CSRF-токен**
- CORS
- XSS фильтры
- JSONP

Шаг 03:

Миссия 3

Иван хочет подключить фронтенд к стороннему API с другого домена. Что нужно учитывать?

Подсказка: Подумайте о политике браузера, ограничивающей доступ к чужим ресурсам.

Ответ:

- **CORS (Cross-Origin Resource Sharing)**

- CSRF
- XSS
- WebSockets

Шаг 04:

Миссия 4

Светлана работает с публичным API и хочет защитить секретный ключ. Как лучше его хранить?

Подсказка: Подумайте, где ключ недоступен пользователю и JavaScript.

Ответ:

- **На сервере, передавать через защищенный endpoint**
- В коде фронтенда
- В localStorage
- В URL параметре

Шаг 05:

Миссия 5

Сопоставьте угрозу с методом защиты API:

Ответ:

- Подделка запросов
 - Перехват токена
 - Доступ с других доменов
 - Передача секретного ключа на клиент
1. Использование CSRF-токена
 2. HTTPS и HTTP-only cookie
 3. Настройка CORS на сервере
 4. Хранение ключа на сервере, а не на фронтенде

Подсказка: Подумайте, какой метод предотвращает каждую конкретную проблему.

- **Подделка запросов** — Использование CSRF-токена
- **Перехват токена** — HTTPS и HTTP-only cookie
- **Доступ с других доменов** — Настройка CORS на сервере
- **Передача секретного ключа на клиент** — Хранение ключа на сервере, а не на фронтенде

Шаг 06:

Миссия 6

Сопоставьте типы токенов с их особенностями:

Ответ:

- JWT
 - Оpaque token
 - Refresh token
 - Access token
1. Не расшифровывается клиентом, используется как идентификатор сессии
 2. Содержит закодированную информацию, может проверяться на фронтенде
 3. Используется для получения нового access token

4. Используется для доступа к защищенному ресурсу на короткий срок

Подсказка: Подумайте, где и как каждый токен используется и какие данные содержит.

- **JWT** — Содержит закодированную информацию, может проверяться на фронтенде
- **Opaque token** — Не расшифровывается клиентом, используется как идентификатор сессии
- **Refresh token** — Используется для получения нового access token
- **Access token** — Используется для доступа к защищенному ресурсу на короткий срок

Шаг 07:

Миссия 7

Сопоставьте заголовки HTTP с их назначением для безопасности API:

Ответ:

- Authorization
 - Content-Type
 - Access-Control-Allow-Origin
 - Set-Cookie
1. Передает токен или учетные данные для доступа
 2. Определяет формат передаваемых данных
 3. Разрешает доступ к ресурсу с другого домена
 4. Устанавливает cookie на клиенте с настройкой HttpOnly/secure

Подсказка: Подумайте, какой заголовок решает конкретную задачу безопасности.

- **Authorization** — Передает токен или учетные данные для доступа
- **Content-Type** — Определяет формат передаваемых данных
- **Access-Control-Allow-Origin** — Разрешает доступ к ресурсу с другого домена
- **Set-Cookie** — Устанавливает cookie на клиенте с настройкой HttpOnly/secure

Шаг 08:

Миссия 8

Расположите шаги безопасной работы с API:

1. Сервер проверяет токен доступа
2. Клиент отправляет запрос с токеном через HTTPS
3. Если токен устарел, клиент использует refresh token
4. Сервер отвечает с данными или ошибкой

Подсказка: Подумайте о последовательности: сначала клиент формирует запрос, сервер проверяет, потом используется refresh token при необходимости.

Ответ:

- Клиент отправляет запрос с токеном через HTTPS
- Сервер проверяет токен доступа
- Сервер отвечает с данными или ошибкой
- Если токен устарел, клиент использует refresh token

Шаг 09:

Миссия 9

Аня делает фронтенд для API. Какой метод аутентификации будет безопаснее всего для публичного API?

Подсказка: Подумайте о токенах и ключах, которые не раскрывают ваши данные.

Ответ:

- **Использовать API-ключи или OAuth токены**
- Передавать логин и пароль в URL
- Хранить пароль в localStorage
- Не использовать авторизацию

Шаг 10:

Миссия 10

Петя хочет, чтобы его фронтенд безопасно общался с API на другом домене. Что нужно учитывать?

Подсказка: Подумайте о политиках браузера, ограничивающих кросс-доменные запросы.

Ответ:

- **Настройку CORS на сервере**
- Игнорирование ошибок кросс-домена
- Отправку данных через GET-запросы без шифрования
- Хранение токенов в cookies без защиты

Шаг 11:

Миссия 11

Игорь разрабатывает фронтенд, который должен безопасно отправлять POST-запросы. Какой заголовок поможет защититься от CSRF?

Подсказка: Подумайте о специальных токенах и заголовках.

Ответ:

- **X-CSRF-Token**
- X-Ignore-CSRF
- Authorization: Basic
- Content-Type: text/plain

Шаг 12:

Миссия 12

Сопоставьте методы защиты API с их описанием:

Ответ:

- OAuth
 - API ключи
 - JWT
 - CORS
1. Механизм безопасной передачи прав доступа через токены
 2. Простой секретный ключ для идентификации клиента
 3. Токен, содержащий закодированные данные о пользователе
 4. Политика браузера для контроля кросс-доменных запросов

Подсказка: Подумайте о том, какие технологии помогают идентифицировать и авторизовать пользователей.

- **OAuth** — Механизм безопасной передачи прав доступа через токены
- **API ключи** — Простой секретный ключ для идентификации клиента
- **JWT** — Токен, содержащий закодированные данные о пользователе
- **CORS** — Политика браузера для контроля кросс-доменных запросов

Шаг 13:

Миссия 13

Сопоставьте потенциальные угрозы API с примерами:

Ответ:

- Утечка токенов
 - Неавторизованный доступ
 - Replay атака
 - SQL Injection через API
1. Использование перехваченных токенов для доступа к API
 2. Отправка GET-запросов без проверки прав
 3. Повторная отправка старого запроса для выполнения действий
 4. Вставка вредоносных данных в параметры запроса

Подсказка: Подумайте, какие атаки направлены на авторизацию и данные.

- **Утечка токенов** — Использование перехваченных токенов для доступа к API
- **Неавторизованный доступ** — Отправка GET-запросов без проверки прав
- **Replay атака** — Повторная отправка старого запроса для выполнения действий
- **SQL Injection через API** — Вставка вредоносных данных в параметры запроса

Шаг 14:

Миссия 14

Расположите шаги безопасного обращения с API в правильном порядке:

1. Получение токена и его хранение в защищенном месте
2. Валидация и фильтрация данных перед отправкой
3. Отправка запроса к API с нужными заголовками
4. Обработка ответа и ошибок безопасным способом

Подсказка: Подумайте о логической последовательности действий для безопасного запроса.

Ответ:

- Получение токена и его хранение в защищенном месте
- Валидация и фильтрация данных перед отправкой
- Отправка запроса к API с нужными заголовками
- Обработка ответа и ошибок безопасным способом

Шаг 15:

Миссия 15

Расположите шаги предотвращения CSRF атак при работе с API:

1. Генерация уникального CSRF токена на сервере
2. Включение токена в каждый защищенный запрос с фронтенда
3. Проверка токена на сервере при получении запроса

4. Отклонение запросов без корректного токена

Подсказка: Думайте о механизме проверки подлинности каждого запроса.

Ответ:

- Генерация уникального CSRF токена на сервере
- Включение токена в каждый защищенный запрос с фронтенда
- Проверка токена на сервере при получении запроса
- Отклонение запросов без корректного токена

Урок 3 - Sanitization и Content Security Policy

Шаг 01:

Миссия 1

Оля добавляет комментарии пользователей на страницу. Какой способ предотвратить XSS будет правильным?

Подсказка: Подумайте, как безопасно отобразить текст, введённый пользователем.

Ответ:

- **Экранировать HTML-теги и спецсимволы**
- Вставлять текст через innerHTML без обработки
- Удалять все символы кроме букв
- Использовать eval для рендеринга

Шаг 02:

Миссия 2

Миша хочет, чтобы его форма была безопасной. Какой приём поможет избежать CSRF-атак?

Подсказка: Подумайте о проверке, что запрос пришёл от вашего приложения.

Ответ:

- **Добавление уникального CSRF-токена к форме**
- Отправка формы через GET-запрос
- Игнорирование проверки сессий
- Хранение пароля в localStorage

Шаг 03:

Миссия 3

Ксюша настраивает CSP (Content Security Policy). Что она должна запретить, чтобы предотвратить XSS?

Подсказка: Подумайте, какие источники скриптов могут быть опасны.

Ответ:

- **Не доверять inline-скриптам и сторонним источникам**
- Разрешить все внешние скрипты
- Использовать eval для динамических скриптов
- Игнорировать все ошибки CSP

Шаг 04:

Миссия 4

Саня работает с вводом пользователя для поиска. Как лучше защитить приложение?

Подсказка: Подумайте, как безопасно обрабатывать данные перед отправкой на сервер.

Ответ:

- **Санитизация и проверка вводимых данных**
- Вставлять текст напрямую в DOM
- Использовать eval для обработки поисковых запросов
- Игнорировать спецсимволы в запросах

Шаг 05:

Миссия 5

Сопоставьте понятие с его функцией:

Ответ:

- Sanitization
 - CSP
 - Nonce
 - Hash для скрипта
1. Очистка пользовательского ввода от опасного кода
 2. Политика безопасности контента для ограничения источников
 3. Уникальный токен для разрешения конкретного скрипта
 4. Контроль целостности скрипта через хэш

Подсказка: Свяжите технологию с её целью безопасности.

- **Sanitization** — Очистка пользовательского ввода от опасного кода
- **CSP** — Политика безопасности контента для ограничения источников
- **Nonce** — Уникальный токен для разрешения конкретного скрипта
- **Hash для скрипта** — Контроль целостности скрипта через хэш

Шаг 06:

Миссия 6

Сопоставьте сценарий с типом угрозы:

Ответ:

- Вставка вредоносного скрипта в форму
 - Подделка формы с чужого сайта
 - Использование опасного внешнего скрипта
 - Повторный запуск перехваченного запроса
1. XSS
 2. CSRF
 3. CSP нарушения
 4. Replay атака

Подсказка: Подумайте, как каждая атака работает в веб-приложении.

- **Вставка вредоносного скрипта в форму** — XSS
- **Подделка формы с чужого сайта** — CSRF
- **Использование опасного внешнего скрипта** — CSP нарушения
- **Повторный запуск перехваченного запроса** — Replay атака

Шаг 07:

Миссия 7

Сопоставьте метод защиты с его особенностью:

Ответ:

- Экранирование
 - Санитизация
 - CSP с nonce
 - Хэширование скриптов
1. Замена спецсимволов на безопасные HTML-сущности
 2. Удаление или очистка опасных тегов и атрибутов
 3. Разрешение конкретного скрипта через токен
 4. Контроль неизменности скрипта через хэш

Подсказка: Свяжите метод с его действием.

- **Экранирование** — Замена спецсимволов на безопасные HTML-сущности
- **Санитизация** — Удаление или очистка опасных тегов и атрибутов
- **CSP с nonce** — Разрешение конкретного скрипта через токен
- **Хэширование скриптов** — Контроль неизменности скрипта через хэш

Шаг 08:

Миссия 8

Расположите шаги безопасной работы с пользовательским вводом:

1. Сбор данных от пользователя
2. Санитизация и экранирование ввода
3. Отправка данных на сервер
4. Рендеринг безопасного контента на странице

Подсказка: Сначала собираем, потом очищаем, потом отправляем, потом показываем безопасно.

Ответ:

- Сбор данных от пользователя
- Санитизация и экранирование ввода
- Отправка данных на сервер
- Рендеринг безопасного контента на странице

Шаг 09:

Миссия 9

Расположите шаги внедрения CSP для защиты:

1. Определение источников скриптов и стилей
2. Добавление директив CSP в заголовки сервера
3. Тестирование приложения на ошибки CSP
4. Мониторинг и корректировка правил при необходимости

Подсказка: Сперва определяем источники, потом добавляем политику, тестируем и поддерживаем.

Ответ:

- Определение источников скриптов и стилей
- Добавление директив CSP в заголовки сервера
- Тестирование приложения на ошибки CSP
- Мониторинг и корректировка правил при необходимости

 Шаг 10:

Миссия 10

Маша добавляет комментарии пользователей на сайт. Какой метод поможет обезопасить ввод от XSS?

Подсказка: Подумайте о том, как браузер воспринимает HTML и JS в строках.

Ответ:

- **Экранировать специальные символы HTML**
- Отправлять данные напрямую в innerHTML
- Игнорировать проверку и доверять пользователю
- Использовать eval для обработки текста

 Шаг 11:

Миссия 11

Дима настраивает Content Security Policy (CSP). Что обеспечит максимальную защиту от внедрения скриптов?

Подсказка: Подумайте, какие источники контента безопасны для включения на страницу.

Ответ:

- **Разрешать только доверенные домены и избегать unsafe-inline**
- Разрешать все скрипты через *
- Игнорировать CSP
- Вставлять скрипты через eval

 Шаг 12:

Миссия 12

Игорь хочет, чтобы форма отправки данных была безопасной. Что следует сделать перед отправкой данных на сервер?

Подсказка: Подумайте о проверке и очистке данных пользователя.

Ответ:

- **Провести санитизацию и валидацию ввода**
- Отправлять данные напрямую без проверки
- Использовать eval для форматирования
- Хранить данные в localStorage без шифрования

 Шаг 13:

Миссия 13

Катя хочет использовать CSP для защиты сайта. Что нужно запретить, чтобы избежать XSS?

Подсказка: Подумайте о небезопасных источниках скриптов.

Ответ:

- **Запретить unsafe-inline скрипты**
- Разрешить все домены для скриптов
- Игнорировать CSP заголовки
- Использовать eval для вставки скриптов

🔗 Шаг 14:

Миссия 14

Сопоставьте термин с его назначением:

Ответ:

- Санитизация (Sanitization)
 - Валидация (Validation)
 - Content Security Policy (CSP)
 - unsafe-inline
1. Очистка данных от потенциально вредоносного кода
 2. Проверка данных на соответствие правилам
 3. Политика безопасности для контроля источников контента
 4. Опасное разрешение внедрять скрипты в HTML

Подсказка: Думайте о том, что предотвращает XSS на уровне ввода и на уровне политики браузера.

- **Санитизация (Sanitization)** — Очистка данных от потенциально вредоносного кода
- **Валидация (Validation)** — Проверка данных на соответствие правилам
- **Content Security Policy (CSP)** — Политика безопасности для контроля источников контента
- **unsafe-inline** — Опасное разрешение внедрять скрипты в HTML

🔗 Шаг 15:

Миссия 15

Сопоставьте пример с типом угрозы:

Ответ:

- Незашифрованный ввод
 - Вставка в комментарий
 - Разрешение скриптов с любого домена
 - Повторное выполнение старого запроса
1. XSS
 2. Replay атака
 3. Слабая CSP
 4. Отсутствие санитизации

Подсказка: Подумайте о том, как браузер и сервер обрабатывают ввод и скрипты.

- **Незашифрованный ввод** — Отсутствие санитизации
- **Вставка в комментарий** — XSS
- **Разрешение скриптов с любого домена** — Слабая CSP
- **Повторное выполнение старого запроса** — Replay атака

Шаг 16:

Миссия 16

Сопоставьте тип CSP-директивы с её назначением:

Ответ:

- script-src
- style-src
- img-src
- connect-src

1. Контроль источников JavaScript
2. Контроль источников CSS
3. Контроль источников изображений
4. Контроль источников AJAX-запросов

Подсказка: Каждая директива отвечает за определённый тип контента.

- **script-src** — Контроль источников JavaScript
- **style-src** — Контроль источников CSS
- **img-src** — Контроль источников изображений
- **connect-src** — Контроль источников AJAX-запросов

Урок 4 - Безопасное хранение токенов

Шаг 01:

Миссия 1

Аня хранит токены аутентификации на клиенте. Какой способ безопаснее всего?

Подсказка: Подумайте о том, где браузер ограничивает доступ к данным.

Ответ:

- **Использовать HttpOnly cookies**
- Хранить в localStorage
- Хранить в sessionStorage без шифрования
- Вставлять токен прямо в HTML

Шаг 02:

Миссия 2

Игорь разрабатывает SPA и думает, где хранить JWT. Что снизит риск XSS?

Подсказка: Подумайте, какой способ хранения недоступен JS на странице.

Ответ:

- **HttpOnly cookie**
- localStorage
- sessionStorage
- window.name

Шаг 03:

Миссия 3

Сергей хочет защитить токены от CSRF. Какой механизм поможет?

Подсказка: Подумайте о проверке происхождения запроса на сервере.

Ответ:

- **Использовать SameSite cookies**
- Хранить токен в localStorage
- Открыть доступ ко всем доменам
- Отправлять токен в GET-запросе

🔗 **Шаг 04:**

Миссия 4

Катя хочет, чтобы токен нельзя было украсть через XSS. Что нужно сделать?

Подсказка: Подумайте о свойствах cookie.

Ответ:

- **Установить HttpOnly и Secure флаги**
- Хранить токен в localStorage
- Отправлять токен в URL
- Сохранять токен в JS-переменной

🔗 **Шаг 05:**

Миссия 5

Сопоставьте термин с его назначением:

Ответ:

- HttpOnly cookie
 - Secure cookie
 - localStorage
 - JWT
1. Недоступна через JavaScript
 2. Отправляется только по HTTPS
 3. Хранение данных на стороне клиента, доступно JS
 4. Токен для аутентификации

Подсказка: Думайте, что защищает токен от XSS и CSRF.

- **HttpOnly cookie** — Недоступна через JavaScript
- **Secure cookie** — Отправляется только по HTTPS
- **localStorage** — Хранение данных на стороне клиента, доступно JS
- **JWT** — Токен для аутентификации

🔗 **Шаг 06:**

Миссия 6

Сопоставьте угрозу с защитным механизмом:

Ответ:

- XSS
 - CSRF
 - Перехват токена в HTTP
 - Повторное использование старого токена
1. HttpOnly cookie и санитизация данных

2. SameSite cookie
3. HTTPS
4. Использование короткоживущих токенов и refresh token

Подсказка: Подумайте, какой механизм защищает от какой угрозы.

- **XSS** — HttpOnly cookie и санитизация данных
- **CSRF** — SameSite cookie
- **Перехват токена в HTTP** — HTTPS
- **Повторное использование старого токена** — Использование короткоживущих токенов и refresh token

Шаг 07:

Миссия 7

Сопоставьте тип токена с его использованием:

Ответ:

- Access token
 - Refresh token
 - Session cookie
 - JWT в localStorage
1. Доступ к API на короткое время
 2. Обновление access token без повторного входа
 3. Поддержка сессии на сервере
 4. Риск XSS, не рекомендуется

Подсказка: Думайте о сроке жизни и безопасности хранения.

- **Access token** — Доступ к API на короткое время
- **Refresh token** — Обновление access token без повторного входа
- **Session cookie** — Поддержка сессии на сервере
- **JWT в localStorage** — Риск XSS, не рекомендуется

Шаг 08:

Миссия 8

Расположите шаги безопасного хранения и обновления токенов:

1. Сохранить access token в HttpOnly cookie
2. Сохранить refresh token в HttpOnly cookie с длительным сроком жизни
3. Использовать HTTPS для передачи токенов
4. При истечении срока access token, использовать refresh token для обновления
5. Удалять токены при выходе пользователя

Подсказка: Подумайте, что происходит сначала, что потом, и как заканчивается сессия.

Ответ:

- Сохранить access token в HttpOnly cookie
- Сохранить refresh token в HttpOnly cookie с длительным сроком жизни
- Использовать HTTPS для передачи токенов
- При истечении срока access token, использовать refresh token для обновления
- Удалять токены при выходе пользователя

Шаг 09:

Миссия 9

Расположите действия разработчика для защиты токенов на клиенте:

1. Определить, какие токены нужны клиенту
2. Выбрать способ хранения (HttpOnly cookie, localStorage и т.д.)
3. Настроить Secure и SameSite флаги для cookie
4. Обеспечить передачу токенов только по HTTPS
5. Регулярно обновлять токены и очищать их при выходе

Подсказка: Начните с планирования, затем настройка безопасности и мониторинг.

Ответ:

- Определить, какие токены нужны клиенту
- Выбрать способ хранения (HttpOnly cookie, localStorage и т.д.)
- Настроить Secure и SameSite флаги для cookie
- Обеспечить передачу токенов только по HTTPS
- Регулярно обновлять токены и очищать их при выходе

Шаг 10:

Миссия 10

Аня хранит токены доступа в localStorage. Почему это может быть опасно?

Подсказка: Подумайте, какие скрипты могут получить доступ к localStorage.

Ответ:

- **JavaScript с сайта может прочитать токены и украсть их**
- localStorage автоматически шифрует данные
- Токены в localStorage не могут устареть
- Никаких рисков нет

Шаг 11:

Миссия 11

Ваня думает хранить токены в cookie. Что повысит безопасность токенов?

Подсказка: Подумайте о свойствах cookie, которые защищают их от JS.

Ответ:

- **Установить HttpOnly и Secure флаги**
- Разрешить доступ через JavaScript
- Хранить токен в localStorage вместо cookie
- Отправлять токен через GET-параметры

Шаг 12:

Миссия 12

Саша хочет, чтобы токены автоматически устаревали. Как лучше это сделать?

Подсказка: Подумайте о времени жизни токена и механизме обновления.

Ответ:

- **Установить срок действия токена на сервере и в cookie**
- Хранить токен навсегда без срока

- Разрешить пользователю самому решать, когда токен устаревает
- Не проверять срок действия токена

🔮 Шаг 13:

Миссия 13

Лена хранит токены в `sessionStorage`. Как это влияет на безопасность?

Подсказка: Подумайте о жизни токена и доступе через другие вкладки.

Ответ:

- **Токены исчезают при закрытии вкладки, но доступны для скриптов на странице**
- Токены защищены от всех скриптов
- Токены живут бесконечно
- Токены могут быть прочитаны сервером без авторизации

🔮 Шаг 14:

Миссия 14

Сопоставьте способ хранения токенов с его характеристикой:

Ответ:

- `localStorage`
 - `sessionStorage`
 - `HttpOnly cookie`
 - Local variable (JS memory)
1. Доступен только серверу или скрипту с `HttpOnly`
 2. Хранение токена на вкладку, исчезает при закрытии браузера
 3. Доступен всем скриптам на странице, хранится навсегда
 4. Доступен только текущей сессии скрипта, не сохраняется между перезагрузками

Подсказка: Думайте о времени жизни и доступе к токenu.

- **`localStorage`** — Доступен всем скриптам на странице, хранится навсегда
- **`sessionStorage`** — Хранение токена на вкладку, исчезает при закрытии браузера
- **`HttpOnly cookie`** — Доступен только серверу или скрипту с `HttpOnly`
- **`Local variable (JS memory)`** — Доступен только текущей сессии скрипта, не сохраняется между перезагрузками

🔮 Шаг 15:

Миссия 15

Сопоставьте угрозу с её причиной:

Ответ:

- XSS
 - CSRF
 - Утечка токена через `localStorage`
 - Прослушка HTTP
1. Маловероятно при использовании `HttpOnly cookie`
 2. Атакующий вставляет скрипт, который крадет токен
 3. Токен отправляется с каждой поддельной формой
 4. Передача токена по незащищённому каналу

Подсказка: Думайте, как токен может быть украден или подделан.

- **XSS** — Атакующий вставляет скрипт, который крадет токен
- **CSRF** — Токен отправляется с каждой поддельной формой
- **Утечка токена через localStorage** — Маловероятно при использовании HttpOnly cookie
- **Прослушка HTTP** — Передача токена по незащищённому каналу

Шаг 16:

Миссия 16

Сопоставьте метод защиты токенов с эффектом:

Ответ:

- Использовать HttpOnly cookie
 - Шифрование токена на клиенте
 - Регулярная ротация токенов
 - Использование короткоживущих токенов
1. Предотвращает кражу токена через JS
 2. Снижает риск использования украденного токена
 3. Обновление токенов повышает безопасность
 4. Токен быстро устаревает после кражи

Подсказка: Подумайте о временных и технических способах защиты.

- **Использовать HttpOnly cookie** — Предотвращает кражу токена через JS
- **Шифрование токена на клиенте** — Снижает риск использования украденного токена
- **Регулярная ротация токенов** — Обновление токенов повышает безопасность
- **Использование короткоживущих токенов** — Токен быстро устаревает после кражи

Модуль 13 - Инструменты разработчика

Урок 1 - ESLint и Prettier

Шаг 01:

Миссия 1

Иван хочет, чтобы его код был аккуратным и понятным для команды. Что ему стоит использовать?

Подсказка: Подумайте о правилах написания кода и автоматическом форматировании.

Ответ:

- **ESLint и Prettier**
- Только браузерные консоли
- Ручная проверка кода коллегами
- Использовать alert() для отладки

Шаг 02:

Миссия 2

Катя хочет автоматически исправлять форматирование кода. Что ей поможет?

Подсказка: Думайте о инструменте, который не только проверяет, но и исправляет стиль.

Ответ:

- **Prettier**
- ESLint без правил форматирования
- Git commit hooks без линтеров
- npm install без пакетов

🔧 **Шаг 03:**

Миссия 3

Максим замечает, что коллеги используют разные кавычки и отступы. Что поможет избежать конфликтов?

Подсказка: Подумайте о стандарте и автоматическом применении правил.

Ответ:

- **ESLint + Prettier с едиными правилами**
- Игнорировать форматирование
- Каждый пишет как хочет
- Использовать alert() для проверки отступов

🔧 **Шаг 04:**

Миссия 4

Сопоставьте инструмент с его функцией:

Ответ:

- ESLint
 - Prettier
 - npm script
 - VSCode extension
1. Автоматическая проверка кода на ошибки и несоответствие стандартам
 2. Автоформатирование кода по заданным правилам
 3. Автоматизация запуска команд и линтеров
 4. Интеграция инструментов прямо в редактор для удобства

Подсказка: Думайте о задачах каждого инструмента: проверка, форматирование, автоматизация, интеграция.

- **ESLint** — Автоматическая проверка кода на ошибки и несоответствие стандартам
- **Prettier** — Автоформатирование кода по заданным правилам
- **npm script** — Автоматизация запуска команд и линтеров
- **VSCode extension** — Интеграция инструментов прямо в редактор для удобства

🔧 **Шаг 05:**

Миссия 5

Сопоставьте файл конфигурации с его назначением:

Ответ:

- .eslintrc.json
- .prettierrc
- package.json

- settings.json (VSCode)

1. Правила линтинга и проверки кода
2. Настройки автоформатирования
3. Скрипты для запуска линтеров и форматоров
4. Интеграция линтеров/форматоров в редактор

Подсказка: Думайте о каждом типе файла как о месте хранения правил или настроек.

- **.eslintrc.json** — Правила линтинга и проверки кода
- **.prettierrc** — Настройки автоформатирования
- **package.json** — Скрипты для запуска линтеров и форматоров
- **settings.json (VSCode)** — Интеграция линтеров/форматоров в редактор

Шаг 06:

Миссия 6

Расположите шаги настройки ESLint + Prettier для нового проекта:

1. Инициализация npm проекта
2. Установка ESLint и Prettier
3. Создание конфигурационных файлов
4. Настройка правил в .eslintrc и .prettierrc
5. Добавление скриптов для проверки и автоформатирования
6. Интеграция в редактор и тестирование работы

Подсказка: Думайте логически: сначала проект, потом пакеты, потом файлы, правила, скрипты и интеграция.

Ответ:

- Инициализация npm проекта
- Установка ESLint и Prettier
- Создание конфигурационных файлов
- Настройка правил в .eslintrc и .prettierrc
- Добавление скриптов для проверки и автоформатирования
- Интеграция в редактор и тестирование работы

Шаг 07:

Миссия 7

Дима заметил, что в проекте много разных стилей кода. Как ему лучше стандартизировать код?

Подсказка: Подумайте о автоматических инструментах проверки кода.

Ответ:

- **Использовать ESLint с единым конфигом**
- Игнорировать разные стили
- Писать код по настроению
- Сравнивать вручную каждый файл

Шаг 08:

Миссия 8

Катя хочет, чтобы все файлы автоматически форматировались при сохранении. Что поможет в этом?

Подсказка: Подумайте о инструменте, который исправляет стиль кода.

Ответ:

- **Использовать Prettier с плагином для редактора**
- Исправлять код вручную
- Отключить проверку стиля
- Использовать ESLint только для ошибок синтаксиса

Шаг 09:

Миссия 9

Игорь добавил ESLint в проект, но он ругается на ошибки, которые он считает неважными. Что сделать?

Подсказка: Подумайте о том, как конфигурация ESLint может быть гибкой.

Ответ:

- **Настроить правила ESLint под проект**
- Игнорировать ошибки ESLint
- Удалить ESLint
- Писать код так, чтобы ESLint не ругался, но без правил

Шаг 10:

Миссия 10

Марина хочет, чтобы ESLint проверял и форматирование кода. Как это настроить?

Подсказка: Подумайте о совместной работе ESLint и Prettier.

Ответ:

- **Использовать плагин eslint-config-prettier**
- Отключить Prettier
- Игнорировать форматирование
- Писать код по вкусу

Шаг 11:

Миссия 11

Сопоставьте инструмент с его функцией:

Ответ:

- ESLint
 - Prettier
 - eslint-config-prettier
 - eslint-plugin-react
1. Форматирование кода автоматически
 2. Проверка кода на ошибки и несоответствие стандартам
 3. Совместимость ESLint и Prettier без конфликтов
 4. Правила специфичные для React

Подсказка: Думайте, что делает проверка кода, что — форматирование, а что — интеграция с библиотеками.

- **ESLint** — Проверка кода на ошибки и несоответствие стандартам
- **Prettier** — Форматирование кода автоматически

- **eslint-config-prettier** — Совместимость ESLint и Prettier без конфликтов
- **eslint-plugin-react** — Правила специфичные для React

Шаг 12:

Миссия 12

Расположите шаги настройки ESLint и Prettier в правильном порядке:

1. Установить ESLint и Prettier
2. Создать конфигурационный файл ESLint
3. Настроить плагины и правила
4. Установить плагин eslint-config-prettier
5. Применить автоформатирование через Prettier

Подсказка: Начните с установки, потом конфиг, плагины, интеграция, и только потом форматирование.

Ответ:

- Установить ESLint и Prettier
- Создать конфигурационный файл ESLint
- Настроить плагины и правила
- Установить плагин eslint-config-prettier
- Применить автоформатирование через Prettier

Шаг 13:

Миссия 12

Саша написал новый компонент, но хочет убедиться, что код соответствует стандартам. Что использовать?

Подсказка: Подумайте о статическом анализе кода.

Ответ:

- **ESLint**
- Prettier
- npm run build
- console.log

Шаг 14:

Миссия 13

Наташа хочет, чтобы код автоматически форматировался при сохранении. Какой инструмент поможет?

Подсказка: Думайте о стиле кода, а не об ошибках.

Ответ:

- **Prettier**
- ESLint
- Git
- Node.js

Шаг 15:

Миссия 14

Иван хочет одновременно проверять ошибки и форматировать код. Как правильно организовать процесс?

Подсказка: Можно комбинировать разные инструменты через один workflow.

Ответ:

- **Использовать ESLint + Prettier**
- Использовать только ESLint
- Использовать только Prettier
- Игнорировать ошибки и форматирование

Шаг 16:

Миссия 15

Алина хочет, чтобы код форматировался и проверялся перед коммитом. Что подключить?

Подсказка: Подумайте о связке Git hooks и автоматизации.

Ответ:

- **Использовать Husky + lint-staged с ESLint и Prettier**
- Вручную запускать ESLint
- Вручную запускать Prettier
- Просто коммитить код

Урок 2 - Husky, Lint-staged, Git hooks

Шаг 01:

Миссия 1

Дима хочет автоматизировать проверку кода перед коммитом. Какой инструмент ему поможет?

Подсказка: Подумайте о инструменте, который запускает скрипты перед действиями Git.

Ответ:

- **Husky**
- Prettier
- ESLint
- npm run

Шаг 02:

Миссия 2

Катя хочет запускать ESLint только на изменённых файлах перед коммитом. Какой инструмент ей нужен?

Подсказка: Подумайте о инструменте, который фильтрует файлы перед запуском скриптов.

Ответ:

- **lint-staged**
- Husky
- Prettier
- Git hooks

Шаг 03:

Миссия 3

Игорь хочет, чтобы при каждом push выполнялись тесты и линтер. Что нужно настроить?

Подсказка: Подумайте о типе Git hook, который срабатывает перед отправкой кода на сервер.

Ответ:

- **pre-push hook через Husky**
- post-commit hook
- lint-staged без Husky
- npm start

🔮 Шаг 04:

Миссия 4

Марина хочет объединить Husky и lint-staged для удобства. Как это сделать?

Подсказка: Подумайте, кто запускает скрипт, а кто определяет файлы для проверки.

Ответ:

- **Husky запускает Git hook, lint-staged проверяет изменённые файлы**
- lint-staged запускает Git hook, Husky проверяет файлы
- Оба инструмента проверяют одинаково все файлы
- Никак нельзя объединить

🔮 Шаг 05:

Миссия 5

Сопоставьте инструмент с его функцией:

Ответ:

- Husky
 - lint-staged
 - pre-commit hook
 - pre-push hook
1. Запускает скрипты перед коммитом
 2. Фильтрует изменённые файлы для линтинга/форматирования
 3. Git hook перед коммитом
 4. Git hook перед push

Подсказка: Думайте, что делает инструмент, а что — тип Git hook.

- **Husky** — Запускает скрипты перед коммитом
- **lint-staged** — Фильтрует изменённые файлы для линтинга/форматирования
- **pre-commit hook** — Git hook перед коммитом
- **pre-push hook** — Git hook перед push

🔮 Шаг 06:

Миссия 6

Сопоставьте ошибку с инструментом, который её предотвращает:

Ответ:

- Неправильный формат кода
- Ошибки синтаксиса
- Незакоммиченные файлы с ошибками

- Отправка невалидного кода на сервер

1. lint-staged + Prettier
2. ESLint
3. Husky + lint-staged
4. Husky pre-push hook

Подсказка: Подумайте, кто отвечает за стиль, кто за ошибки кода и кто — за Git workflow.

- **Неправильный формат кода** — lint-staged + Prettier
- **Ошибки синтаксиса** — ESLint
- **Незакоммиченные файлы с ошибками** — Husky + lint-staged
- **Отправка невалидного кода на сервер** — Husky pre-push hook

Шаг 07:

Миссия 7

Расположите шаги настройки Husky и lint-staged:

1. Установить Husky
2. Инициализировать Git hooks с помощью Husky
3. Установить lint-staged
4. Настроить скрипты для изменённых файлов
5. Добавить скрипты в pre-commit hook

Подсказка: Сначала установка, потом инициализация, настройка lint-staged и интеграция с hook.

Ответ:

- Установить Husky
- Инициализировать Git hooks с помощью Husky
- Установить lint-staged
- Настроить скрипты для изменённых файлов
- Добавить скрипты в pre-commit hook

Шаг 08:

Миссия 8

Расположите шаги работы разработчика с Git hooks и линтером:

1. Писать код
2. lint-staged проверяет изменённые файлы
3. Husky запускает pre-commit hook
4. Исправлять ошибки и форматирование
5. Коммитить код
6. Husky pre-push hook запускает тесты перед push
7. Push в репозиторий

Подсказка: Думайте о последовательности: код → проверка → исправление → commit → push.

Ответ:

- Писать код
- lint-staged проверяет изменённые файлы
- Husky запускает pre-commit hook
- Исправлять ошибки и форматирование
- Коммитить код

- Husky pre-push hook запускает тесты перед push
- Push в репозиторий

🔧 Шаг 09:

Миссия 9

Саша хочет, чтобы перед каждым коммитом код автоматически проверялся на ошибки. Какой инструмент поможет?

Подсказка: Подумайте о Git hooks и npm пакетах для автоматизации.

Ответ:

- **Husky**
- Prettier
- ESLint
- npm init

🔧 Шаг 10:

Миссия 10

Наташа хочет, чтобы только изменённые файлы проверялись перед коммитом. Что ей использовать?

Подсказка: Подумайте о том, как ускорить проверки перед коммитом.

Ответ:

- **lint-staged**
- ESLint
- Prettier
- npm test

🔧 Шаг 11:

Миссия 11

Иван хочет автоматически форматировать код перед коммитом. Как связать Prettier с Git hooks?

Подсказка: Подумайте о комбинации инструментов Husky и lint-staged.

Ответ:

- **Использовать Husky + lint-staged с Prettier**
- Запускать Prettier вручную
- Использовать только ESLint
- Добавить скрипт в package.json без hooks

🔧 Шаг 12:

Миссия 12

Алина хочет запретить коммит, если есть ошибки ESLint. Как это реализовать?

Подсказка: Подумайте о пред-коммит hooks.

Ответ:

- **Настроить Husky pre-commit hook с ESLint**
- Игнорировать ошибки и коммитить
- Запускать ESLint вручную после коммита
- Использовать lint-staged без Husky

Шаг 13:

Миссия 13

Сопоставьте инструмент с его задачей:

Ответ:

- Husky
 - lint-staged
 - Prettier
 - ESLint
1. Форматирование кода
 2. Проверка кода на ошибки и стандарты
 3. Выполнение скриптов на изменённых файлах перед коммитом
 4. Создание Git hooks для автоматизации

Подсказка: Думайте о том, кто отвечает за ошибки, стиль и интеграцию с Git.

- **Husky** — Создание Git hooks для автоматизации
- **lint-staged** — Выполнение скриптов на изменённых файлах перед коммитом
- **Prettier** — Форматирование кода
- **ESLint** — Проверка кода на ошибки и стандарты

Шаг 14:

Миссия 14

Сопоставьте действие с результатом:

Ответ:

- Добавление Husky pre-commit hook
 - Настройка lint-staged
 - Запуск Prettier через lint-staged
 - Ошибки ESLint блокируют коммит
1. Автоматическое форматирование только изменённых файлов
 2. Коммит не будет выполнен при ошибках
 3. Git hooks будут активны перед коммитом
 4. Скрипты запускаются только на изменённых файлах

Подсказка: Думайте о цепочке автоматизации: Git hook → lint-staged → Prettier/ESLint.

- **Добавление Husky pre-commit hook** — Git hooks будут активны перед коммитом
- **Настройка lint-staged** — Скрипты запускаются только на изменённых файлах
- **Запуск Prettier через lint-staged** — Автоматическое форматирование только изменённых файлов
- **Ошибки ESLint блокируют коммит** — Коммит не будет выполнен при ошибках

Шаг 15:

Миссия 15

Расположите шаги настройки Husky и lint-staged:

1. Установить Husky и lint-staged
2. Инициализировать Husky в проекте
3. Создать pre-commit hook

4. Настроить lint-staged для изменённых файлов
5. Подключить Prettier и ESLint к lint-staged

Подсказка: Начните с установки, потом инициализация, hooks, настройка файлов, интеграция с инструментами.

Ответ:

- Установить Husky и lint-staged
- Инициализировать Husky в проекте
- Создать pre-commit hook
- Настроить lint-staged для изменённых файлов
- Подключить Prettier и ESLint к lint-staged

Шаг 16:

Миссия 16

Расположите шаги работы разработчика с Husky и lint-staged:

1. Писать код
2. Сохранять изменения
3. pre-commit hook запускает lint-staged
4. lint-staged запускает Prettier и ESLint
5. Исправить ошибки и коммитить

Подсказка: Подумайте о полном цикле проверки кода перед коммитом.

Ответ:

- Писать код
- Сохранять изменения
- pre-commit hook запускает lint-staged
- lint-staged запускает Prettier и ESLint
- Исправить ошибки и коммитить

Урок 3 - Source maps, DevTools

Шаг 01:

Миссия 1

Алексей хочет отлаживать минифицированный код. Какой инструмент ему поможет увидеть исходный код?

Подсказка: Подумайте о том, как отображается исходник вместо минифицированного кода.

Ответ:

- **Source maps**
- Prettier
- Webpack
- Babel

Шаг 02:

Миссия 2

Наташа хочет отследить, где именно в исходном коде произошла ошибка в браузере. Что использовать?

Подсказка: Ищите инструмент, который связывает ошибки в минифицированном коде с исходником.

Ответ:

- **Source maps**
- DevTools Console
- Node.js debugger
- ESLint

Шаг 03:

Миссия 3

Иван хочет проанализировать производительность веб-приложения в реальном времени. Какой инструмент открыть?

Подсказка: Подумайте о встроенных инструментах браузера.

Ответ:

- **DevTools Performance**
- Prettier
- ESLint
- Node.js profiler

Шаг 04:

Миссия 4

Алина хочет отследить сетевые запросы и ответы API. Какой раздел DevTools открыть?

Подсказка: Думайте о вкладке, которая показывает HTTP-трафик.

Ответ:

- **Network**
- Console
- Elements
- Sources

Шаг 05:

Миссия 5

Сопоставьте понятие с описанием:

Ответ:

- Source maps
 - DevTools Console
 - Network panel
 - Performance panel
1. Позволяет видеть исходный код вместо минифицированного
 2. Выводит ошибки и предупреждения JavaScript
 3. Отслеживает сетевые запросы и ответы
 4. Анализирует производительность и время загрузки

Подсказка: Думайте о связке инструмент → цель.

- **Source maps** — Позволяет видеть исходный код вместо минифицированного
- **DevTools Console** — Выводит ошибки и предупреждения JavaScript

- **Network panel** — Отслеживает сетевые запросы и ответы
- **Performance panel** — Анализирует производительность и время загрузки

🔧 Шаг 06:

Миссия 6

Сопоставьте действие разработчика с инструментом:

Ответ:

- Проверка ошибок JS
- Отслеживание HTTP-запросов
- Анализ рендеринга страницы
- Отладка минифицированного кода

1. Console
2. Network
3. Performance
4. Source maps

Подсказка: Подумайте, что используется для ошибок, что для сети, а что для производительности.

- **Проверка ошибок JS** — Console
- **Отслеживание HTTP-запросов** — Network
- **Анализ рендеринга страницы** — Performance
- **Отладка минифицированного кода** — Source maps

🔧 Шаг 07:

Миссия 7

Сопоставьте тип панели DevTools с её назначением:

Ответ:

- Elements
- Console
- Network
- Sources

1. Просмотр и редактирование DOM
2. Отладка и вывод ошибок JS
3. Слежение за сетевыми запросами
4. Отладка кода и source maps

Подсказка: Вспомните, какая панель для DOM, какая для кода и сети.

- **Elements** — Просмотр и редактирование DOM
- **Console** — Отладка и вывод ошибок JS
- **Network** — Слежение за сетевыми запросами
- **Sources** — Отладка кода и source maps

🔧 Шаг 08:

Миссия 8

Расположите шаги отладки минифицированного кода с source maps:

1. Подключить source maps при сборке
2. Открыть DevTools → Sources
3. Найти ошибку в минифицированном коде
4. Проверить исходный код через source maps
5. Исправить ошибку в исходниках

Подсказка: Думайте о порядке: подключение → поиск → проверка → исправление.

Ответ:

- Подключить source maps при сборке
- Открыть DevTools → Sources
- Найти ошибку в минифицированном коде
- Проверить исходный код через source maps
- Исправить ошибку в исходниках

Шаг 09:

Миссия 9

Расположите шаги анализа производительности веб-страницы:

1. Открыть DevTools → Performance
2. Запустить запись действий страницы
3. Выполнить действия, вызывающие нагрузку
4. Остановить запись
5. Проанализировать результаты и найти узкие места

Подсказка: Думайте о последовательности записи действий и анализа.

Ответ:

- Открыть DevTools → Performance
- Запустить запись действий страницы
- Выполнить действия, вызывающие нагрузку
- Остановить запись
- Проанализировать результаты и найти узкие места

Шаг 10:

Миссия 10

Аня только начала дебажить код и хочет видеть исходный код вместо минифицированного. Что ей поможет?

Подсказка: Думайте о связи с компиляцией и сборкой.

Ответ:

- **Source maps**
- Prettier
- Husky
- Webpack DevServer

Шаг 11:

Миссия 11

Борис хочет поставить точку останова прямо в исходном TypeScript-коде в браузере. Что нужно подключить?

Подсказка: Подумайте о связи компилированного кода и исходника.

Ответ:

- **Source maps**
- ESLint
- Prettier
- Node.js debugger

Шаг 12:

Миссия 12

Света хочет отслеживать ошибки и значения переменных прямо в браузере. Какой инструмент использовать?

Подсказка: Думайте о встроенных инструментах браузера для разработчика.

Ответ:

- **DevTools**
- ESLint
- Husky
- Node.js CLI

Шаг 13:

Миссия 13

Дима хочет ускорить отладку и видеть, какой код выполняется пошагово. Какой режим DevTools ему нужен?

Подсказка: Подумайте о режиме, который позволяет пошаговое выполнение и проверку состояния переменных.

Ответ:

- **Debugger**
- Console.log
- Prettier
- Husky hook

Шаг 14:

Миссия 14

Сопоставьте термин с его описанием:

Ответ:

- Source maps
- DevTools
- Breakpoints
- Console

1. Позволяет видеть оригинальный код при минификации
2. Инструменты разработчика в браузере
3. Точки останова для отладки
4. Вывод логов и ошибок

Подсказка: Думайте о назначении каждого инструмента в процессе отладки.

- **Source maps** — Позволяет видеть оригинальный код при минификации
- **DevTools** — Инструменты разработчика в браузере
- **Breakpoints** — Точки останова для отладки
- **Console** — Вывод логов и ошибок

Шаг 15:

Миссия 15

Сопоставьте вкладку DevTools с её функцией:

Ответ:

- Elements
 - Network
 - Sources
 - Performance
1. Просмотр и редактирование DOM и CSS
 2. Отслеживание загрузки ресурсов и API-запросов
 3. Отладка JS-кода и работа с breakpoints
 4. Анализ производительности и таймингов

Подсказка: Вспомните, где какие данные показываются в DevTools.

- **Elements** — Просмотр и редактирование DOM и CSS
- **Network** — Отслеживание загрузки ресурсов и API-запросов
- **Sources** — Отладка JS-кода и работа с breakpoints
- **Performance** — Анализ производительности и таймингов

Шаг 16:

Миссия 16

Сопоставьте событие с его назначением:

Ответ:

- Pause on exceptions
 - Step over
 - Step into
 - Step out
1. Останавливает выполнение при возникновении ошибки
 2. Выполняет текущую строку и переходит к следующей
 3. Заходит внутрь вызова функции
 4. Выходит из текущей функции

Подсказка: Различайте остановку на ошибке и навигацию по коду.

- **Pause on exceptions** — Останавливает выполнение при возникновении ошибки
- **Step over** — Выполняет текущую строку и переходит к следующей
- **Step into** — Заходит внутрь вызова функции
- **Step out** — Выходит из текущей функции

Урок 4 - Профилирование производительности

Шаг 01:

Миссия 1

Алиса хочет понять, какие функции её приложения потребляют больше всего времени. Что ей поможет?

Подсказка: Думайте о средствах браузера для измерения производительности.

Ответ:

- **Performance профилирование в DevTools**
- ESLint
- Prettier
- Husky

Шаг 02:

Миссия 2

Борис хочет измерить время выполнения конкретного блока кода. Какой инструмент использовать?

Подсказка: Подумайте о встроенных возможностях DevTools для тайминга.

Ответ:

- **Console.time / Console.timeEnd**
- Source maps
- Debugger
- Linting

Шаг 03:

Миссия 3

Света хочет отследить нагрузку на память приложения при взаимодействии пользователя с интерфейсом. Что выбрать?

Подсказка: Думайте о вкладках профилирования в DevTools.

Ответ:

- **Memory вкладка в DevTools**
- Network вкладка
- Console
- Elements

Шаг 04:

Миссия 4

Сопоставьте термин профилирования с его назначением:

Ответ:

- CPU профайлинг
- Memory профайлинг
- Network профайлинг
- Performance timeline

1. Анализ использования процессора функциями

2. Отслеживание потребления памяти
3. Мониторинг запросов к серверу и загрузки ресурсов
4. Запись событий и измерение производительности страницы

Подсказка: Разделяйте измерение CPU, памяти, сети и событий.

- **CPU профайлинг** — Анализ использования процессора функциями
- **Memory профайлинг** — Отслеживание потребления памяти
- **Network профайлинг** — Мониторинг запросов к серверу и загрузки ресурсов
- **Performance timeline** — Запись событий и измерение производительности страницы

Шаг 05:

Миссия 5

Сопоставьте событие профилирования с его целью:

Ответ:

- Function Call
 - Recalculate Style
 - Layout
 - Paint
1. Вызов функции и измерение времени её выполнения
 2. Пересчет стилей после изменений DOM
 3. Определение размеров и позиций элементов
 4. Отрисовка элементов на экране

Подсказка: Подумайте о цепочке от функции до визуальной отрисовки.

- **Function Call** — Вызов функции и измерение времени её выполнения
- **Recalculate Style** — Пересчет стилей после изменений DOM
- **Layout** — Определение размеров и позиций элементов
- **Paint** — Отрисовка элементов на экране

Шаг 06:

Миссия 6

Сопоставьте вкладку DevTools с её профилирующей функцией:

Ответ:

- Performance
 - Memory
 - Network
 - Console
1. Анализ производительности и таймингов
 2. Проверка утечек памяти и потребления ресурсов
 3. Мониторинг HTTP-запросов
 4. Вывод ошибок и логов

Подсказка: Каждая вкладка DevTools отвечает за свой тип данных.

- **Performance** — Анализ производительности и таймингов
- **Memory** — Проверка утечек памяти и потребления ресурсов
- **Network** — Мониторинг HTTP-запросов

- **Console** — Вывод ошибок и логов

Шаг 07:

Миссия 7

Расположите шаги CPU профилирования в правильном порядке:

1. Открыть DevTools и перейти на вкладку Performance
2. Нажать Start Recording
3. Выполнить действия на странице
4. Нажать Stop Recording
5. Проанализировать результаты

Подсказка: Сначала включаем запись, потом выполняем действия, затем анализируем.

Ответ:

- Открыть DevTools и перейти на вкладку Performance
- Нажать Start Recording
- Выполнить действия на странице
- Нажать Stop Recording
- Проанализировать результаты

Шаг 08:

Миссия 8

Расположите шаги для выявления утечек памяти:

1. Перейти на вкладку Memory
2. Сделать снимок Heap Snapshot
3. Выполнить действия на странице
4. Сравнить снимки
5. Найти объекты, которые не освобождаются

Подсказка: Сначала делаем снимок, потом действия, затем анализируем различия.

Ответ:

- Перейти на вкладку Memory
- Сделать снимок Heap Snapshot
- Выполнить действия на странице
- Сравнить снимки
- Найти объекты, которые не освобождаются

Шаг 09:

Миссия 9

Расположите шаги для анализа производительности страницы через Performance Timeline:

1. Открыть DevTools
2. Перейти на вкладку Performance
3. Начать запись
4. Выполнить необходимые действия на странице
5. Остановить запись и изучить Flame Chart

Подсказка: Запись действий и последующий анализ — ключевой процесс.

Ответ:

- Открыть DevTools
- Перейти на вкладку Performance
- Начать запись
- Выполнить необходимые действия на странице
- Остановить запись и изучить Flame Chart

 Шаг 10:

Миссия 10

Ирина заметила, что её приложение медленно загружается. Какой инструмент поможет увидеть, что замедляет работу страницы?

Подсказка: Подумайте о встроенных инструментах браузера для анализа производительности.

Ответ:

- **Performance в DevTools**
- ESLint
- Prettier
- Husky hook

 Шаг 11:

Миссия 11

Петя хочет проверить, какая функция занимает больше всего времени при выполнении. Какой подход использовать?

Подсказка: Думайте о инструменте, который замеряет выполнение функций и создаёт таймлайны.

Ответ:

- **Профилирование через вкладку Performance**
- Source maps
- Console.log
- Step over

 Шаг 12:

Миссия 12

Светлана хочет узнать, какие ресурсы загружаются дольше всего на странице. Где это можно увидеть?

Подсказка: Подумайте о вкладке, где показана загрузка CSS, JS и изображений.

Ответ:

- **Network в DevTools**
- Sources
- Console
- Elements

 Шаг 13:

Миссия 13

Алекс хочет отследить, сколько времени занимает выполнение цикла в его функции. Что ему поможет?

Подсказка: Подумайте о замерах времени выполнения конкретных участков кода.

Ответ:

- **Метод `console.profile()` и вкладка Performance**
- ESLint
- Prettier
- Breakpoints

Шаг 14:

Миссия 14

Сопоставьте вкладку или метод с его назначением:

Ответ:

- Network
 - Performance
 - Console
 - `console.profile()`
1. Отслеживание загрузки ресурсов и API-запросов
 2. Анализ производительности JS и рендеринга
 3. Вывод логов и ошибок
 4. Создание профиля выполнения функции

Подсказка: Вспомните, где показываются сетевые ресурсы и где создаются профили выполнения.

- **Network** — Отслеживание загрузки ресурсов и API-запросов
- **Performance** — Анализ производительности JS и рендеринга
- **Console** — Вывод логов и ошибок
- **`console.profile()`** — Создание профиля выполнения функции

Шаг 15:

Миссия 15

Сопоставьте понятие профилирования с его функцией:

Ответ:

- Call Tree
 - Flame Chart
 - Timing
 - Memory
1. Дерево вызовов функций с временем выполнения
 2. Визуализация времени выполнения функций в виде графика
 3. Замер времени выполнения конкретных операций
 4. Анализ использования памяти и объектов

Подсказка: Разделите визуализацию и количественные показатели.

- **Call Tree** — Дерево вызовов функций с временем выполнения
- **Flame Chart** — Визуализация времени выполнения функций в виде графика
- **Timing** — Замер времени выполнения конкретных операций
- **Memory** — Анализ использования памяти и объектов

Шаг 16:

Миссия 16

Сопоставьте этапы анализа производительности с их описанием:

Ответ:

- Запись профиля
 - Анализ результатов
 - Оптимизация
 - Перепроверка
1. Запуск инструментов профилирования и выполнение действий на странице
 2. Просмотр Call Tree, Flame Chart, выявление узких мест
 3. Исправление проблемных участков кода
 4. Снова тестируем после внесённых изменений

Подсказка: Сначала замеряем, потом анализируем, исправляем и перепроверяем.

- **Запись профиля** — Запуск инструментов профилирования и выполнение действий на странице
- **Анализ результатов** — Просмотр Call Tree, Flame Chart, выявление узких мест
- **Оптимизация** — Исправление проблемных участков кода
- **Перепроверка** — Снова тестируем после внесённых изменений

Модуль 14 - Работа с формами и валидацией

Урок 1 - Валидация ручная и библиотечная (Yup)

Шаг 01:

Миссия 1

Оля создает форму регистрации. Какой первый шаг для проверки, что пользователь ввел все обязательные поля?

Подсказка: Подумайте о простейшей проверке без сторонних библиотек.

Ответ:

- **Ручная проверка значений полей в JavaScript**
- Использовать console.log
- Добавить CSS классы
- Запуск тестов через Jest

Шаг 02:

Миссия 2

Игорь хочет убедиться, что email введён корректно. Какой подход будет правильным?

Подсказка: Думайте о шаблонах для проверки текста.

Ответ:

- **Регулярное выражение для проверки формата email**
- console.log(email)
- alert(email)
- CSS border red

🔮 Шаг 03:

Миссия 3

Катя решает использовать библиотеку Yup для валидации формы. Что она сможет делать с Yup?

Подсказка: Подумайте о том, как библиотека упрощает проверку значений.

Ответ:

- **Определять схемы и проверять значения автоматически**
- Рисовать форму
- Запускать сервер
- Минифицировать код

🔮 Шаг 04:

Миссия 4

Миша хочет, чтобы поле пароля было обязательно и длиной минимум 8 символов. Как Yup может помочь?

Подсказка: Подумайте о методах схемы Yup для ограничений.

Ответ:

- **yup.string().required().min(8)**
- yup.number()
- yup.boolean()
- yup.array()

🔮 Шаг 05:

Миссия 5

Сопоставьте тип проверки с его примером:

Ответ:

- Обязательное поле
- Формат email
- Минимальная длина
- Максимальная длина

1. Поле должно быть заполнено пользователем
2. example@mail.com
3. Пароль не менее 8 символов
4. Имя не более 20 символов

Подсказка: Вспомните, какие проверки касаются формата, а какие — длины.

- **Обязательное поле** — Поле должно быть заполнено пользователем
- **Формат email** — example@mail.com
- **Минимальная длина** — Пароль не менее 8 символов
- **Максимальная длина** — Имя не более 20 символов

🔮 Шаг 06:

Миссия 6

Сопоставьте методы Yup с их функцией:

Ответ:

- `required()`
 - `min()`
 - `max()`
 - `email()`
1. Проверка, что значение введено
 2. Минимальная длина строки или число
 3. Максимальная длина строки или число
 4. Проверка формата email

Подсказка: Подумайте о прямом назначении каждого метода схемы.

- **`required()`** — Проверка, что значение введено
- **`min()`** — Минимальная длина строки или число
- **`max()`** — Максимальная длина строки или число
- **`email()`** — Проверка формата email

 Шаг 07:

Миссия 7

Расположите шаги валидации формы в правильном порядке:

1. Пользователь вводит данные
2. Проверка данных на соответствие правилам Yup
3. Вывод ошибок на экран, если есть
4. Отправка данных на сервер, если ошибок нет

Подсказка: Сначала ввод, потом проверка, потом реакция и отправка.

Ответ:

- Пользователь вводит данные
- Проверка данных на соответствие правилам Yup
- Вывод ошибок на экран, если есть
- Отправка данных на сервер, если ошибок нет

 Шаг 08:

Миссия 8

Аня создаёт форму регистрации. Какой первый шаг при ручной валидации формы?

Подсказка: Подумайте о том, с чего обычно начинается проверка данных пользователя.

Ответ:

- **Проверка обязательных полей на заполнение**
- Подключение Yup
- Стилизация кнопки submit
- Отправка формы на сервер

 Шаг 09:

Миссия 9

Борис использует Yup для валидации email. Как правильно определить правило?

Подсказка: Подумайте, какой метод Yup проверяет формат текста.

Ответ:

- **Yup.string().email().required()**
- Yup.number().min(0)
- Yup.boolean().required()
- Yup.date().required()

Шаг 10:

Миссия 10

Катя хочет проверить, что пароль не короче 8 символов. Как это реализовать с Yup?

Подсказка: Подумайте о методе Yup для проверки длины строки.

Ответ:

- **Yup.string().min(8).required()**
- Yup.number().min(8)
- Yup.string().max(8)
- Yup.boolean()

Шаг 11:

Миссия 11

Дима хочет проверить, что форма отправляется только при отсутствии ошибок валидации. Что нужно сделать?

Подсказка: Подумайте о проверке статуса валидации перед submit.

Ответ:

- **Проверять isValid или errors перед отправкой**
- Игнорировать ошибки
- Сразу отправлять данные на сервер
- Вызывать alert без проверки

Шаг 12:

Миссия 12

Сопоставьте тип проверки с её назначением:

Ответ:

- required
- email
- min
- matches

1. Проверка, что поле заполнено
2. Проверка формата электронной почты
3. Проверка минимальной длины строки
4. Проверка по регулярному выражению

Подсказка: Подумайте, какой метод Yup проверяет формат, а какой длину.

- **required** — Проверка, что поле заполнено
- **email** — Проверка формата электронной почты
- **min** — Проверка минимальной длины строки
- **matches** — Проверка по регулярному выражению

Шаг 13:

Миссия 13

Сопоставьте элемент формы с типичной ошибкой валидации:

Ответ:

- Поле email
- Поле пароль
- Поле возраст
- Поле имя

1. Некорректный формат email
2. Меньше 8 символов
3. Меньше минимального числа
4. Поле пустое

Подсказка: Подумайте, какие ошибки чаще всего встречаются для разных типов данных.

- **Поле email** — Некорректный формат email
- **Поле пароль** — Меньше 8 символов
- **Поле возраст** — Меньше минимального числа
- **Поле имя** — Поле пустое

Шаг 14:

Миссия 14

Сопоставьте этапы работы с формой и валидацией:

Ответ:

- Ручная проверка
- Использование Yup
- Отображение ошибок
- Отправка данных

1. Проверка значений с помощью JS-кода
2. Создание схемы и проверка через validate
3. Показ пользователю, что заполнено неверно
4. Если ошибок нет, данные отправляются на сервер

Подсказка: Сначала проверяем, потом показываем ошибки, затем отправляем.

- **Ручная проверка** — Проверка значений с помощью JS-кода
- **Использование Yup** — Создание схемы и проверка через validate
- **Отображение ошибок** — Показ пользователю, что заполнено неверно
- **Отправка данных** — Если ошибок нет, данные отправляются на сервер

Шаг 15:

Миссия 15

Расположите шаги валидации формы в правильном порядке:

1. Проверка обязательных полей
2. Проверка формата данных
3. Отображение ошибок
4. Использование Yup или кастомной функции

5. Отправка данных при отсутствии ошибок

Подсказка: Сначала проверяем поля, затем показываем ошибки и только после этого отправляем.

Ответ:

- Проверка обязательных полей
- Проверка формата данных
- Отображение ошибок
- Использование Yup или кастомной функции
- Отправка данных при отсутствии ошибок

Урок 2 - Сложные формы вложенные поля, динамика

Шаг 01:

Миссия 1

Антон создаёт форму заказа с вложенным объектом 'адрес'. Какой первый шаг при ручной валидации вложенного поля?

Подсказка: Сначала убедитесь, что все ключевые свойства объекта заполнены.

Ответ:

- **Проверить наличие street, city и zip**
- Сразу отправить данные на сервер
- Игнорировать вложенный объект
- Сделать submit без проверки

Шаг 02:

Миссия 2

Оля использует массив 'товары' в форме. Как правильно проверить, что каждый товар содержит название и цену?

Подсказка: Подумайте о переборе массива и проверке каждого элемента.

Ответ:

- **Перебрать массив и проверить свойства каждого объекта**
- Проверять только первый элемент массива
- Игнорировать вложенные объекты
- Отправлять форму без проверки

Шаг 03:

Миссия 3

Игорь добавляет динамические поля для контактов. Как проверить их валидность?

Подсказка: Динамические поля могут добавляться пользователем, проверяйте их все.

Ответ:

- **Перебрать все динамические элементы и проверить значения**
- Проверять только первый элемент
- Игнорировать динамику
- Заполнять пустые значения автоматически

Шаг 04:

Миссия 4

Сопоставьте тип поля с примером вложенной или динамической структуры:

Ответ:

- Вложенный объект
 - Массив объектов
 - Динамическое поле
 - Простое поле
1. Адрес с ключами `street`, `city`, `zip`
 2. Список товаров с `name` и `price`
 3. Контакт, который пользователь может добавить несколько раз
 4. Имя пользователя

Подсказка: Подумайте о структуре данных, которые требуют вложенности или могут повторяться.

- **Вложенный объект** — Адрес с ключами `street`, `city`, `zip`
- **Массив объектов** — Список товаров с `name` и `price`
- **Динамическое поле** — Контакт, который пользователь может добавить несколько раз
- **Простое поле** — Имя пользователя

Шаг 05:

Миссия 5

Сопоставьте шаги работы с динамическими полями формы:

Ответ:

- Добавление поля
 - Валидация
 - Удаление поля
 - Отправка данных
1. Пользователь добавляет новый элемент
 2. Проверяем все текущие значения
 3. Удаляем ненужный элемент
 4. Если ошибок нет, отправляем на сервер

Подсказка: Динамика требует внимательного порядка действий.

- **Добавление поля** — Пользователь добавляет новый элемент
- **Валидация** — Проверяем все текущие значения
- **Удаление поля** — Удаляем ненужный элемент
- **Отправка данных** — Если ошибок нет, отправляем на сервер

Шаг 06:

Миссия 6

Расположите шаги обработки сложной формы с вложенными и динамическими полями:

1. Сбор данных из формы
2. Валидация всех полей
3. Отображение ошибок пользователю

4. Подготовка данных для API
5. Отправка данных

Подсказка: Сначала собираем и проверяем данные, потом показываем ошибки и отправляем.

Ответ:

- Сбор данных из формы
- Валидация всех полей
- Отображение ошибок пользователю
- Подготовка данных для API
- Отправка данных

Шаг 07:

Миссия 7

Расположите порядок действий при работе с массивом вложенных объектов:

1. Добавление нового объекта пользователем
2. Заполнение полей объекта
3. Валидация объекта
4. Сохранение всех объектов для отправки
5. Отправка данных на сервер

Подсказка: Думайте о логике действий от создания до отправки.

Ответ:

- Добавление нового объекта пользователем
- Заполнение полей объекта
- Валидация объекта
- Сохранение всех объектов для отправки
- Отправка данных на сервер

Шаг 08:

Миссия 8

Расположите действия при динамическом удалении поля формы:

1. Пользователь нажимает 'удалить'
2. Удаляем элемент из состояния формы
3. Пересчитываем ошибки и предупреждения
4. Обновляем интерфейс

Подсказка: Сначала инициируем удаление, потом обновляем данные и интерфейс.

Ответ:

- Пользователь нажимает 'удалить'
- Удаляем элемент из состояния формы
- Пересчитываем ошибки и предупреждения
- Обновляем интерфейс

Шаг 09:

Миссия 9

Лена создает форму заказа с вложенными адресами доставки. Что нужно сделать первым шагом?

Подсказка: Подумайте, с чего обычно начинается работа с вложенными объектами в форме.

Ответ:

- **Инициализировать начальное состояние вложенных полей**
- Сразу отправлять форму на сервер
- Добавить кнопки динамического добавления адресов
- Сделать валидацию каждого поля вручную

🔮 **Шаг 10:**

Миссия 10

Игорь хочет добавить возможность динамически добавлять несколько контактов в форме. Какой подход правильный?

Подсказка: Подумайте о том, как динамически управлять массивами полей.

Ответ:

- **Использовать массив объектов в состоянии формы**
- Создавать отдельную форму для каждого контакта
- Игнорировать динамику и фиксировать одно поле
- Отправлять контакты без полей формы

🔮 **Шаг 11:**

Миссия 11

Саша хочет валидировать вложенные поля с помощью Yup. Какой метод подходит для массива объектов?

Подсказка: Подумайте, как Yup обрабатывает массивы и объекты внутри схемы.

Ответ:

- **Yup.array().of(Yup.object({...}))**
- Yup.string()
- Yup.boolean()
- Yup.date()

🔮 **Шаг 12:**

Миссия 12

Маша хочет, чтобы пользователь мог добавлять новые адреса, а форма автоматически их валидировала. Что следует сделать?

Подсказка: Подумайте, как связать динамику с схемой Yup.

Ответ:

- **Создать Yup.array().of(Yup.object({...})) и обновлять массив при добавлении**
- Валидацию добавлять вручную через alert
- Просто дублировать объект адреса несколько раз
- Не проверять новые поля до отправки

🔮 **Шаг 13:**

Миссия 13

Сопоставьте элемент формы с типичной динамической задачей:

Ответ:

- Массив адресов
 - Массив телефонов
 - Список участников
 - Список файлов
1. Добавление/удаление элементов массива
 2. Валидация каждого элемента массива
 3. Обновление состояния при изменении
 4. Загрузка и проверка файлов

Подсказка: Подумайте, какие операции чаще всего требуются для каждого типа динамических полей.

- **Массив адресов** — Добавление/удаление элементов массива
- **Массив телефонов** — Валидация каждого элемента массива
- **Список участников** — Обновление состояния при изменении
- **Список файлов** — Загрузка и проверка файлов

 Шаг 14:

Миссия 14

Сопоставьте методы управления динамическими формами с их назначением:

Ответ:

- push
 - remove
 - map
 - setValue
1. Добавляет новый элемент в массив полей
 2. Удаляет элемент из массива
 3. Перебирает элементы для рендера
 4. Обновляет значение конкретного поля

Подсказка: Подумайте, какие методы массива связаны с динамикой полей.

- **push** — Добавляет новый элемент в массив полей
- **remove** — Удаляет элемент из массива
- **map** — Перебирает элементы для рендера
- **setValue** — Обновляет значение конкретного поля

 Шаг 15:

Миссия 15

Расположите шаги работы с динамическими вложенными полями в правильном порядке:

1. Инициализация состояния формы
2. Добавление нового элемента массива
3. Валидация добавленного элемента
4. Отображение ошибок пользователю
5. Отправка данных на сервер при отсутствии ошибок

Подсказка: Сначала создаем форму, потом добавляем элементы, проверяем и только после этого отправляем.

Ответ:

- Инициализация состояния формы
- Добавление нового элемента массива
- Валидация добавленного элемента
- Отображение ошибок пользователю
- Отправка данных на сервер при отсутствии ошибок

 Шаг 16:

Миссия 16

Расположите действия пользователя и формы в правильном порядке при динамическом вводе:

1. Пользователь вводит данные в новое поле
2. Форма обновляет состояние
3. Валидация нового поля
4. Отображение ошибок рядом с полем
5. Если ошибок нет, форма готова к отправке

Подсказка: Действия пользователя инициируют цепочку обновлений и проверок.

Ответ:

- Пользователь вводит данные в новое поле
- Форма обновляет состояние
- Валидация нового поля
- Отображение ошибок рядом с полем
- Если ошибок нет, форма готова к отправке

 Урок 3 - UX паттерны для форм

 Шаг 01:

Миссия 1

Разработчик Лена создает форму регистрации. Какой UX-паттерн улучшит опыт пользователя при вводе электронной почты?

Подсказка: Подумайте о том, как минимизировать ошибки при вводе.

Ответ:

- **Подсказка формата ввода и проверка в реальном времени**
- Отправка формы без проверки и уведомление об ошибке позже
- Скрытие поля email и подсказок
- Использование сложного CAPTCHA на каждом шаге

 Шаг 02:

Миссия 2

Артем делает форму обратной связи. Как улучшить восприятие обязательных полей?

Подсказка: Подумайте, как визуально выделить важные поля.

Ответ:

- **Использовать звездочки или цветовое выделение для обязательных полей**
- Ставить обязательность в виде всплывающего окна после отправки
- Не указывать обязательные поля вовсе
- Скрывать обязательные поля под кнопкой «Подробнее»

Шаг 03:

Миссия 3

В форме заказа Игорь хочет ускорить ввод данных о доставке. Что поможет пользователю заполнить форму быстрее?

Подсказка: Подумайте о автозаполнении и сокращении кликов.

Ответ:

- **Использование автозаполнения и предложений из истории адресов**
- Запрашивать адрес полностью вручную без подсказок
- Скрывать поле города
- Заполнять адрес случайными данными

Шаг 04:

Миссия 4

Юлия создает форму подписки на рассылку. Как сделать поле email более дружелюбным?

Подсказка: Подумайте, как дать пользователю подсказку прямо в поле.

Ответ:

- **Использовать placeholder с примером адреса**
- Не подписывать поле и не давать подсказок
- Вставить CAPTCHA перед вводом email
- Использовать поле скрытым

Шаг 05:

Миссия 5

Алексей работает над формой логина. Как снизить вероятность ошибок при вводе пароля?

Подсказка: Подумайте о том, как помочь пользователю увидеть, что он вводит.

Ответ:

- **Добавить возможность показать/скрыть пароль**
- Сразу блокировать форму при любой ошибке
- Писать подсказку пароля на экране
- Отправлять пароль на email

Шаг 06:

Миссия 6

Сопоставьте UX-паттерн формы с его описанием:

Ответ:

- Инлайн-валидация
 - Placeholder
 - Группировка полей
 - Прогресс-бар заполнения
1. Подсказка внутри поля ввода, показывающая пример
 2. Проверка введенных данных прямо при вводе
 3. Разделение логических блоков формы для удобства
 4. Визуальное отображение, сколько шагов осталось

Подсказка: Обратите внимание, где пользователь получает мгновенную обратную связь, а где просто визуальное руководство.

- **Инлайн-валидация** — Проверка введенных данных прямо при вводе
- **Placeholder** — Подсказка внутри поля ввода, показывающая пример
- **Группировка полей** — Разделение логических блоков формы для удобства
- **Прогресс-бар заполнения** — Визуальное отображение, сколько шагов осталось

Шаг 07:

Миссия 7

Сопоставьте элементы формы с их преимуществами для пользователя:

Ответ:

- Выпадающий список
 - Чекбоксы
 - Радиокнопки
 - Автодополнение
1. Позволяет выбрать несколько вариантов
 2. Сокращает количество ошибок при выборе и ускоряет ввод
 3. Позволяет выбрать только один вариант из группы
 4. Представляет список заранее определенных опций

Подсказка: Подумайте, какие элементы подходят для одного выбора, а какие для нескольких.

- **Выпадающий список** — Представляет список заранее определенных опций
- **Чекбоксы** — Позволяет выбрать несколько вариантов
- **Радиокнопки** — Позволяет выбрать только один вариант из группы
- **Автодополнение** — Сокращает количество ошибок при выборе и ускоряет ввод

Шаг 08:

Миссия 8

Сопоставьте виды ошибок в форме с подходящими UX-паттернами:

Ответ:

- Ошибка формата email
 - Пропущенное обязательное поле
 - Слабый пароль
 - Несовпадение пароля и подтверждения
1. Инлайн-валидация с подсказкой правильного формата
 2. Выделение поля и сообщение о необходимости заполнения
 3. Показ индикатора силы пароля
 4. Сравнение полей и предупреждение при несовпадении

Подсказка: Ошибки лучше ловить сразу при вводе, чтобы пользователь не пересекал всю форму.

- **Ошибка формата email** — Инлайн-валидация с подсказкой правильного формата
- **Пропущенное обязательное поле** — Выделение поля и сообщение о необходимости заполнения
- **Слабый пароль** — Показ индикатора силы пароля
- **Несовпадение пароля и подтверждения** — Сравнение полей и предупреждение при несовпадении

Шаг 09:

Миссия 9

Сопоставьте термины UX с их описанием:

Ответ:

- Минимизация кликов
 - Мгновенная обратная связь
 - Прогрессивное раскрытие
 - Согласованность интерфейса
1. Показывать пользователю только необходимое и открывать остальное по мере надобности
 2. Сохранять привычные паттерны для легкого восприятия
 3. Сокращение действий пользователя до минимума
 4. Пользователь сразу видит результат своего действия

Подсказка: Подумайте, что уменьшает усилия пользователя и что сразу информирует о результате.

- **Минимизация кликов** — Сокращение действий пользователя до минимума
- **Мгновенная обратная связь** — Пользователь сразу видит результат своего действия
- **Прогрессивное раскрытие** — Показывать пользователю только необходимое и открывать остальное по мере надобности
- **Согласованность интерфейса** — Сохранять привычные паттерны для легкого восприятия

Шаг 10:

Миссия 10

Сопоставьте UX-элементы с типичными ошибками:

Ответ:

- Инлайн-подсказка
 - Многоступенчатая форма
 - Автозаполнение
 - Доступная ошибка
1. Снижение вероятности опечаток и неправильного формата
 2. Разделение длинной формы на логические шаги
 3. Показ ошибок прямо у поля с понятным сообщением
 4. Ускорение заполнения формы за счет истории данных

Подсказка: Подумайте, какой элемент предотвращает ошибки, а какой ускоряет работу.

- **Инлайн-подсказка** — Снижение вероятности опечаток и неправильного формата
- **Многоступенчатая форма** — Разделение длинной формы на логические шаги
- **Автозаполнение** — Ускорение заполнения формы за счет истории данных
- **Доступная ошибка** — Показ ошибок прямо у поля с понятным сообщением

Шаг 11:

Миссия 11

Расположите шаги создания удобной формы регистрации:

1. Добавление проверки полей и подсказок
2. Определение обязательных и опциональных полей
3. Тестирование формы с реальными пользователями
4. Разработка структуры и логики полей

Подсказка: Сначала нужно спланировать структуру, затем выделить обязательные поля, потом добавить подсказки и в конце тестировать.

Ответ:

- Разработка структуры и логики полей
- Определение обязательных и опциональных полей
- Добавление проверки полей и подсказок
- Тестирование формы с реальными пользователями

Шаг 12:

Миссия 12

Расположите шаги для реализации автодополнения в форме:

1. Подключение источника данных для подсказок
2. Отслеживание ввода пользователя в реальном времени
3. Отображение предложенных вариантов в выпадающем списке
4. Обработка выбора пользователем одного из вариантов

Подсказка: Автодополнение начинается с данных, затем обрабатывается ввод и отображается список, после чего пользователь выбирает вариант.

Ответ:

- Подключение источника данных для подсказок
- Отслеживание ввода пользователя в реальном времени
- Отображение предложенных вариантов в выпадающем списке
- Обработка выбора пользователем одного из вариантов

Шаг 13:

Миссия 13

Анна создает форму регистрации на сайте игры. Какой UX-паттерн поможет пользователю сразу понять, какие поля обязательны?

Подсказка: Обратите внимание на визуальные обозначения.

Ответ:

- **Использование звездочек * возле обязательных полей**
- Скрытие обязательных полей до конца формы
- Использование серого цвета для всех полей
- Добавление длинного текста-пояснения к каждому полю

Шаг 14:

Миссия 14

Максим добавляет форму обратной связи. Как сделать так, чтобы пользователь не забыл заполнить email?

Подсказка: Подумайте о моментальной обратной связи при вводе.

Ответ:

- **Показывать ошибку, если поле осталось пустым после фокуса**
- Не показывать ничего до отправки формы
- Сделать поле очень маленьким
- Удалить поле email

🔗 **Шаг 15:**

Миссия 15

Вася делает форму заказа с несколькими шагами. Какой паттерн улучшит UX?

Подсказка: Думайте о визуальном ориентире и прогрессе пользователя.

Ответ:

- **Пошаговая форма с индикатором прогресса**
- Все поля на одной длинной странице
- Скрывать поля, пока пользователь не отправит форму
- Добавить случайные всплывающие подсказки

🔗 **Шаг 16:**

Миссия 16

Лена разрабатывает форму логина. Какой UX-паттерн снизит количество ошибок при вводе пароля?

Подсказка: Подумайте о визуальной проверке правильности ввода.

Ответ:

- **Показ пароля при нажатии на значок глаза**
- Ничего не показывать до отправки
- Ставить случайные ограничения на длину
- Писать подсказку с паролем в письме

Модуль 15 - Навигация и роутинг

Урок 1 - history.pushState, location.hash

🔗 **Шаг 01:**

Миссия 1

Игорь создает одностраничное приложение (SPA). Какой метод JavaScript позволяет изменить URL без перезагрузки страницы?

Подсказка: Подумайте о методе, который управляет историей браузера.

Ответ:

- **history.pushState**
- location.reload
- window.open
- document.write

🔗 **Шаг 02:**

Миссия 2

Мария хочет, чтобы при нажатии на кнопку в SPA URL менялся на #about. Какой подход следует использовать?

Подсказка: Думайте о якорях и фрагментах в URL.

Ответ:

- **location.hash = 'about'**
- history.pushState({}, '', 'about')
- window.open('#about')
- document.write('#about')

Шаг 03:

Миссия 3

Петя хочет, чтобы при клике на ссылку страница не перезагружалась, но URL менялся. Какой метод подходит?

Подсказка: Подумайте, как SPA обновляет адрес без полной загрузки.

Ответ:

- **history.pushState**
- location.reload
- alert('clicked')
- window.location.href = URL

Шаг 04:

Миссия 4

Настя разрабатывает меню с вкладками. Как можно сделать так, чтобы при выборе вкладки URL обновлялся и можно было делиться ссылкой?

Подсказка: Подумайте о простом способе обозначить текущую вкладку в URL.

Ответ:

- **Использовать location.hash для каждой вкладки**
- Менять только внутренний div без URL
- Перезагружать страницу при каждой вкладке
- Использовать alert для отображения вкладки

Шаг 05:

Миссия 5

Саша хочет, чтобы пользователь мог вернуться к предыдущей вкладке с помощью кнопки браузера. Что нужно использовать?

Подсказка: Думайте о истории посещений в SPA.

Ответ:

- **history.pushState и события popstate**
- location.hash без слушателя событий
- window.alert('Назад')
- document.write('назад')

Шаг 06:

Миссия 6

Формулировка: Сопоставьте методы и свойства с их описанием.

Понятия:

Ответ:

- `history.pushState`
- `history.replaceState`
- `location.hash`
- `window.onpopstate`

Определения:

1. Добавляет новую запись в историю браузера без перезагрузки
2. Меняет текущую запись истории без добавления новой
3. Изменяет часть URL после `#`
4. Событие, которое срабатывает при навигации назад/вперед

 **Подсказка:**

Подумайте, какой метод управляет историей, а какой — только фрагментом URL.

- **`history.pushState`** — Добавляет новую запись в историю браузера без перезагрузки
- **`history.replaceState`** — Меняет текущую запись истории без добавления новой
- **`location.hash`** — Изменяет часть URL после `#`
- **`window.onpopstate`** — Событие, которое срабатывает при навигации назад/вперед

 **Шаг 07:**

Миссия 7

Формулировка: Сопоставьте типы URL с их поведением в SPA.

Ответ:

- hash URL
 - pushState URL
 - reload page URL
 - external link
1. Меняется часть после `#` без перезагрузки
 2. Меняется путь и параметры без перезагрузки
 3. Полная перезагрузка страницы
 4. Открывает другой сайт

 **Подсказка:**

Обратите внимание, какие действия требуют перезагрузки, а какие — нет.

- **hash URL** — Меняется часть после `#` без перезагрузки
- **pushState URL** — Меняется путь и параметры без перезагрузки
- **reload page URL** — Полная перезагрузка страницы
- **external link** — Открывает другой сайт

 **Шаг 08:**

Миссия 8

Формулировка: Сопоставьте события и действия с их эффектом в истории браузера.

Ответ:

- `history.pushState()`
- `history.replaceState()`

- location.hash change
- click back button

1. Добавляет запись в историю
2. Заменяет текущую запись
3. Меняет URL после #, может вызвать hashchange
4. Триггерит событие popstate

🔗 Подсказка:

Обратите внимание на разницу между добавлением и заменой записи истории.

- **history.pushState()** — Добавляет запись в историю
- **history.replaceState()** — Заменяет текущую запись
- **location.hash change** — Меняет URL после #, может вызвать hashchange
- **click back button** — Триггерит событие popstate

🔗 Шаг 09:

Миссия 9

Формулировка: Сопоставьте типы навигации с их примерами.

Ответ:

- hash navigation
 - history navigation
 - full page reload
 - external navigation
1. mysite.com/#section2
 2. mysite.com/about (через pushState)
 3. mysite.com/contact.html
 4. google.com

🔗 Подсказка:

Смотрите на символы # и использование pushState.

- **hash navigation** — mysite.com/#section2
- **history navigation** — mysite.com/about (через pushState)
- **full page reload** — mysite.com/contact.html
- **external navigation** — google.com

🔗 Шаг 10:

Миссия 10

Формулировка: Сопоставьте методы работы с URL и их ограничения.

Ответ:

- location.hash
 - history.pushState
 - history.replaceState
 - window.location.href
1. Только часть после #, перезагрузка не требуется
 2. Можно менять путь и параметры, без перезагрузки
 3. Заменяет текущий state без добавления новой записи

4. Полная загрузка нового URL

Подсказка:

Подумайте, какие методы вызывают перезагрузку, а какие нет.

- **location.hash** — Только часть после #, перезагрузка не требуется
- **history.pushState** — Можно менять путь и параметры, без перезагрузки
- **history.replaceState** — Заменяет текущий state без добавления новой записи
- **window.location.href** — Полная загрузка нового URL

Шаг 11:

Миссия 11

Вопрос: Расположите шаги внедрения hash-навигации в SPA в правильном порядке:

1. Добавить события для отслеживания изменения location.hash
2. Создать ссылки с href='#section'
3. Обработать изменение хэша и отобразить соответствующий контент
4. Тестировать навигацию без перезагрузки страницы

Подсказка:

Сначала создаются ссылки, затем добавляются слушатели и логика обработки.

Ответ:

- Создать ссылки с href='#section'
- Добавить события для отслеживания изменения location.hash
- Обработать изменение хэша и отобразить соответствующий контент
- Тестировать навигацию без перезагрузки страницы

Шаг 12:

Миссия 12

Вопрос: Расположите шаги внедрения history.pushState навигации:

1. Создать обработчик popstate
2. Вызвать history.pushState с новым URL
3. Обновить контент страницы в соответствии с URL
4. Тестировать работу кнопок вперед/назад

Подсказка:

Сначала настраиваем обработчик событий, потом меняем URL и контент, потом тестируем.

Ответ:

- Создать обработчик popstate
- Вызвать history.pushState с новым URL
- Обновить контент страницы в соответствии с URL
- Тестировать работу кнопок вперед/назад

Шаг 13:

Миссия 13

Вопрос: Расположите шаги реализации универсальной навигации с hash и pushState:

1. Определить поддерживаемые пути и хэши
2. Добавить обработчики изменения hash и popstate

3. Рендерить контент в зависимости от текущего URL или hash
4. Обновлять URL через pushState при внутренних переходах
5. Тестировать работу кнопок назад/вперед

🔗 Подсказка:

Сначала планируем пути, потом добавляем слушатели и логику, затем тестируем.

Ответ:

- Определить поддерживаемые пути и хэши
- Добавить обработчики изменения hash и popstate
- Рендерить контент в зависимости от текущего URL или hash
- Обновлять URL через pushState при внутренних переходах
- Тестировать работу кнопок назад/вперед

🔗 Шаг 14:

Миссия 14

Саша делает одностраничное приложение (SPA). Какой метод JavaScript позволит изменить URL без перезагрузки страницы?

Подсказка: Подумайте о методах, которые работают с историей браузера.

Ответ:

- **history.pushState**
- document.location.reload
- window.alert
- console.log

🔗 Шаг 15:

Миссия 15

Лена хочет, чтобы при переходе к секции страницы URL изменялся на #about. Какой способ лучше использовать?

Подсказка: Подумайте о коротких идентификаторах в адресной строке.

Ответ:

- **location.hash = 'about'**
- history.pushState(null,null,'/home')
- document.write('about')
- console.log('#about')

🔗 Шаг 16:

Миссия 15

Сергей использует location.hash для навигации. Что произойдет, если присвоить location.hash значение '#contact'?

Подсказка: Подумайте, как браузер реагирует на якоря.

Ответ:

- **Браузер прокрутит страницу к элементу с id='contact'**
- Перезагрузится вся страница
- Выведется alert с текстом 'contact'
- Скрипт завершится с ошибкой

Урок 2 - Создание своего Router-а на JS

Шаг 01:

Миссия 1

Аня хочет, чтобы при переходе между страницами SPA URL менялся без перезагрузки. Какой метод JavaScript лучше использовать?

Подсказка: Подумайте о методах работы с историей браузера.

Ответ:

- **history.pushState**
- location.reload
- console.log('page')
- window.alert('change')

Шаг 02:

Миссия 2

Борис хочет реализовать переход на секцию #about без перезагрузки страницы. Какой вариант правильный?

Подсказка: Подумайте о коротких идентификаторах в URL.

Ответ:

- **location.hash = 'about'**
- history.pushState(null,null,'/home')
- document.write('about')
- alert('#about')

Шаг 03:

Миссия 3

Кирилл хочет сохранить состояние фильтров на странице и обновлять URL при изменении без перезагрузки. Что ему подходит?

Подсказка: Подумайте, какой метод позволяет добавлять новые записи в историю браузера.

Ответ:

- **history.pushState**
- location.reload
- alert('saved')
- document.getElementById('filters')

Шаг 04:

Миссия 4

Марина хочет, чтобы при нажатии кнопки "Назад" пользователь возвращался к предыдущей секции SPA. Какой подход правильный?

Подсказка: Подумайте о событиях, связанных с историей браузера.

Ответ:

- **Использовать событие window.onpopstate**
- window.scrollTo(0,0)
- alert('Back pressed')
- location.reload()

🔗 Шаг 05:

Миссия 5

Саша использует `location.hash` для навигации. Что произойдет, если присвоить `location.hash` значение `'#contact'`?

Подсказка: Подумайте, как браузер реагирует на якоря.

Ответ:

- **Браузер прокрутит страницу к элементу с `id='contact'`**
- Перезагрузится вся страница
- Выведется `alert` с текстом `'contact'`
- Скрипт завершится с ошибкой

🔗 Шаг 06:

Миссия 6

Формулировка: Сопоставьте методы Router-а с их назначением.

Ответ:

- `history.pushState`
 - `history.replaceState`
 - `location.hash`
 - `window.onpopstate`
1. Добавляет новую запись в историю без перезагрузки
 2. Заменяет текущую запись истории
 3. Меняет часть URL после `#` и прокручивает к элементу
 4. Событие, вызываемое при изменении состояния истории

🔗 **Подсказка:**

Подумайте, какие методы создают новые записи истории, а какие заменяют существующие.

- **`history.pushState`** — Добавляет новую запись в историю без перезагрузки
- **`history.replaceState`** — Заменяет текущую запись истории
- **`location.hash`** — Меняет часть URL после `#` и прокручивает к элементу
- **`window.onpopstate`** — Событие, вызываемое при изменении состояния истории

🔗 Шаг 07:

Миссия 7

Формулировка: Сопоставьте элементы Router-а с их функцией.

Ответ:

- `routes`
 - `currentPath`
 - `navigate()`
 - `render()`
1. Содержит объект с URL и соответствующими функциями для отрисовки
 2. Хранит текущий путь из адресной строки
 3. Меняет путь и вызывает отрисовку
 4. Отвечает за отображение контента для текущего пути

🔗 **Подсказка:**

Подумайте, какой элемент хранит данные, а какой отвечает за действия.

- **routes** — Содержит объект с URL и соответствующими функциями для отрисовки
- **currentPath** — Хранит текущий путь из адресной строки
- **navigate()** — Меняет путь и вызывает отрисовку
- **render()** — Отвечает за отображение контента для текущего пути

🔗 Шаг 08:

Миссия 8

Формулировка: Сопоставьте события Router-а с их описанием.

Ответ:

- hashchange
 - popstate
 - DOMContentLoaded
 - click on link
1. Вызывается при изменении location.hash
 2. Вызывается при навигации назад/вперед
 3. Срабатывает при полной загрузке HTML документа
 4. Вызывается при клике на ссылку для внутреннего роутинга

🔗 Подсказка:

Подумайте, какие события связаны с URL, а какие с загрузкой страницы.

- **hashchange** — Вызывается при изменении location.hash
- **popstate** — Вызывается при навигации назад/вперед
- **DOMContentLoaded** — Срабатывает при полной загрузке HTML документа
- **click on link** — Вызывается при клике на ссылку для внутреннего роутинга

🔗 Шаг 09:

Миссия 9

Вопрос: Расположите шаги навигации Router-а при клике на ссылку в правильном порядке:

1. Событие click перехвачено Router-ом
2. Вызов navigate() с новым путем
3. Обновление currentPath
4. Вызов render() для отображения нового контента

🔗 Подсказка:

Подумайте, что происходит сначала: событие, затем изменение состояния, потом рендер.

Ответ:

- Событие click перехвачено Router-ом
- Вызов navigate() с новым путем
- Обновление currentPath
- Вызов render() для отображения нового контента

🔗 Шаг 10:

Миссия 10

Саша хочет, чтобы его SPA мог менять адрес без перезагрузки. Какой метод лучше использовать?

Подсказка: Подумайте о работе с историей браузера.

Ответ:

- **history.pushState**
- location.reload
- document.write
- alert

Шаг 11:

Миссия 11

Лена хочет добавить возможность перехода к секции #about без перезагрузки страницы. Что использовать?

Подсказка: Думайте о якорях в URL.

Ответ:

- **location.hash = 'about'**
- history.replaceState(null,null,'/about')
- document.write('about')
- alert('#about')

Шаг 12:

Миссия 12

Петя хочет, чтобы его Router сохранял состояние фильтров при смене URL без перезагрузки. Какой метод использовать?

Подсказка: Смотрите на методы, которые изменяют историю без обновления страницы.

Ответ:

- **history.pushState**
- location.reload
- alert('state saved')
- document.getElementById('filters')

Шаг 13:

Миссия 13

Ира хочет, чтобы при нажатии кнопки "Назад" пользователь возвращался к предыдущей секции SPA. Что делать?

Подсказка: Думайте о событиях, связанных с историей браузера.

Ответ:

- **window.onpopstate = () => {}**
- window.scrollTo(0,0)
- alert('Back pressed')
- location.reload()

Шаг 14:

Миссия 14

Сергей использует `location.hash` для навигации. Что произойдет, если присвоить `location.hash` значение `'#contact'`?

Подсказка: Подумайте, как браузер реагирует на якоря.

Ответ:

- **Браузер прокрутит страницу к элементу с `id='contact'`**
- Перезагрузится вся страница
- Выведется `alert` с текстом `'contact'`
- Скрипт завершится с ошибкой

Шаг 15:

Миссия 15

Формулировка: Сопоставьте методы Router-а с их описанием.

Ответ:

- `history.pushState`
 - `history.replaceState`
 - `location.hash`
 - `window.onpopstate`
1. Добавляет новую запись в историю без перезагрузки
 2. Заменяет текущую запись истории
 3. Меняет часть URL после `#` и прокручивает к элементу
 4. Событие, вызываемое при изменении состояния истории

 **Подсказка:**

Подумайте, какие методы добавляют новые записи, а какие просто заменяют существующие.

- **`history.pushState`** — Добавляет новую запись в историю без перезагрузки
- **`history.replaceState`** — Заменяет текущую запись истории
- **`location.hash`** — Меняет часть URL после `#` и прокручивает к элементу
- **`window.onpopstate`** — Событие, вызываемое при изменении состояния истории

Шаг 16:

Миссия 16

Формулировка: Сопоставьте элементы Router-а с их задачей.

Ответ:

- `Route`
 - `Router`
 - `navigate()`
 - `onPopState`
1. Определяет, какой компонент рендерить для URL
 2. Хранит и управляет всеми маршрутами
 3. Метод для перехода между маршрутами
 4. Обрабатывает переход назад/вперед в истории

 **Подсказка:**

Подумайте, кто отвечает за логику, а кто за обработку событий браузера.

- **`Route`** — Определяет, какой компонент рендерить для URL

- **Router** — Хранит и управляет всеми маршрутами
- **navigate()** — Метод для перехода между маршрутами
- **onPopState** — Обрабатывает переход назад/вперед в истории

Урок 3 - Основы SPA-навигации

Шаг 01:

Миссия 1

Максим создает одностраничное приложение и хочет изменять URL без перезагрузки страницы. Какой метод ему выбрать?

Подсказка: Подумайте о методах, работающих с историей браузера.

Ответ:

- **history.pushState**
- location.reload
- document.write
- alert

Шаг 02:

Миссия 2

Аня хочет, чтобы при переходе к секции #portfolio страница не перезагружалась. Как ей это сделать?

Подсказка: Думайте о якорях и hash в URL.

Ответ:

- **location.hash = 'portfolio'**
- history.replaceState(null,null,'/portfolio')
- document.write('portfolio')
- location.reload()

Шаг 03:

Миссия 3

Игорь хочет сохранять состояние фильтров при смене маршрута без перезагрузки. Какой метод лучше использовать?

Подсказка: Смотрите на методы, изменяющие историю без обновления страницы.

Ответ:

- **history.pushState**
- location.reload
- alert('state saved')
- document.getElementById('filters')

Шаг 04:

Миссия 4

Света хочет, чтобы при нажатии кнопки "Назад" пользователь возвращался к предыдущей секции SPA. Как это реализовать?

Подсказка: Думайте о событиях изменения истории браузера.

Ответ:

- **window.onpopstate = () => { }**
- window.scrollTo(0,0)
- alert('Back pressed')
- location.reload()

 **Шаг 05:**

Миссия 5

Вадим использует location.hash для навигации. Что произойдет, если присвоить location.hash значение '#services'?

Подсказка: Подумайте, как браузер реагирует на якоря.

Ответ:

- **Браузер прокрутит страницу к элементу с id='services'**
- Перезагрузится вся страница
- Выведется alert с текстом 'services'
- Скрипт завершится с ошибкой

 **Шаг 06:**

Миссия 6

Формулировка: Сопоставьте методы SPA-навигации с их действием.

Ответ:

- history.pushState
 - history.replaceState
 - location.hash
 - window.onpopstate
1. Добавляет новое состояние в историю без перезагрузки
 2. Заменяет текущее состояние истории
 3. Изменяет часть URL после # и прокручивает к элементу
 4. Событие, вызываемое при изменении состояния истории

 **Подсказка:**

Обратите внимание на разницу между добавлением и заменой состояния.

- **history.pushState** — Добавляет новое состояние в историю без перезагрузки
- **history.replaceState** — Заменяет текущее состояние истории
- **location.hash** — Изменяет часть URL после # и прокручивает к элементу
- **window.onpopstate** — Событие, вызываемое при изменении состояния истории

 **Шаг 07:**

Миссия 7

Формулировка: Сопоставьте элементы Router-а с их задачей.

Ответ:

- Route
- Router
- navigate()

- `onPopState`

1. Определяет компонент для конкретного URL
2. Хранит и управляет маршрутами
3. Метод перехода между маршрутами
4. Обработка нажатия "Назад/Вперед" в браузере

🔗 Подсказка:

Подумайте, кто отвечает за логику, а кто за события браузера.

- **Route** — Определяет компонент для конкретного URL
- **Router** — Хранит и управляет маршрутами
- **navigate()** — Метод перехода между маршрутами
- **onPopState** — Обработка нажатия "Назад/Вперед" в браузере

🔗 Шаг 08:

Миссия 8

Формулировка: Сопоставьте методы и их эффект.

Ответ:

- `history.pushState`
 - `history.replaceState`
 - `location.hash`
 - `window.onpopstate`
1. Добавление нового состояния в историю
 2. Замена текущего состояния
 3. Прокрутка к элементу с `id`
 4. Обработка клика "Назад" в браузере

🔗 Подсказка:

Различайте добавление и замену состояния.

- **history.pushState** — Добавление нового состояния в историю
- **history.replaceState** — Замена текущего состояния
- **location.hash** — Прокрутка к элементу с `id`
- **window.onpopstate** — Обработка клика "Назад" в браузере

🔗 Шаг 09:

Миссия 9

Вопрос: Расположите шаги создания SPA Router-а на JS в правильном порядке:

1. Определение маршрутов и компонентов
2. Создание метода `navigate()`
3. Обработка событий истории (`popstate`)
4. Инициализация Router и отрисовка текущего компонента

🔗 Подсказка:

Сначала маршруты, потом методы навигации и события, в конце инициализация.

Ответ:

- Определение маршрутов и компонентов
- Создание метода `navigate()`

- Обработка событий истории (popstate)
- Инициализация Router и отрисовка текущего компонента

🔗 Шаг 10:

Миссия 10

Вопрос: Расположите действия при клике на ссылку внутри SPA:

1. Вызов `navigate()` с новым URL
2. `history.pushState` добавляет запись в историю
3. Компонент для нового URL рендерится
4. URL обновляется без перезагрузки

🔗 Подсказка:

От клика до рендеринга компонента последовательность должна быть логичной.

Ответ:

- Вызов `navigate()` с новым URL
- `history.pushState` добавляет запись в историю
- Компонент для нового URL рендерится
- URL обновляется без перезагрузки

🔗 Шаг 11:

Миссия 11

Аня хочет, чтобы её SPA менял адрес в браузере без перезагрузки страницы. Какой метод использовать?

Подсказка: Подумайте о возможностях History API.

Ответ:

- **`history.pushState`**
- `location.reload`
- `document.write`
- `alert('update')`

🔗 Шаг 12:

Миссия 12

Ваня хочет использовать якорь для перехода к секции `#about` без перезагрузки. Что делать?

Подсказка: Думайте о свойствах `location`.

Ответ:

- **`location.hash = 'about'`**
- `history.pushState(null,null,'/about')`
- `document.write('about')`
- `alert('#about')`

🔗 Шаг 13:

Миссия 13

Костя хочет, чтобы при смене URL SPA сохранял состояние фильтров. Что использовать?

Подсказка: Подумайте о методах, изменяющих историю без перезагрузки.

Ответ:

- **history.pushState**
- location.reload
- alert('state saved')
- document.getElementById('filters')

🔗 **Шаг 14:**

Миссия 14

Даша хочет, чтобы кнопка "Назад" возвращала пользователя к предыдущей секции SPA. Что сделать?

Подсказка: Думайте о событиях браузера.

Ответ:

- **window.onpopstate = () => {}**
- window.scrollTo(0,0)
- alert('Back pressed')
- location.reload()

🔗 **Шаг 15:**

Миссия 15

Саша использует location.hash = '#contact'. Что произойдет?

Подсказка: Подумайте, как браузер работает с якорями.

Ответ:

- **Страница прокрутится к элементу с id='contact'**
- Перезагрузится страница
- Выведется alert с текстом 'contact'
- Скрипт завершится с ошибкой

🔗 **Шаг 16:**

Миссия 16

Формулировка: Сопоставьте методы History API с их функцией.

Ответ:

- history.pushState
 - history.replaceState
 - location.hash
 - window.onpopstate
1. Добавляет запись в историю без перезагрузки
 2. Заменяет текущую запись истории
 3. Меняет якорь и прокручивает к элементу
 4. Вызывается при изменении состояния истории

🔗 **Подсказка:**

Обратите внимание на различие между добавлением и заменой записи.

- **history.pushState** — Добавляет запись в историю без перезагрузки
- **history.replaceState** — Заменяет текущую запись истории
- **location.hash** — Меняет якорь и прокручивает к элементу

- **window.onpopstate** — Вызывается при изменении состояния истории

Модуль 16 - TypeScript для JS-разработчиков

Урок 1 - Типы, интерфейсы, функции

Шаг 01:

Миссия 1

Аня хочет указать тип переменной, которая хранит возраст пользователя. Какой тип выбрать?

Подсказка: Возраст — это число.

Ответ:

- **number**
- string
- boolean
- any

Шаг 02:

Миссия 2

Ваня хочет, чтобы функция возвращала строку с приветствием. Как правильно указать тип возвращаемого значения?

Подсказка: Смотрите на то, что возвращает функция.

Ответ:

- **string**
- number
- void
- boolean

Шаг 03:

Миссия 3

Костя создаёт объект пользователя с полями name и age. Как указать тип для этого объекта?

Подсказка: Думайте об интерфейсе или типе объекта.

Ответ:

- **{ name: string; age: number }**
- [string, number]
- string
- any

Шаг 04:

Миссия 4

Даша хочет объявить массив чисел. Какой тип использовать?

Подсказка: Подумайте о синтаксисе массивов в TypeScript.

Ответ:

- **number[]**
- string[]
- Array

- `boolean[]`

🔮 Шаг 05:

Миссия 5

Саша хочет, чтобы функция принимала два числа и возвращала их сумму. Как указать тип функции?

Подсказка: Смотрите на аргументы и возвращаемое значение.

Ответ:

- **`(a: number, b: number) => number`**
- `(a: string, b: string) => string`
- `(a: number, b: number) => void`
- `any`

🔮 Шаг 06:

Миссия 6

Формулировка: Сопоставьте типы с примерами:

Ответ:

- `string`
 - `number`
 - `boolean`
 - `any`
1. `'Hello World'`
 2. `42`
 3. `true`
 4. `{ name: 'Alice' }`

🔮 Подсказка:

Подумайте, какой тип соответствует значению.

- **`string`** — `'Hello World'`
- **`number`** — `42`
- **`boolean`** — `true`
- **`any`** — `{ name: 'Alice' }`

🔮 Шаг 07:

Миссия 7

Формулировка: Сопоставьте интерфейс с его описанием:

Ответ:

- `User`
 - `Product`
 - `Order`
1. Определяет свойства пользователя
 2. Содержит характеристики товара
 3. Хранит информацию о заказе

🔮 Подсказка:

Подумайте, кто является субъектом, а кто объектом.

- **User** — Определяет свойства пользователя
- **Product** — Содержит характеристики товара
- **Order** — Хранит информацию о заказе

Шаг 08:

Миссия 8

Формулировка: Сопоставьте функции TypeScript с их типами:

Ответ:

- sum
- greet
- isAdult

1. (a: number, b: number) => number
2. (name: string) => string
3. (age: number) => boolean

Подсказка:

Думайте, что возвращает каждая функция.

- **sum** — (a: number, b: number) => number
- **greet** — (name: string) => string
- **isAdult** — (age: number) => boolean

Шаг 09:

Миссия 9

Вопрос: Расположите шаги при создании интерфейса TypeScript:

1. Определяем свойства объекта
2. Указываем типы свойств
3. Создаём интерфейс с ключевым словом interface
4. Используем интерфейс в функции или объекте

Подсказка:

Сначала планируем свойства, потом создаём интерфейс, затем применяем.

Ответ:

- Определяем свойства объекта
- Указываем типы свойств
- Создаём интерфейс с ключевым словом interface
- Используем интерфейс в функции или объекте

Шаг 10:

Миссия 10

Вопрос: Расположите шаги при описании функции с типами:

1. Определяем аргументы функции
2. Указываем тип возвращаемого значения
3. Реализуем тело функции
4. Проверяем использование функции

Подсказка:

Сначала объявляем сигнатуру, потом тело и тестируем.

Ответ:

- Определяем аргументы функции
- Указываем тип возвращаемого значения
- Реализуем тело функции
- Проверяем использование функции

Шаг 11:

Миссия 11

Ирина хочет объявить переменную, которая может содержать только строку. Какой тип указать в TypeScript?

Подсказка: Типы указываются после двоеточия.

Ответ:

- **string**
- number
- boolean
- any

Шаг 12:

Миссия 12

Петя хочет создать функцию, которая возвращает число. Как правильно указать тип возвращаемого значения?

Подсказка: Синтаксис: имяФункции(): тип { }

Ответ:

- **function getNumber(): number { return 42; }**
- function getNumber() { return 42; }
- let getNumber: number = 42;
- function getNumber(): string { return '42'; }

Шаг 13:

Миссия 13

Антон хочет, чтобы переменная могла быть числом или строкой. Какой тип использовать?

Подсказка: TypeScript поддерживает объединение типов с помощью |.

Ответ:

- **number | string**
- any
- object
- boolean

Шаг 14:

Миссия 14

Оля хочет описать объект с полями name (строка) и age (число). Какой синтаксис использовать?

Подсказка: TypeScript позволяет указывать типы полей через :.

Ответ:

- **{ name: string; age: number }**
- { name = string, age = number }
- [name: string, age: number]
- object(name: string, age: number)

Шаг 15:

Миссия 15

Саша хочет сделать поле optional в интерфейсе TypeScript. Как это указать?

Подсказка: Optional-поля используют специальный знак после имени поля.

Ответ:

- **age?: number**
- age!: number
- age: number | undefined
- optional age: number

Шаг 16:

Миссия 16

Формулировка: Сопоставьте типы TypeScript с их описанием.

Ответ:

- string
 - number
 - boolean
 - any
1. Текстовые значения
 2. Числовые значения
 3. Логические значения
 4. Любой тип значения

Подсказка:

Подумайте о том, какие данные хранят строки, числа и логика.

- **string** — Текстовые значения
- **number** — Числовые значения
- **boolean** — Логические значения
- **any** — Любой тип значения

Урок 2 - Работа с DOM и API в TS

Шаг 01:

Миссия 1

Аня хочет получить элемент кнопки по id в TypeScript. Какой синтаксис будет правильным?

Подсказка: Используйте document.getElementById и приведите тип, если нужно.

Ответ:

- **const btn = document.getElementById('myBtn') as HTMLButtonElement**
- const btn: string = document.getElementById('myBtn')

- `const btn = document.querySelector('#myBtn') as string`
- `const btn: any = document.getElementById('myBtn')`

🔗 Шаг 02:

Миссия 2

Максим хочет навесить обработчик события `click` на кнопку в TypeScript. Какой тип использовать для события?

Подсказка: Подумайте, что `document` и элементы имеют события определённых типов.

Ответ:

- **`(e: MouseEvent) => void`**
- `(e: EventTarget) => void`
- `(e: string) => void`
- `(e: HTMLElement) => void`

🔗 Шаг 03:

Миссия 3

Оля хочет получить значение `input` поля типа `text`. Как правильно указать тип в TypeScript?

Подсказка: Не забудьте привести тип к `HTMLInputElement` для доступа к `value`.

Ответ:

- **`const input = document.getElementById('name') as HTMLInputElement`**
- `const input: string = document.getElementById('name')`
- `const input = document.querySelector('input') as string`
- `const input: any = document.getElementById('name')`

🔗 Шаг 04:

Миссия 4

Дима хочет выполнить `fetch`-запрос к API и получить JSON. Как правильно указать тип ответа?

Подсказка: В TypeScript полезно указать ожидаемую структуру объекта.

Ответ:

- **`fetch('/api').then(res => res.json() as Promise)`**
- `fetch('/api').then(res => res.json() as string)`
- `fetch('/api').then(res => res.text() as Promise)`
- `fetch('/api').then(res => res.json() as any[])`

🔗 Шаг 05:

Миссия 5

Наташа хочет использовать `querySelector` для `div` и быть уверенной в типе элемента. Как правильно написать?

Подсказка: Приведение типов позволяет избежать ошибок компиляции.

Ответ:

- **`const div = document.querySelector('div') as HTMLDivElement`**
- `const div: HTMLElement = document.querySelector('div')`
- `const div = document.querySelector('div') as string`
- `const div: any = document.querySelector('div')`

🔮 Шаг 06:

Миссия 6

Формулировка: Сопоставьте методы работы с DOM с их назначением:

Ответ:

- `getElementById`
- `querySelector`
- `querySelectorAll`
- `addEventListener`

1. Получить один элемент по CSS-селектору
2. Получить все элементы по CSS-селектору
3. Получить элемент по `id`
4. Навесить обработчик события на элемент

🔮 Подсказка:

Смотрите на название метода и его назначение.

- **`getElementById`** — Получить элемент по `id`
- **`querySelector`** — Получить один элемент по CSS-селектору
- **`querySelectorAll`** — Получить все элементы по CSS-селектору
- **`addEventListener`** — Навесить обработчик события на элемент

🔮 Шаг 07:

Миссия 7

Формулировка: Сопоставьте типы событий с примерами:

Ответ:

- `MouseEvent`
- `KeyboardEvent`
- `FocusEvent`
- `InputEvent`

1. Клик мышкой
2. Нажатие клавиши
3. Получение фокуса на `input`
4. Изменение текста в `input`

🔮 Подсказка:

Смотрите на источник события: мышь, клавиатура, поле ввода.

- **`MouseEvent`** — Клик мышкой
- **`KeyboardEvent`** — Нажатие клавиши
- **`FocusEvent`** — Получение фокуса на `input`
- **`InputEvent`** — Изменение текста в `input`

🔮 Шаг 08:

Миссия 8

Формулировка: Сопоставьте методы `fetch` с их назначением:

Ответ:

- `fetch(url)`
 - `res.json()`
 - `res.text()`
 - `res.blob()`
1. Получение JSON объекта
 2. Получение текста
 3. Получение бинарного объекта
 4. Отправка запроса на сервер

 Подсказка:

Подумайте, что возвращает каждый метод и для чего он предназначен.

- **`fetch(url)`** — Отправка запроса на сервер
- **`res.json()`** — Получение JSON объекта
- **`res.text()`** — Получение текста
- **`res.blob()`** — Получение бинарного объекта

 Шаг 09:

Миссия 9

Вопрос: Расположите шаги получения значения `input` и отправки его на API:

1. Получаем элемент `input`
2. Навешиваем обработчик события
3. Берем `value` и формируем объект
4. Отправляем объект через `fetch`

 Подсказка:

Сначала получаем элемент, затем реагируем на событие, формируем данные и отправляем.

Ответ:

- Получаем элемент `input`
- Навешиваем обработчик события
- Берем `value` и формируем объект
- Отправляем объект через `fetch`

 Шаг 10:

Миссия 10

Аня хочет получить доступ к элементу с `id 'app'` на странице. Какой тип использовать в TypeScript?

Подсказка: DOM-элементы можно типизировать с помощью `HTMLElement`.

Ответ:

- **`const app: HTMLElement | null = document.getElementById('app')`**
- `const app: string = document.getElementById('app')`
- `const app: number = document.getElementById('app')`
- `const app: any = document.querySelector('div')`

🔗 Шаг 11:

Миссия 11

Иван хочет добавить обработчик события 'click' на кнопку с id 'submit'. Как правильно типизировать аргумент event?

Подсказка: События в TypeScript имеют свои типы, например MouseEvent для клика.

Ответ:

- **button.addEventListener('click', (event: MouseEvent) => {})**
- button.addEventListener('click', (event: EventTarget) => {})
- button.addEventListener('click', (event: any) => {})
- button.addEventListener('click', (event: string) => {})

🔗 Шаг 12:

Миссия 12

Мария хочет получить значение input с id 'username'. Какой тип будет правильным?

Подсказка: Значения input доступны через свойство value, которое является строкой.

Ответ:

- **const username: string = (document.getElementById('username') as HTMLInputElement).value**
- const username: number = document.getElementById('username').value
- const username: HTMLElement = document.getElementById('username').value
- const username: any = document.getElementById('username')

🔗 Шаг 13:

Миссия 13

Сергей хочет сделать fetch-запрос к API и получить JSON-ответ. Как типизировать данные?

Подсказка: Ответ API можно типизировать с помощью интерфейса, описывающего структуру данных.

Ответ:

- **interface User { id: number; name: string } const response = await fetch(url); const data: User = await response.json()**
- const data: any = await response.json()
- const data: string = await response.json()
- const data: number = await response.json()

🔗 Шаг 14:

Миссия 14

Елена хочет обработать ошибку fetch-запроса. Как правильно типизировать объект ошибки?

Подсказка: Ошибки обычно имеют тип Error.

Ответ:

- **try { ... } catch (error: unknown) { if (error instanceof Error) console.log(error.message) }**
- catch (error: string)
- catch (error: any)
- catch (error: number)

🔮 Шаг 15:

Миссия 15

Формулировка: Сопоставьте методы DOM с их назначением.

Ответ:

- `document.getElementById`
- `document.querySelector`
- `document.createElement`
- `element.appendChild`

1. Находит элемент по id
2. Находит первый элемент по CSS-селектору
3. Создает новый элемент
4. Добавляет элемент внутрь другого

🔮 Подсказка:

Подумайте, какой метод выбирает элемент, а какой создает новый.

- **`document.getElementById`** — Находит элемент по id
- **`document.querySelector`** — Находит первый элемент по CSS-селектору
- **`document.createElement`** — Создает новый элемент
- **`element.appendChild`** — Добавляет элемент внутрь другого

🔮 Шаг 16:

Миссия 16

Формулировка: Сопоставьте типы событий с их описанием.

Ответ:

- `click`
- `input`
- `submit`
- `load`

1. Срабатывает при клике пользователя
2. Изменение значения `input`
3. Отправка формы
4. Загрузка документа или ресурса

🔮 Подсказка:

Подумайте о том, когда каждое событие возникает.

- **`click`** — Срабатывает при клике пользователя
- **`input`** — Изменение значения `input`
- **`submit`** — Отправка формы
- **`load`** — Загрузка документа или ресурса

📖 Урок 3 - Конфигурация и миграция с JS

🔮 Шаг 01:

Миссия 1

Андрей хочет подключить TypeScript к существующему проекту на JavaScript. Что нужно сделать в первую очередь?

Подсказка: Настройка TS начинается с создания конфигурационного файла.

Ответ:

- **Создать файл `tsconfig.json`**
- Сразу переименовать все `.js` файлы в `.ts`
- Установить только `@types/node`
- Игнорировать настройки компилятора

Шаг 02:

Миссия 2

Виктор хочет мигрировать функцию из JS в TS. Как правильно добавить типы для аргументов и возвращаемого значения?

Подсказка: Начните с простых типов, например `number` или `string`.

Ответ:

- **`function sum(a: number, b: number): number { return a + b }`**
- `function sum(a, b) { return a + b }`
- `function sum(a: any, b: any) { return a + b }`
- `function sum(a: string, b: string): number { return a + b }`

Шаг 03:

Миссия 3

Ольга хочет запретить использование неинициализированных переменных в проекте. Какой флаг `tsconfig` нужно включить?

Подсказка: Смотрите на строгие проверки типов.

Ответ:

- **`"strict": true`**
- `"allowJs": true`
- `"noEmit": true`
- `"target": "ES6"`

Шаг 04:

Миссия 4

Игорь хочет включить поддержку модулей ES в TypeScript. Какой параметр `tsconfig` нужно использовать?

Подсказка: Проверьте, как настроены `module` и `target`.

Ответ:

- **`"module": "ESNext"`**
- `"module": "CommonJS"`
- `"allowJs": true`
- `"strict": true`

Шаг 05:

Миссия 5

Наталья хочет игнорировать файлы JS при компиляции TypeScript. Какой параметр tsconfig ей поможет?

Подсказка: Иногда проект содержит смешанный код.

Ответ:

- **"allowJs": false**
- "strict": true
- "esModuleInterop": true
- "target": "ES6"

Шаг 06:

Миссия 6

Формулировка: Сопоставьте флаги tsconfig с их значением.

Ответ:

- allowJs
 - strict
 - target
 - module
1. Разрешает компиляцию файлов .js
 2. Включает строгую проверку типов
 3. Определяет версию ECMAScript для вывода
 4. Определяет систему модулей

Подсказка:

Подумайте, что делает каждый параметр и на что влияет.

- **allowJs** — Разрешает компиляцию файлов .js
- **strict** — Включает строгую проверку типов
- **target** — Определяет версию ECMAScript для вывода
- **module** — Определяет систему модулей

Шаг 07:

Миссия 7

Формулировка: Сопоставьте параметры компилятора с их эффектом.

Ответ:

- esModuleInterop
 - noEmit
 - outDir
 - rootDir
1. Позволяет импортировать CommonJS модули как ES
 2. Запрещает генерацию файлов JS
 3. Папка для вывода скомпилированных файлов
 4. Корневая папка исходников

Подсказка:

Обратите внимание на настройку пути и поведения компилятора.

- **esModuleInterop** — Позволяет импортировать CommonJS модули как ES

- **noEmit** — Запрещает генерацию файлов JS
- **outDir** — Папка для вывода скомпилированных файлов
- **rootDir** — Корневая папка исходников

🔗 Шаг 08:

Миссия 8

Формулировка: Сопоставьте файлы миграции с их назначением.

Ответ:

- index.js
- app.ts
- utils.ts
- config.json

1. Точка входа JS-проекта
2. Главный файл TS-приложения
3. Утилитарные функции
4. Конфигурация проекта

🔗 Подсказка:

Подумайте, какой файл обычно содержит основной код, а какой вспомогательный.

- **index.js** — Точка входа JS-проекта
- **app.ts** — Главный файл TS-приложения
- **utils.ts** — Утилитарные функции
- **config.json** — Конфигурация проекта

🔗 Шаг 09:

Миссия 9

Вопрос: Расположите шаги миграции проекта с JS на TS в правильном порядке:

1. Создать tsconfig.json
2. Переименовать файлы .js в .ts
3. Добавить типы для функций и переменных
4. Запустить компиляцию и исправить ошибки

🔗 Подсказка:

Сначала настраивается проект, потом идут изменения в коде, в конце проверка компиляции.

Ответ:

- Создать tsconfig.json
- Переименовать файлы .js в .ts
- Добавить типы для функций и переменных
- Запустить компиляцию и исправить ошибки

🔗 Шаг 10:

Миссия 10

Алексей хочет добавить TypeScript в существующий проект на JavaScript. Что нужно сделать в первую очередь?

Подсказка: Сначала нужно установить необходимые пакеты и создать конфигурацию.

Ответ:

- **Установить typescript и создать tsconfig.json**
- Сразу переписать все файлы на .ts
- Удалить все package.json зависимости
- Игнорировать конфигурацию TypeScript

🔧 **Шаг 11:**

Миссия 11

Мария хочет, чтобы TypeScript проверял только некоторые файлы проекта на JavaScript. Как это сделать?

Подсказка: В tsconfig.json можно указать include и exclude.

Ответ:

- **Указать пути файлов в include и exclude в tsconfig.json**
- Удалить ненужные файлы из проекта
- Переименовать все файлы в .ts
- Игнорировать файлы, TypeScript сам выберет

🔧 **Шаг 12:**

Миссия 12

Иван хочет использовать типы из существующих JavaScript библиотек. Что ему поможет?

Подсказка: Для многих JS-библиотек существуют пакеты с типами.

Ответ:

- **Установить @types/* для нужных библиотек**
- Игнорировать любые типы
- Писать свои типы для всех функций
- Менять код библиотеки под TypeScript

🔧 **Шаг 13:**

Миссия 13

Сопоставьте поля tsconfig.json с их назначением:

Ответ:

- compilerOptions
 - include
 - exclude
 - files
1. Настройки компилятора
 2. Файлы и директории, включаемые в компиляцию
 3. Файлы и директории, исключаемые из компиляции
 4. Явно указанные файлы для компиляции

🔧 **Подсказка:**

Подумайте, какие настройки влияют на поведение компилятора, а какие на выбор файлов.

- **compilerOptions** — Настройки компилятора
- **include** — Файлы и директории, включаемые в компиляцию
- **exclude** — Файлы и директории, исключаемые из компиляции

- **files** — Явно указанные файлы для компиляции

🔗 Шаг 14:

Миссия 14

Сопоставьте опции компилятора TypeScript с их эффектом:

Ответ:

- strict
 - target
 - module
 - outDir
1. Включает строгую проверку типов
 2. Версия JavaScript для компиляции
 3. Система модулей (CommonJS, ES6 и др.)
 4. Папка для скомпилированных файлов

🔗 Подсказка:

Подумайте, какая настройка влияет на проверку типов, а какая на формат вывода.

- **strict** — Включает строгую проверку типов
- **target** — Версия JavaScript для компиляции
- **module** — Система модулей (CommonJS, ES6 и др.)
- **outDir** — Папка для скомпилированных файлов

🔗 Шаг 15:

Миссия 15

Сопоставьте шаги миграции с их описанием:

Ответ:

- Установка TypeScript
 - Создание tsconfig.json
 - Переименование файлов .js в .ts
 - Добавление типов
1. Установить пакет typescript
 2. Настроить параметры компиляции
 3. Подготовить файлы к проверке типов
 4. Обеспечить строгую типизацию кода

🔗 Подсказка:

Подумайте о порядке, в котором безопаснее всего добавлять TS в проект.

- **Установка TypeScript** — Установить пакет typescript
- **Создание tsconfig.json** — Настроить параметры компиляции
- **Переименование файлов .js в .ts** — Подготовить файлы к проверке типов
- **Добавление типов** — Обеспечить строгую типизацию кода

🔗 Шаг 16:

Миссия 16

Вопрос: Расположите шаги безопасной миграции проекта с JS на TS:

1. Установка TypeScript и tsconfig.json
2. Проверка работы проекта с allowJs:true
3. Постепенное переименование файлов .js в .ts
4. Добавление строгой типизации и исправление ошибок

🔗 **Подсказка:**

Начните с установки и базовой настройки, потом постепенная миграция.

Ответ:

- Установка TypeScript и tsconfig.json
- Проверка работы проекта с allowJs:true
- Постепенное переименование файлов .js в .ts
- Добавление строгой типизации и исправление ошибок